2026 EDİSYONU · EĞİTİM & REHBER

AI Çağında Junior Yazılım Mühendisliği

Sıfırdan Uzmanlığa Yapay Zeka Destekli Mühendislik Rehberi

Yazar / Geliştirici: AI-Driven Junior Software Engineering Team **Kapsam:** 10 Ana Bölüm, 60 Kapsamlı Alt Bölüm **Format:** NotebookLM & Udemy Uyumlu Sesli/Videolu Müfredat Rehberi

İçindekiler Tablosu

AI Çağında Junior Yazılım Mühendisliği 15
Sıfırdan Uzmanlığa Yapay Zeka Destekli Mühendislik Rehberi 15
🎯 Kitabın Amacı ve AI Çağında Hayatta Kalma Rehberi 15
📋 Detaylı Müfredat & İçindekiler Tablosu 15
Bölüm 1: Yeni Dünyaya Giriş & AI Zihniyeti 19
1. İlk Gün Heyecanı: 'Yazılımcılık Bitti mi?' Mitini Yıkmak 19
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 20
🧠 2. AI Çağında Yeni Zihniyet Modeli 20
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 21
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 21
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 22
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 22
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 23
2. Geleneksel Junior vs. AI-Augmented Junior: 10x Zihniyet 23
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 24
🧠 2. AI Çağında Yeni Zihniyet Modeli 24
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 24
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 25
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 26
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 27
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 27
3. Modern AI Araç Setimiz: Copilot, Cursor, Claude Code ve Ötesi 28
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 28
🧠 2. AI Çağında Yeni Zihniyet Modeli 28
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 29
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 29
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 30
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 30
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 31
4. AI Halüsinasyonu ve Güven Tuzağı: Kod Denetçisi Olmak 31
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 32
🧠 2. AI Çağında Yeni Zihniyet Modeli 32
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 33

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 34
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 34
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 35
5. Prompt Mühendisliği Değil 'Context Mühendisliği': LLM'lere Nasıl Brif Verilir? 35
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 36
🧠 2. AI Çağında Yeni Zihniyet Modeli 36
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 37
KISITLAR: 37
BEKLENEN ÇIKTI TİPİ: 37
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 38
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 38
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 39
6. Geliştirme Ortamını (IDE) Turbo Şarj Etmek: Cursor Rules ve Kısayollar 39
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 40
🧠 2. AI Çağında Yeni Zihniyet Modeli 40
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 41
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 41
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 42
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 42
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 43
Bölüm 2: Algoritmik Düşünce, Veri Yapıları & AI ile Problem Çözme 43
7. Sözde Koddan (Pseudocode) Gerçek Koda: AI ile Algoritma Tasarlamak 44
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 44
🧠 2. AI Çağında Yeni Zihniyet Modeli 44
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 45
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 45
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 46
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 47
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 47
8. Veri Yapıları AI Çağında Neden Hâlâ Hayati? 48
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 48
🧠 2. AI Çağında Yeni Zihniyet Modeli 48
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 49
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 49
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 50
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 51
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 51
9. Zaman ve Bellek Karmaşıklığı (Big O): AI Üretti, Ama Verimli mi? 52
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 52
🧠 2. AI Çağında Yeni Zihniyet Modeli 52

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 53
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 54
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 55
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 55
10. Tip Sistemleri ve Clean Code: AI Çıktısını Type-Safe Kılmak 56
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 56
🧠 2. AI Çağında Yeni Zihniyet Modeli 56
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 57
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 59
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 60
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 60
11. Hata Mesajlarını ve Stack Trace'i AI ile Okuma ve Akıl Yürütme Sanatı 61
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 61
🧠 2. AI Çağında Yeni Zihniyet Modeli 62
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 62
İLGİLİ KOD (src/utils/card.ts - Satır 30-40): 62
GÖREV: 63
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 64
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 64
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 65
12. Kod Okuma Süper Gücü: Yabancı Bir Kod Tabanını AI ile 10 Dakikada Haritalamak 65
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 66
🧠 2. AI Çağında Yeni Zihniyet Modeli 66
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 67
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 67
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 68
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 68
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 69
Bölüm 3: Modern Git, Takım Çalışması & AI-Destekli Kod İnceleme 69
13. Git Temelleri & Akıllı Commit Stratejisi: AI ile Anlamlı Geçmiş 70
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 70
🧠 2. AI Çağında Yeni Zihniyet Modeli 70
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 71
GÖREV: 71
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 72
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 72
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 73
14. Merge Conflict Kabusunu AI ile 60 Saniyede Çözmek 73
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 74
🧠 2. AI Çağında Yeni Zihniyet Modeli 74

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 75
GÖREV: 75
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 76
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 76
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 77
15. Kusursuz Pull Request (PR) Yazma Sanatı: AI ile PR Açıklamaları 77
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 78
🧠 2. AI Çağında Yeni Zihniyet Modeli 78
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 79
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 79
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 80
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 81
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 81
16. Code Review'de Senior'ları Etkilemek: Kendi Kodunun Şeytanının Avukatı Ol 82
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 82
🧠 2. AI Çağında Yeni Zihniyet Modeli 82
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 83
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 85
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 85
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 85
17. GitHub Actions ve Otomasyon: AI Destekli CI/CD Boru Hatları 86
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 86
🧠 2. AI Çağında Yeni Zihniyet Modeli 87
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 87
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 88
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 90
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 90
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 90
18. Dokümantasyon Artık Yük Değil: Koddan Canlı Dokümantasyon Üretmek 91
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 91
🧠 2. AI Çağında Yeni Zihniyet Modeli 92
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 92
GÖREV: 92
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 94
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 94
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 94
Bölüm 4: Yazılım Mimarisi, Tasarım Kalıpları & Temiz Kod 95
19. Mimari Nedir ve Junior Neden Bilmelidir? Monolit vs. Mikroservis 95
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 96
🧠 2. AI Çağında Yeni Zihniyet Modeli 96

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 97
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 97
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 99
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 99
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 99
20. SOLID Prensipleri AI Promptlarına Nasıl Çevrilir? 100
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 100
🧠 2. AI Çağında Yeni Zihniyet Modeli 101
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 101
GÖREV: 101
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 103
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 104
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 104
21. Tasarım Kalıpları (Design Patterns): Factory, Strategy, Observer Pratikte 105
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 105
🧠 2. AI Çağında Yeni Zihniyet Modeli 105
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 106
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 106
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 108
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 109
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 109
22. Katmanlı Mimari & Separation of Concerns: Sorumlulukları Ayırmak 110
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 110
🧠 2. AI Çağında Yeni Zihniyet Modeli 110
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 111
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 111
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 113
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 113
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 113
23. State Yönetimi & Veri Akışı: Kaosu Önlemek 114
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 114
🧠 2. AI Çağında Yeni Zihniyet Modeli 115
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 115
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 116
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 117
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 118
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 118
24. API Mimarisi: REST, GraphQL ve gRPC Karşılaştırması 119
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 119

🧠 2. AI Çağında Yeni Zihniyet Modeli 119
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 120
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 120
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 122
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 122
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 122
Bölüm 5: Veritabanları, Veri Modelleme & Backend Temelleri 123
25. İlişkisel (SQL) vs. İlişkisel Olmayan (NoSQL): Hangisi Ne Zaman? 123
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 124
🧠 2. AI Çağında Yeni Zihniyet Modeli 124
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 125
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 125
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 126
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 127
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 127
26. Veri Modelleme & ER Diyagramlarını AI ile Tasarlamak 128
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 128
🧠 2. AI Çağında Yeni Zihniyet Modeli 128
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 129
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 129
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 133
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 133
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 133
27. SQL Ustalığı: Karmaşık Sorgular, İndeksleme ve AI Query Optimizer 134
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 135
🧠 2. AI Çağında Yeni Zihniyet Modeli 135
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 135
EXPLAIN ANALYZE ÇIKTISI: 136
GÖREV: 136
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 137
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 137
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 138
28. ORM'ler (Prisma, TypeORM, SQLAlchemy) ve Gizli Tehlike: N+1 Problemi 138
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 139
🧠 2. AI Çağında Yeni Zihniyet Modeli 139
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 140
GÖREV: 140
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 141
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 142
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 142

29. Cache Mekanizmaları: Redis ile Yıldırım Hızında Yanıtlar 143
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 143
🧠 2. AI Çağında Yeni Zihniyet Modeli 144
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 144
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 144
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 147
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 147
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 147
30. Kimlik Doğrulama ve Yetkilendirme: JWT, OAuth2 ve Session Yönetimi 148
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 148
🧠 2. AI Çağında Yeni Zihniyet Modeli 149
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 149
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 149
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 152
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 152
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 152
Bölüm 6: Test Odaklı Geliştirme (TDD) & Kalite Güvencesi 153
31. Neden Test Yazarız? Test Piramidini Anlamak 153
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 154
🧠 2. AI Çağında Yeni Zihniyet Modeli 154
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 155
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 155
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 156
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 156
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 157
32. Birim Testlerini AI ile Hızlandırmak: Jest, PyTest ve Mocklama 157
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 158
🧠 2. AI Çağında Yeni Zihniyet Modeli 158
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 159
GÖREV: 159
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 162
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 162
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 162
33. Entegrasyon ve E2E Testleri: Playwright ile Uçtan Uca Doğrulama 163
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 163
🧠 2. AI Çağında Yeni Zihniyet Modeli 164
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 164
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 164
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 166

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 166
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 166
34. Test Driven Development (TDD) + AI Döngüsü: Red-Green-Refactor 167
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 167
🧠 2. AI Çağında Yeni Zihniyet Modeli 168
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 168
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 169
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 170
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 171
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 171
35. Edge Case (Uç Durum) Avcılığı: AI ile Olmayacak Senaryoları Bulmak 172
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 172
🧠 2. AI Çağında Yeni Zihniyet Modeli 172
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 173
GÖREV: 173
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 175
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 175
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 175
36. Kod Kalite Metrikleri: SonarQube, Linter'lar ve Statik Analiz 176
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 176
🧠 2. AI Çağında Yeni Zihniyet Modeli 177
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 177
GÖREV: 178
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 180
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 180
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 180
Bölüm 7: Hata Ayıklama (Debugging), Profiling & Gözlemlenebilirlik 181
37. Dedektif Gibi Düşünmek: Debugging'in Psikolojisi ve Bilimsel Metodu 181
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 182
🧠 2. AI Çağında Yeni Zihniyet Modeli 182
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 183
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 183
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 184
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 185
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 185
38. Breakpoint'ler, Loglar ve Debugger Araçlarını Efektif Kullanmak 186
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 186
🧠 2. AI Çağında Yeni Zihniyet Modeli 187
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 187
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 187

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 188
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 189
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 189
39. Heisenbug'lar ve Race Condition: Asenkron Hataları Yakalamak 190
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 190
🧠 2. AI Çağında Yeni Zihniyet Modeli 191
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 191
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 191
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 192
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 193
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 193
40. Bellek Sızıntısı (Memory Leak) ve CPU Boğulması Teşhisi 194
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 194
🧠 2. AI Çağında Yeni Zihniyet Modeli 194
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 195
GÖREV: 195
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 197
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 197
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 197
41. Post-Mortem Kültürü: 'Kimi Suçlayacağız?' Değil, 'Nasıl Öğreneceğiz?' 198
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 198
🧠 2. AI Çağında Yeni Zihniyet Modeli 199
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 199
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 200
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 201
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 202
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 202
42. Canlı Ortam Gözlemlenebilirliği: Sentry, Datadog ve Log Analizi 203
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 203
🧠 2. AI Çağında Yeni Zihniyet Modeli 203
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 204
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 204
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 205
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 206
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 206
Bölüm 8: Güvenlik, OWASP & AI Kod Hijyeni 207
43. AI'nın Ürettiği Kod Güvenli mi? Gizli Güvenlik Açıkları 207
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 207
🧠 2. AI Çağında Yeni Zihniyet Modeli 208

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 208
GÖREV: 209
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 210
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 210
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 211
44. OWASP Top 10 Pratik Kılavuzu: SQL Injection, XSS, CSRF 212
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 212
🧠 2. AI Çağında Yeni Zihniyet Modeli 213
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 213
GÖREV: 213
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 214
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 215
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 215
45. API Güvenliği & Rate Limiting: Sistemini Korumak 216
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 216
🧠 2. AI Çağında Yeni Zihniyet Modeli 217
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 217
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 218
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 219
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 220
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 220
46. Gizli Bilgi (Secret) Yönetimi: API Anahtarlarını GitHub'a Yüklememek! 221
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 221
🧠 2. AI Çağında Yeni Zihniyet Modeli 222
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 222
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 222
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 223
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 224
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 224
47. Telif Hakları, Lisanslar ve Kodun Hukuki Güvenliği 225
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 225
🧠 2. AI Çağında Yeni Zihniyet Modeli 225
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 226
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 226
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 227
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 227
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 228
48. Savunmacı Programlama (Defensive Programming) Sanatı 228
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 229

🧠 2. AI Çağında Yeni Zihniyet Modeli 229
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 230
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 230
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 233
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 233
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 233
Bölüm 9: DevOps, Konteynerizasyon & Bulut Dağıtımı 234
49. 'Benim Makinemde Çalışıyordu': Docker ve Konteyner Mantığı 234
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 235
🧠 2. AI Çağında Yeni Zihniyet Modeli 235
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 236
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 236
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 237
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 238
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 238
50. Docker Compose ile Çok Servisli Yerel Geliştirme Ortamı 239
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 239
🧠 2. AI Çağında Yeni Zihniyet Modeli 240
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 240
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 241
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 242
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 243
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 243
51. Bulut Bilişime Giriş: AWS, GCP ve Azure Temel Kavramları 244
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 244
🧠 2. AI Çağında Yeni Zihniyet Modeli 245
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 245
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 246
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 246
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 247
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 247
52. Sunucusuz (Serverless) Mimariler & Edge Computing 248
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 248
🧠 2. AI Çağında Yeni Zihniyet Modeli 248
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 249
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 249
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 251
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 251
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 251

53. Infrastructure as Code (IaC): Terraform ve Pulumi'ye AI Bakışı 252
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 252
🧠 2. AI Çağında Yeni Zihniyet Modeli 253
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 253
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 254
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 257
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 257
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 257
54. Sıfır Kesintili Dağıtım (Zero-Downtime Deployment): Blue-Green & Canary 258
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 258
🧠 2. AI Çağında Yeni Zihniyet Modeli 259
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 259
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 260
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 261
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 261
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 261
Bölüm 10: Kariyer Hızlandırma, İletişim & Kıdemli Zihniyeti 262
55. Junior'dan Mid-Level'a Geçişin Haritası: Hangi Metriklerle Ölçülürsün? 262
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 263
🧠 2. AI Çağında Yeni Zihniyet Modeli 263
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 264
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 264
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 265
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 265
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 266
56. Teknik İletişim: Ürün Yöneticileri ve Tasarımcılarla Konuşma Sanatı 266
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 267
🧠 2. AI Çağında Yeni Zihniyet Modeli 267
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 268
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 268
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 269
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 269
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 270
57. AI Araçlarıyla Portföy ve Açık Kaynak Projesi İnşa Etmek 270
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 271
🧠 2. AI Çağında Yeni Zihniyet Modeli 271
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 272
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 272
🏗️ Mimari Akış Şeması 273

🧪 Testler ve Doğrulama 273
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 275
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 276
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 277
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 277
59. Sürekli Öğrenme Sistemi: Değişen Teknolojide Geri Kalmama Formülü 278
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 278
🧠 2. AI Çağında Yeni Zihniyet Modeli 278
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 279
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 279
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 280
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 280
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 281
60. Mezuniyet ve Manifesto: AI'ın Efendisi Olan Yeni Nesil Mühendis 281
📖 1. Giriş Hikayesi & Günlük Hayat Analojisi 282
🧠 2. AI Çağında Yeni Zihniyet Modeli 282
🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt) 283
💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi 283
⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı 284
🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri 284
🎯 7. Günün Uygulamalı Görevi & Mini Sınav 285

AI Çağında Junior Yazılım Mühendisliği

Sıfırdan Uzmanlığa Yapay Zeka Destekli Mühendislik Rehberi

Yazar: AI-Driven Software Engineering Collective
Baskı: 2026 İlkbahar / Kapsamlı Başvuru & Eğitim Edisyonu
Hedef Kitle: Yazılıma yeni başlayanlar, stajyerler, junior geliştiriciler ve ekibini modernize etmek isteyen teknik liderler.
Özellik: NotebookLM sesli genel bakış (Audio Overview Podcast) ve Udemy kurs üretimi için özel olarak tasarlanmıştır.

🎯 Kitabın Amacı ve AI Çağında Hayatta Kalma Rehberi

Bu kitap, yapay zekanın kod yazabildiği bir dünyada junior bir mühendisin nasıl vazgeçilmez, yüksek etki yaratan ve aranan bir teknik profesyonele dönüşebileceğini adım adım anlatmaktadır.

Geleneksel yazılım eğitimleri söz dizimi (syntax) ezberletmeye odaklanırken, bu kitap sizi **mimari düşünen, AI ajanlarını denetleyen, test odaklı çalışan ve güvenlik açıklarını koklayabilen bir orkestra şefine** dönüştürmeyi amaçlar.

📋 Detaylı Müfredat & İçindekiler Tablosu

Bölüm 1: Yeni Dünyaya Giriş & AI Zihniyeti

Yapay zekanın yazılım dünyasını nasıl değiştirdiğini anlayın; klavye ameleliğinden mimari orkestrasyona geçiş yapın.

- **01. İlk Gün Heyecanı: 'Yazılımcılık Bitti mi?' Mitini Yıkmak** — *Kod Yazmanın Değil, Problem Çözmenin Yeni Altın Çağı*
- **02. Geleneksel Junior vs. AI-Augmented Junior: 10x Zihniyet** — *Kod Yazma Hamallığından Değer Üretme Hızına Geçiş*
- **03. Modern AI Araç Setimiz: Copilot, Cursor, Claude Code ve Ötesi** — *Hangi Görev İçin Hangi AI Aracını Seçmelisin?*
- **04. AI Halüsinasyonu ve Güven Tuzağı: Kod Denetçisi Olmak** — *Var Olmayan Kütüphaneleri ve Sahte Metotları Yakalama Sanatı*
- **05. Prompt Mühendisliği Değil 'Context Mühendisliği': LLM'lere Nasıl Brif Verilir?** — *Süslü Cümleleri Unutun: Şemalar, Tipler ve Dosya Sınırlarıyla Konuşun*

- **06. Geliştirme Ortamını (IDE) Turbo Şarj Etmek: Cursor Rules ve Kısayollar** — *Her Seferinde Aynı Şeyleri Tekrarlamaktan Kurtulun*

Bölüm 2: Algoritmik Düşünce, Veri Yapıları & AI ile Problem Çözme

Ezber kodlamanın ötesine geçip temel algoritmik mantığı kavrayın; yapay zekanın karmaşıklık analizlerini denetleyin.

- **07. Sözde Koddan (Pseudocode) Gerçek Koda: AI ile Algoritma Tasarlamak** — *Klavye Başına Geçmeden Önce Düşünceyi Yapılandırma Rehberi*
- **08. Veri Yapıları AI Çağında Neden Hâlâ Hayati?** — *Array, Hash Map, Set ve Tree Arasındaki Seçim Sisteminizi Nasıl Kurtarır?*
- **09. Zaman ve Bellek Karmaşıklığı (Big O): AI Üretti, Ama Verimli mi?** — *5 Elemanla Test Edilen Kodun Canlıda 5 Milyon Elemanda Çökmesini Önlemek*
- **10. Tip Sistemleri ve Clean Code: AI Çıktısını Type-Safe Kılmak** — *'Any' Kullanımını Yasaklayarak Çalışma Zamanı Hatalarını Derleme Zamanında Yakalamak*
- **11. Hata Mesajlarını ve Stack Trace'i AI ile Okuma ve Akıl Yürütme Sanatı** — *Terminaldeki Kırmızı Yazılardan Korkmayı Bırakıp Dedektif Gibi İz Sürmek*
- **12. Kod Okuma Süper Gücü: Yabancı Bir Kod Tabanını AI ile 10 Dakikada Haritalamak** — *200.000 Satırlık Monolite İlk Kez Bakan Junior'ın Hayatta Kalma Kılavuzu*

Bölüm 3: Modern Git, Takım Çalışması & AI-Destekli Kod İnceleme

Takım ortamında kusursuz sürüm kontrolü, rebase, branch stratejileri ve AI destekli PR süreçlerini yönetin.

- **13. Git Temelleri & Akıllı Commit Stratejisi: AI ile Anlamlı Geçmiş** — *'WIP', 'Fixed Bug', 'asdasd' Commitlerinden Kurtulma Rehberi*
- **14. Merge Conflict Kabusunu AI ile 60 Saniyede Çözmek** — *<<<<<<< HEAD ve >>>>>>> Çatışmalarını Korkusuzca Yönetmek*
- **15. Kusursuz Pull Request (PR) Yazma Sanatı: AI ile PR Açıklamaları** — *Senior Mühendislerin İlk Bakışta Onaylayacağı (LGTM) PR'lar Hazırlamak*
- **16. Code Review'de Senior'ları Etkilemek: Kendi Kodunun Şeytanının Avukatı Ol** — *PR'ı Ekibe Göndermeden Önce AI ile Sanal Code Review Simülasyonu*
- **17. GitHub Actions ve Otomasyon: AI Destekli CI/CD Boru Hatları** — *Her Push'ta Otomatik Test, Lint ve AI Kod Kalitesi Denetimi*
- **18. Dokümantasyon Artık Yük Değil: Koddan Canlı Dokümantasyon Üretmek** — *Eski ve Güncelliğini Yitirmiş Wiki Sayfalarına Son Verme Rehberi*

Bölüm 4: Yazılım Mimarisi, Tasarım Kalıpları & Temiz Kod

SOLID prensipleri, tasarım kalıpları ve katmanlı mimarilerle spagetti kodları önleyin; AI çıktısını yapılandırın.

- **19. Mimari Nedir ve Junior Neden Bilmelidir? Monolit vs. Mikroservis** — *Erken Dönem Mikroservis Çılgınlığına Kapılmadan Doğru Mimarisi Seçmek*
- **20. SOLID Prensipleri AI Promptlarına Nasıl Çevrilir?** — *Yapay Zekanın Spagetti Kod Üretmesini Engelleyen 5 Altın Mimari Kural*

- **21. Tasarım Kalıpları (Design Patterns): Factory, Strategy, Observer Pratikte** — *Tekerleği Yeniden İcat Etmeden Endüstri Standartlarında Kod Yazmak*
- **22. Katmanlı Mimari & Separation of Concerns: Sorumlulukları Ayırmak** — *Controller, Service, Repository Katmanlarını Birbirine Karıştırmama Kılavuzu*
- **23. State Yönetimi & Veri Akışı: Kaosu Önlemek** — *Prop Drilling, Global State ve Asenkron Durum Yönetimi Rehberi*
- **24. API Mimarisi: REST, GraphQL ve gRPC Karşılaştırması** — *Hangi Protokolü Ne Zaman Seçmelisin? OpenAPI Şemalarıyla Tip Güvenliği*

Bölüm 5: Veritabanları, Veri Modelleme & Backend Temelleri

İlişkisel ve NoSQL modelleme, indeksleme, N+1 sorgu optimizasyonu ve dayanıklı API tasarım ilkelerini öğrenin.

- **25. İlişkisel (SQL) vs. İlişkisel Olmayan (NoSQL): Hangisi Ne Zaman?** — *PostgreSQL mi MongoDB mi? Ezbere Seçimlerden Kurtulma Kılavuzu*
- **26. Veri Modelleme & ER Diyagramlarını AI ile Tasarlamak** — *1:1, 1:N ve N:M İlişkilerini Doğru Kurarak Veri Tutarlılığını Sağlamak*
- **27. SQL Ustalığı: Karmaşık Sorgular, İndeksleme ve AI Query Optimizer** — *EXPLAIN ANALYZE Okuyarak 10 Saniyelik Sorguları 5 Milisaniyeye Düşürmek*
- **28. ORM'ler (Prisma, TypeORM, SQLAlchemy) ve Gizli Tehlike: N+1 Problemi** — *Tek Bir Sayfa İçin Veritabanına 1000 Sorgu Gönderme Tuzağından Kurtulmak*
- **29. Cache Mekanizmaları: Redis ile Yıldırım Hızında Yanıtlar** — *Cache-Aside Deseni ve 'Cache Invalidation' Zorluğunu Aşmak*
- **30. Kimlik Doğrulama ve Yetkilendirme: JWT, OAuth2 ve Session Yönetimi** — *Authentication (Sen Kimsin?) vs. Authorization (Neye İznin Var?) Ayrımı*

Bölüm 6: Test Odaklı Geliştirme (TDD) & Kalite Güvencesi

Birim, entegrasyon ve E2E test stratejileri kurun; yapay zekanın ürettiği kodun en büyük savunma kalkanı olan testleri yönetin.

- **31. Neden Test Yazarız? Test Piramidini Anlamak** — *'Kodum Zaten Çalışıyor' Tuzağından Kurtulup Gece Rahat Uyuma Sanatı*
- **32. Birim Testlerini AI ile Hızlandırmak: Jest, PyTest ve Mocklama** — *Gerçek Veritabanına veya API'ye Bağlanmadan Test Yazma Sanatı*
- **33. Entegrasyon ve E2E Testleri: Playwright ile Uçtan Uca Doğrulama** — *Tüm Parçaların Bir Araya Geldiğinde Doğru Çalıştığını Kanıtlamak*
- **34. Test Driven Development (TDD) + AI Döngüsü: Red-Green-Refactor** — *Yapay Zekanın Halüsinasyon Görmesini Engelleyen Nihai Geliştirme Ritmi*
- **35. Edge Case (Uç Durum) Avcılığı: AI ile Olmayacak Senaryoları Bulmak** — *29 Şubat, Artık Saniyeler, Boş Stringler ve Sonsuz Sayılarla Sistemi Kırmak*
- **36. Kod Kalite Metrikleri: SonarQube, Linter'lar ve Statik Analiz** — *Teknik Borç Faizini Ödemeden Sistemi Sürdürülebilir Kılmak*

Bölüm 7: Hata Ayıklama (Debugging), Profiling & Gözlemlenebilirlik

Gelişmiş loglama, metrikler, trace takibi, bellek sızıntısı tespiti ve AI ile kök neden analizi gerçekleştirin.

- **37. Dedektif Gibi Düşünmek: Debugging'in Psikolojisi ve Bilimsel Metodu** — *Rastgele Kod Değiştirme Çaresizliğinden Hipotez Testine Geçiş*
- **38. Breakpoint'ler, Loglar ve Debugger Araçlarını Efektif Kullanmak** — *'console.log' Kirliliğinden Kurtulup Zamanı Durdurma Sanatı*
- **39. Heisenbug'lar ve Race Condition: Asenkron Hataları Yakalamak** — *İncelemeye Çalıştığınızda Ortadan Kaybolan Hayalet Hatalarla Mücadele*
- **40. Bellek Sızıntısı (Memory Leak) ve CPU Boğulması Teşhisi** — *Sunucunun Her 4 Saatte Bir Şişip Çökmesini Node Profiler ile Önlemek*
- **41. Post-Mortem Kültürü: 'Kimi Suçlayacağız?' Değil, 'Nasıl Öğreneceğiz?'** — *Production Çöktüğünde Mühendisleri Suçlamayan Şirketlerin Yükselişi*
- **42. Canlı Ortam Gözlemlenebilirliği: Sentry, Datadog ve Log Analizi** — *Kullanıcılar 'Site Bozuk' Demeden Önce Hataları Canlıda Yakalamak*

Bölüm 8: Güvenlik, OWASP & AI Kod Hijyeni

OWASP Top 10 açıkları, gizli anahtar yönetimi, prompt enjeksiyonları ve yapay zeka halüsinasyon risklerine karşı sisteminizi zırhlandırın.

- **43. AI'nın Ürettiği Kod Güvenli mi? Gizli Güvenlik Açıkları** — *Yapay Zekanın Koda Gizlice Enjekte Ettiği Güvenlik Açıklarını Avlamak*
- **44. OWASP Top 10 Pratik Kılavuzu: SQL Injection, XSS, CSRF** — *Web Güvenliğinin En Tehlikeli 10 Açığını Kod Seviyesinde Kapatmak*
- **45. API Güvenliği & Rate Limiting: Sistemini Korumak** — *Brute-Force ve DoS Saldırılarına Karşı Hız Sınırları ve Güvenlik Başlıkları*
- **46. Gizli Bilgi (Secret) Yönetimi: API Anahtarlarını GitHub'a Yüklememek!** — *GitHub'a Yanlışlıkla AWS Anahtarı Yükleyip 1 Saatte 50.000 Dolar Fatura Yeme Kabusu*
- **47. Telif Hakları, Lisanslar ve Kodun Hukuki Güvenliği** — *GPL, MIT, Apache Lisansları ve Şirketinizi Davalardan Koruma Rehberi*
- **48. Savunmacı Programlama (Defensive Programming) Sanatı** — *'Murphy Kanunları'na Göre Kod Yazmak: Yanlış Gidebilecek Her Şey Yanlış Gidecektir*

Bölüm 9: DevOps, Konteynerizasyon & Bulut Dağıtımı

Docker çok aşamalı derlemeler, CI/CD pipeline otomasyonu ve bulut dağıtım mimarilerini hayata geçirin.

- **49. 'Benim Makinemde Çalışıyordu': Docker ve Konteyner Mantığı** — *Yazılım Dünyasının En Meşhur Mazeretini Tarihe Gömen Konteyner Devrimi*
- **50. Docker Compose ile Çok Servisli Yerel Geliştirme Ortamı** — *Tek Bir 'docker compose up' Komutuyla Tüm Şirket Altyapısını Ayağa Kaldırmak*
- **51. Bulut Bilişime Giriş: AWS, GCP ve Azure Temel Kavramları** — *Kendi Sunucunu Yönetme Zahmetinden Bulutun Sınırsız Gücüne Geçiş*
- **52. Sunucusuz (Serverless) Mimariler & Edge Computing** — *Sıfır Sunucu Yönetimi, Soğuk Başlangıçlar (Cold Start) ve Küresel Dağıtım*
- **53. Infrastructure as Code (IaC): Terraform ve Pulumi'ye AI Bakışı** — *Bulut Konsolunda Elle Tıklamayı Bırakıp Altyapıyı Kod Olarak Sürümlemek*
- **54. Sıfır Kesintili Dağıtım (Zero-Downtime Deployment): Blue-Green & Canary** — *Cuma Akşamı Bile Korkusuzca Deploy Çıkma Sanatı*

Bölüm 10: Kariyer Hızlandırma, İletişim & Kıdemli Zihniyeti

Teknik liderlik, mülakat hazırlığı, mentörlük ve AI çağında aranan 10x mühendis olma vizyonunu edinin.

- **55. Junior'dan Mid-Level'a Geçişin Haritası: Hangi Metriklerle Ölçülürsün?** — *Sürekli Yardım İsteyen Geliştiriciden Bağımsız Çözüm Üreten Mühendise Dönüşmek*
- **56. Teknik İletişim: Ürün Yöneticileri ve Tasarımcılarla Konuşma Sanatı** — *'Jargon' Duvarını Yıkıp İş Değeriyle Konuşan Mühendis Olmak*
- **57. AI Araçlarıyla Portföy ve Açık Kaynak Projesi İnşa Etmek** — *Sıradan 'To-Do App' Projelerini Çöpe Atıp Fark Yaratan İşler Üretmek*
- **58. AI Çağında Teknik Mülakatlara Hazırlık: Live Coding & System Design** — *Ezber LeetCode Sorularından Zihniyeti ve Akıl Yürütmeyi Kanıtlama Sanatına*
- **59. Sürekli Öğrenme Sistemi: Değişen Teknolojide Geri Kalmama Formülü** — *Her Hafta Yeni Bir Framework Çıkarken Akıl Sağlığını Korumak*
- **60. Mezuniyet ve Manifesto: AI'ın Efendisi Olan Yeni Nesil Mühendis** — *Yapay Zekanın Değiştiremediği Tek Şey: İnsan Merakı, Vizyonu ve Yargı Yeteneği*

Bölüm 1: Yeni Dünyaya Giriş & AI Zihniyeti

Yapay zekanın yazılım dünyasını nasıl değiştirdiğini anlayın; klavye ameleliğinden mimari orkestrasyona geçiş yapın.

1. İlk Gün Heyecanı: 'Yazılımcılık Bitti mi?' Mitini Yıkmak

Kod Yazmanın Değil, Problem Çözmenin Yeni Altın Çağı

Bölüm 1 • Zorluk Derecesi: Başlangıç Seviyesi • Tahmini Okuma Süresi: 11 dk

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Yapay zeka tüm kodları birkaç saniyede yazabiliyorsa, bir şirketin yeni mezun veya junior bir yazılımcıya maaş ödemesi için mantıklı tek bir sebep kaldı mı?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Yapay zeka yazılımcılığı öldürmedi; söz dizimi (syntax) ezberciliğini öldürüp mimari düşünceyi öne çıkardı. - 📌 Junior mühendis artık 'klavye amelesi' değil, yapay zeka stajyerlerini yöneten bir orkestra şefidir. - 📌 Değeriniz yazdığınız kod satır sayısıyla değil, doğruluğunu garanti ettiğiniz sistemlerle ölçülür.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Ofise veya uzaktan çalışma masanıza oturduğunuz o ilk günü hayal edin. Ekranda yanıp sönen imleç ve kulağınızda sosyal medyanın uğultusu: *"Yapay zeka varken artık junior'lara gerek yok!"*

Bir an durup aşçılık tarihine bakalım. Mikserler, akıllı fırınlar ve otomatik hamur yoğurma makineleri icat edildiğinde şefler yok oldu mu? Tam aksine! Önceden saatlerce yumurta çırpmak zorunda kalan çıraklar, artık lezzet dengesine, sunuma ve menü tasarımına odaklanmaya başladı.

Yazılımda da tam olarak bu devrim yaşanıyor. Eskiden günlerce 'for döngüsü' veya karmaşık CSS hizalamasıyla boğuşan junior, bugün dakikalar içinde prototip çıkarıp işin mantığına, güvenliğine ve kullanıcı deneyimine kafa yorabiliyor. Korku duymak yerine fırının kontrol düğmelerini elinize almanın zamanı geldi.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Geleneksel yazılımcı zihniyeti **"Nasıl yazarım?"** sorusuyla başlardı. Syntax hatırlamak, StackOverflow'da kaybolmak ve dökümantasyon sayfalarında saatler harcamak bir meziyetti.

AI-Driven mühendis zihniyeti ise **"Ne inşa ediyoruz ve bunu nasıl doğrularım?"** sorusunu merkeze alır:

Özellik	Geleneksel Junior	AI Destekli (AI-Driven) Junior
Odak Noktası	Dil söz dizimi (Syntax) ve kütüphane API'leri	Problem tanımı, sistem akışı ve doğrulama
Hızlandırıcı	Google arama + StackOverflow kopyala-yapıştır	LLM ve Agent tabanlı akıllı asistanlar
En Büyük Risk	Yavaş öğrenme ve motivasyon kaybı	AI halüsinasyonunu fark edemeyip körü körüne güvenme
Katma Değer	Basit görevleri manuel kodlama	Görevi AI ile hızla çözüp kenar durumları test etme

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Kıdemli Yazılım Mühendisi Mentoru
Sen deneyimli, temiz kod (Clean Code) standartlarına bağlı bir kıdemli mühendissin.
Ben yazılım ekibine yeni katılmış bir junior mühendisim.

## GÖREV:
Aşağıdaki kullanıcı hikayesi (User Story) için bana doğrudan kod YAZMA.
Bunun yerine:
1. Çözmem gereken temel iş mantığı maddelerini listele.
2. Karşılaşabileceğim 3 potansiyel uç durumu (edge case) belirt.
3. Kodu yazarken dikkat etmem gereken mimari prensibi açıkla.

## KULLANICI HİKAYESİ:
"Bir e-ticaret sepetinde kullanıcı indirim kuponu uyguladığında sepet toplamını
güncelleyen fonksiyon."
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```python
# Kontrolsüz AI Çıktısı: Sınır değerleri ve tipleri kontrol edilmemiş fonksiyon
def apply_coupon(cart_total, coupon_discount):
    # Eğer indirim sepetten büyükse sepet eksiye düşüyor!
    # Tip kontrolü yok: coupon_discount string gelirse crash olur.
    return cart_total - coupon_discount
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```python
from decimal import Decimal
from typing import Union

def apply_discount(
    cart_total: Union[Decimal, float],
    discount_amount: Union[Decimal, float]
) -> Decimal:
    """
    Sepet toplamına indirim uygular.
    Sepet tutarının sıfırın altına düşmesini engeller ve Decimal hassasiyetinde
çalışır.
    """
    total = Decimal(str(cart_total))
    discount = Decimal(str(discount_amount))

    if total < Decimal("0.00"):
        raise ValueError("Sepet tutarı negatif olamaz.")
    if discount < Decimal("0.00"):
        raise ValueError("İndirim tutarı negatif olamaz.")

    new_total = total - discount
    return max(new_total, Decimal("0.00"))
```

💡 **Kod Analizi:** Acemi yaklaşımda float tipi kullanılarak finansal yuvarlama hataları ve eksi bakiye bug'ı gözden kaçırılmıştır. Kıdemli mühendis denetimindeki sürümde ise `Decimal` tipiyle yuvarlama hassasiyeti sağlanmış, negatif değer kontrolleri ve `max(..., 0)` ile defansif koruma eklenmiştir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** AI'ın ürettiği ilk kodu çalıştığı an PR olarak açmak.
  - **Çözüm:** Her zaman 'Bu kod neden patlayabilir?' sorusunu sorup en az 3 uç durum testi yazın.
- **Tuzak:** Temel algoritma ve mantığı bilmeden her şeyi yapay zekaya devretmek.
  - **Çözüm:** AI'ı bir 'otopilot' değil, yan koltukta oturan 'yardımcı pilot' olarak görün.
- **Tuzak:** Halüsinasyon riskini unutup kütüphane fonksiyonlarını doğrulamamak.
  - **Çözüm:** AI tarafından önerilen kütüphane metotlarının resmi dokümantasyonunu her zaman hızlıca kontrol edin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

🎥 **Ders Başlığı:** ***Ders 1: AI Çağında Junior Olmak - İlk Adımlar ve Zihniyet Değişimi***

- 🖥️ **Slayt 1:** Neden 'Yazılımcılık Bitti' söylemi gerçeği yansıtmıyor?
- 🖥️ **Slayt 2:** Kod ezberciliğinden Problem Mimarisine geçiş.
- 🖥️ **Slayt 3:** AI-Driven geliştiricinin 3 temel ayağı: Bağlam, Doğrulama, İletişim.
- 🖥️ **Slayt 4:** Günün canlı demo özeti: Hatalı AI çıktısını senior seviyeye dönüştürmek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Cursor veya VS Code'u açın. Basit bir e-ticaret indirim fonksiyonu isteyin. AI'ın ilk verdiği kontrolsüz çıktıyı inceleyin, eksiklerini tespit edin ve prompt ile defansif kontroller ekleterek test suite oluşturun.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bugün bir yapay zeka aracına (Claude, ChatGPT veya Cursor) daha önce yazdığınız bir fonksiyonu verin ve şu soruyu sorun: 'Bu fonksiyonu production ortamında çökertebilecek 3 senaryo nedir?'

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: AI çağında bir junior yazılım mühendisinin en değerli ayırt edici yeteneği nedir? - A) En hızlı kod yazan kişi olmak - B) Üretilen kodun mantığını, sınır durumlarını ve doğruluğunu denetleyebilmek - C) Dillerin tüm kütüphane fonksiyonlarını ezbere bilmek - D) Kodlama yapmadan tüm işi yapay zekaya bırakmak > **Doğru Yanıt:** B > **Açıklama:** Yapay zeka hızlı kod üretebilir, ancak üretilen kodun doğruluğunu, sınır durumlarını ve sistemle uyumunu denetlemek mühendisin sorumluluğundadır.

Soru 2: Float tipi yerine Decimal kullanılmasının temel finansal sebebi nedir? - A) Decimal'ın bellekte daha az yer kaplaması - B) Kayan noktalı (float) aritmetiğindeki ikili taban yuvarlama hassasiyet kayıplarını önlemek - C) Sadece Python'da geçerli olması - D) Kodun derleme hızını artırması > **Doğru Yanıt:** B > **Açıklama:** Kayan noktalı sayılar ikili tabanda temsil edilirken kuruş hassasiyetinde yuvarlama hatalarına (örn. 0.1 + 0.2 != 0.3) yol açar. Finansal işlemlerde Decimal zorunludur.

---

2. Geleneksel Junior vs. AI-Augmented Junior: 10x Zihniyet

Kod Yazma Hamallığından Değer Üretme Hızına Geçiş
Bölüm 1 • Zorluk Derecesi: Başlangıç Seviyesi • Tahmini Okuma Süresi: 12 dk

---

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Bir junior mühendis yapay zekayı etkin kullandığında gerçekten bir mid-level veya senior gibi çıktı üretebilir mi, yoksa bu sadece bir illüzyon mu?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 10x yazılımcı miti bireysel deha ile değil, doğru araç ve geri bildirim döngüsüyle gerçeğe dönüşür. - 📌 Geleneksel junior saatlerce dokümantasyon ararken, AI-augmented junior bağlamı besleyip doğrudan prototipe geçer. - 📌 Hız tek başına başarı değildir; test edilmemiş hızlı kod, şirketi batıracak teknik borç demektir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

İki yeni mezun mühendis hayal edin: Can ve Elif. İkisi de aynı fintech şirketinde aynı gün işe başladı.

İlk görevleri: *"Kullanıcıların adres bilgilerini doğrulayan ve posta koduna göre şehri getiren bir servis yazmak."*

Can geleneksel yöntemi seçti. 3 saat boyunca regex (düzenli ifade) yazmaya çalıştı, posta kodu API dokümantasyonunda kayboldu ve günün sonunda sadece birkaç satır kod yazıp yorgun düştü.

Elif ise AI-augmented yaklaşımla ilerledi. Önce iş gereksinimlerini listeledi. AI asistanına Türkiye ve Avrupa posta kodu formatlarını doğrulayan test senaryolarını yazdırdı. Ardından bu testleri geçecek temiz, tip korumalı servisi ürettirdi. 45 dakika içinde çalışan, test edilmiş ve dokümante edilmiş bir Pull Request sundu. Senior mühendis Elif'in PR'ını gördüğünde gözlerine inanamadı.

🧠 2. AI Çağında Yeni Zihniyet Modeli

AI-Augmented mühendis olmanın 3 temel ayağı vardır: 1. **İterasyon Hızı:** 0'dan 1'e gitmek dakikalar sürer. 2. **Aktif Doğrulama:** Yazılan her kodun hemen unit testi olmalıdır. 3. **Kritik Sorgulama:** "AI bu kütüphaneyi neden önerdi? Daha modern bir alternatif var mı?"

```
[İş Gereksinimi] —> [AI ile Test Senaryosu] —> [AI ile Kod Üretimi] —> [İnsan Doğrulaması & Refactor]
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# GÖREV: Test-Driven Development (TDD) İle Servis Tasarımı
Bir TypeScript projesinde çalışıyoruz. Kullanıcı posta kodu doğrulama servisi
yazacağız.

Lütfen bana doğrudan implementasyon KODUNU VERME.
Öncelikle aşağıdaki senaryoları kapsayan Jest test dosyasını hazırla:
1. Geçerli formatlar (Örn: '34340', '06100')
2. Hatalı uzunluk veya harf içeren durumlar
3. Boş veya tanımsız (null/undefined) girdiler
4. API zaman aşımı (timeout) durumu
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Geleneksel/Aceleci Junior: Hata fırlatmayan, belirsiz tip
export async function getCityByZip(zip: any) {
  const res = await fetch("https://api.zippopotam.us/tr/" + zip);
  const data = await res.json();
  return data.places[0]["place name"]; // Eğer API hata dönerse patlar!
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// AI-Augmented Junior: Tip güvenli, Result Pattern ile hata yönetimi
export interface CityResult {
  city: string;
  region: string;
}

export type PostalLookupResponse =
  | { success: true; data: CityResult }
  | { success: false; error: string };

export async function getCityByPostalCode(
  postalCode: string,
  fetcher: typeof fetch = fetch
): Promise<PostalLookupResponse> {
  const cleaned = postalCode.trim();
  if (!/^\d{5}$/.test(cleaned)) {
    return { success: false, error: "Geçersiz posta kodu formatı (5 haneli rakam
beklenir)." };
  }

  try {
    const res = await fetcher(`https://api.zippopotam.us/tr/${cleaned}`, { signal:
AbortSignal.timeout(3000) });
    if (!res.ok) {
      return { success: false, error: `Sunucu hatası: ${res.statusText}` };
    }
    const data = await res.json();
    return {
      success: true,
      data: {
        city: data.places[0]["place name"],
        region: data.places[0]["state"]
      }
    };
  } catch (err: unknown) {
    return { success: false, error: "Servise erişilemedi veya zaman aşımı
oluştu." };
  }
}
```

💡 **Kod Analizi:** Kıdemli AI yaklaşımında; `any` tipi ortadan kaldırılmış, Result Pattern ile hata durumları açıkça modellenmiş, AbortSignal ile timeout eklenmiş ve bağımlılık enjeksiyonu (fetcher) sayesinde test edilebilirlik sağlanmıştır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** AI tarafından üretilen kodu okumadan commit etmek.

  - **Çözüm:** Her satırı sesli okuyabilecek kadar anlamadan kod tabanına dahil etmeyin.
- **Tuzak:** Dış API çağrılarında zaman aşımı (timeout) belirlememek.
  - **Çözüm:** Tüm HTTP isteklerine AbortSignal.timeout() veya benzeri sınırlar ekleyin.
- **Tuzak:** Gevşek tip kullanımı (any veya Object).
  - **Çözüm:** TypeScript'te strictly typed arayüzler ve Union tipleri kullanın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 2: 10x Junior Mühendisin Anatomisi - Hız ve Kalite Dengesi*

- 🖥️ **Slayt 1:** Geleneksel geliştirici vs AI-Augmented geliştirici iş akışı.
- 🖥️ **Slayt 2:** Result Pattern ile güvenli hata yönetimi.
- 🖥️ **Slayt 3:** TDD yaklaşımını AI ile birleştirmek: Önce test, sonra kod.
- 🖥️ **Slayt 4:** Pull Request hazırlarken senior mühendisleri etkileyen detaylar.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Boş bir TypeScript dosyası açın. Posta kodu doğrulama servisini önce test dosyasıyla birlikte AI'a prompt edin. Mock fetch kullanarak testleri çalıştırın ve 0 hata ile yeşil ışık alın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Mevcut projenizdeki bir fonksiyona `try/catch` ve Result Pattern ekleyerek olası bir HTTP 500 hatasında çökmesini engelleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Result Pattern kullanmanın geleneksel try/catch fırlatmaya göre en büyük avantajı nedir? - A) Kodun daha hızlı çalışması - B) Fonksiyonu çağıran tarafın olası başarı ve hata durumlarını tip düzeyinde zorunlu olarak ele almasını sağlaması - C) Sadece JavaScript'e özel olması - D) Bellek kullanımını yarı yarıya düşürmesi > **Doğru Yanıt:** B > **Açıklama:** Result Pattern (success: true | success: false) sayesinde TypeScript derleyicisi çağrıyı yapan mühendisi hata durumunu kontrol etmeye zorlar; beklenmedik çalışma zamanı çökmelerini önler.

Soru 2: AI-Augmented bir junior mühendis ile geleneksel junior arasındaki en belirgin fark nedir? - A) AI-Augmented junior hiç kod yazmaz - B) AI-Augmented junior iş mantığını planlayıp doğrulamaya odaklanırken mekanik işleri AI ile hızlandırır - C) Geleneksel junior daha güvenli kod yazar - D) AI-Augmented junior test yazmayı gereksiz bulur > **Doğru Yanıt:** B > **Açıklama:** AI-Augmented mühendis zamanını mekanik boilerplate yazmak yerine sistem mimarisine, testlere ve sınır durumlarına harcar.

3. Modern AI Araç Setimiz: Copilot, Cursor, Claude Code ve Ötesi

Hangi Görev İçin Hangi AI Aracını Seçmelisin?
Bölüm 1 • Zorluk Derecesi: Başlangıç Seviyesi • Tahmini Okuma Süresi: 13 dk

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Cursor varken Copilot gereksiz mi? Ya da Claude Code terminal asistanları geliştiricinin yerini mi alıyor?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Her yapay zeka aracının bir 'tatlı noktası' (sweet spot) vardır; tek bir araca bağımlı kalmayın.
> - 📌 Satır içi tamamlama (Ghost Text) için Copilot, tüm kod tabanı indeksleme için Cursor, otonom refactoring için CLI agent'lar idealdir. - 📌 En pahalı araç değil, bağlamı (context) en iyi yönettiğiniz araç en iyi sonucu verir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Marangoz atölyesine girdiğinizi düşünün. Duvarda testere, rende, zımpara ve çekiç asılı. Bir marangoz vida sıkmak için çekiç kullanırsa ne olur? Tahta çatlar ve vida bozulur.

Yazılım dünyasında da bugün benzer bir hata yapılıyor: Bir junior, 50 dosyalık bir mimari refactoring'i GitHub Copilot'un satır içi otomatik tamamlamasına yaptırmaya çalışıyor. Veya basit bir `console.log` eklemek için 5 dakika otonom CLI agent çalıştırıp jeton (token) yakıyor.

Doğru aleti doğru işte kullanmak, profesyonel bir mühendisi amatörden ayıran ilk beceridir.

🧠 2. AI Çağında Yeni Zihniyet Modeli

AI Araçlarının Katmanlı Mimarisi:

Katman	Araç Örnekleri	En İyi Kullanım Alanı	Dikkat Edilmesi Gereken
Micro (Satır İçi)	GitHub Copilot, Supermaven	Boilerplate tamamlama, döngüler, DTO tanımları	Körü körüne Tab'a basıp yanlış mantığı onaylamak
Meso (Editör İçi)	Cursor (Composer), Windsurf	Dosyalar arası ilişki, `@files`, `@docs` bağlamı	Çok fazla gereksiz dosya ekleyip token limitini aşmak
Macro (CLI / Agent)	Claude Code, Aider, Antigravity	Terminal komutları, testleri çalıştırıp otomatik düzeltme	Yanlış komut çalıştırma veya git geçmişini bozma riski

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Senior Full-Stack Mimar
Projemizde PostgreSQL, Prisma ORM, Next.js ve TailwindCSS kullanıyoruz.

## BAĞLAM (@schema.prisma ve @auth.ts):
Aşağıdaki yeni gereksinimi karşılamak için hangi dosyaları değiştirmem gerektiğini
listele:
"Kullanıcıların profil sayfasına iki faktörlü kimlik doğrulama (2FA) açma/kapama
butonu ekleme."

Lütfen hemen kod yazma; sadece:
1. Değişecek veritabanı alanları
2. Backend API endpoint'i
3. Frontend bileşeni
adım adım bir mimari plan sun.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# Kötü Yaklaşım: AI'ın çalıştırdığı terminal komutunu kontrol etmeden çalıştırmak
# Yapay zekaya "Projeyi sıfırla ve yeniden kur" dediniz ve şunu önerdi:
rm -rf node_modules package-lock.json && git clean -fdx && npm install
# DİKKAT: Henüz commit etmediğiniz tüm yerel dosyalarınız sonsuza dek silindi!
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
# Güvenli Yaklaşım: Önce durumu doğrula, dry-run yap ve korumalı çalış
# 1. Değişiklikleri güvene al
git status
git stash save "proje-temizlik-oncesi"

# 2. Sadece hedeflenen paketleri temizle
rm -rf node_modules
npm cache verify
npm install

# 3. Testleri çalıştırarak ortamı doğrula
npm test
```

💡 **Kod Analizi:** Terminal üzerinde çalışan otonom AI agent'ları kullanırken `git clean -fdx` veya `rm -rf` gibi yıkıcı komutları asla gözünüz kapalı onaylamayın. Her zaman önce `git stash` veya branch yedeği alın.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Aynı anda 3 farklı AI eklentisini açıp birbirleriyle çakıştırmak.
  - **Çözüm:** Birincil editörünüzü ve tek bir tamamlama aracını belirleyip odaklanın.
- **Tuzak:** Büyük projelerde tüm kod tabanını prompta yapıştırmak.
  - **Çözüm:** Sadece ilgili servis ve interface dosyalarını bağlam (@file) olarak ekleyin.
- **Tuzak:** AI CLI araçlarına sudo veya sınırsız terminal izni vermek.
  - **Çözüm:** Hassas terminal komutları için her zaman manuel onay modunu açık tutun.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📹 Ders Başlığı: ***Ders 3: AI Araç Kutunuzu Kurmak - Copilot, Cursor ve CLI Entegrasyonu***

- 🖥️ **Slayt 1:** 3 Katmanlı AI Mimarisi: Micro, Meso, Macro araçlar.
- 🖥️ **Slayt 2:** Cursor'da @sembolleri (@files, @docs, @codebase) ile bağlam yönetimi.
- 🖥️ **Slayt 3:** Terminal asistanlarını güvenli kullanma kılavuzu.
- 🖥️ **Slayt 4:** Canlı karşılaştırma: Hangi araç nerede kazandırır?

Canlı Kodlama / Ekran Kaydı Senaryosu:
Cursor IDE'de `@codebase` kullanarak projedeki bir veritabanı modelinin nerede kullanıldığını sorgulatın. Ardından `@file` ile sadece iki dosyayı seçerek hedefli bir refactor gerçekleştirin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bugün IDE'nizde en sık kullandığınız 3 dosyayı `@` kısayoluyla bağlama ekleyerek küçük bir fonksiyon geliştirin ve token kullanımını gözlemleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Cursor veya benzeri modern AI IDE'lerinde '@' sembolünün temel amacı nedir? - A) E-posta göndermek - B) Modele yalnızca belirli dosyaları, dokümanları veya kod tabanını bağlam olarak iletmek - C) Kodu otomatik olarak derlemek - D) GitHub şifresini kaydetmek > **Doğru Yanıt:** B > **Açıklama:** @ sembolü sayesinde LLM'e projenin sadece ilgili parçaları bağlam olarak gönderilir; böylece hem maliyet düşer hem de doğruluk artar.

Soru 2: Otonom terminal agent'ları (CLI) kullanırken en kritik güvenlik kuralı nedir? - A) İnternet bağlantısını kesmek - B) Dosya silen ve git geçmişini etkileyen komutları manuel olarak denetlemek - C) Sadece Python kullanmak - D) Bilgisayarı yeniden başlatmak > **Doğru Yanıt:** B > **Açıklama:** Terminal agent'ları geri dönülemez dosya silme veya branch ezme komutları önerebilir. Kritik komutlar mutlaka insan onayından geçmelidir.

---

4. AI Halüsinasyonu ve Güven Tuzağı: Kod Denetçisi Olmak

Var Olmayan Kütüphaneleri ve Sahte Metotları Yakalama Sanatı
Bölüm 1 • Zorluk Derecesi: Başlangıç Seviyesi • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Bir yapay zeka modeli son derece mantıklı görünen ancak gerçekte hiç var olmayan bir npm paketi önerdiğinde ne olur? Bu halüsinasyon bir siber güvenlik saldırısına dönüşebilir mi?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Yapay zeka gerçeği bilmez; olasılıksal olarak en mantıklı gelen bir sonraki kelimeyi tahmin eder. - 📌 Slopsquatting (paket halüsinasyonu), saldırganların sahte kütüphaneler yükleyip şirketleri hacklediği gerçek bir tehdittir. - 📌 Mühendisin en büyük görevi kod yazmak değil, AI çıktısına karşı 'Şüpheci Müfettiş' rolü üstlenmektir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

2023 yılında bir junior geliştirici, Python ile AWS S3 dosyalarını sıkıştırmak için ChatGPT'ye danıştı. Model kendinden çok emin bir dille şu cevabı verdi:

`pip install aws-s3-fast-compressor`

Geliştirici kodu projeye ekledi. Ancak bir gariplik vardı: Böyle bir kütüphane Python Package Index'te (PyPI) henüz yoktu! Bir siber güvenlik araştırmacısı bu durumu fark etti ve yapay zekaların sıkça uydurduğu bu sahte kütüphane ismini tescilleyerek içine bir test kodu koydu. Birkaç gün içinde dünyanın önde gelen teknoloji şirketlerinden yüzlerce sunucunun bu sahte paketi indirmeye çalıştığı görüldü!

İşte buna **AI Halüsinasyon Tuzağı** denir. Yapay zeka yalan söylemez; ama var olmayan şeyleri gerçekmiş gibi hayal eder.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Halüsinasyon Tespiti İçin 4 Aşamalı Doğrulama Protokolü:

```
[AI Yanıtı]
   |
   ├── 1. Paket Doğrulaması: NPM / PyPI üzerinde gerçekten var mı? İndirme sayısı
kaç?
   |
   ├── 2. Dokümantasyon Kontrolü: Fonksiyon parametreleri resmi sitede yazıyor mu?
   |
   ├── 3. Tip Kontrolü (Compiler): TypeScript / MyPy hatasız derleniyor mu?
   |
   └── 4. Çalışma Zamanı Testi (Runtime): Unit test uç durumları geçiyor mu?
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Şüpheci Güvenlik ve Kod Denetçisi
Aşağıda bir yapay zekanın önerdiği kod parçası bulunmaktadır.

## GÖREV:
Bu koda asla hemen güvenme. Aşağıdaki analizleri gerçekleştir:
1. Bu kodda kullanılan fonksiyon ve kütüphaneler güncel mi yoksa 'uydurulmuş'
(halüsinasyon) olabilir mi?
2. Hangi satırda sessiz bir mantık hatası (silent bug) veya güvenlik açığı
bulunmaktadır?
3. Bu kodun doğruluğunu kanıtlayacak bağımsız bir test fonksiyonu yaz.

## KOD PARÇASI:
```javascript
const crypto = require('crypto');
function createToken(userId) {
  // AI önerisi:
  return crypto.createCipher('des', userId).final('hex');
}
```
```

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```javascript
// Halüsinasyon / Eski API Tuzağı: createCipher güvensizdir ve Node.js'te deprecated
edilmiştir!
const crypto = require('crypto');

function generateSecureToken(userId) {
  // 'des' algoritması kırılmıştır ve createCipher IV (başlatma vektörü) kullanmaz!
  const cipher = crypto.createCipher('des', 'gizli_anahtar');
  let token = cipher.update(userId, 'utf8', 'hex');
  token += cipher.final('hex');
  return token;
}
```
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// Güncel & Güvenli Standart: AES-256-GCM ve rastgele IV kullanımı
const crypto = require('crypto');

function generateSecureToken(userId, secretKeyBuffer) {
  if (!secretKeyBuffer || secretKeyBuffer.length !== 32) {
    throw new Error("Geçersiz anahtar uzunluğu: 32 baytlık buffer zorunludur.");
  }

  // 12 baytlık benzersiz IV (Initialization Vector)
  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', secretKeyBuffer, iv);

  let encrypted = cipher.update(String(userId), 'utf8', 'hex');
  encrypted += cipher.final('hex');
  const authTag = cipher.getAuthTag().toString('hex');

  // IV + AuthTag + Şifreli metin bir arada paketlenir
  return `${iv.toString('hex')}:${authTag}:${encrypted}`;
}
```

💡 **Kod Analizi:** Eski AI modelleri eğitim verilerindeki 10 yıllık eski kodları hatırlayarak güvensiz `crypto.createCipher` metodunu önerebilir. Modern denetim; güvenli `createCipheriv`, AES-256-GCM algoritması ve kimlik doğrulama etiketi (Auth Tag) kullanımını zorunlu kılar.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** AI'ın önerdiği her yeni paketi sorgulamadan package.json'a eklemek.
  - **Çözüm:** npm trends veya Snyk üzerinden paketin yaşını, yapımcısını ve indirilme sayısını kontrol edin.
- **Tuzak:** Derleme hatası vermeyen kodun doğru çalıştığını varsaymak.
  - **Çözüm:** Tip doğrulaması yalnızca şekli kontrol eder; iş mantığını doğrulamak için runtime testi yazın.
- **Tuzak:** AI'a 'Bu kod kesin çalışıyor mu?' diye sorup 'Evet efendim' yanıtına inanmak.
  - **Çözüm:** AI kendi hatasını savunma eğilimindedir; doğrulama için bağımsız test çalıştırın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 4: Yapay Zekayı Yakalamak - Halüsinasyon ve Güvenlik Denetimi*

- 🖥️ **Slayt 1:** Halüsinasyon nedir? Olasılıksal dil modelleri nasıl çalışır?

- 🖥️ **Slayt 2:** Paket halüsinasyonu (Slopsquatting) tehdidi ve tedbirler.
- 🖥️ **Slayt 3:** Eski (deprecated) API'leri tespit etme teknikleri.
- 🖥️ **Slayt 4:** Adım adım kod denetimi: AI çıktısını teste tabi tutmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Modelden bilerek var olmayan bir özellik için kütüphane isteyin. Modelin uydurduğu ismi npm'de aratarak sahte olduğunu gösterin. Ardından aynı işi standart kütüphanelerle güvenli şekilde kodlatın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki `package.json` dosyasını açın. En az 1 kütüphanenin npm sayfasını ziyaret edin; son güncelleme tarihini ve haftalık indirme sayısını inceleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Paket halüsinasyonu (Slopsquatting) saldırısı nasıl gerçekleşir? - A) Hackerların kullanıcının bilgisayarına fiziksel olarak girmesiyle - B) Saldırganların, yapay zekaların sıkça uydurduğu sahte paket isimlerini tescilleyip içine zararlı kod yerleştirmesiyle - C) Sadece C++ dilinde derleyicinin çökmesiyle - D) Bilgisayar ekranının donmasıyla > **Doğru Yanıt:** B > **Açıklama:** Saldırganlar LLM'lerin uydurmaya meyilli olduğu paket isimlerini takip eder ve bu isimlerle zararlı paketler yayınlayarak dikkatsiz geliştiricilerin indirmesini bekler.

Soru 2: Bir yapay zeka çıktısının güvenilir olduğunu anlamanın en sağlam yolu nedir? - A) AI'a 'Bu doğru mu?' diye sormak - B) Çıktıyı otomatik unit testler, linter'lar ve resmi dokümantasyon ile bağımsızca doğrulamak - C) Kodun çok uzun ve karmaşık görünmesi - D) Kodun StackOverflow'da bir yanıta benzemesi > **Doğru Yanıt:** B > **Açıklama:** Teknik doğruluk ancak derleyiciler, bağımsız test koşucuları ve resmi dokümantasyon ile garanti altına alınabilir.

---

5. Prompt Mühendisliği Değil 'Context Mühendisliği': LLM'lere Nasıl Brif Verilir?

Süslü Cümleleri Unutun: Şemalar, Tipler ve Dosya Sınırlarıyla Konuşun
Bölüm 1 • Zorluk Derecesi: Başlangıç Seviyesi • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"'Sen bir dünya çapında yazılımcısın' gibi süslü prompt şablonları gerçekten işe yarıyor mu, yoksa modele doğru TypeScript interface'ini vermek çok daha mı etkili?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Prompt mühendisliği sözcüklerle ilgilidir; Context mühendisliği ise sistemin durumunu ve sınırlarını modele aktarmakla ilgilidir. - 📌 Çöp bağlam (Garbage Context) verirseniz, en gelişmiş model bile çöp kod üretir. - 📌 Bir modele verilebilecek en kaliteli bağlam: Tip tanımları, girdi/çıktı örnekleri ve kabul kriterleridir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir mimarlık ofisine stajyer girdiğini düşünün. Patron stajyere: *"Bana harika, modern, ödül alacak bir bina çiz!"* derse ne olur? Stajyer uzay mekiğine benzeyen ama inşa edilmesi imkansız bir gökdelen çizer.

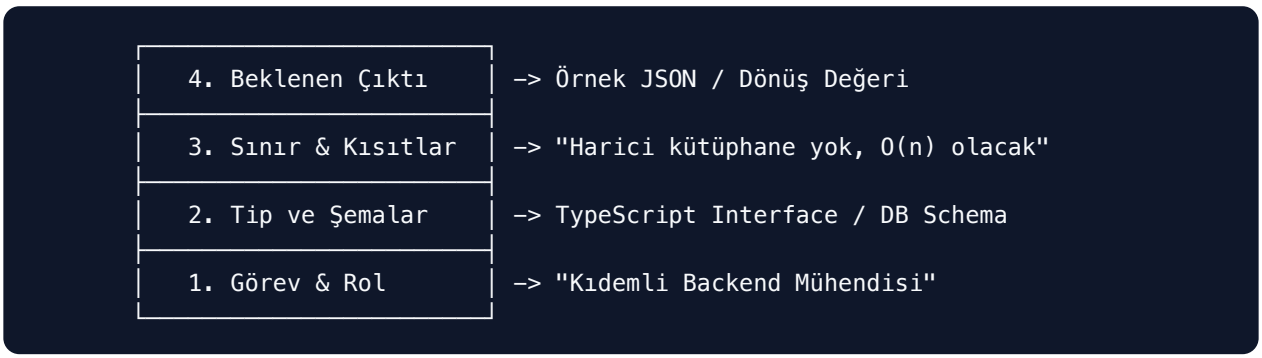
Peki patron şöyle brif verirse: - Arsa boyutu: 500 m² - İmar izni: En fazla 3 kat - Malzeme: Prefabrik betonarme ve cam - Hedef: 4 kişilik 6 adet daire

Stajyer tam istenen, hemen inşa edilebilir projeyi getirir!

LLM'ler de tıpkı bu stajyer gibidir. Onlara *"Bana harika bir sepet fonksiyonu yaz"* derseniz halüsinasyon görürler. Ama şemayı, tipleri ve kısıtları verirseniz kusursuz üretim yaparlar.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Context Mühendisliğinin 4B Formülü (Bağlam Piramidi):



🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Senior Backend Engineer
# GÖREV: Verilen kullanıcı listesini yaşa göre filtreleyip departmana göre gruplayan
fonksiyon yaz.

## GİRDİ TİP TANIMI (TypeScript):
```typescript
interface Employee {
  id: string;
  name: string;
  age: number;
  department: 'ENGINEERING' | 'SALES' | 'MARKETING';
}
```

KISITLAR:

- Hiçbir üçüncü taraf (Lodash vb.) kütüphane kullanılmayacak.
- Saf JavaScript/TypeScript `reduce` veya `Map` kullanılacak.
- Yaşı 18'den küçük olanlar elenecek.

BEKLENEN ÇIKTI TİPİ:

```
type GroupedResult = Record<Employee['department'], Employee[]>;
```

```
---

## 🖥️ 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Zayıf Bağlam Verildiğinde AI Çıktısı: Lodash kütüphanesi şart koşulmuş ve tipler
eksik
// Prompt: "Çalışanları departmana göre grupla"
const _ = require('lodash');

function groupStaff(data: any) {
  // Lodash yüklemediyseniz projeniz patlar!
  return _.groupBy(data, 'department');
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// Context Mühendisliği ile Elde Edilen Çıktı: Sıfır bağımlılık, tam tip güvenliği
export interface Employee {
  id: string;
  name: string;
  age: number;
  department: 'ENGINEERING' | 'SALES' | 'MARKETING';
}

export type GroupedEmployees = Partial<Record<Employee['department'], Employee[]>>;

export function groupEmployeesByDepartment(
  employees: readonly Employee[]
): GroupedEmployees {
  return employees.reduce<GroupedEmployees>((acc, emp) => {
    if (emp.age < 18) {
      return acc; // Kısıt: 18 yaş altı elenir
    }

    const dept = emp.department;
    if (!acc[dept]) {
      acc[dept] = [];
    }
    acc[dept]!.push(emp);
    return acc;
  }, {});
}
```

💡 **Kod Analizi:** Modele tipler ve kısıtlar (‘Lodash kullanma’) verildiğinde, fazladan 50KB bağımlılık eklemek yerine modern dil standartlarını kullanarak hafif ve güvenli kod üretir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Prompt içine 10 paragraflık hikaye yazıp asıl teknik kısıtları unutmak.
  - **Çözüm:** Laf kalabalığını atın; kod şemalarını ve kısıtları maddeler halinde yazın.
- **Tuzak:** Modele veri tabanı şemasını vermeden SQL veya ORM kodu istemek.
  - **Çözüm:** Her zaman ilgili tablo şemasını (schema.prisma veya CREATE TABLE) bağlam olarak ekleyin.
- **Tuzak:** Hata aldığınızda sadece ‘Hata verdi düzelt’ yazmak.
  - **Çözüm:** Terminal çıktısını ve çağrı yığınını (stack trace) tam olarak bağlama ekleyin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 5: Context Mühendisliği - LLM'lere Yazılımcı Gibi Brif Vermek*

- 🖥️ **Slayt 1:** Neden Prompt Mühendisliği yetersiz kalıyor?
- 🖥️ **Slayt 2:** 4B Formülü: Rol, Şemalar, Kısıtlar, Çıktı Formatı.
- 🖥️ **Slayt 3:** TypeScript Interface'lerini en güçlü prompt aracı olarak kullanmak.
- 🖥️ **Slayt 4:** Canlı uygulama: Kötü prompt vs Bağlamı zengin prompt karşılaştırması.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Aynı işi iki farklı pencerede yaptırın: Birinde 'Bana e-posta doğrulayan fonksiyon yaz' deyin; diğerinde regex kısıtları ve TypeScript tipleri verin. Çıkan iki kodu yan yana inceleyin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Yapay zekaya göndereceğiniz bir sonraki görevde hiçbir Türkçe/İngilizce açıklama yapmadan sadece `interface Input` ve `interface Output` vererek kod üretmesini isteyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Context mühendisliğinde bir yapay zeka modeline verilebilecek en yüksek sinyalli (en kaliteli) bilgi hangisidir? - A) 'Sen dahi bir yazılımcısın' cümlesi - B) Tip tanımları, veri tabanı şeması ve girdi/çıktı kısıtları - C) 5 sayfalık genel şirket tanıtım metni - D) Emojilerle süslenmiş motivasyon mesajı > **Doğru Yanıt:** B > **Açıklama:** Modeller matematiksel olarak semantik sınırlara göre çalışır; tip tanımları ve şemalar en kesin kısıtları sağlayarak sapmayı önler.

Soru 2: Yapay zekanın gereksiz üçüncü taraf kütüphaneler eklemesini önlemenin en pratik yolu nedir? - A) İnterneti kapatmak - B) Prompt kısıtlarına 'Sıfır harici kütüphane, sadece yerel dil özellikleri kullanılacaktır' şartı eklemek - C) Sadece C dilinde kod yazmak - D) Kütüphane eklediğinde ona kızmak > **Doğru Yanıt:** B > **Açıklama:** Açıkça belirtilen kısıtlar (constraints), modelin token tahmin alanını sınırlar ve gereksiz bağımlılıkları engeller.

---

6. Geliştirme Ortamını (IDE) Turbo Şarj Etmek: Cursor Rules ve Kısayollar

Her Seferinde Aynı Şeyleri Tekrarlamaktan Kurtulun
Bölüm 1 • Zorluk Derecesi: Başlangıç Seviyesi • Tahmini Okuma Süresi: 13 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Bir projede `.cursorrules` dosyası kullanmak ile kullanmamak arasındaki fark, junior bir mühendisin günlük üretkenliğini 2 kat artırabilir mi?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Cursor Rules (.cursorrules), takım standartlarınızı ve mimari kurallarınızı yapay zekanın kalıcı hafızasına kazır. - 📌 Her prompta 'Türkçe yaz, Tailwind kullan, test ekle' demek yerine bu kuralları otomatikleştirin. - 📌 Klavye kısayolları (Cmd+K, Cmd+L, Cmd+I) refleks haline geldiğinde kod yazma hızınız düşünme hızınıza yaklaşır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Her sabah kahve almaya gittiğiniz kafeyi hayal edin. Her gün baristaya: *"Benim adım Ali, laktozsuz sütlü, tek shot espressolu, kağıt bardakta bir latte istiyorum"* demek mi daha verimli; yoksa baristanın sizi gördüğü an: *"Ali Bey, her zamankinden mi?"* deyip 30 saniyede kahvenizi vermesi mi?

`.cursorrules` veya IDE kural dosyaları işte o baristadır!

Projenizin kök dizinine koyacağınız küçük bir metin dosyası, yapay zekaya sizin kim olduğunuzu, projenin hangi dille yazıldığını, hangi mimari kalıpları kullandığınızı ve nelere ASLA izin vermediğinizi her saniye fısıldar.

🧠 2. AI Çağında Yeni Zihniyet Modeli

İdeal Bir `.cursorrules` Dosyası Anatomisi:

```
# 1. Proje Kimliği ve Teknoloji Yığını (Stack)
- Next.js 14 (App Router), TypeScript Strict Mode, TailwindCSS.

# 2. Mimari Kurallar
- Asla 'any' kullanma; daima tam tip tanımla.
- React Server Component (RSC) varsayılandır; 'use client' sadece gerekli yerlerde.

# 3. Test ve Doğrulama
- Her yeni servis fonksiyonu için aynı isimde `.test.ts` dosyası üret.

# 4. Stil ve Format
- Erken dönüş (Early Return) desenini uygula.
- İç içe geçmiş if-else bloklarından kaçın.
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# GÖREV: Projemiz İçin Özel .cursorrules Dosyası Oluştur
Yeni bir Full-Stack projeye başlıyoruz.
Teknoloji Yığını:
- FastAPI (Python 3.12)
- Pydantic V2
- SQLAlchemy 2.0 (Async)
- PostgreSQL
- PyTest

Lütfen bu proje için AI'ın her zaman uyması gereken 5 maddelik katı bir
`.cursorrules` içeriği hazırla.
Özellikle tip güvenliğine, async/await standartlarına ve test yazımına odaklansın.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# Kötü .cursorrules: Çok uzun, laf kalabalığı dolu ve kural içermeyen metin
Sen harika bir yardımcısın. Benim için en iyi python kodlarını yaz.
Lütfen hata yapma ve her zaman nazik ol. Kodların çok temiz olsun.
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
# PROJE KURALLARI (.cursorrules)
## TEKNOLOJİ YIĞINI:
- Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2.0 (Async).

## KODLAMA STANDARTLARI:
1. TIP GÜVENLİĞİ: Tüm fonksiyonlarda type hints zorunludur. 'Any' yasaktır.
2. VERİTABANI: Asla senkron sorgu yazma; daima `await session.execute(...)` kullan.
3. MODELLER: Giriş/Çıkış için daima Pydantic şemalarını SQLAlchemy modellerinden
ayır.
4. HATA YÖNETİMİ: HTTPException fırlatırken daima standart JSON hata yapısı kullan.
5. TESTLER: Her endpoint için en az 1 başarı ve 2 hata durumunu kapsayan pytest
fonksiyonu yaz.
```

💡 **Kod Analizi:** Etkili kural dosyaları doğrudan emir kipleriyle, net yasaklarla ve teknoloji sürümleriyle yazılır. Model bu kuralları her işlemde 'Sistem Mesajı' olarak okur.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Kural dosyasına 50 sayfa dokümantasyon yapıştırıp context penceresini şişirmek.
  - **Çözüm:** Kuralları 30-50 satır arasında, madde madde ve öz tutun.
- **Tuzak:** Kural dosyasını yazıp git'e eklememek.
  - **Çözüm:** `.cursorrules` dosyasını depoya commit edin, böylece tüm ekip arkadaşlarınız aynı AI davranışından faydalansın.
- **Tuzak:** Kısayolları öğrenmeyip her şeyi fareyle tıklayarak yapmak.
  - **Çözüm:** Günde en az 30 dakika farenizi bir kenara bırakıp IDE kısayollarını zorlayın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 6: IDE'nizi Kişiselleştirmek - .cursorrules ile Takım Standartları*

- 🖥️ **Slayt 1:** .cursorrules dosyası nedir ve nasıl çalışır?
- 🖥️ **Slayt 2:** Kalıcı bağlam oluşturmanın faydaları.
- 🖥️ **Slayt 3:** En kritik IDE kısayolları ve geliştirici iş akışı.
- 🖥️ **Slayt 4:** Bölüm 1 Özeti: AI Çağında Yeni Nesil Yazılımcının Zihniyet Dönüşümü.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Proje kökünde `.cursorrules` oluşturun. 'Tüm fonksiyonlar Türkçe açıklama içermelidir ve TypeScript strict olmalıdır' kuralını yazın. Ardından basit bir fonksiyon isteyerek kuralın anında uygulandığını gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bugün çalıştığınız projenin kök dizininde bir `.cursorrules` dosyası oluşturun ve projenizde uyulması gereken en önemli 3 mimari kuralı içine yazın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Proje kök dizinindeki `.cursorrules` dosyasının temel işlevi nedir? - A) Bilgisayara yeni RAM taktırmak - B) Projedeki her AI etkileşimine otomatik olarak mimari kuralları ve kodlama standartlarını sistem bağlamı olarak eklemek - C) Sadece git commitlerini silmek - D) CSS dosyalarını sıkıştırmak > **Doğru Yanıt:** B > **Açıklama:** Kural dosyaları, geliştiricinin her promptta aynı kuralları tekrarlamasına gerek kalmadan yapay zekayı proje standartlarına bağlı tutar.

Soru 2: Cursor IDE'sinde dosya içinde doğrudan kod düzenleme penceresini açan temel kısayol hangisidir? - A) Ctrl + Alt + Del - B) Cmd + K (Windows'ta Ctrl + K) - C) F5 - D) Shift + Escape > **Doğru Yanıt:** B > **Açıklama:** Cmd+K (veya Ctrl+K) editörde seçili kod üzerinde doğrudan satır içi düzenleme yapmayı sağlayan en yaygın AI kısayoludur.

Bölüm 2: Algoritmik Düşünce, Veri Yapıları & AI ile Problem Çözme

Ezber kodlamanın ötesine geçip temel algoritmik mantığı kavrayın; yapay zekanın karmaşıklık analizlerini denetleyin.

7. Sözde Koddan (Pseudocode) Gerçek Koda: AI ile Algoritma Tasarlamak

Klavye Başına Geçmeden Önce Düşünceyi Yapılandırma Rehberi
Bölüm 2 • Zorluk Derecesi: Temel / Orta Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ **NotebookLM & Audio Overview Podcast Notları**

Podcast Tartışma Sorusu:
"Yapay zekaya doğrudan kod yazdırmak yerine neden önce adım adım sözde kod (pseudocode) tartıştırmalıyız?"

Bölümden Çıkarılacak 3 Altın Cümle:
- 📌 Sözde kod (Pseudocode), insan zihni ile yapay zekanın akıl yürütme motoru arasındaki ortak dildir. - 📌 Doğrudan kod istemek modelin aceleci varsayımlar yapmasına; sözde kod ise mantık adımlarını önceden planlamasına yol açar. - 📌 İyi bir sözde kod dile bağımsızdır; Python, Go veya Rust'a eşit kolaylıkla derlenebilir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir inşaat şantiyesine gittiğinizi düşünün. Şef mühendis ustaların yanına gelip: *"Hadi hemen tuğlaları üst üste koymaya başlayın, bina yükselsin!"* der mi? Tabii ki hayır! Önce mavi ozalit planlar (blueprints) incelenir, kolonların taşıma kapasitesi hesaplanır.

Kodlama dünyasında sözde kod (pseudocode), o mavi ozalit plandır.

Junior yazılımcıların en büyük hatası, problemi tam anlamadan yapay zekaya *"Hemen bana kod yaz"* demektir. Oysa önce mantık akışını sözde kodla kurduğunuzda, yapay zeka hem mantık hatalarını hemen fark eder hem de yazdığı kod ilk seferde çalışır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Sözde Kod ile Düşünme Döngüsü:

```
[Problem Tanımı]
      |
      ▼
[Sözde Kod (Pseudocode) Taslağı] ──> "Adım 1: Girdiyi doğrula, Adım 2: Set
kullanarak tekilleştir"
      |
      ▼
[AI İle Mantık & Karmaşıklık Denetimi] ──> "Burada O(N^2) var, hash map kullanalım
mı?"
      |
      ▼
[Temiz Kod Üretimi & Testler]
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Algoritma Uzmanı
Bir kullanıcı listesindeki mükerrer (duplicate) e-posta adreslerini temizleyip en
son kayıt olanı saklayan bir sistem tasarlayacağız.

Lütfen hemen Python veya JavaScript kodu YAZMA.
Bunun yerine:
1. Problemin adım adım SÖZDE KODUNU (pseudocode) çıkar.
2. Zaman karmaşıklığını (Time Complexity) hesapla.
3. Bellek kullanımını minimize etmek için alternatif bir yaklaşım var mı tartış.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ **Acemi / Kontrolsüz AI Çıktısı (Riskli):**

```
# Kötü: İç içe döngülerle O(n^2) çalışan kontrolsüz arama
def deduplicate_users(users):
    unique = []
    for u in users:
        # Her eleman için listeyi baştan sona tarar: 100,000 kullanıcıda sistem
kilitlenir!
        found = False
        for ex in unique:
            if ex['email'] == u['email']:
                found = True
                break
        if not found:
            unique.append(u)
    return unique
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
# Sözde Koddan Üretilmiş O(n) Performanslı Versiyon
from typing import List, Dict, Any

def deduplicate_users_latest(users: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
    """
    Kullanıcıları e-posta adresine göre tekilleştirir.
    En güncel 'created_at' damgasına sahip olan kaydı O(N) zaman karmaşıklığında
saklar.
    """
    latest_by_email: Dict[str, Dict[str, Any]] = {}

    for user in users:
        email = user.get("email", "").strip().lower()
        if not email:
            continue  # Geçersiz e-postaları atla

        existing = latest_by_email.get(email)
        if existing is None or user.get("created_at", 0) >
existing.get("created_at", 0):
            latest_by_email[email] = user

    return list(latest_by_email.values())
```

💡 **Kod Analizi:** Sözde kod aşamasında hash map (sözlük) kullanmaya karar verildiği için arama süresi O(N^2)'den O(N)'e düşürülmüştür. Ayrıca e-posta normalizasyonu (lowercase) ve boş değer koruması eklenmiştir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** AI'a 'bunu çöz' deyip çıkan karmaşık kodu anlamadan kabul etmek.
  - **Çözüm:** Her zaman önce algoritmanın Türkçe/İngilizce adımlarını yazdırın.
- **Tuzak:** Büyük veri setlerinde nested loop (iç içe döngü) tuzaklarına düşmek.
  - **Çözüm:** Veri yapısı olarak liste yerine Hash Map (Dict/Set) tercih edin.
- **Tuzak:** E-posta veya kullanıcı adı karşılaştırmalarında 'case-sensitivity'yi unutmak.
  - **Çözüm:** String verileri her zaman `.strip().lower()` ile normalize edin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 7: Sözde Kod ile Düşünmek - AI ile Algoritma Tasarımı*

- 🖥️ **Slayt 1:** Neden doğrudan kod yazmak en büyük junior hatasıdır?
- 🖥️ **Slayt 2:** Sözde kod (Pseudocode) nedir ve nasıl yapılandırılır?
- 🖥️ **Slayt 3:** O(N^2) felaketinden O(N) zarafetine geçiş.
- 🖥️ **Slayt 4:** Canlı demo: E-ticaret sepet hesaplama algoritmasını sözde kodla tasarlamak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Terminal veya chat ekranında bir LeetCode orta seviye problem verin. 'Kod yazma, sadece 4 maddelik sözde kod çıkar' deyin. Çıkan sözde kodu adım adım onaylayıp ardından tek seferde temiz fonksiyonu ürettirin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bir sonraki geliştireceğiniz fonksiyonda tek satır kod yazmadan önce defterinize veya not uygulamanıza 5 maddelik sözde kod yazın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Algoritma tasarımında sözde kod (pseudocode) kullanmanın en kritik faydası nedir? - A) Kodun derlenme süresini sıfıra indirmesi - B) Dil söz dizimi karmaşasına girmeden saf iş mantığını ve sınır durumlarını doğrulamaya imkan tanıması - C) Sadece Python dilinde çalışması - D) Veritabanı lisans maliyetlerini düşürmesi > **Doğru Yanıt:** B > **Açıklama:** Sözde kod, geliştiricinin ve yapay zekanın söz dizimi (syntax) yerine mantık akışına ve karmaşıklığa odaklanmasını sağlar.

Soru 2: 100.000 kullanıcılı bir listede tekilleştirme yaparken iç içe iki döngü kullanmanın tehlikesi nedir? - A) Hiçbir tehlikesi yoktur, en modern yöntemdir - B) O(N^2) zaman karmaşıklığı sebebiyle 10 milyar işlem gerektirip sunucuyu kilitlemesi - C) Sadece CSS hatalarına yol açması - D) Değişken isimlerinin

silinmesi > **Doğru Yanıt:** B > **Açıklama:** 100.000 eleman için N^2 işlem 10.000.000.000 adıma tekabül eder ve işlemciyi dakikalarca dondurur. O(N) hash map yaklaşımı ise milisaniyeler sürer.

8. Veri Yapıları AI Çağında Neden Hâlâ Hayati?

Array, Hash Map, Set ve Tree Arasındaki Seçim Sisteminizi Nasıl Kurtarır?
Bölüm 2 • Zorluk Derecesi: Temel / Orta Seviye • Tahmini Okuma Süresi: 13 dk

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Yapay zeka her şeyi optimize edebiliyorsa, bir yazılımcının Array ile Set arasındaki bellek farkını bilmesine neden hâlâ gerek var?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Yapay zeka varsayılan olarak en kolayına gelen veri yapısını (çoğunlukla Array) seçme eğilimindedir. - 📌 Yanlış veri yapısı seçildiğinde, dünyanın en gelişmiş AI modeli bile o kodu verimli hale getiremez. - 📌 Arama için Set/Map, sıralı erişim için Array, hiyerarşik ilişkiler için Tree kullanmak mimari bir karardır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Evinizde 10.000 adet kitap olduğunu düşünün.

Bu kitapları odanın ortasına rastgele devasa bir dağ gibi yığarsanız ne olur? Belli bir kitabı bulmak için tek tek 10.000 kitabı elden geçirmeniz gerekir. İşte bu **Dizi (Array)** üzerindeki doğrusal aramadır (O(N)).

Peki bu kitapları harf sırasına göre dizilmiş kütüphane raflarına yerleştirirseniz? İstediğiniz kitaba 10 saniyede ulaşırsınız! İşte bu **Hash Map / Ağaç (Tree)** veri yapısıdır.

Yapay zeka size kitapları taşımada yardım eden hızlı bir işçidir. Ama kitapları odaya yığacak mısınız yoksa raflara mı dizeceksiniz? Bu kararı sadece siz verebilirsiniz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Hangi Veri Yapısı Ne Zaman Seçilmeli?

Veri Yapısı	Arama Hızı	Ekleme/Silme	En İyi Senaryo	Dikkat
Array / List	O(N)	Sona ekleme O(1), Araya ekleme O(N)	Sıralı listeleme, indeksle doğrudan erişim	Büyük listelerde `includes()` veya `in` yapmak performansı öldürür
Set	O(1)	O(1)	Benzersizlik kontrolü, "Bu kullanıcı listede var mı?"	Sıralama garantisi yoktur
Hash Map / Dict	O(1)	O(1)	Anahtar-değer eşlemeleri, önbellek (cache), indeksleme	Bellek tüketimi diziye göre biraz daha fazladır
Tree / Graph	O(log N)	O(log N)	Hiyerarşik veri (kategori ağacı, organizasyon şeması)	Düğümleri yönetmek ve dengelemek uzmanlık ister

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Kıdemli Veri Yapıları Uzmanı
# GÖREV: 500.000 aktif oturum ID'si (Session Token) içeren bir listede gelen
isteklerin geçerli olup olmadığını kontrol edeceğiz.

Lütfen bana bir veri yapısı önerisi ve karşılaştırması sun:
1. Python'da `list` kullanmak ile `set` kullanmak arasındaki bellek ve zaman farkı
nedir?
2. 500.000 eleman için sorgu süresi karşılaştırması (Benchmark) senaryosu açıkla.
3. Kodu en optimize veri yapısıyla yaz.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ **Acemi / Kontrolsüz AI Çıktısı (Riskli):**

```
// Kötü Veri Yapısı: Dizi (Array) üzerinde O(n) arama yapmak
export class NaiveSessionManager {
  private activeTokens: string[] = [];

  addToken(token: string) {
    this.activeTokens.push(token);
  }

  // 100.000 aktif kullanıcıda her istekte tüm diziyi gezer: Felaket!
  isValid(token: string): boolean {
    return this.activeTokens.includes(token);
  }
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// Doğru Veri Yapısı: O(1) Set ile Anında Üyelik Kontrolü
export class OptimizedSessionManager {
  private activeTokens: Set<string> = new Set();

  addToken(token: string): void {
    this.activeTokens.add(token);
  }

  removeToken(token: string): boolean {
    return this.activeTokens.delete(token);
  }

  // 1.000.000 eleman olsa bile mikrosaniye düzeyinde O(1) yanıt
  isValid(token: string): boolean {
    return this.activeTokens.has(token);
  }

  get count(): number {
    return this.activeTokens.size;
  }
}
```

💡 **Kod Analizi:** Array üzerindeki `.includes()` araması O(N) iken, Set üzerindeki `.has()` kontrolü O(1) zaman karmaşıklığındadır. 100.000 öğede bu fark 50 milisaniyeden 0.001 milisaniyeye (50.000 kat) hızlanma anlamına gelir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Sadece varlık kontrolü (membership check) yaparken Array kullanmak.
  - **Çözüm:** Elemanın varlığını sorguluyorsanız her zaman Set veya Map tercih edin.

- **Tuzak:** Hash Map anahtarlarında çok büyük ve karmaşık objeler kullanıp hash çarpışması yaratmak.
  - **Çözüm:** Anahtar olarak her zaman benzersiz string veya ID kullanın.
- **Tuzak:** AI'a 'bunu hızlı yap' deyip arka plandaki veri yapısını incelememek.
  - **Çözüm:** AI'ın seçtiği veri tipinin zaman ve alan karmaşıklığını (Big-O) mutlaka sorgulayın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 8: Veri Yapıları Ustalığı - Doğru Koleksiyonu Seçmek*

- 🖥️ **Slayt 1:** Neden AI bile kötü bir veri yapısı seçimini kurtaramaz?
- 🖥️ **Slayt 2:** Array vs Set vs Map: 50.000 katlık performans farkı.
- 🖥️ **Slayt 3:** Bellek tüketimi vs Arama hızı dengesi.
- 🖥️ **Slayt 4:** Canlı Benchmark: 100.000 kayıt üzerinde List ve Set hız testi.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Node.js veya Python'da bir micro-benchmark scripti yazın. 100.000 elemanlı bir listede `in` / `.includes()` aramasının kaç ms sürdüğünü, Set ile `.has()` aramasının kaç ms sürdüğünü canlı konsolda gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Mevcut kod tabanınızda bir döngü içinde `.includes()` veya `in` kontrolü yapılan en az bir yer bulun ve bunu `Set` kullanarak O(1)'e refactor edin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: 100.000 elemanlı bir koleksiyonda bir elemanın var olup olmadığını en hızlı şekilde kontrol etmek için hangi veri yapısı kullanılmalıdır? - A) Dizi (Array / List) - B) Hash Set - C) Bağlı Liste (Linked List) - D) Yığıt (Stack) > **Doğru Yanıt:** B > **Açıklama:** Hash Set, verinin hash koduna göre O(1) sabit zamanda doğrudan konumuna erişir; tüm elemanları gezmek zorunda kalmaz.

Soru 2: Array yerine Hash Map kullanmanın potansiyel tek dezavantajı nedir? - A) Asla çalışmaması - B) Hash tablolarının ve kova (bucket) yapısının bellekte biraz daha fazla yer kaplaması - C) Sadece Python'da geçerli olması - D) Kodun okunmasını imkansız kılması > **Doğru Yanıt:** B > **Açıklama:** Hash Map'ler hızlı erişim sağlamak için bellekten bir miktar ödün verir (Time-Space Tradeoff).

---

9. Zaman ve Bellek Karmaşıklığı (Big O): AI Üretti, Ama Verimli mi?

5 Elemanla Test Edilen Kodun Canlıda 5 Milyon Elemanda Çökmesini Önlemek
Bölüm 2 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 13 dk

🎙️ **NotebookLM & Audio Overview Podcast Notları**

Podcast Tartışma Sorusu:
"AI modelleri lokal bilgisayarınızda anında çalışan kodlar üretir. Peki o kod canlı ortamda 1 milyon kullanıcıya ulaştığında neden sunucuları eritir?"

Bölümden Çıkarılacak 3 Altın Cümle:
- 📌 Lokal ortamda çalışan her kod ölçeklenebilir değildir; Big-O analizi olmadan canlıya çıkmak kumardır. - 📌 AI modelleri genellikle en az kod satırı gerektiren çözümleri üretir; bu çözümler ise çoğunlukla O(N^2) veya O(2^N) gizli tuzaklar taşır. - 📌 İyi bir mühendis, AI çıktısına bakar bakmaz döngü derinliğini ve bellek tahsisini (allocation) zihninde hesaplayabilmelidir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir kargo dağıtım şirketinde çalıştığınızı düşünün.

Eğer şehrinizde sadece 3 ev varsa, kargo görevlisi evlerin sırasını rastgele dolaşarak paketleri 15 dakikada dağıtabilir. Ancak şehirde 50.000 ev olduğunda, rotayı akıllıca planlamazsanız kargo görevlisi aynı sokaktan yüzlerce kez geçer ve teslimat aylar sürer!

Big-O notasyonu işte bu kargo rotasının matematiksel ifadesidir: - Girdi 10 katına çıktığında harcanan süre 10 katına mı çıkıyor (O(N))? - Yoksa 100 katına mı fırlıyor (O(N^2))? - Yoksa sabit mi kalıyor (O(1))?

Yapay zeka genellikle 3 evlik senaryolar için kod yazar. 50.000 evi hesaba katmak sizin işinizdir.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Big-O Skalası: Cennetten Cehenneme

```
O(1)      ──> [MÜKEMMEL] Sabit Zaman (Hash Map erişimi)
O(log N)  ──> [HARİKA] İkili Arama (Binary Search)
O(N)      ──> [İYİ / KABUL EDİLEBİLİR] Tekil döngü ile geçiş
O(N log N)──> [ORTALAMA] Hızlı Sıralama (Merge Sort / Timsort)
──────────────────────────────────────────────────────────────
O(N^2)    ──> [TEHLİKELİ] İç içe iki döngü (Nested Loops)
O(2^N)    ──> [FELAKET] Üstel artış, sistemi eritir
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: Yüksek Performanslı Sistem Mühendisi
Aşağıdaki Python fonksiyonunu analiz et.

## GÖREV:
1. Zaman (Time) ve Alan (Space) Big-O karmaşıklığını tam olarak hesapla.
2. Bu fonksiyon 100.000 elemanlı bir listede çalıştırılırsa ne olur?
3. Fonksiyonu O(N) veya O(N log N) seviyesine çekecek optimize alternatifi üret.

## KOD:
```python
def find_pairs(arr, target_sum):
    pairs = []
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] + arr[j] == target_sum:
                pairs.append((arr[i], arr[j]))
    return pairs
````
````

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```python
# AI'ın Ürettiği Naive Kod: O(N^2) Zaman Karmaşıklığı
def find_pairs_quadratic(arr: list[int], target: int) -> list[tuple[int, int]]:
    pairs = []
    # İç içe iki döngü: N=100,000 olduğunda 5 milyar karşılaştırma!
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] + arr[j] == target:
                pairs.append((arr[i], arr[j]))
    return pairs
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```python
# Optimize Edilmiş Versiyon: O(N) Zaman ve O(N) Alan Karmaşıklığı
def find_pairs_linear(arr: list[int], target: int) -> list[tuple[int, int]]:
    seen = set()
    pairs = set()

    # Tek geçiş (Single Pass): N=100,000 olduğunda sadece 100,000 işlem
    for num in arr:
        complement = target - num
        if complement in seen:
            # Sıralı tuple ekleyerek tekrarları önleriz
            pairs.add((min(num, complement), max(num, complement)))
        seen.add(num)

    return list(pairs)
```

💡 **Kod Analizi:** Hash Set kullanımı sayesinde tamamlayıcı sayı (complement = target - num) O(1) sürede bulunur. İç içe döngü kaldırılarak algoritma O(N^2)'den O(N)'e çekilmiştir. 100.000 elemanda işlem süresi 15 dakikadan 0.05 saniyeye düşer.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Küçük mock verilerle (3-5 eleman) test yapıp performansın harika olduğunu sanmak.
  - **Çözüm:** Test senaryolarınıza en az 50.000 rastgele kayıt üreten yük testleri ekleyin.
- **Tuzak:** Döngü içinde farkında olmadan gizli O(N) işlemleri çalıştırmak (Örn: Array.splice veya List.pop(0)).
  - **Çözüm:** Listenin başından eleman silmenin O(N) olduğunu unutmayın; deque kullanın.
- **Tuzak:** Premature Optimization (Erken optimizasyon) yaparak kodu okunaksız hale getirmek.
```

- **Çözüm:** Önce temiz ve doğru çalışan kodu yazın; profil çıkarıp darboğazı ölçtükten sonra optimize edin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 9: Big-O Analizi - AI Kodunun Performansını Ölçmek*

- 🖥️ **Slayt 1:** Lokalde çalışan kod neden prod ortamında çöker?
- 🖥️ **Slayt 2:** Big-O karmaşıklığını 30 saniyede gözle tespit etme formülü.
- 🖥️ **Slayt 3:** O(N^2) vs O(N) Two-Sum algoritması vaka analizi.
- 🖥️ **Slayt 4:** Python cProfile ve Node.js console.time ile darboğaz tespiti.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Python'da `time` modülünü kullanarak 100.000 rastgele sayı üretin. `find_pairs_quadratic` fonksiyonunun süresini ölçün (birkaç saniye sonra manuel durdurun). Ardından `find_pairs_linear` fonksiyonunu çalıştırıp 0.04 saniyede bittiğini gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki bir döngüyü inceleyin. İçinde başka bir döngü veya `.find()` / `.indexOf()` var mı kontrol edin. Eğer varsa bunu bir Map veya Set ile O(N)'e dönüştürün.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: İç içe iki adet `for` döngüsünün aynı dizi (N elemanlı) üzerinde koştuğu bir algoritmanın zaman karmaşıklığı nedir? - A) O(1) - B) O(N) - C) O(N^2) - D) O(log N) > **Doğru Yanıt:** C > **Açıklama:** Her eleman için diğer tüm elemanlar taranıyorsa N * N = N^2 işlem yapılır; bu da O(N^2) kuadratik karmaşıklıktır.

Soru 2: Bir Python listesinin en başından eleman silmek (`list.pop(0)`) neden O(N) karmaşıklıktadır? - A) Python yavaş bir dil olduğu için - B) İlk eleman silindiğinde geriye kalan tüm elemanların bellekte bir sola kaydırılması gerektiği için - C) Sadece sayılarda geçerli olduğu için - D) Yanlış bilgi, O(1)'dir > **Doğru Yanıt:** B > **Açıklama:** Diziler bellekte bitişik bloklar halindedir. Baştan eleman silindiğinde arkadaki tüm elemanlar boşluğu doldurmak için bir adım ötelenir.

---

10. Tip Sistemleri ve Clean Code: AI Çıktısını Type-Safe Kılmak

'Any' Kullanımını Yasaklayarak Çalışma Zamanı Hatalarını Derleme Zamanında Yakalamak
Bölüm 2 • Zorluk Derecesi: Temel / Orta Seviye • Tahmini Okuma Süresi: 12 dk

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Yapay zeka modelleri neden sürekli 'any' veya belirsiz tipler kullanmaya meyillidir ve bu tembellik projeyi nasıl çökertir?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 TypeScript veya statik tipler lüks değil; yapay zekanın saçmalamasını engelleyen en güçlü gardiyandır. - 📌 `any` kullanmak, tip denetleyicisini (type checker) kör etmek ve çalışma zamanı hatalarına davetiye çıkarmaktır. - 📌 Zod veya Pydantic gibi doğrulama kütüphaneleri, dış dünyadan gelen verinin AI kodunu patlatmasını engeller.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Havalimanı gümrük kapısını düşünün.

Gümrük memuru gelen her bavulu açıp kontrol etmek yerine: *"Geç geç, hepiniz geçin, ne olursa olsun kabulüm"* derse ne olur? Ülkeye yasaklı maddeler, kaçak eşyalar dolar. İşte TypeScript'te `any` kullanmak tam olarak budur.

Oysa sıkı bir tip sistemi (Strict Typing), bavulları x-ray cihazından geçiren memurdur: *"Bu bavulda sadece 'User' tipi kıyafetler olabilir. 'undefined' veya 'null' tespit edildi, içeri giremezsiniz!"*

Yapay zeka modelleri bazen kolaya kaçıp `any` yazar. Bir mühendisin görevi, o gümrük kapısını asla nöbetçisiz bırakmamaktır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Tip Güvenliği Zinciri:

```
[Dış Dünya: API / Form / DB]
       |
       ▼
[Çalışma Zamanı Doğrulama (Zod / Pydantic)] ──> "Girdi şemaya uyuyor mu?"
       |
       ▼
[Derleme Zamanı Güvencesi (Strict Types)] ──> "Asla undefined veya any yok"
       |
       ▼
[Saf & Temiz İş Mantığı (Clean Code)]
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: TypeScript / Clean Code Gurusu
Aşağıdaki gevşek ve güvensiz yazılmış JavaScript/TypeScript fonksiyonunu incele.

## GÖREV:
1. Kodda `any` kullanımını tamamen kaldır.
2. Zod kullanarak gelen JSON verisini doğrulayan (runtime validation) bir şema yaz.
3. 'Early Return' ve Clean Code prensiplerine uygun, tip güvenli hale getir.

## KOD:
```typescript
function processOrder(data: any) {
  if (data) {
    if (data.items) {
      return data.items.map((i: any) => i.price * i.qty);
    }
  }
}
```
```

````
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Kötü / Gevşek Tip: 'any' kullanımı ve tanımsız değer çökmesi riski
export function processOrderNaive(data: any): any {
  // Eğer data.items boşsa veya price string geldiyse ('100' * 2 = NaN) patlar!
  return data.items.map((item: any) => {
    return item.price * item.quantity;
  });
}
````
````

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
import { z } from "zod";

// 1. Çalışma Zamanı Şeması (Runtime Schema)
export const OrderItemSchema = z.object({
  id: z.string().uuid(),
  price: z.number().positive("Fiyat pozitif bir sayı olmalıdır."),
  quantity: z.number().int().positive("Adet pozitif tam sayı olmalıdır.")
});

export const OrderSchema = z.object({
  orderId: z.string().min(1),
  items: z.array(OrderItemSchema).min(1, "En az bir sipariş kalemi bulunmalıdır.")
});

// 2. TypeScript Tipleri Otomatik Türetilir (Inference)
export type Order = z.infer<typeof OrderSchema>;
export type OrderItem = z.infer<typeof OrderItemSchema>;

export interface OrderTotalResult {
  orderId: string;
  total: number;
  itemCount: number;
}

// 3. Tip Güvenli ve Temiz Fonksiyon
export function calculateOrderTotal(rawData: unknown): OrderTotalResult {
  // Veriyi kapıda doğrula: Geçersizse anında açıklayıcı hata fırlatır
  const validated = OrderSchema.parse(rawData);

  const total = validated.items.reduce((sum, item) => {
    return sum + (item.price * item.quantity);
  }, 0);

  return {
    orderId: validated.orderId,
    total: Math.round(total * 100) / 100,
    itemCount: validated.items.length
  };
}
```

💡 **Kod Analizi:** `unknown` tipi ile dışarıdan gelen verinin belirsizliği açıkça kabul edilmiş, ardından Zod ile kapıda doğrulanmıştır. `z.infer` ile tek bir kaynaktan hem derleme hem çalışma zamanı güvenliği sağlanmıştır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Tip hatasını çözemeyince `// @ts-ignore` veya `as any` yazıp geçmek.

  - **Çözüm:** Sorunu halının altına süpürmeyin; tip daraltma (type narrowing) veya Zod kullanın.
- **Tuzak:** Sadece interface yazıp dışarıdan gelen JSON'ın buna uyduğunu varsaymak.
  - **Çözüm:** TypeScript tipleri derlendikten sonra yok olur (type erasure); runtime'da Zod veya Pydantic şarttır.
- **Tuzak:** İç içe 5 kat if-else piramitleri oluşturmak.
  - **Çözüm:** Guard Clause (Early Return) desenini uygulayarak kodu düzleştirin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 10: Tip Sistemleri ve Clean Code - Zod ile Kusursuz Doğrulama*

- 🖥️ **Slayt 1:** Neden 'any' yazılım dünyasının en tehlikeli anahtar kelimesidir?
- 🖥️ **Slayt 2:** Type Erasure nedir? Derleme zamanı vs Çalışma zamanı.
- 🖥️ **Slayt 3:** Zod ile tek kaynaktan şema ve tip türetme.
- 🖥️ **Slayt 4:** Guard Clause deseni ile iç içe if-else piramitlerinden kurtulmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir API endpoint simülasyonu yapın. Gelen bozuk JSON verisini (örneğin fiyatı string veya negatif olan bir obje) önce naive fonksiyona yollayın, `NaN` aldığını gösterin. Ardından Zod şemasından geçirin ve hatanın detaylı fırlatıldığını gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde `any` kullanılan en az 2 yeri bulun. `any` yerine katı bir interface veya `unknown` + Zod doğrulaması ekleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: TypeScript kodunda `any` yerine `unknown` kullanmanın temel farkı nedir? - A) Hiçbir farkı yoktur - B) `unknown` tipi, üzerinde işlem yapılmadan önce tip kontrolü (typeof, instanceof veya Zod) yapılmasını zorunlu kılar - C) `unknown` sadece sayılar için kullanılır - D) `unknown` kodun derlenmesini engeller > **Doğru Yanıt:** B > **Açıklama:** `any` tüm kontrolleri devre dışı bırakırken, `unknown` geliştiriciyi veriyi kullanmadan önce tipini daraltmaya zorlar.

Soru 2: Zod veya Pydantic gibi kütüphanelere neden ihtiyaç duyarız? - A) CSS yazmak için - B) TypeScript tipleri derleme sonrasında silindiği için, dışarıdan gelen verileri çalışma zamanında (runtime) doğrulamak için - C) Sadece SQL sorgularını hızlandırmak için - D) Bilgisayarın fan hızını ayarlamak için

> **Doğru Yanıt:** B > **Açıklama:** TypeScript derlendiğinde tüm tip tanımları silinir. Çalışma zamanında gelen verinin güvenliği ancak runtime validator'lar ile sağlanabilir.

11. Hata Mesajlarını ve Stack Trace'i AI ile Okuma ve Akıl Yürütme Sanatı

Terminaldeki Kırmızı Yazılardan Korkmayı Bırakıp Dedektif Gibi İz Sürmek
Bölüm 2 • Zorluk Derecesi: Temel / Orta Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ **NotebookLM & Audio Overview Podcast Notları**

Podcast Tartışma Sorusu:
"Junior mühendisler terminaldeki 50 satırlık kırmızı hata mesajını görünce neden panikler ve AI bu panikten nasıl kurtarır?"

Bölümden Çıkarılacak 3 Altın Cümle:
- 📌 Stack Trace düşmanınız değil; hatanın nerede, ne zaman ve hangi çağrı zincirinde gerçekleştiğini söyleyen yol haritasıdır. - 📌 AI'a sadece 'Kod çalışmıyor' demek yerine tüm stack trace'i ve ilgili fonksiyon kodunu vermek çözümü 10 saniyeye indirir. - 📌 En üstteki kütüphane satırlarına değil, kendi yazdığınız kodun bulunduğu satır numarasına odaklanın.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir uçağın kokpitini hayal edin. Aniden kırmızı bir ikaz ışığı yanıp sönüyor ve siren çalıyor.

Acemi bir pilot ne yapar? Panikleyip ellerini havaya kaldırır veya gözünü kapatır. Tecrübeli bir kaptan pilot ne yapar? Sakinlikle ikaz panelindeki hata kodunu okur: *"Motor 2 yağ basıncı uyarısı, Sayfa 4B kontrol listesini açın."*

Terminalinizde patlayan o 50 satırlık kırmızı Stack Trace de uçağın ikaz panelidir. O kırmızı yazılar size kızmıyor; sadece: *"Filanca dosyanın 42. satırında tanımsız bir objenin özelliğini okumaya çalıştın"* diyor.

Yapay zekayı yanınıza yardımcı pilot olarak aldığınızda, o karmaşık logları 5 saniyede tercüme edebilirsiniz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Stack Trace Okuma Filtresi:

```
[50 Satırlık Kırmızı Terminal Çıktısı]
              |
              ▼
[1. Hata Türü & Mesajı]: TypeError: Cannot read properties of undefined (reading
'email')
              |
              ▼
[2. Kütüphane Gürültüsünü Ele]: /node_modules/express/lib/router/... (ATLA!)
              |
              ▼
[3. Kendi Kodunun İlk Satırı]: at UserService.getUserProfile (src/services/
user.ts:42:18)
              |
              ▼
[4. AI'a Beslenecek Odak]: "src/services/user.ts dosyasının 42. satırındaki çağrı
bağlamı"
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: Debugging ve Kök Neden Analiz Uzmanı
Uygulamamız aşağıdaki hatayı fırlattı.

## HATA MESAJI & STACK TRACE:
```text
TypeError: Cannot read properties of undefined (reading 'name')
    at formatCustomerCard (/app/src/utils/card.ts:34:25)
    at Array.map (<anonymous>)
    at getCustomersDashboard (/app/src/controllers/dashboard.ts:18:32)
````

İLGİLİ KOD (src/utils/card.ts - Satır 30-40):
````

```
export function formatCustomerCard(customer: Customer) {
  return {
    id: customer.id,
    displayName: customer.profile.name.toUpperCase(),
  };
}
```

GÖREV:

1. Bu hatanın gerçek kök nedeni (Root Cause) nedir?
2. `profile` veya `name` alanının eksik geldiği durumları güvenle ele alacak defansif düzeltmeyi yap.
3. Bu durumun gelecekte tekrar etmemesi için TypeScript tipini güncelle.

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Hata Fırlatan Kod: Profil veya Name tanımsız olduğunda patlar
export function formatCustomerCardNaive(customer: any) {
  // Eğer customer.profile null veya undefined ise uygulama çöker!
  return {
    id: customer.id,
    displayName: customer.profile.name.toUpperCase(),
  };
}
```
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
export interface CustomerProfile {
  name?: string;
  avatarUrl?: string;
}

export interface Customer {
  id: string;
  profile?: CustomerProfile | null;
}

export interface FormattedCustomerCard {
  id: string;
  displayName: string;
}

export function formatCustomerCard(customer: Customer): FormattedCustomerCard {
  // Optional Chaining (?.) ve Nullish Coalescing (??) ile tam koruma
  const rawName = customer.profile?.name?.trim();
  const displayName = rawName ? rawName.toUpperCase() : "İSİMSİZ MÜŞTERİ";

  return {
    id: customer.id,
    displayName
  };
}
```

💡 **Kod Analizi:** Stack trace incelendiğinde `profile` nesnesinin null geldiği hemen anlaşılır. `customer.profile?.name` ile güvenli zincirleme yapılmış ve varsayılan bir geri dönüş değeri (`İSİMSİZ MÜŞTERİ`) atanarak çökme engellenmiştir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Hata çıktısını okumadan tüm dosyayı silip baştan yazdırmak.
  - **Çözüm:** Hatanın işaret ettiği dosya ve satır numarasını bulun, sadece o bloğu inceleyin.
- **Tuzak:** Terminaldeki hatayı kopyalamadan AI'a 'kod hata verdi' demek.
  - **Çözüm:** Her zaman hem hata türünü hem de ilk 10 satırlık stack trace'i eksiksiz yapıştırın.
- **Tuzak:** Hatanın sadece sonucunu gizlemek (Örn: Boş try/catch ile hatayı yutmak).
  - **Çözüm:** Hataları asla sessizce yutmayın; loglayın ve kök nedeni düzeltin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 11: Stack Trace Okuma Sanatı - Kök Neden Analizi*

- 🖥️ **Slayt 1:** Stack Trace anatomisi: Hata türü, mesajı ve çağrı yığını.
- 🖥️ **Slayt 2:** node_modules gürültüsünü filtreleyip uygulama kodunu bulmak.
- 🖥️ **Slayt 3:** Optional Chaining (?.) ve Nullish Coalescing (??) ile defansif kodlama.
- 🖥️ **Slayt 4:** Canlı vaka: Production çökmesini AI ile 60 saniyede teşhis edip çözmek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Kasıtlı olarak `null.toUpperCase()` hatası fırlatan bir API yazın. Terminalde beliren stack trace'i kopyalayın, AI asistanına besleyip kök neden analizini ve çözümünü tek adımda gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bugün çalıştığınız projede bilerek bir hata fırlatın (`throw new Error('Test Hatası')`). Oluşan stack trace'i satır satır inceleyerek çağrı zincirini takip edin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Bir stack trace incelenirken ilk bakılması gereken yer neresidir? - A) En alttaki işletim sistemi çekirdek çağrıları - B) Hata mesajının kendisi (Error Signature) ve kendi proje dizinimize işaret eden ilk satır numarası - C) Bilgisayarın IP adresi - D) package.json dosyasının son satırı > **Doğru Yanıt:** B > **Açıklama:** Hatanın türü (örn. TypeError) ve kendi kaynak kodumuzun yer aldığı dosya/satır numarası sorunun kaynağını doğrudan gösterir.

Soru 2: JavaScript/TypeScript'te `customer?.profile?.name ?? 'Anonim'` ifadesinin sağladığı fayda nedir? - A) Kodu 10 kat hızlandırır - B) Eğer customer veya profile tanımsız (undefined/null) ise çökmeden güvenle 'Anonim' değerini döndürür - C) Sadece sayılarda çalışır - D) Belleği tamamen sıfırlar > **Doğru Yanıt:** B > **Açıklama:** Optional Chaining (`?.`) zincirdeki ara eleman null/undefined ise hata fırlatılmasını önler; Nullish Coalescing (`??`) ise varsayılan değer sağlar.

---

12. Kod Okuma Süper Gücü: Yabancı Bir Kod Tabanını AI ile 10 Dakikada Haritalamak

200.000 Satırlık Monolite İlk Kez Bakan Junior'ın Hayatta Kalma Kılavuzu
Bölüm 2 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 13 dk

> **🎙️ NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Bir şirkette ilk iş gününüzde 200.000 satırlık devasa bir kod tabanıyla karşılaştığınızda, günlerce kaybolmak yerine AI ile sistemi nasıl 1 saatte çözebilirsiniz?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Yazılım mühendisleri zamanlarının %70'ini kod yazarak değil, mevcut kodu okuyarak ve anlayarak geçirir. - 📌 Bilinmeyen bir projeyi incelerken ilk bakılacak yerler: `package.json`, giriş noktası (`index/main`) ve veritabanı şemalarıdır. - 📌 AI'a 'Tüm projeyi oku' demek yerine; 'Giriş isteği (request) geldiğinde hangi katmanlardan geçiyor?' gibi akış soruları sorun.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Kendinizi yabancı bir metropolde, örneğin Tokyo'da bir sokak ortasında hayal edin. Elinizde harita yok, tabelalardaki yazıları okuyamıyorsunuz ve binalar gökyüzüne uzanıyor.

Geleneksel bir junior mühendis 500 dosyalık bir şirket projesini açtığında tam olarak bu Tokyo sendromunu yaşar: *"Ben neredeyim? Bu dosya nereye bağlanıyor?"*

Akıllı bir gezgin ne yapar? Önce metro haritasını açar, ana hatları (kırmızı, mavi hatlar) öğrenir.

Kod tabanının metro haritası: 1. Giriş kapısı (Main / Entry Point) 2. Veritabanı modelleri (Data Models) 3. API yönlendiricileri (Router / Controllers)

Yapay zeka ise elinizdeki akıllı navigasyon cihazıdır. Doğru soruları sorduğunuzda sizi 10 dakikada şehrin merkezine ulaştırır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Kod Tabanı Haritalama Hiyerarşisi (Top-Down Yaklaşım):

```
1. MANİFESTO: package.json / pyproject.toml
   └── Teknoloji yığını, bağımlılıklar, scriptler

2. VERİ MODELİ: schema.prisma / models/
   └── Bu sistem neyi saklıyor? (Kullanıcı, Ürün, Fatura)

3. GİRİŞ KAPISI: src/main.ts / app.py
   └── Sunucu nasıl ayağa kalkıyor? Hangi middleware'ler devrede?

4. İŞ MANTIĞI: services/ / use-cases/
   └── Gerçek dünya kuralları nerede işleniyor?
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Kıdemli Yazılım Mimarı Onboarding Rehberi
Projeye yeni katılan bir mühendisim. Projemizde aşağıdaki 3 ana dosya bulunmaktadır:
1. @package.json
2. @schema.prisma
3. @src/server.ts

## GÖREV:
Bu 3 dosyayı inceleyerek bana şu konularda 1 sayfalık yönetici özeti sun:
1. Bu sistem temel olarak ne işe yarar (iş modeli nedir)?
2. Bir HTTP isteği geldiğinde izlenen tipik akış mimarisi nasıldır?
3. Projeyi yerel makinemde ayağa kaldırmak için çalıştırmam gereken komut sırası
nedir?
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# Kötü Yaklaşım: Nereden başlayacağını bilmeden rastgele dosyaları açıp kaybolmak
# 500 dosya arasında kaybolup 3 gün boyunca hiçbir şey anlayamamak...
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
# Akıllı Haritalama İş Akışı:
# 1. Proje bağımlılıklarını ve çalıştırma komutlarını incele
cat package.json | grep -A 10 "scripts"

# 2. Veritabanı modellerini haritala
cat prisma/schema.prisma | grep "model "

# 3. Testleri çalıştırarak canlı doğrulama yap
npm run test:unit

# 4. Cursor / AI aracında @codebase ile sistem mimarisi diyagramı iste:
# "Generate a mermaid diagram illustrating the lifecycle of an order creation
request."
```

💡 **Kod Analizi:** Yeni bir projeye girdiğinizde rastgele dosya okumak yerine `scripts` alanını ve veritabanı modellerini incelemek size 10 kat daha hızlı resmin bütününü verir. AI'a Mermaid formatında mimari diyagram çizdirmek akılda kalıcılığı artırır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** İlk günden tüm kod tabanını baştan sona okumaya çalışıp tükenmek.
  - **Çözüm:** Sadece üzerinde çalışacağınız ilk biletin (ticket) dokunduğu dosyaları izole edin.
- **Tuzak:** Projeyi lokalde çalıştırmadan sadece statik kod okumak.
  - **Çözüm:** İlk gün hedefiniz projeyi yerel makinenizde hatasız derleyip testleri yeşile döndürmek olsun.
- **Tuzak:** Ekip arkadaşlarına sormadan önce AI ile kodun amacını analiz etmemek.
  - **Çözüm:** Senior'a gitmeden önce AI ile temel akışı çıkarın; soru sorarken 'X dosyasını inceledim, Y akışını teyit etmek istiyorum' deyin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 12: Kod Okuma Süper Gücü - Kod Tabanını Haritalamak*

- 🖥️ **Slayt 1:** Yazılımcılık neden %70 kod okuma, %30 kod yazmadır?
- 🖥️ **Slayt 2:** 3 Nokta Kılavuzu: Manifestolar, Veri Modelleri, Giriş Noktaları.
- 🖥️ **Slayt 3:** Mermaid.js ile AI destekli anlık mimari akış diyagramları üretmek.
- 🖥️ **Slayt 4:** Bölüm 2 Özeti: Algoritmik Düşünceden Büyük Resme.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Açık kaynaklı popüler bir repoyu (örneğin Redux Toolkit veya tRPC) açın. Cursor'da `@codebase` kulla-

narak: 'Generate a Mermaid sequence diagram for a basic request' komutunu verin. Çıkan diyagramı Markdown önizlemesinde inceleyerek akışı anlatın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

GitHub'da yıldızı yüksek bir açık kaynak projesini klonlayın. AI asistanınızı kullanarak 15 dakika içinde projenin ne işe yaradığını ve veri tabanı modellerini özetleyen bir `ARCHITECTURE.md` dosyası çıkartın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Yabancı bir yazılım projesine ilk kez katıldığınızda sistemin ne yaptığını anlamak için en stratejik başlangıç noktası neresidir? - A) Rastgele bir CSS dosyası - B) Proje manifestosu (package.json / pyproject.toml) ve veritabanı şemaları (schema / models) - C) .gitignore dosyası - D) README dosyasındaki telif hakları bölümü > **Doğru Yanıt:** B > **Açıklama:** Manifesto projenin teknoloji yığınını ve araçlarını gösterirken, veritabanı şemaları sistemin işlediği gerçek dünya nesnelerini (User, Order vb.) ortaya koyar.

Soru 2: AI yardımıyla bir kod tabanının akışını görselleştirmek için en yaygın kullanılan markdown tabanlı diyagram dili hangisidir? - A) HTML5 - B) Mermaid.js - C) SASS - D) Dockerfile > **Doğru Yanıt:** B > **Açıklama:** Mermaid.js, doğrudan markdown dosyaları içinde sequence, flowchart ve class diyagramları oluşturmaya yarayan endüstri standardı bir araçtır.

Bölüm 3: Modern Git, Takım Çalışması & AI-Destekli Kod İnceleme

> **Takım ortamında kusursuz sürüm kontrolü, rebase, branch stratejileri ve AI destekli PR süreçlerini yönetin.**

---

13. Git Temelleri & Akıllı Commit Stratejisi: AI ile Anlamlı Geçmiş

'WIP', 'Fixed Bug', 'asdasd' Commitlerinden Kurtulma Rehberi
Bölüm 3 • Zorluk Derecesi: Temel / Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Git geçmişi neden bir şirketin en değerli dokümantasyonudur ve yapay zeka ile Conventional Commits standardı nasıl otomatikleştirilir?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Git commit mesajları sadece ne yaptığınızı değil, o kararı neden aldığınızı anlatmalıdır. - 📌 Conventional Commits (feat, fix, refactor, chore) standardı, yapay zekanın kod tabanını daha iyi anlamasını sağlar. - 📌 AI destekli Git hook'ları (Husky + Git hooks) ile kalitesiz commit mesajları daha terminalden çıkmadan engellenebilir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir adli tıp uzmanının suç mahallini incelediğini düşünün. Masanın üzerinde bir günlük var; ancak sayfaların hepsinde sadece *"Bir şeyler yaptım"*, *"Düzeltme"*, *"Son hali"* yazıyor. Katili bu notlarla bulabilir misiniz? İmkansız!

İşte canlı sunucunuz gece saat 03:00'te çöktüğünde ve acil geri alma (rollback) yapmanız gerektiğinde, o 50 adet *"fixed bug"* veya *"asdasd"* yazan commit mesajları tam olarak o işe yaramaz günlüktür.

Oysa *"fix(auth): prevent token expiration crash by validating clock skew"* yazan bir commit, sistemi saniyeler içinde kurtarır.

Yapay zeka sayesinde artık harika commit mesajları yazmak 2 saniye sürüyor; tembellik için hiçbir mazeret kalmadı.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Conventional Commits Standardı Formülü:

```
<tür>(<kapsam>): <kısa özet (emir kipi)>

[İsteğe bağlı: Neden bu değişiklik yapıldı? (Motivation & Why)]
[İsteğe bağlı: Varsa kırıcı değişiklik (BREAKING CHANGE)]

Türler:
- feat: Yeni bir kullanıcı özelliği
- fix: Bir hatanın düzeltilmesi
- refactor: Davranışı değiştirmeden kodu temizleme
- test: Test ekleme veya düzeltme
- chore: Bağımlılık güncellemesi, yapılandırma
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: Git ve Sürüm Kontrol Uzmanı
Aşağıda `git diff` çıktım bulunmaktadır.

## GİRDİ (git diff):
```diff
diff --git a/src/auth.ts b/src/auth.ts
--- a/src/auth.ts
+++ b/src/auth.ts
@@ -12,3 +12,6 @@ export function verifyToken(token: string) {
+  if (!token || token.trim() === '') {
+    throw new InvalidTokenError('Token boş olamaz');
+  }
````

GÖREV:

Bu diff için Conventional Commits standardına tam uyumlu: 1. Başlık satırı (Title - en fazla 50 karakter) 2. Açıklama gövdesi (Body - neden yapıldığı ve çözülen güvenlik riski) çıkar.
````

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```bash
# Kötü Alışkanlık: Anlamsız, tek satırlık acele commit
git add .
git commit -m "auth duzeltildi"
git push origin main
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
# AI Destekli Profesyonel Conventional Commit
git add src/auth.ts
git commit -m "fix(auth): validate empty tokens before verification

Guard against empty and whitespace-only authorization tokens.
Prevents unhandled runtime crashes and rejects malicious empty bearer headers
at the perimeter. Closes #104."

# CLI AI Aracıyla Tek Komutta Üretmek:
# npx czg veya aicommits kullanarak otomatik üretin
```

💡 **Kod Analizi:** Kıdemli commit mesajı; hatanın kapsamını (`auth`), eylemi (`validate`), gerekçeyi (çökme ve güvenlik açığı engelleme) ve ilişkili bilet numarasını (`#104`) içerir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Tek bir devasa commit içine 20 farklı ilgisiz işi koyup 'tüm işler bitti' yazmak.
  - **Çözüm:** Atomik commit prensibini uygulayın; her commit tek bir mantıksal değişikliği temsil etmelidir.
- **Tuzak:** Doğrudan `main` veya `master` branch'ine commit atmak.
  - **Çözüm:** Daima `feat/kullanici-adi` veya `fix/sorun` şeklinde bir feature branch açarak çalışın.
- **Tuzak:** Commit mesajını geçmiş zaman ('düzeltildi') ile yazmak.
  - **Çözüm:** Emir kipi ('düzelt', 'ekle' / 'fix', 'add') kullanarak yazın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 13: Git Ustalığı - AI Destekli Anlamlı Commitler*
```

- 🖥️ **Slayt 1:** Neden kötü commit mesajları şirketlere milyonlarca dolar kaybettirir?
- 🖥️ **Slayt 2:** Conventional Commits standardının anatomisi.
- 🖥️ **Slayt 3:** Atomik commit felsefesi: Küçük, odaklı, geri alınabilir adımlar.
- 🖥️ **Slayt 4:** AICOMMITS ve IDE eklentileriyle tek tuşla commit mesajı üretmek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir dosyada bilinçli bir değişiklik yapın. Terminalde `git diff` alın. Cursor'ın entegre AI commit butonuna veya `aicommits` CLI aracına basarak diff'i analiz ettirip mükemmel bir Conventional Commit başlığı ürettirin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bugünden itibaren yapacağınız sonraki 5 commit için Conventional Commits formatını uygulayın (Örn: `feat:`, `fix:`, `refactor:`).

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Conventional Commits standardında yeni bir kullanıcı özelliğini sisteme eklerken hangi ön ek kullanılır? - A) fix: - B) feat: - C) chore: - D) docs: > **Doğru Yanıt:** B > **Açıklama:** `feat` (feature) sisteme yeni bir kullanıcı özelliği veya fonksiyon eklendiğini belirtir.

Soru 2: Atomik commit (Atomic Commit) yapmanın birincil teknik faydası nedir? - A) Sabit diski daha hızlı çalıştırması - B) Bir hata durumunda sadece o küçük değişikliği `git revert` ile sistemi bozmadan geri alabilmek - C) Sadece Python projelerinde zorunlu olması - D) GitHub yıldız sayısını otomatik artırması > **Doğru Yanıt:** B > **Açıklama:** Atomik commitler sayesinde bir hata tespit edildiğinde 1000 satırlık devasa bir blok yerine sadece sorunlu 5 satırlık commit güvenle geri alınabilir.

---

14. Merge Conflict Kabusunu AI ile 60 Saniyede Çözmek

<<<<<<< HEAD ve >>>>>>> Çatışmalarını Korkusuzca Yönetmek
Bölüm 3 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Junior mühendisler neden merge conflict çıktığında panikleyip branch'i silme noktasına gelir ve AI her iki tarafın niyetini nasıl birleştirebilir?"

Bölümden Çıkarılacak 3 Altın Cümle:
- 📌 Merge Conflict bir hata değil; iki geliştiricinin aynı kod bloğunu aynı anda değiştirdiğini bildiren bir işbirliği sinyalidir. - 📌 Çözüm 'benimkini al' ya da 'onunkini al' ikileminde değil; her iki tarafın iş mantığını harmanlamaktadır. - 📌 AI modelleri, conflict marker'larını (`<<<<<<<` , `=======` , `>>>>>>>`) okuyup her iki mantığı kusursuzca sentezleyebilir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

İki ressamın aynı tuval üzerinde çalıştığını hayal edin.

Ahmet tuvalin sağ köşesine güzel bir çınar ağacı çiziyor. Aynı anda Zeynep de tuvalin tam o köşesine bir göl ve kuğu çiziyor. Akşam olup resimleri birleştirmeye kalktıklarında boyalar birbirine giriyor!

İşte bu bir **Merge Conflict**'tir. Git boyaları karıştırmak istemediği için fırçayı elinizden alır ve sorar: *"Burada hem ağaç hem kuğu mu olacak, yoksa birinden vaz mı geçeceksiniz?"*

Junior yazılımcılar genellikle panikleyip rastgele tuşlara basar veya başkasının kodunu siler. Oysa yapay zeka harika bir sanat eleştirmeni gibi bakar: *"Gölün kıyısına ağacı yerleştirelim, ikisi de kalsın"* diyerek çatışmayı çözer.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Merge Conflict Anatomisi:

```
<<<<<<< HEAD (Senin Değişikliğin - Current Change)
const taxRate = 0.20; // Türkiye KDV güncellemesi
=======
const taxRate = 0.18;
const hasCustomDiscount = true; // Takım arkadaşının eklediği yeni özellik
>>>>>>> incoming-branch (Gelen Değişiklik)

_____________________________________________________

AI Sentezi (Doğru Çözüm):
const taxRate = 0.20; // Güncel oran korundu
const hasCustomDiscount = true; // Arkadaşının özelliği dahil edildi
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Git Merge ve Çatışma Çözüm Uzmanı
Aşağıdaki dosyada bir Git Merge Conflict meydana geldi.

## ÇATIŞAN KOD BLOĞU:
```typescript
<<<<<<< HEAD
export async function fetchUserOrders(userId: string) {
  // Sıralama özelliği eklendi
  return await db.orders.findMany({ where: { userId }, orderBy: { createdAt:
'desc' } });
}
=======
export async function fetchUserOrders(userId: string, limit: number = 10) {
  // Sayfalama (pagination) limiti eklendi
  return await db.orders.findMany({ where: { userId }, take: limit });
}
>>>>>>> feature/pagination
```

GÖREV:

1. Her iki tarafın ne amaçladığını ayrı ayrı açıkla.
2. Her iki özelliği de (hem orderBy hem take limiti) içeren birleşik ve hatasız nihai fonksiyonu yaz.
3. Conflict marker'larını tamamen temizle.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Kötü / Aceleci Çözüm: Takım arkadaşının limit parametresini silip sadece kendi
kodunu bırakmak
export async function fetchUserOrders(userId: string) {
  // Arkadaşının yaptığı tüm sayfalama mantığı çöpe gitti!
  return await db.orders.findMany({ where: { userId }, orderBy: { createdAt:
'desc' } });
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// AI ile Sentezlenmiş Doğru Çözüm: Her iki özellik korunur
export interface FetchOrdersOptions {
  limit?: number;
  direction?: 'asc' | 'desc';
}

export async function fetchUserOrders(
  userId: string,
  options: FetchOrdersOptions = {}
) {
  const { limit = 10, direction = 'desc' } = options;

  return await db.orders.findMany({
    where: { userId },
    orderBy: { createdAt: direction },
    take: limit
  });
}
```

💡 **Kod Analizi:** Çatışma çözülürken tek bir tarafın kodunu ezmek yerine, fonksiyon bir seçenekler objesi (`FetchOrdersOptions`) alacak şekilde genişletilmiş; böylece hem sıralama hem de sayfalama özellikleri kusursuzca birleştirilmiştir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Panikleyip `git checkout --ours` veya `git checkout --theirs` diyerek bir tarafı tamamen ezmek.
  - **Çözüm:** Her zaman iki tarafın kodunu da dikkatle okuyup birleştirin.
- **Tuzak:** Conflict marker'larını (`<<<<<<<` , `=======`) dosyada unutup commit etmek.
  - **Çözüm:** Commit etmeden önce mutlaka `git diff` ve testleri (`npm test`) çalıştırın.
- **Tuzak:** Haftalarca ana branch'ten (`main`) pull almayıp çatışmaların devasa bir boyuta ulaşmasına izin vermek.
  - **Çözüm:** Her sabah çalışmaya başlarken `git pull origin main` ile yerel branch'inizi güncel tutun.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

🎥 **Ders Başlığı:** ***Ders 14: Korkusuz Git - Merge Conflict'leri AI ile Çözmek***

- 🖥️ **Slayt 1:** Merge Conflict nedir ve neden korkulmamalıdır?
- 🖥️ **Slayt 2:** HEAD vs Incoming: Çatışma etiketlerini okumak.
- 🖥️ **Slayt 3:** Cursor ve VS Code 3-Way Merge Editörünü kullanmak.
- 🖥️ **Slayt 4:** AI ile çatışan iki iş mantığını tek bir fonksiyonda sentezlemek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Lokalde iki farklı branch açın. Aynı dosyanın 10. satırını iki branch'te farklı değiştirip merge çatışması üretin. IDE'nin conflict arayüzünü ve AI 'Resolve Conflict' butonunu kullanarak çatışmayı 30 saniyede çözün ve testleri çalıştırın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bir test reposunda bilerek bir merge conflict yaratın. Conflict etiketlerini AI'a vererek çözdürmeyi deneyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Git merge conflict çıktığında `<<<<<<< HEAD` etiketi neyi temsil eder? - A) Uzaktaki sunucuda silinen kodları - B) Şu an üzerinde bulunduğunuz mevcut branch'teki yerel değişiklikleri - C) GitHub şifrenizi - D) Hatalı yazılmış CSS kodunu > **Doğru Yanıt:** B > **Açıklama:** `HEAD`, şu an aktif olarak bulunduğunuz branch'teki (mevcut) değişiklikleri simgeler.

Soru 2: Büyük ve sancılı merge conflict'lerini önlemenin en etkili takım alışkanlığı nedir? - A) Hiç kod yazmamak - B) Küçük branch'lerle çalışmak ve her gün ana branch'ten (`main`) güncellemeleri çekmek (Pull/Rebase) - C) Sadece ayda bir commit atmak - D) Tüm ekibin aynı bilgisayardan kod yazması > **Doğru Yanıt:** B > **Açıklama:** Sık aralıklarla ana branch ile senkronize olmak ve küçük branch'ler kullanmak çatışmaların büyümesini engeller.

---

15. Kusursuz Pull Request (PR) Yazma Sanatı: AI ile PR Açıklamaları

Senior Mühendislerin İlk Bakışta Onaylayacağı (LGTM) PR'lar Hazırlamak
Bölüm 3 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Senior mühendisler neden boş veya 'değişiklikler yapıldı' yazan PR'ları görmekten nefret eder ve AI ile nasıl kusursuz bir PR şablonu doldurulur?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Bir Pull Request sadece kod değişikliği değil; takım arkadaşlarınıza yaptığınız bir satış sunumudur. - 📌 İyi bir PR: 'Neyi değiştirdim?', 'Neden değiştirdim?' ve 'Nasıl test ettim?' sorularına eksiksiz yanıt verir. - 📌 AI CLI araçları veya GitHub Actions, git diff'ini okuyarak ekran görüntüleri hariç PR açıklamasını saniyeler içinde taslaklaştırabilir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir restorana gittiğinizi ve garsonun masanıza kapağı kapalı, dumanlar tüten bir tabak bırakıp tek kelime etmeden gittiğini düşünün. İçinde ne olduğunu, alerjen içerip içermediğini, et mi sebze mi olduğunu bilmiyorsunuz. O yemeği hemen yer misiniz? Asla!

Açıklaması boş bırakılmış veya *"bug fix"* yazılmış bir Pull Request de o kapalı tabaktır. Kıdemli mühendis o PR'ı incelemek için saatlerce kodun içine dalıp *"Bu çocuk burada ne yapmaya çalışmış?"* diye dedektiflik yapmak zorunda kalır.

Oysa tabağı açıp: *"Şefimizin özel mantarlı risotto yemeği, glütensizdir ve fırında 20 dakika pişti"* diyen bir garson güven verir.

Kusursuz bir PR açıklaması, kıdemli mühendise *"Gözüm kapalı onaylayabilirim (LGTM)"* dedirten en büyük profesyonellik göstergesidir.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Altın Standart PR Şablonu:

```
## 🎯 Amaç & Çözülen Problem
- [Jira-123]: Kullanıcı şifre sıfırlama akışında yaşanan zaman aşımı hatası giderildi.

## 🛠️ Yapılan Değişiklikler
- `authService.ts`: Token geçerlilik süresi 15 dakikadan 30 dakikaya çıkarıldı.
- `mailer.ts`: Şifre sıfırlama e-postasına yeni şablon entegre edildi.

## 🧪 Nasıl Test Edildi?
- [x] Unit testler çalıştırıldı (`npm run test:auth`).
- [x] Yerel ortamda MailHog ile sahte e-posta gönderimi doğrulandı.

## 📸 Ekran Görüntüleri / Kanıt (UI Değişiklikleri İçin)
[Ekran Görüntüsü / GIF]
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Kıdemli Yazılım Ekip Lideri
Aşağıdaki `git diff --stat` ve kod değişikliklerimi incele.

## DEĞİŞİKLİK ÖZETİ:
- `src/controllers/payment.ts`: İki adımlı ödeme doğrulama eklendi.
- `src/services/stripe.ts`: Webhook imza kontrolü eklendi.
- `tests/payment.test.ts`: 4 yeni unit test eklendi.

## GÖREV:
GitHub için profesyonel, markdown formatında eksiksiz bir Pull Request (PR) açıklaması hazırla.
İçinde:
1. Problem tanımı
2. Teknik çözüm özeti
3. Test adımları kontrol listesi (checkbox)
4. Güvenlik notu
bulunsun.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ **Acemi / Kontrolsüz AI Çıktısı (Riskli):**

```
# Kötü PR Açıklaması:
odeme duzeltildi testler geciyor
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
## 🎯 Problem & Çözüm Özeti
Stripe üzerinden gelen mükerrer webhook bildirimleri nedeniyle bazı kullanıcıların
kartından iki kez para çekilme riski bulunuyordu.

Bu PR ile **Idempotency Key (Tekillik Anahtarı)** deseni uygulanmış ve webhook
istekleri için HMAC imza doğrulaması zorunlu kılınmıştır.

## 🛠️ Değişiklik Maddeleri
- `payment.ts`: İşlem öncesinde Redis üzerinden `idempotencyKey` varlığı sorgulanır.
- `stripe.ts`: `stripe.webhooks.constructEvent` kullanılarak sahte bildirimler
engellenir.
- `payment.test.ts`: Aynı payload iki kez gönderildiğinde ikincisinin engellendiği
test edildi.

## 🧪 Test Kontrol Listesi
- [x] Tüm unit testler başarıyla geçti (`npm test`).
- [x] Stripe CLI ile yerel ortamda webhook tetiklendi (`stripe trigger
payment_intent.succeeded`).
- [x] Kötü niyetli (imzasız) isteklerin 400 Bad Request aldığı doğrulandı.

## 🔒 Güvenlik Notu
Yeni bir kütüphane eklenmemiştir; Stripe SDK'nın yerel doğrulama metodu
kullanılmıştır.
```

💡 **Kod Analizi:** Bu derece ayrıntılı bir PR açıklaması gören bir tech lead, kod tabanına güven duyar ve gereksiz soru-cevap turlarına girmeden onay verir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** 1000'den fazla satır içeren devasa ('canavar') PR'lar açmak.
  - **Çözüm:** Bir PR'ın maksimum 200-400 satır arasında kalmasına özen gösterin; büyük işleri küçük parçalara bölün.
- **Tuzak:** Test sonuçlarını veya ekran görüntülerini PR'a eklememek.
  - **Çözüm:** Özellikle frontend değişikliklerinde öncesi/sonrası ekran görüntüsü ekleyin.
- **Tuzak:** PR açtıktan sonra CI/CD boru hattının (GitHub Actions) kırmızı yandığını fark etmemek.
  - **Çözüm:** PR açtıktan sonra en az 3 dakika testlerin yeşile dönmesini ekrandan takip edin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 15: Kusursuz Pull Request - Senior'ların Güvenini Kazanmak*

- 🖥️ **Slayt 1:** PR nedir? Bir kod değişikliğinden öte bir iletişim aracı.
- 🖥️ **Slayt 2:** Altın Standart PR şablonunun bileşenleri.
- 🖥️ **Slayt 3:** AI ile git diff'inden otomatik PR açıklaması üretmek.
- 🖥️ **Slayt 4:** Büyük PR'ları küçük parçalara bölme stratejisi.

Canlı Kodlama / Ekran Kaydı Senaryosu:
GitHub üzerinde yeni bir PR açma ekranını gösterin. Terminaldeki diff'i AI'a kopyalayıp 15 saniyede markdown PR metni ürettirin. PR açıklamasına yapıştırıp önizlemesini gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Açacağınız bir sonraki PR'da 'Altın Standart PR Şablonu'nu kullanın ve en az bir test ekran görüntüsü veya terminal test çıktısı ekleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Bir Pull Request açılırken kıdemli bir mühendisin en çok görmek istediği bilgi hangisidir? - A) Geliştiricinin en sevdiği müzik - B) Değişikliğin neden yapıldığı (problem tanımı) ve nasıl test edildiğinin kanıtı - C) PR'ın kaç dakikada yazıldığı - D) Bilgisayarın işlemci markası > **Doğru Yanıt:** B > **Açıklama:** Kıdemli mühendis kodun ne olduğunu diff'te görür; ancak 'neden' yapıldığını ve güvenilirliğini ancak PR açıklamasından ve test kanıtlarından anlar.

Soru 2: Bir Pull Request'in ideal büyüklüğü (satır sayısı) ne kadar olmalıdır? - A) En az 5.000 satır - B) 10.000 satır - C) 200 - 400 satır arası odaklanmış küçük parçalar - D) Sadece 1 karakter > **Doğru Yanıt:** C > **Açıklama:** Araştırmalar, 400 satırdan büyük PR'larda inceleme kalitesinin dramatik şekilde düştüğünü ve hataların gözden kaçtığını göstermektedir.

---

16. Code Review'de Senior'ları Etkilemek: Kendi Kodunun Şeytanının Avukatı Ol

PR'ı Ekibe Göndermeden Önce AI ile Sanal Code Review Simülasyonu
Bölüm 3 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Kendi kodunuzu PR olarak açmadan önce yapay zekaya 'Beni acımasızca eleştir' dediğinizde yakaladığınız hatalar, kariyerinizi nasıl hızlandırır?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 En iyi mühendisler kodlarını başkası incelemeden önce kendi kendilerine en sert eleştiriyi yapanlardır. - 📌 PR açmadan önce AI'a 'Kıdemli Güvenlik Mimarı' rolü vererek diff'i inceletmek basit hataları eler. - 📌 Code Review yorumlarına kişisel bir saldırı gibi değil; ücretsiz bir mentorluk fırsatı gibi yaklaşın.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Uzaya roket fırlatan bir NASA ekibini düşünün. Mühendisler roketi fırlatma rampasına koyup *"Hadi fırlatalım, bakalım havada ne olacak"* der mi?

Asla! Fırlatmadan önce 'Kırmızı Takım' (Red Team) adı verilen bir grup uzman gelir. Görevleri roketi kırmak, patlatmak ve en küçük zayıflığı bulmaktır. Bu testleri geçen roket güvenle uçar.

Yazılımda da Pull Request açmadan önce kendi 'Kırmızı Takım'ınızı kurabilirsiniz. Yapay zekaya: *"Sen şirketin en huysuz, en titiz kıdemli mimarısın. Bu koddaki tüm açıkları bul"* dediğinizde, gerçek code review'e çıktığınızda senior'lar kodunuzun mükemmelliğine hayran kalır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Pre-Flight (Uçuş Öncesi) Code Review Döngüsü:

```
[Yerel Kod Değişiklikleri]
           |
           ▼
[Adım 1: git diff Çıktısını Al]
           |
           ▼
[Adım 2: AI'a "Kıdemli Mimar Gözüyle İncele" Promptu Gönder]
           |
           ├── "Burada bellek sızıntısı var mı?"
           ├── "SQL Injection veya tip kaçağı var mı?"
           └── "İsimlendirmeler temiz kod standartlarına uyuyor mu?"
           |
           ▼
[Adım 3: Düzeltmeleri Yap & Testleri Çalıştır]
           |
           ▼
[Adım 4: Kusursuz PR'ı Takıma Sun]
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: Titiz ve Huysuz Kıdemli Yazılım Mimarı
Ben ekibinde çalışan bir junior mühendisim. Aşağıda hazırladığım Pull Request diff'i
yer alıyor.

## GÖREV:
Bana kibar davranma; koddaki teknik eksiklikleri acımasızca ortaya çıkar:
1. Hangi sınır durum (edge case) gözden kaçmış?
2. Performans veya bellek tüketimi açısından nerede patlayabilir?
3. Kod okunabilirliği ve SOLID prensipleri açısından hangi 2 kritik eleştirin var?

## DIFF İÇERİĞİ:
```typescript
export async function updateUserEmail(userId: string, newEmail: string) {
  const user = await db.user.findUnique({ where: { id: userId } });
  user.email = newEmail;
  await db.user.update({ where: { id: userId }, data: { email: newEmail } });
  await sendVerificationEmail(newEmail);
  return { success: true };
}
````
````

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Pre-Flight Öncesi: E-posta formatı kontrol edilmemiş, mükerrer kontrolü yok,
transaction yok!
export async function updateUserEmailNaive(userId: string, newEmail: string) {
  // Eğer newEmail başka bir kullanıcıda zaten varsa veritabanı çöker!
  // Eğer e-posta servisi hata verirse DB güncellenmiş kalır (inconsistent state)!
  return await db.user.update({ where: { id: userId }, data: { email: newEmail } });
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```typescript
import { z } from "zod";

const EmailSchema = z.string().email("Geçersiz e-posta formatı").toLowerCase();

export async function updateUserEmailRobust(userId: string, rawEmail: string) {
  // 1. Giriş Doğrulaması
  const validatedEmail = EmailSchema.parse(rawEmail);

  // 2. İşlem bütünlüğü (Database Transaction)
  return await db.$transaction(async (tx) => {
    // Çakışma kontrolü
    const existing = await tx.user.findUnique({ where: { email: validatedEmail } });
    if (existing && existing.id !== userId) {
      throw new ConflictError("Bu e-posta adresi başka bir hesap tarafından
kullanılıyor.");
    }

    const updatedUser = await tx.user.update({
      where: { id: userId },
      data: { email: validatedEmail, emailVerified: false }
    });

    // 3. Yan etki (Outbox deseni veya güvenli e-posta kuyruğu)
    await emailQueue.add("send-verification", { userId, email: validatedEmail });

    return { success: true, user: updatedUser };
  });
}
```

💡 **Kod Analizi:** Sanal review sonrasında: E-posta normalizasyonu (lowercase), mükerrer kayıt kontrolü, `db.$transaction` ile veri tutarlılığı ve doğrudan API çağırmak yerine kuyruk (queue) kullanımı eklenmiştir.
```

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Code review'de gelen eleştirilere savunmaya geçip kırılmak.
  - **Çözüm:** Eleştirilen siz değilsiniz, koddur. Her yorumu öğrenme fırsatı olarak görün.
- **Tuzak:** AI pre-flight review yapmadan doğrudan PR açıp basit hatalarla senior'ın vaktini almak.
  - **Çözüm:** Her PR öncesi en az 1 kez AI'a diff kontrolü yaptırın.
- **Tuzak:** Review yorumunu uygulayıp testleri tekrar çalıştırmadan push etmek.
  - **Çözüm:** Her review revizyonundan sonra tüm test paketini yeniden çalıştırın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 16: Code Review Başarısı - Kendi Kodunun Şeytanının Avukatı Olmak*

- 🖥️ **Slayt 1:** Neden en iyi mühendisler kendi kodlarını acımasızca eleştirir?
- 🖥️ **Slayt 2:** Pre-Flight Check: AI ile Sanal Kıdemli Mimar Simülasyonu.
- 🖥️ **Slayt 3:** Review yorumlarına profesyonel yanıt verme sanatı.
- 🖥️ **Slayt 4:** Veri tutarlılığı: Database Transaction ve Hata Sınırları.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Basit bir veritabanı güncelleme fonksiyonunu AI'a verin. 'Bu koddaki race condition ve transaction açıklarını bul' deyin. AI'ın sunduğu eleştirileri inceleyip kodu refactor edin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bugün yazacağınız bir kodu GitHub'a push etmeden önce AI'a 'Sen bir güvenlik müfettişisin, bu diff'i eleştir' diyerek taratın ve bulduğu en az 1 noktayı düzeltin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Bir veritabanı işleminde birden fazla ilişkili adımın ya hep ya hiç (atomik) olarak çalışmasını garanti eden mekanizma nedir? - A) CSS Flexbox - B) Database Transaction ($transaction) - C) setTimeout fonksiyonu - D) console.warn > **Doğru Yanıt:** B > **Açıklama:** Veritabanı işlemleri (Transaction), içindeki adımlardan biri başarısız olduğunda tüm değişiklikleri geri alarak (rollback) veri tutarsızlığını önler.

Soru 2: Code Review sürecinde gelen teknik bir eleştiriye profesyonel bir junior mühendis nasıl yaklaşmalıdır? - A) Eleştiriyi kişisel bir hakaret sayıp PR'ı silmelidir - B) Eleştirinin arkasındaki mimari nedeni anlamaya çalışıp yapıcı şekilde tartışmalı veya kodu iyileştirmelidir - C) Yorum yapan senior'ı

şikayet etmelidir - D) Yorumları görmezden gelip PR'ı doğrudan merge etmelidir > **Doğru Yanıt:** B > **Açıklama:** Code Review, yazılım mühendisliğinde en hızlı öğrenme ve mentorluk kanalıdır; yapıcı yaklaşım profesyonelliğin temelidir.

17. GitHub Actions ve Otomasyon: AI Destekli CI/CD Boru Hatları

Her Push'ta Otomatik Test, Lint ve AI Kod Kalitesi Denetimi
Bölüm 3 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Şirketler neden 'benim makinemde çalışıyor' sözüne güvenmez ve GitHub Actions gibi CI/CD sistemleri insan denetiminin yerini nasıl alır?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 CI/CD (Continuous Integration / Continuous Delivery), kodun her commit'te izole bir sunucuda otomatik test edilmesidir. - 📌 Eğer testler CI ortamında geçmiyorsa, o kod 'çalışmıyor' kabul edilir; mazeret geçerli değildir. - 📌 GitHub Actions YAML dosyalarını yapay zekaya yazdırmak, karmaşık pipeline kurulumlarını saatlerden dakikalara indirir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir otomobil fabrikasındaki montaj hattını düşünün.

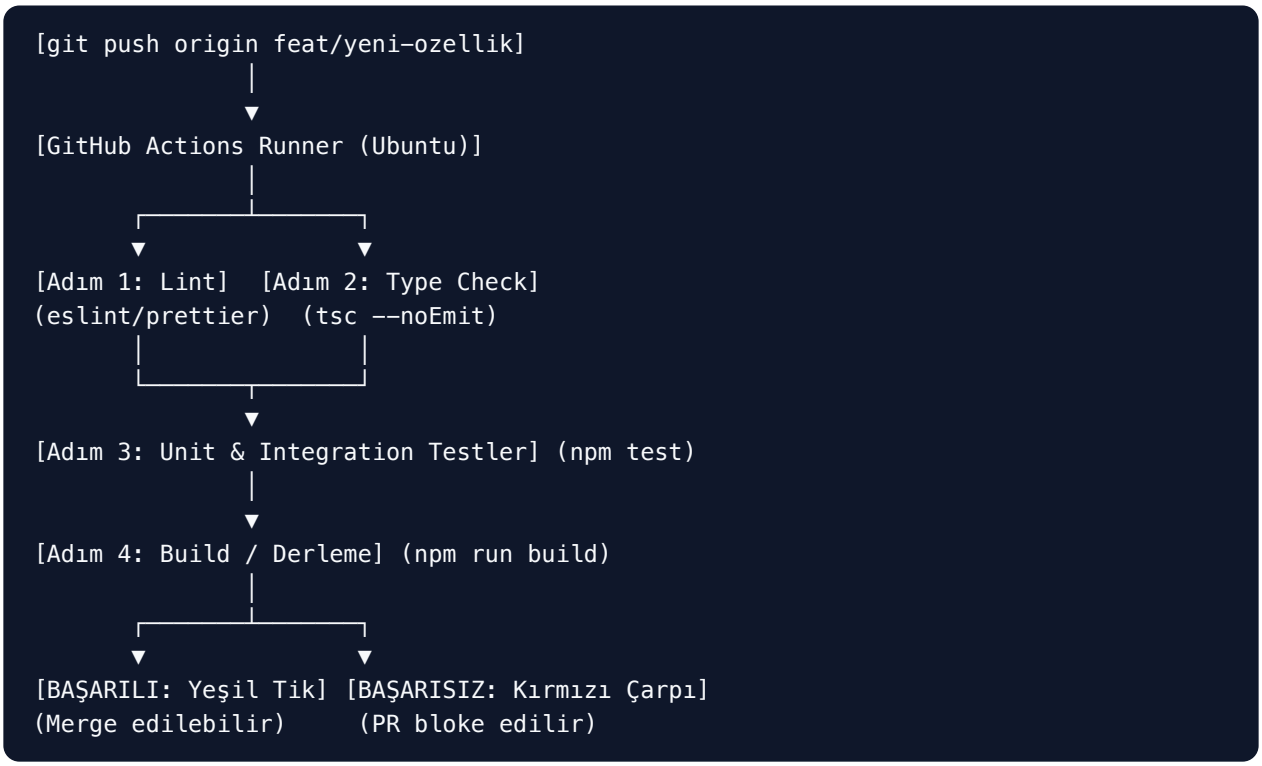
Her araba hattan çıkmadan önce otomatik bir test istasyonuna girer: Robot kollar kapıları 1000 kez çarpar, su fıskiyeleri su sızdırmazlığını kontrol eder, bilgisayarlar fren hidroliğini ölçer. Eğer araç bu testlerin tek birinden bile kalırsa hattan dışarı çıkarılamaz.

GitHub Actions işte o otomatik test robotudur!

Siz kodunuzu GitHub'a gönderdiğiniz anda, bulutta tertemiz sanal bir sunucu (Ubuntu) açılır. Projenizi kurar, linter'ı çalıştırır, tüm testleri koşar ve derler. Eğer her şey yeşil yanarsa kodunuz birleşmeye hak kazanır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

CI/CD Boru Hattı Akışı:



🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: DevOps ve CI/CD Mühendisi
Next.js ve TypeScript kullanan bir projemiz var.

## GÖREV:
GitHub Actions için `.github/workflows/ci.yml` dosyasını hazırla.
Boru hattı şu gereksinimleri karşılamalıdır:
1. `main` ve `develop` branch'lerine açılan PR'larda ve push'larda otomatik
tetiklensin.
2. Node.js 20 sürümünü kullansın ve `npm ci` ile bağımlılıkları önbelleklesin
(cache).
3. Sırasıyla şu adımları çalıştırsın:
   - Lint kontrolü (`npm run lint`)
   - TypeScript derleme kontrolü (`npx tsc --noEmit`)
   - Testler (`npm test`)
4. Herhangi bir adım çökerse pipeline anında dursun.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```yaml
# Kötü CI Dosyası: Önbellekleme yok, sürüm belirtilmemiş, testleri atlıyor
name: CI
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - run: npm install # npm ci yerine install kullanmak lockfile'ı bozar!
      - run: npm run build
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```yaml
name: Continuous Integration (CI)

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main, develop ]

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  quality-gate:
    name: Lint, Typecheck & Tests
    runs-on: ubuntu-latest

    steps:
      - name: 📥 Depoyu Klonla
        uses: actions/checkout@v4

      - name: 🟢 Node.js Kurulumu & Önbellek
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: 📦 Bağımlılıkları Kur (Deterministic)
        run: npm ci

      - name: 🔍 Linter & Format Kontrolü
        run: npm run lint

      - name: 🛡️ TypeScript Tip Kontrolü
        run: npx tsc --noEmit

      - name: 🧪 Unit Testleri Çalıştır
        run: npm test -- --ci --maxWorkers=2

      - name: 🏗️ Production Build Doğrulaması
        run: npm run build
```

💡 **Kod Analizi:** `npm ci` kullanılarak `package-lock.json` ile birebir uyumlu kurulum sağlanmış, `concurrency` bloğu ile aynı branch'e yeni push geldiğinde eski gereksiz CI koşuları otomatik iptal edilerek fatura maliyeti düşürülmüştür.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** CI ortamında `npm install` kullanmak.
  - **Çözüm:** CI sunucularında her zaman deterministik olan `npm ci` komutunu kullanın.
- **Tuzak:** CI kırmızı yanarken 'nasılsa yerelde çalışıyor' deyip PR'ı zorla merge etmek.
  - **Çözüm:** CI kırmızıysa kod bozuktur; hatayı CI loglarından okuyup düzeltin.
- **Tuzak:** Secret (API anahtarları) bilgilerini CI yaml dosyasına düz metin yazmak.
  - **Çözüm:** Hassas bilgileri her zaman GitHub Secrets (`${{ secrets.API_KEY }}`) üzerinden çekin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 17: GitHub Actions ile CI/CD - Kalite Kapılarını Kurmak*

- 🖥️ **Slayt 1:** CI/CD nedir ve şirketler için neden hayati önem taşır?
- 🖥️ **Slayt 2:** GitHub Actions anatomisi: Workflow, Event, Job, Step.
- 🖥️ **Slayt 3:** Deterministik kurulum: npm install vs npm ci.
- 🖥️ **Slayt 4:** Canlı Demo: Sıfırdan GitHub Actions CI pipeline kurmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Reponuzda `.github/workflows/ci.yml` oluşturun. Kasıtlı olarak bir TypeScript tip hatası yapıp push edin. GitHub Actions sekmesinde build'in kırmızıya düştüğünü ve PR'ın kilitlendiğini gösterin. Hatayı düzeltip yeşil tiki alın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Kişisel bir GitHub projenizde `.github/workflows/ci.yml` oluşturun ve testleri her push'ta otomatik çalıştıracak şekilde yapılandırın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: CI sunucularında `npm install` yerine `npm ci` kullanılmasının temel sebebi nedir? - A) `npm ci` komutunun daha renkli çıktılar vermesi - B) `package-lock.json` dosyasını değiştirmeden, kilitli sürümleri birebir ve temiz kurarak determinizm sağlaması - C) Sadece Mac bilgisayarlarda çalışması - D) İnternet bağlantısı gerektirmemesi > **Doğru Yanıt:** B > **Açıklama:** `npm ci` mevcut lockfile'ı asla değiştirmez ve sıfırdan temiz bir kurulum yaparak beklenmedik sürüm uyuşmazlıklarını önler.

Soru 2: GitHub Actions'ta `concurrency: cancel-in-progress: true` ayarının sağladığı fayda nedir? - A) Bilgisayarı kapatması - B) Aynı branch'e yeni bir commit geldiğinde eski ve artık gereksiz olan çalışan CI işini otomatik iptal ederek sunucu süresi tasarrufu sağlaması - C) Kodları otomatik silmesi - D) Testleri yapmadan geçmesi > **Doğru Yanıt:** B > **Açıklama:** Geliştirici arka arkaya commit attığında önceki eski test koşularını iptal etmek hem kaynak tasarrufu sağlar hem de CI kuyruklarını rahatlatır.

18. Dokümantasyon Artık Yük Değil: Koddan Canlı Dokümantasyon Üretmek

Eski ve Güncelliğini Yitirmiş Wiki Sayfalarına Son Verme Rehberi
Bölüm 3 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Geliştiriciler neden dokümantasyon yazmaktan nefret eder ve AI ile dokümantasyonu kodun doğal bir yan ürünü haline nasıl getirebiliriz?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Güncellenmeyen dokümantasyon, hiç yazılmamış dokümantasyondan daha tehlikelidir çünkü yanlış yönlendirir. - 📌 En iyi dokümantasyon kodun hemen yanında yaşayan dokümantasyondur (JSDoc, TypeDoc, OpenAPI/Swagger). - 📌 AI araçları, TypeScript tiplerini ve API endpoint'lerini okuyarak Markdown belgelerini ve API referanslarını anında güncelleyebilir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Tarihi bir kaleye girdiğinizi ve elinizde 1850 yılından kalma bir harita olduğunu düşünün. Haritada *"Burada gizli bir altın odası var"* yazıyor; gidiyorsunuz ama orada derin bir uçurum var! Harita güncellenmediği için hayatınızı kaybedebilirsiniz.

Yazılım şirketlerindeki eski Confluence sayfaları ve unutulmuş Word dosyaları tam olarak o 1850 yılı haritasıdır. Kod değişir, mimari gelişir ama doküman 2 yıl önceki haliyle kalır.

Modern mühendislikte çözüm **Canlı Dokümantasyon (Living Documentation)** yaklaşımıdır: Dokümantasyon kodun içine yazılır (JSDoc ve Şemalar). Yapay zeka bu şemaları okuyup saniyeler içinde güncel web sitelerine ve markdown sayfalarına dönüştürür.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Koddan Dokümantasyona Otomasyon Piramidi:

```
[Tip ve Fonksiyon Açıklamaları (JSDoc / TypeDoc)]
                    |
                    ▼
[Otomatik Şema Çıkarımı (OpenAPI / Swagger / Zod)]
                    |
                    ▼
[AI İle Zenginleştirme: Kullanım Örnekleri & Senaryolar]
                    |
                    ▼
[Canlı Dokümantasyon Sitesi (Mintlify / Docusaurus / README.md)]
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: Teknik Yazar ve API Mimarı
Aşağıdaki TypeScript servis fonksiyonunu incele.

## KOD:
```typescript
export async function calculateShippingCost(
  weightKg: number,
  destinationZone: 'LOCAL' | 'NATIONAL' | 'INTERNATIONAL',
  isExpress: boolean = false
): Promise<number> {
  // Mantık...
}
```

## GÖREV:

Bu fonksiyon için: 1. Standartlara tam uyumlu TSDoc / JSDoc bloğu oluştur (@param, @returns, @throws). 2. Bir junior geliştiricinin kolayca anlayacağı pratik bir kullanım örneği (code example) ekle. 3. API dokümantasyonu için bir Markdown tablosu ve açıklama paragrafı hazırla.
```

````

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Kötü Dokümantasyon: Ne parametrelerin anlamı belli ne de olası hatalar
// Kargo hesaplar
export function calc(w: number, z: string, e: boolean) {
  // ...
}
````

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

````
/**
 * Bir siparişin ağırlığına ve teslimat bölgesine göre kargo ücretini hesaplar.
 *
 * @remarks
 * Uluslararası kargolarda ekspres seçeneği ekstra gümrük işlem ücreti içerir.
 *
 * @param weightKg - Paketin kilogram cinsinden ağırlığı (0.01 - 100.0 arası).
 * @param destinationZone - Teslimat bölgesi ('LOCAL', 'NATIONAL', 'INTERNATIONAL').
 * @param isExpress - Hızlı teslimat seçeneği (Varsayılan: false).
 *
 * @returns KDV dahil toplam kargo tutarı (TRY cinsinden).
 *
 * @throws {RangeError} Ağırlık 0 veya negatifse ya da 100 kg'dan büyükse.
 *
 * @example
 * ```typescript
 * const cost = await calculateShippingCost(2.5, 'NATIONAL', true);
 * console.log(`Ödenecek tutar: ${cost} TL`);
 * ```
 */
export async function calculateShippingCost(
  weightKg: number,
  destinationZone: 'LOCAL' | 'NATIONAL' | 'INTERNATIONAL',
  isExpress: boolean = false
): Promise<number> {
  if (weightKg <= 0 || weightKg > 100) {
    throw new RangeError("Paket ağırlığı 0 ile 100 kg arasında olmalıdır.");
  }
  // Hesaplama implementasyonu...
  return 150.0;
}
````

💡 **Kod Analizi:** TSDoc formatında yazılan bu açıklamalar hem IDE içinde fareyle üzerine gelindiğinde zengin ipucu (IntelliSense) olarak görünür, hem de TypeDoc gibi araçlarla tek komutta HTML dokümantasyonuna dönüştürülebilir.
````

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Dokümantasyonu Confluence veya Word dosyalarında tutup koddan koparmak.
  - **Çözüm:** Dokümantasyonu daima kodun içinde (TSDoc) veya aynı repoda markdown dosyalarında tutun.
- **Tuzak:** Kodun ne yaptığını bariz şekilde tekrarlayan anlamsız yorumlar yazmak (Örn: `i++; // i'yi bir artır`).
  - **Çözüm:** Yorumlarda 'ne' yapıldığını değil, 'neden' yapıldığını ve iş kuralı gerekçelerini açıklayın.
- **Tuzak:** API değişikliklerinde dokümantasyonu güncellemeyi unutmak.
  - **Çözüm:** Zod veya OpenAPI şemalarını kullanarak dokümanı koddan otomatik türetin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 18: Canlı Dokümantasyon - Koddan Otomatik Belge Üretmek*

- 🖥️ **Slayt 1:** Neden geleneksel dokümantasyon ölmeye mahkumdur?
- 🖥️ **Slayt 2:** Canlı Dokümantasyon (Living Docs) felsefesi.
- 🖥️ **Slayt 3:** TSDoc, TypeDoc ve Mintlify ile otomatik dokümantasyon üretimi.
- 🖥️ **Slayt 4:** Bölüm 3 Özeti: Takım İşbirliği, Git ve Otomasyonun Gücü.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir TypeScript fonksiyonuna JSDoc ekleyin. TypeDoc veya AI kullanarak bu fonksiyondan anında profesyonel bir `API_REFERENCE.md` sayfası üretin ve önizlemesini gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki en kritik 2 servis fonksiyonuna eksiksiz TSDoc açıklamaları (@param, @returns, @example) ekleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Yazılım geliştirmede 'Canlı Dokümantasyon' (Living Documentation) kavramı neyi ifade eder? - A) Dokümanın canlı yayında okunmasını - B) Dokümantasyonun doğrudan kodun içindeki tiplerden ve açıklamalardan otomatik olarak türetilerek her zaman güncel kalmasını - C) Sadece video şeklinde doküman hazırlanmasını - D) Dokümanın her gün elle yeniden yazılmasını > **Doğru Yanıt:** B > **Açıklama:** Canlı dokümantasyon, kod değiştikçe otomatik güncellenen; tek gerçeklik kaynağının (Single Source of Truth) kod olduğu yaklaşımdır.

Soru 2: TSDoc veya JSDoc bloklarında `@example` etiketinin temel amacı nedir? - A) Fonksiyonu silmek - B) Diğer geliştiricilere fonksiyonun nasıl çağrılacağına dair çalışan, somut bir kod örneği sunmak - C) Sadece CSS renklerini tanımlamak - D) Bilgisayarı hızlandırmak > **Doğru Yanıt:** B > **Açıklama:** `@example` etiketi, geliştiricilerin API'yi nasıl kullanacağını hemen görmesini sağlayan en etkili dokümantasyon aracıdır.

Bölüm 4: Yazılım Mimarisi, Tasarım Kalıpları & Temiz Kod

SOLID prensipleri, tasarım kalıpları ve katmanlı mimarilerle spagetti kodları önleyin; AI çıktısını yapılandırın.

19. Mimari Nedir ve Junior Neden Bilmelidir? Monolit vs. Mikroservis

Erken Dönem Mikroservis Çılgınlığına Kapılmadan Doğru Mimarisi Seçmek
Bölüm 4 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 13 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Bir şirketin 5 kişilik junior/mid ekibi varken neden Netflix gibi 200 mikroservis yapmaya çalışması ölümcül bir hatadır?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Yazılım mimarisi, projenin ilerleyen aşamalarında değiştirilmesi pahalı olan kararlar bütünüdür.
- 📌 Monolit mimari kötü kod demek değildir; modüler bir monolit başlangıç aşamasındaki ekipler için en hızlı ve güvenli yoldur. - 📌 Yapay zeka araçları tek bir monolit kod tabanında (monorepo) çok daha yüksek bağlam (context) başarısı gösterir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir kahve dükkanı açtığınızı düşünün.

İlk gün dükkanda sadece siz ve bir arkadaşınız var. Şöyle bir plan yapar mısınız: - Kahve çekirdeği öğütme departmanı ayrı bir binada - Süt ısıtma departmanı arka sokakta - Kasa ayrı bir binada - Her departman birbirine telsizle (HTTP/gRPC) sipariş numarası yollasın!

Böyle yaparsanız bir fincan kahveyi 45 dakikada servis edemezsiniz ve batarsınız! İşte erken aşamada mikroservise bölünmek tam olarak budur.

Oysa tek bir tezgahın arkasında (Monolit) yan yana durursanız, birbirinizle konuşarak saniyede 1 kahve çıkarırsınız. Dükkanınız Starbucks gibi 1000 şubeye ulaştığında departmanları ayırmayı düşünürsünüz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Monolit vs Mikroservis Karşılaştırma Matrisi:

Kriter	Modüler Monolit (Modular Monolith)	Dağıtık Mikroservisler (Microservices)
Geliştirme Hızı	Çok yüksek (Tek repo, tek derleme)	Düşük (Servisler arası sözleşmeler, RPC)
Dağıtım (Deployment)	Basit (Tek bir Docker imajı / sunucu)	Karmaşık (Kubernetes, servis ağı, API Gateway)
Hata Ayıklama (Debug)	Kolay (Tek stack trace, yerel breakpoint)	Zor (Distributed tracing, Jaeger, correlation ID)
Ölçeklenme	Dikey ölçeklenme (CPU/RAM artırma)	Yatay bağımsız ölçeklenme (servis bazlı)
AI Uyumluluğu	Mükemmel (Tüm modeller ve tipler aynı bağlamda)	Parçalı (Servis sınırları prompt sınırlarını zorlar)

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Baş Sistem Mimarı (Chief Architect)
Yeni kurulan bir e-ticaret platformunun teknik mimarisini tartışıyoruz.
Ekip büyüklüğümüz: 4 Junior, 1 Kıdemli Mühendis.

## GÖREV:
Mikroservis mi yoksa Modüler Monolit mi tercih etmeliyiz?
Lütfen şu 4 başlık altında net bir karar raporu hazırla:
1. Geliştirici üretkenliği ve yerel ortam kurulum süresi
2. Operasyonel maliyet (AWS / DevOps altyapısı)
3. Dağıtık veri tutarlılığı ve distributed transaction riskleri
4. Nihai mimari tavsiyen ve geçiş stratejisi
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Mikroservis Çılgınlığı: Basit bir sipariş için 3 farklı HTTP servisini çağırmak
// Ağ gecikmesi, servis kesintisi ve tutarsızlık riski tavan!
export async function createOrderMicroservices(userId: string, itemId: string) {
  // 1. Kullanıcı servisi
  const user = await fetch(`http://user-service/users/${userId}`).then(r =>
r.json());
  // 2. Stok servisi (Eğer çökerse ne olacak?)
  const stock = await fetch(`http://inventory-service/items/${itemId}`).then(r =>
r.json());
  // 3. Ödeme servisi (Para çekilip sipariş servisi çökerse para buharlaşır!)
  const payment = await fetch(`http://payment-service/charge`, { method:
'POST' }).then(r => r.json());

  return { success: true };
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
// Modüler Monolit: Aynı işlem belleğinde (in-memory) güvenli transaction
export class OrderService {
  constructor(
    private readonly userRepo: UserRepository,
    private readonly inventoryRepo: InventoryRepository,
    private readonly paymentGateway: PaymentGateway,
    private readonly db: DatabaseClient
  ) {}

  async createOrder(userId: string, itemId: string): Promise<OrderResult> {
    // Tüm adımlar tek bir veritabanı transaction'ında atomik yürütülür
    return await this.db.transaction(async (tx) => {
      const user = await this.userRepo.findById(userId, tx);
      if (!user) throw new NotFoundError("Kullanıcı bulunamadı.");

      const item = await this.inventoryRepo.reserveItem(itemId, tx);
      if (!item.inStock) throw new StockError("Ürün stokta kalmadı.");

      const charge = await this.paymentGateway.authorize(user, item.price);
      const order = await tx.orders.create({ data: { userId, itemId, chargeId:
charge.id } });

      return { success: true, orderId: order.id };
    });
  }
}
```

💡 **Kod Analizi:** Modüler monolit sayesinde 3 farklı ağ çağrısı (network round-trip) ve servis çökme riski yerine, tek bir veritabanı işlemiyle atomik başarı garanti altına alınır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Ekipte tek bir DevOps mühendisi bile yokken Kubernetes ve 15 mikroservis kurmaya çalışmak.
  - **Çözüm:** İlk 1-2 yıl modüler monolit ile ilerleyin; iş modeli kanıtlanınca gerektiğinde servisleri ayırın.
- **Tuzak:** Monolit kod yazarken modülleri birbirine çorba yapmak (Spagetti Monolit).
  - **Çözüm:** Monolit içinde katı modül sınırları (User, Order, Inventory) belirleyin; servisler sadece interface üzerinden konuşsun.
- **Tuzak:** Mikroservisler arasında distributed transaction (2PC / Saga) karmaşasını yönetememek.
  - **Çözüm:** Mümkün olan her yerde tek veritabanı ve ACID transaction gücünü kullanın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 19: Mimari Kararlar - Monolit vs Mikroservis Rehberi*

- 🖥️ **Slayt 1:** Yazılım mimarisi nedir ve junior mühendis neden bilmelidir?
- 🖥️ **Slayt 2:** Mikroservis hipotezi: Ne zaman gereklidir, ne zaman felakettir?
- 🖥️ **Slayt 3:** Modüler Monolit anatomisi: Hız ve düzenin birleşimi.
- 🖥️ **Slayt 4:** AI araçlarının monolit projelerdeki bağlam üstünlüğü.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Tek bir monorepo içinde `modules/user` ve `modules/order` yapısını gösterin. Modüllerin birbirinin iç detaylarına doğrudan erişmek yerine dışa açılan servis interface'i üzerinden nasıl konuştuğunu IDE'de gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Mevcut projenizin mimarisini inceleyin. Kodlar katmanlara ve modüllere ayrılmış mı, yoksa tüm fonksiyonlar tek bir devasa klasörde mi toplanmış? 3 maddelik bir analiz çıkarın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Erken aşamadaki (startup) küçük bir yazılım ekibi için 'Modüler Monolit' mimarinin en büyük avantajı nedir? - A) Asla kod yazılmaması - B) Dağıtık ağ karmaşıklığı ve DevOps maliyeti olmadan hızlı geliştirme, kolay hata ayıklama ve atomik transaction imkanı sunması - C) Sadece C++ dilinde derlenebilmesi - D) İnternet gerektirmemesi > **Doğru Yanıt:** B > **Açıklama:** Modüler monolit, mikroservislerin getirdiği ağ gecikmesi ve operasyonel yük olmadan yüksek geliştirici hızı ve veri bütünlüğü sağlar.

Soru 2: Mikroservis mimarisinde karşılaşılan en karmaşık teknik problem hangisidir? - A) CSS yazmak - B) Servisler arası ağ gecikmeleri, kısmi çöküşler (partial failures) ve dağıtık veri tutarlılığı - C) Bilgisayar monitörünün çözünürlüğü - D) Klavye tuşlarının sertliği > **Doğru Yanıt:** B > **Açıklama:** Dağıtık sistemlerde ağ güvenilmezdir. Bir servisin çökmesi veya gecikmesi zincirleme hatalara (cascading failure) yol açabilir.

20. SOLID Prensipleri AI Promptlarına Nasıl Çevrilir?

Yapay Zekanın Spagetti Kod Üretmesini Engelleyen 5 Altın Mimari Kural
Bölüm 4 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

🎙️ **NotebookLM & Audio Overview Podcast Notları**

Podcast Tartışma Sorusu:
"SOLID prensipleri 20 yıl önce nesne yönelimli programlama için çıktı. Peki 2026'da LLM'lere kod yazdırırken SOLID neden hâlâ en büyük kurtarıcımız?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 AI modelleri serbest bırakıldığında her işi tek bir fonksiyona yığma (God Function) eğilimindedir. - 📌 SOLID prensiplerini prompt kısıtı olarak vermek (Örn: 'Single Responsibility uygula'), AI'ın modüler kod üretmesini sağlar. - 📌 Bağımlılıkların Tersine Çevrilmesi (Dependency Inversion), test edilebilir ve mocklanabilir kodun anahtarıdır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

İsviçre çakısını elinize alın: Üzerinde bıçak, makas, tirbüşon, testere ve tırnak törpüsü var. Küçük bir piknik için harikadır; ancak bir ameliyathaneye girdiğinizde cerrahın elinde İsviçre çakısı görürseniz masadan kaçarsınız!

Cerrahın elinde sadece tek bir işi mükemmel yapan neşter olmalıdır.

Yapay zekaya *"Kullanıcı kaydı yap"* dediğinizde size tam bir İsviçre çakısı üretir: Aynı fonksiyon hem şifreyi doğrular, hem veritabanına yazar, hem e-posta atar, hem PDF fatura üretir! Kod 300 satır olur ve ilk değişiklikte patlar.

SOLID prensiplerini prompt'a eklediğinizde ise yapay zeka neşterler üretir: Her sınıf ve fonksiyon tek bir işi kusursuz yapar.

🧠 2. AI Çağında Yeni Zihniyet Modeli

SOLID Prensiplerinin Prompt Karşılıkları:

```
S - Single Responsibility: "Bu fonksiyon sadece tek bir iş yapmalı. E-posta
gönderimini ayrı bir servise ayır."
O - Open/Closed: "Yeni bir ödeme yöntemi eklendiğinde mevcut koda dokunulmamalı; bir
interface implemente edilmeli."
L - Liskov Substitution: "Alt sınıflar üst sınıfın davranışını bozmamalı,
beklenmedik hata fırlatmamalı."
I - Interface Segregation: "Müşteriyi kullanmadığı metotları içermeye zorlayan
devasa interfaceler yazma; küçük arayüzler kur."
D - Dependency Inversion: "Doğrudan somut sınıfa (Concrete Class) değil, bir
Interface/Abstraksiyona bağımlı ol."
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: SOLID ve Temiz Mimari Uzmanı
Aşağıdaki 'God Object' (her işi yapan) JavaScript fonksiyonunu SOLID prensiplerine
göre refactor et.

## MEVCUT KOD:
```typescript
class OrderProcessor {
  async process(order: any) {
    // 1. Doğrulama
    if (order.amount <= 0) throw new Error("Hatalı tutar");
    // 2. SQL kaydı
    await db.query(`INSERT INTO orders VALUES (${order.id}, ${order.amount})`);
    // 3. E-posta bildirimi
    await mailer.send(order.email, "Sipariş alındı");
  }
}
````

GÖREV:
````

1. Single Responsibility (SRP) uyarınca doğrulama, veritabanı ve bildirim işlerini ayrı servislere böl.
2. Dependency Inversion (DIP) uyarınca `OrderProcessor` sınıfına bağımlılıkları constructor üzerinden interface olarak enjekte et.

````
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// SOLID'e Aykırı: Her işi tek sınıfta yapan God Object
export class OrderProcessorNaive {
  async processOrder(order: any) {
    // SRP ihlali: Validasyon, DB erişimi ve Mail tek yerde
    if (!order.id || order.amount <= 0) throw new Error("Geçersiz");
    // SQL Injection açığı da var!
    await db.query(`INSERT INTO orders VALUES ('${order.id}', ${order.amount})`);
    // DIP ihlali: Dış kütüphaneye doğrudan sıkı bağımlılık (tight coupling)
    await fetch("https://api.sendgrid.com/send", { body: JSON.stringify(order) });
  }
}
````
````

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// SOLID Uyumlu: Modüler, Test Edilebilir, Tip Güvenli Mimari

// 1. Abstraksiyonlar (DIP)
export interface OrderRepository {
  save(order: ValidatedOrder): Promise<void>;
}

export interface NotificationService {
  notifyOrderPlaced(order: ValidatedOrder): Promise<void>;
}

export interface ValidatedOrder {
  readonly id: string;
  readonly amount: number;
  readonly customerEmail: string;
}

// 2. Orkestrasyon Sınıfı (SRP: Sadece iş akışını yönetir)
export class OrderProcessor {
  constructor(
    private readonly repository: OrderRepository,
    private readonly notifier: NotificationService
  ) {}

  async process(order: ValidatedOrder): Promise<void> {
    // Adım 1: Kaydet
    await this.repository.save(order);

    // Adım 2: Bildir
    await this.notifier.notifyOrderPlaced(order);
  }
}
```

💡 **Kod Analizi:** Artık `OrderProcessor` veritabanının PostgreSQL mi MongoDB mi olduğunu veya e-postanın SendGrid ile mi AWS SES ile mi gittiğini bilmez. Test yazarken sahte (mock) repository enjekte etmek 2 satırdır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** SOLID prensiplerini aşırıya kaçırıp 3 satırlık kod için 15 farklı interface ve dosya üretmek (Over-engineering).
  - **Çözüm:** Pragmatik olun; SOLID prensiplerini sistemin büyüyeceği ve değişeceği kritik noktalara uygulayın.
- **Tuzak:** AI'a sadece 'SOLID uygula' deyip hangi prensibe odaklanacağını belirtmemek.
  - **Çözüm:** Özellikle 'SRP ve Dependency Injection uygula' şeklinde spesifik kurallar verin.
- **Tuzak:** Interface yazıp constructor yerine sınıf içinde `new ConcreteClass()` çağırmak.

- **Çözüm:** Bağımlılıkları daima dışarıdan constructor parametresi olarak geçin (Inversion of Control).

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 20: SOLID Prensipleri - AI Kodunu Mimari Seviyeye Taşımak*

- 🖥️ **Slayt 1:** Neden yapay zeka varsayılan olarak 'God Object' üretir?
- 🖥️ **Slayt 2:** 5 SOLID prensibinin sadeleştirilmiş Türkçe açıklaması.
- 🖥️ **Slayt 3:** Dependency Inversion ile test edilebilir kod üretmek.
- 🖥️ **Slayt 4:** Canlı Refactoring: 150 satırlık dev fonksiyonu SOLID ile parçalamak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Tek bir sınıf içinde hem veritabanına yazan hem e-posta atan fonksiyonu AI'a verin. 'Apply Single Responsibility and Dependency Inversion' promptuyla iki ayrı interface ve sınıf oluşturmasını sağlayın. Ardından mock kullanarak Jest testi yazdırın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Kod tabanınızda 100 satırdan uzun olan bir fonksiyon bulun. İçindeki işleri listeleyin ve en az bir işi (örn: e-posta veya loglama) ayrı bir servise çıkartın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: SOLID'in 'Single Responsibility Principle' (Tek Sorumluluk Prensibi) kuralı neyi ifade eder? - A) Şirkette sadece bir yazılımcının çalışmasını - B) Bir sınıfın veya fonksiyonun değişmesi için yalnızca tek bir nedeninin olması gerektiğini - C) Sadece tek bir satır kod yazılmasını - D) Değişkenlerin tek harfle isimlendirilmesini > **Doğru Yanıt:** B > **Açıklama:** SRP, bir modülün sadece tek bir aktöre/iş kuralına karşı sorumlu olması gerektiğini; farklı sorumlulukların birbirine karıştırılmamasını savunur.

Soru 2: Dependency Inversion (Bağımlılıkların Tersine Çevrilmesi) prensibinin en büyük pratik avantajı nedir? - A) Kodun daha yavaş çalışması - B) Sınıfların somut veritabanlarına sıkı sıkıya bağlanmasını engelleyerek birim testlerde (Unit Test) kolayca mocklanabilmesini sağlaması - C) Sadece Java dilinde çalışması - D) CSS hatalarını düzeltmesi > **Doğru Yanıt:** B > **Açıklama:** Interface bağımlılıkları sayesinde test koşarken gerçek veritabanı yerine sahte (mock) objeler verilebilir; testler izole ve yıldırım hızında çalışır.

21. Tasarım Kalıpları (Design Patterns): Factory, Strategy, Observer Pratikte

Tekerleği Yeniden İcat Etmeden Endüstri Standartlarında Kod Yazmak
Bölüm 4 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

🎙️ **NotebookLM & Audio Overview Podcast Notları**

Podcast Tartışma Sorusu:
"Gang of Four tasarım kalıpları 1994 yılında yazıldı. 2026'da AI çağında Factory, Strategy veya Observer kalıpları hâlâ geçerli mi?"

Bölümden Çıkarılacak 3 Altın Cümle:
- 📌 Tasarım kalıpları ezberlenecek formüller değil; yazılımcıların ve yapay zekanın ortak kelime dağarcığıdır (Vocabulary). - 📌 AI'a 'Burada 10 tane if-else yazmak yerine Strategy Pattern kullan' demek, tek kelimeyle mükemmel mimari kurdurur. - 📌 Observer deseni, günümüzün modern olay tabanlı (Event-Driven) sistemlerinin ve bildirim mekanizmalarının temelidir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir restorana gittiğinizi ve garsona sipariş verdiğinizi hayal edin.

Garsona her seferinde: *"Önce bir tabağı dolaptan çıkar, sonra içine 200 gram un koy, fırını 180 dereceye ayarla, fırına sür ve piştikten sonra masama getir"* diyor musunuz? Hayır! Sadece menüden: *"Bir pizza Margherita lütfen"* dersiniz. Mutfak bunu nasıl üreteceğini zaten bilir. İşte bu **Factory (Fabrika)** desenidir!

Peki pizzayı nasıl ödeyeceksiniz? Nakit mi, kredi kartı mı, kripto ile mi? Kasiyer ödeme yöntemine göre pos cihazını uzatır veya nakit alır. İşte bu **Strategy (Strateji)** desenidir!

Pizzanız hazır olduğunda masadaki çağrı cihazı titrer ve size haber verir. İşte bu da **Observer (Gözlemci)** desenidir!

Tasarım kalıpları hayatın içindeki standart çözümlerdir. Kodda da aynısı geçerlidir.

🧠 2. AI Çağında Yeni Zihniyet Modeli

En Çok Kullanılan 3 Tasarım Kalıbı:

```
1. FACTORY PATTERN (Yaratımsal):
   İstemci nesnenin nasıl üretildiğini bilmez; sadece 'Bana X tipinde bir nesne ver'
der.
   Kullanım: Farklı veritabanı sürücüleri, ödeme sağlayıcıları oluşturma.

2. STRATEGY PATTERN (Davranışsal):
   Çalışma zamanında (runtime) algoritmanın veya davranışın değiştirilebilmesini
sağlar.
   Kullanım: Farklı kargo hesaplama algoritmaları, indirim kuponu stratejileri.

3. OBSERVER PATTERN (Davranışsal):
   Bir nesnede durum değiştiğinde, ona abone olan tüm diğer nesneleri otomatik
haberdar eder.
   Kullanım: Event Emitter'lar, Webhook bildirimleri, WebSocket yayınları.
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Yazılım Tasarım Kalıpları Mimarı
Projemizde kullanıcılar 3 farklı yöntemle bildirim alabilmektedir:
- SMS (Twilio)
- E-posta (SendGrid)
- Push Notification (Firebase)

## GÖREV:
Bu yapıyı 'Strategy Pattern' kullanarak TypeScript ile kur.
1. Ortak bir `NotificationStrategy` interface'i tanımla (`send(userId, message)`).
2. Her kanal için ayrı bir somut strateji sınıfı yaz.
3. Bir `NotificationContext` sınıfı yazarak çalışma zamanında stratejinin dinamik
olarak değiştirilebilmesini sağla.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ **Acemi / Kontrolsüz AI Çıktısı (Riskli):**

```
// Kötü: 20 tane if-else ile spagetti kod
export async function sendNotificationSpaghetti(type: string, userId: string, msg:
string) {
  if (type === "SMS") {
    // 30 satır Twilio kodu...
  } else if (type === "EMAIL") {
    // 30 satır SendGrid kodu...
  } else if (type === "PUSH") {
    // 30 satır Firebase kodu...
  } else {
    throw new Error("Bilinmeyen bildirim türü");
  }
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// Strategy Pattern ile Zarif ve Genişletilebilir Mimari

// 1. Strateji Arayüzü (Contract)
export interface NotificationStrategy {
  send(userId: string, message: string): Promise<boolean>;
}

// 2. Somut Stratejiler (Concrete Strategies)
export class EmailNotificationStrategy implements NotificationStrategy {
  async send(userId: string, message: string): Promise<boolean> {
    console.log(`[SendGrid] ${userId} kullanıcısına e-posta gönderildi: ${message}
`);
    return true;
  }
}

export class SmsNotificationStrategy implements NotificationStrategy {
  async send(userId: string, message: string): Promise<boolean> {
    console.log(`[Twilio] ${userId} kullanıcısına SMS atıldı: ${message}`);
    return true;
  }
}

// 3. Bağlam Yöneticisi (Context)
export class NotificationService {
  private strategy: NotificationStrategy;

  constructor(initialStrategy: NotificationStrategy) {
    this.strategy = initialStrategy;
  }

  setStrategy(newStrategy: NotificationStrategy): void {
    this.strategy = newStrategy;
  }

  async dispatch(userId: string, message: string): Promise<boolean> {
    return await this.strategy.send(userId, message);
  }
}
```

💡 **Kod Analizi:** Artık sisteme 'WhatsApp' veya 'Slack' bildirimi eklemek istediğimizde mevcut koda tek bir satır dahi dokunmayız (Open/Closed Principle). Sadece yeni bir `WhatsAppStrategy` sınıfı ekleriz.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Basit 2 durumlu bir kontrol için devasa Factory veya Strategy sınıfları kurmak.

  - **Çözüm:** Eğer durum sayısı 2'yi geçmeyecekse basit bir `if/else` yeterlidir; kalıpları karmaşıklık arttığında devreye sokun.
- **Tuzak:** Observer deseninde abonelikten çıkmayı (unsubscribe) unutup bellek sızıntısına (memory leak) yol açmak.
  - **Çözüm:** Bileşenler yok edildiğinde (unmount) event listener'ları mutlaka temizleyin.
- **Tuzak:** Kalıpların isimlerini projedeki isimlendirmelere yansıtmamak.
  - **Çözüm:** Sınıf isimlerine `PaymentStrategy`, `LoggerFactory` gibi açık ekler verin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 21: Tasarım Kalıpları - Factory, Strategy ve Observer*

- 🖥️ **Slayt 1:** Tasarım kalıpları neden yapay zeka ile geliştiricinin ortak dilidir?
- 🖥️ **Slayt 2:** If-Else canavarından Strategy Pattern ile kurtulmak.
- 🖥️ **Slayt 3:** Factory Pattern ile nesne yaratımını soyutlamak.
- 🖥️ **Slayt 4:** Observer Pattern: Event-Driven mimarinin temeli.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir e-ticaret kargo hesaplayıcısı yazın. 'Aras', 'Yurtiçi' ve 'DHL' için 3 farklı strateji sınıfı kurun. Kullanıcı kargo firmasını seçtiğinde runtime'da stratejinin nasıl değiştiğini ve doğru ücreti hesapladığını konsolda gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde en az 3 farklı `else if` içeren bir kod bloğu bulun ve bunu Strategy Pattern ile refactor edin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Strategy Pattern'in temel amacı nedir? - A) Veritabanındaki tüm tabloları silmek - B) Birbirinin yerine geçebilen algoritmalar ailesi tanımlayarak, algoritmayı onu kullanan istemciden bağımsız şekilde çalışma zamanında değiştirebilmek - C) Sadece HTML dosyalarını sıkıştırmak - D) Bilgisayarın pil ömrünü uzatmak > **Doğru Yanıt:** B > **Açıklama:** Strategy Pattern, çalışma zamanında farklı algoritmaların (örn: ödeme veya sıralama yöntemleri) dinamik olarak birbirinin yerine kullanılmasını sağlar.

Soru 2: Observer Pattern'de bildirimleri dinleyen taraflara ne ad verilir? - A) Subscriber (Abone / Gözlemci) - B) Compiler - C) Terminal - D) Router > **Doğru Yanıt:** A > **Açıklama:** Observer deseninde durum yayınlayan nesne Subject (Yayımcı), bu durumu dinleyen nesneler ise Observer / Subscriber (Abone) olarak adlandırılır.

22. Katmanlı Mimari & Separation of Concerns: Sorumlulukları Ayırmak

Controller, Service, Repository Katmanlarını Birbirine Karıştırmama Kılavuzu
Bölüm 4 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

🎙️ **NotebookLM & Audio Overview Podcast Notları**

Podcast Tartışma Sorusu:
"Bir API Controller'ının içine SQL sorgusu yazmak neden yazılım dünyasının en büyük günahlarından biridir ve katmanlı mimari sistemi nasıl korur?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Separation of Concerns (İlgi Alanlarının Ayrımı), sistemin her katmanının sadece kendi işini bilmesidir. - 📌 Controller sadece HTTP protokolünü (istek/yanıt) yönetir; Service saf iş mantığını yürütür; Repository ise veriye erişir. - 📌 Yapay zekaya bu katmanları açıkça belirttiğinizde, AI çok daha temiz, modüler ve kurumsal kod üretir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Lüks bir oteli hayal edin.

Otelde 3 ana görevli var: 1. **Resepsiyonist (Controller):** Kapıda müşteriyi karşılar, kimliğini kontrol eder ve isteğini alır. Asla mutfağa girip yemek pişirmez! 2. **Şef Aşçı (Service):** Siparişin malzemelerini hazırlar, tarife göre pişirir. Müşterinin kredi kartıyla ilgilenmez! 3. **Kiler Sorumlusu (Repository):** Domatesin, etin hangi rafta olduğunu bilir; şef isteyince raftan çıkarıp verir.

Eğer resepsiyonist masayı bırakıp mutfakta omlet yapmaya, kilerdeki domatesleri saymaya kalkarsa otelde kaos çıkar!

İşte Controller içine veritabanı sorgusu yazmak, o resepsiyonistin mutfakta tencere yıkamasıdır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

3 Katmanlı Mimari Akışı:

```
[İstemci (React/Mobil)] ──(HTTP İsteği)──> [1. CONTROLLER KATMANI]
                                                    |
                                             (İş Talebi)
                                                    ▼
                                            [2. SERVICE KATMANI]
                                             (İş Mantığı & Kurallar)
                                                    |
                                             (Veri Talebi)
                                                    ▼
                                            [3. REPOSITORY KATMANI]
                                                    |
                                            (SQL / ORM Sorgusu)
                                                    ▼
                                            [VERİTABANI (Postgres/Mongo)]
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Temiz Mimari (Clean Architecture) Uzmanı
Node.js / Express ve Prisma kullanan bir kullanıcı oluşturma özelliğimiz var.

## GÖREV:
Bu özelliği 3 katmanlı mimariye uygun şekilde 3 ayrı dosyada tasarla:
1. `user.controller.ts`: Sadece HTTP req/res, durum kodları ve Zod doğrulaması ile ilgilensin.
2. `user.service.ts`: Şifre hashleme ve iş kuralları ile ilgilensin (asla HTTP nesnesi almasın!).
3. `user.repository.ts`: Sadece Prisma sorgusu ile veritabanına kayıt yapsın.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Felaket: Controller içinde hem validasyon, hem hashleme, hem SQL!
app.post("/users", async (req, res) => {
  if (!req.body.email) return res.status(400).send("Email gerekli");
  const hashed = await bcrypt.hash(req.body.password, 10);
  const user = await prisma.user.create({ data: { email: req.body.email, password:
hashed } });
  await sendEmail(user.email);
  res.json(user); // Şifre hash'i bile frontend'e sızdı!
});
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
// 1. REPOSITORY KATMANI (user.repository.ts)
export class UserRepository {
  async create(data: { email: string; passwordHash: string }) {
    return await prisma.user.create({ data });
  }
  async findByEmail(email: string) {
    return await prisma.user.findUnique({ where: { email } });
  }
}

// 2. SERVICE KATMANI (user.service.ts - Saf İş Mantığı)
export class UserService {
  constructor(private readonly userRepo: UserRepository) {}

  async registerUser(dto: RegisterUserDto): Promise<SafeUser> {
    const existing = await this.userRepo.findByEmail(dto.email);
    if (existing) throw new ConflictError("E-posta zaten kayıtlı.");

    const passwordHash = await hashPassword(dto.password);
    const user = await this.userRepo.create({ email: dto.email, passwordHash });

    return { id: user.id, email: user.email }; // Şifre asla dönmez!
  }
}

// 3. CONTROLLER KATMANI (user.controller.ts - Sadece HTTP Protokolü)
export class UserController {
  constructor(private readonly userService: UserService) {}

  async handleRegister(req: Request, res: Response): Promise<void> {
    const validated = RegisterUserSchema.parse(req.body);
    const user = await this.userService.registerUser(validated);
    res.status(201).json({ success: true, data: user });
  }
}
```

💡 **Kod Analizi:** Katmanlar ayrıldığında; veritabanını Prisma'dan TypeORM'a geçirmek sadece Repository'yi etkiler. Web framework'ünü Express'ten Fastify'a geçirmek ise sadece Controller'ı etkiler. Service katmanındaki iş mantığına asla dokunulmaz!

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Service fonksiyonlarına Express `req` ve `res` nesnelerini parametre olarak geçmek.
  - **Çözüm:** Service katmanı HTTP'den habersiz olmalıdır; sadece saf JavaScript objeleri (DTO) almalıdır.
- **Tuzak:** Controller'ı atlayıp doğrudan veritabanı modelini frontend'e döndürerek şifre gibi gizli alanları sızdırmak.
  - **Çözüm:** Daima Response DTO kullanarak sadece güvenli alanları dönün.
- **Tuzak:** Basit CRUD işlemlerinde gereksiz 10 katman oluşturup aşırı karmaşıklık yaratmak.
  - **Çözüm:** İş mantığı olmayan yerlerde katmanları gereksiz derinleştirmeyin; pragmatik olun.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

🎥 Ders Başlığı: *Ders 22: Katmanlı Mimari - Controller, Service ve Repository*

- 🖥️ **Slayt 1:** Separation of Concerns (İlgi Alanlarının Ayrımı) nedir?
- 🖥️ **Slayt 2:** 3 Katmanın Görev Dağılımı: Resepsiyonist, Şef ve Kiler.
- 🖥️ **Slayt 3:** Service katmanını HTTP bağımsız (framework-agnostic) tutmanın gücü.
- 🖥️ **Slayt 4:** Canlı Demo: Spagetti Express rotasını 3 katmana refactor etmek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Tek dosyalık bir `app.post('/register')` kodunu açın. Adım adım `repository`, `service` ve `controller` dosyalarına bölün. Service katmanı için sahte bir repository ile 1 dakikada unit test yazarak çalıştığını gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Mevcut projenizde Controller içinde doğrudan veritabanı sorgusu yapılan bir fonksiyonu bulun ve veritabanı kodunu bir Repository fonksiyonuna taşıyın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: 3 Katmanlı Mimaride 'Service' katmanının temel sorumluluğu nedir? - A) HTTP durum kodlarını (200, 404) belirlemek - B) Şirketin saf iş kurallarını (business logic) ve hesaplamalarını yürütmek - C) Doğrudan SQL sorgusu yazmak - D) CSS butonlarını renklendirmek > **Doğru Yanıt:** B > **Açıklama:** Service katmanı iş kurallarının kalbidir; ne HTTP protokolünden ne de veritabanının türünden etkilenmeden mantığı işletir.

Soru 2: Service fonksiyonuna `(req, res)` nesnelerini parametre olarak geçmek neden kötü bir mimari alışkanlıktır? - A) Fonksiyonu yavaşlatması - B) Service katmanını Express'e ve HTTP protokolüne sıkı sıkıya bağlayarak CLI, kuyruk (queue) veya test ortamlarında yeniden kullanılmasını imkansız kılması - C) TypeScript'in hata vermesi - D) Monitörün kapanması > **Doğru Yanıt:** B > **Açıklama:** Service saf verilerle (DTO) çalışmalıdır; böylece aynı servis hem bir HTTP isteğinden hem de bir cron job veya kuyruk mesajından çağrılabilir.

23. State Yönetimi & Veri Akışı: Kaosu Önlemek

Prop Drilling, Global State ve Asenkron Durum Yönetimi Rehberi
Bölüm 4 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Modern web uygulamalarında en çok bug nerede çıkar? Neden veritabanı veya sunucu değil, istemcideki 'State (Durum)' yönetimi başımızı ağrıtır?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 State (Durum), uygulamanın belirli bir andaki hafızasıdır; kontrolsüz mutasyonlar kaosa yol açar. - 📌 Sunucu Durumu (Server State - React Query / SWR) ile İstemci Durumu (Client State - Zustand) birbirine karıştırılmamalıdır. - 📌 Tek Yönlü Veri Akışı (Unidirectional Data Flow), durum değişikliklerini öngörülebilir ve izlenebilir kılar.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Büyük bir ailenin ortak bir buzdolabı olduğunu düşünün.

Eğer evdeki 5 kişi de buzdolabındaki sütü kafasına göre kafaya dikip içerse, kimse sütün bittiğinden haberdar olmaz. Anne kahve yapacakken süt kutusunun boş olduğunu görünce çığlık atar! İşte bu **kontrolsüz mutable state**'tir.

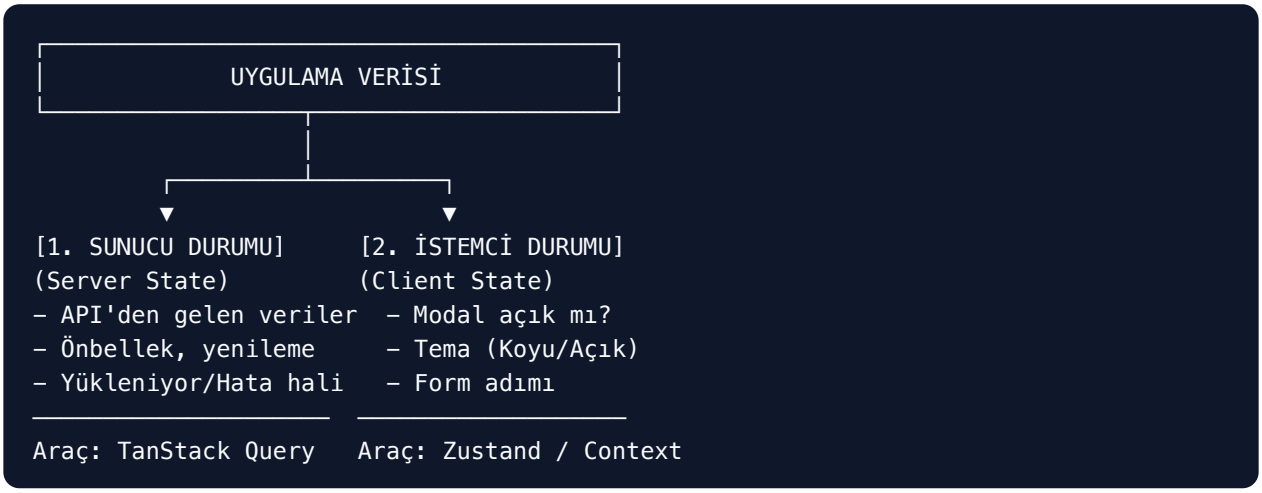
Peki buzdolabının kapağına bir liste koysanız: - Sütü alan kişi listeden bir çizgi çeker. - Son süt bittiğinde otomatik olarak sipariş listesine eklenir.

Artık herkes ne olduğunu bilir ve sürpriz yaşanmaz!

Yazılımda Zustand, Redux veya React Query işte o buzdolabı listesidir. Verinin kim tarafından, ne zaman ve nasıl değiştirildiğini kayıt altına alır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Modern State Ayrımı Matrisi:



🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Frontend Mimarisi ve React Uzmanı
Uygulamamızda hem sunucudan gelen ürün listesi var hem de kullanıcının sepet durumu var.

## GÖREV:
1. Neden ürün listesini `useEffect` ve `useState` ile yönetmenin kötü bir alışkanlık olduğunu açıkla.
2. Ürün listesini TanStack Query (React Query) kullanarak önbellekli ve otomatik yenilenen şekilde kodla.
3. Sepet durumunu (ekle/çıkar/toplam) hafif bir Zustand store'u olarak tasarla.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Kötü: useEffect ile veri çekmek (Race condition, önbellek yok, boilerplate dolu)
function ProductListNaive() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Bileşen unmount olursa memory leak ve race condition riski!
    fetch("/api/products")
      .then(res => res.json())
      .then(data => {
        setProducts(data);
        setLoading(false);
      });
  }, []); // Bağımlılıklar güncellenmediğinde bayat veri!
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// 1. İSTEMCİ DURUMU: Zustand ile Sepet Yönetimi (cartStore.ts)
import { create } from "zustand";

interface CartItem {
  id: string;
  name: string;
  price: number;
}

interface CartStore {
  items: CartItem[];
  addItem: (item: CartItem) => void;
  clearCart: () => void;
  total: () => number;
}

export const useCartStore = create<CartStore>((set, get) => ({
  items: [],
  addItem: (item) => set((state) => ({ items: [...state.items, item] })),
  clearCart: () => set({ items: [] }),
  total: () => get().items.reduce((sum, item) => sum + item.price, 0)
}));

// 2. SUNUCU DURUMU: TanStack Query ile Ürün Çekme (products.hook.ts)
import { useQuery } from "@tanstack/react-query";

export function useProducts() {
  return useQuery({
    queryKey: ["products"],
    queryFn: async () => {
      const res = await fetch("/api/products");
      if (!res.ok) throw new Error("Ürünler yüklenemedi.");
      return (await res.json()) as CartItem[];
    },
    staleTime: 1000 * 60 * 5 // 5 dakika boyunca önbellekten anında sun
  });
}
```

💡 **Kod Analizi:** TanStack Query; otomatik önbellekleme (caching), arka planda yenileme (revalidation) ve yüklenme durumlarını tek satırda çözerken, Zustand yerel sepet hafızasını prop drilling olmadan yönetir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Sunucudan gelen her API verisini Redux veya global state'e kopyalayıp senkronizasyon kaybetmek.

  - **Çözüm:** Sunucu verisi için TanStack Query kullanın; Redux/Zustand'ı sadece gerçek UI durumları için saklayın.
- **Tuzak:** React state'ini doğrudan mutasyona uğratmak (Örn: `state.items.push(item)`).
  - **Çözüm:** State daima immutable (değişmez) güncellenmelidir (`[...state.items, item]`).
- **Tuzak:** Prop drilling yaparak bir veriyi 8 alt bileşene elle aktarmak.
  - **Çözüm:** Derin hiyerarşilerde Zustand veya React Context kullanın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 23: State Yönetimi - TanStack Query ve Zustand ile Düzen*

- 🖥️ **Slayt 1:** Frontend bug'larının %70'i neden State yönetiminden kaynaklanır?
- 🖥️ **Slayt 2:** Server State (Sunucu) vs Client State (İstemci) ayrımı.
- 🖥️ **Slayt 3:** useEffect veri çekme tuzağından TanStack Query'ye geçiş.
- 🖥️ **Slayt 4:** Zustand ile 10 satırda global sepet yönetimi.

Canlı Kodlama / Ekran Kaydı Senaryosu:
React uygulamasında iki bileşen açın. Biri sepete ürün eklesin, diğeri header'da sepet adedini göstersin. Zustand store'u ile prop aktarmadan anında güncellendiğini gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde `useEffect` ile veri çektiğiniz bir yeri bulun ve TanStack Query `useQuery` kullanarak yeniden yazın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: React uygulamalarında sunucudan (API) gelen veriyi yönetmek için modern standart hangi araçtır? - A) Sadece `useState` ve `useEffect` - B) TanStack Query (React Query) veya SWR gibi Sunucu Durumu (Server State) kütüphaneleri - C) jQuery - D) CSS Animations > **Doğru Yanıt:** B > **Açıklama:** TanStack Query önbellekleme, deduplication, retry ve otomatik senkronizasyon özelliklerini otomatik sunar.

Soru 2: React state'inde `state.users.push(newUser)` yazmak neden tehlikelidir? - A) Kodun derlenmemesi - B) Dizinin referansı değişmediği için React'in değişikliği fark edememesi ve arayüzü yeniden render etmemesi (Bailout) - C) Bilgisayarın kilitlenmesi - D) Sadece Python'da hata vermesi > **Doğru Yanıt:** B > **Açıklama:** React sığ karşılaştırma (shallow equality) yapar. Dizi doğrudan mutasyona uğradığında referans değişmediği için render tetiklenmez.

24. API Mimarisi: REST, GraphQL ve gRPC Karşılaştırması

Hangi Protokolü Ne Zaman Seçmelisin? OpenAPI Şemalarıyla Tip Güvenliği
Bölüm 4 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"REST öldü mü? GraphQL her derde deva mı yoksa gRPC mikroservislerin gizli süper gücü mü?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 REST basitliği ve HTTP önbelleklemesiyle kamusal API'lerin (Public API) kralıdır. - 📌 GraphQL, mobil uygulamalarda aşırı veri çekme (Over-fetching) ve yetersiz veri çekme (Under-fetching) sorununu çözer. - 📌 gRPC (Protocol Buffers), mikroservisler arasında ikili (binary) formatta HTTP/2 ile ışık hızında haberleşme sağlar.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Farklı ülkelere mektup gönderdiğinizi düşünün.

1. **REST:** Standart bir posta mektubudur. Zarfın üstünde adres yazar (URL), içine standart kağıt konur (JSON). Dünyadaki her postacı bunu anlar.
2. **GraphQL:** Kişiye özel alışveriş sipariş formudur. Forma tam olarak *"Bana sadece kırmızı kazağın bedenini ve fiyatını getir, kumaş bilgisini getirme"* yazarsınız; kurye tam istediklerinizi getirir.
3. **gRPC:** İki askeri üs arasındaki kriptolu fiber optik kablodur. İngilizce veya Türkçe konuşmaz; ikili (binary 0 ve 1) şifreli sinyallerle mikrosaniyede haberleşir.

Bir e-ticaret sitesi yapıyorsanız REST veya GraphQL seçersiniz. Ancak arka planda binlerce finansal işlemi konuşturan sunucularınız varsa gRPC seçersiniz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Protokol Karar Ağacı:

```
[Yeni Bir API Geliştirilecek]
             |
             ├── Kamuya açık genel API veya standart CRUD mu?
             |   └──> REST API (OpenAPI / Swagger ile)
             |
             ├── Karmaşık mobil ekranlar ve ilişkili veriler mi var?
             |   └──> GraphQL (Apollo / Yoga)
             |
             └── İç mikroservisler arası yüksek hızlı veri transferi mi?
                 └──> gRPC (Protocol Buffers)
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: API Mimarisi ve Entegrasyon Uzmanı
Bir ödeme platformu için 'Müşteri İşlemleri API'si tasarlıyoruz.

## GÖREV:
1. REST, GraphQL ve gRPC arasındaki farkları; gecikme (latency), ağ yükü (payload size) ve önbellekleme (caching) açısından karşılaştır.
2. Bu sistem için OpenAPI 3.0 (Swagger) uyumlu bir `POST /v1/payments` endpoint şeması (YAML) hazırla.
3. Bu şemadan TypeScript client tiplerinin nasıl otomatik türetileceğini (Contract-First) göster.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```yaml
# Kötü / Dokümante Edilmemiş REST: Tip yok, hata şeması belirsiz
/api/pay:
  post:
    description: Para ode
    responses:
      200:
        description: Basarili
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```yaml
openapi: 3.0.3
info:
  title: Core Payments API
  version: 1.0.0
paths:
  /v1/payments:
    post:
      summary: Yeni bir ödeme işlemi başlatır
      operationId: processPayment
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/PaymentRequest'
      responses:
        '201':
          description: Ödeme başarıyla onaylandı
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/PaymentResponse'
        '400':
          description: Geçersiz parametreler veya yetersiz bakiye
components:
  schemas:
    PaymentRequest:
      type: object
      required: [amount, currency, sourceToken]
      properties:
        amount:
          type: number
          format: float
          minimum: 0.01
        currency:
          type: string
          enum: [USD, EUR, TRY]
        sourceToken:
          type: string
          example: "tok_visa_1234"
    PaymentResponse:
      type: object
      properties:
        paymentId:
          type: string
          format: uuid
        status:
          type: string
          enum: [SUCCEEDED, PENDING, FAILED]
```

💡 **Kod Analizi:** Contract-First yaklaşımında bu YAML dosyası yazıldıktan sonra `openapi-typescript` aracıyla hem frontend hem backend için tek tuşla sıfır hatayla TypeScript tipleri ve SDK üretilir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Basit bir blog sitesi için GraphQL kurup gereksiz N+1 sorgu karmaşasıyla boğuşmak.
  - **Çözüm:** İhtiyaç yoksa REST ile başlayın; karmaşık veri grafikleri oluştuğunda GraphQL'e geçin.
- **Tuzak:** REST API'de HTTP durum kodlarını (200, 400, 404, 500) görmezden gelip her hatada 200 OK içinde `{ error: true }` dönmek.
  - **Çözüm:** HTTP standartlarına uyun; hataları doğru durum kodlarıyla iletin.
- **Tuzak:** API sözleşmesini (OpenAPI) elle yazıp koddan koparmak.
  - **Çözüm:** Şemayı ya koddan otomatik türetin ya da şemayı tek gerçeklik kaynağı kabul edip kodu şemadan üretin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 24: Modern API Mimarisi - REST, GraphQL ve gRPC*

- 🖥️ **Slayt 1:** API Protokolleri Karşılaştırması: REST vs GraphQL vs gRPC.
- 🖥️ **Slayt 2:** Over-fetching ve Under-fetching problemlerini anlamak.
- 🖥️ **Slayt 3:** Contract-First Yaklaşımı: OpenAPI ile Tip Güvenliği.
- 🖥️ **Slayt 4:** Bölüm 4 Özeti: Yazılım Mimarisi ve Tasarım Kalıpları.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir OpenAPI YAML dosyasından `openapi-typescript` CLI aracıyla 5 saniyede TypeScript tipleri üretin. Bu tipleri hem Express rotasında hem de fetch çağrısında kullanarak tam tip güvenliğini gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki en önemli 1 REST endpoint'i için Swagger / OpenAPI 3.0 uyumlu şema tanımı yazın veya Zod kullanarak şema türetin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: GraphQL'in geleneksel REST API'ye göre en belirgin avantajı nedir? - A) Asla hata vermemesi - B) İstemcinin (Frontend/Mobil) tam olarak ihtiyaç duyduğu alanları talep edebilmesi (Over-fetching ve Under-fetching'i önlemesi) - C) Sadece Python'da çalışması - D) CSS yüklemesini hızlandırması > **Doğru Yanıt:** B > **Açıklama:** GraphQL istemcinin tek bir sorguyla sadece istediği alanları ve iç içe ilişkili nesneleri tek istekte alabilmesini sağlar.

Soru 2: İç mikroservisler arasında yüksek hız ve düşük gecikme gerektiğinde gRPC'nin tercih edilmesinin temel teknik sebebi nedir? - A) Metin tabanlı JSON yerine Protocol Buffers (ikili veri) kullanması ve HTTP/2 üzerinden çalışması - B) Şifre gerektirmemesi - C) Sadece Google tarafından kullanılması - D) Veritabanı istememesi > **Doğru Yanıt:** A > **Açıklama:** gRPC, binary serialization (Protocol Buffers) ve HTTP/2 multiplexing sayesinde JSON'a kıyasla çok daha küçük paket boyutu ve yüksek hız sunar.

Bölüm 5: Veritabanları, Veri Modelleme & Backend Temelleri

İlişkisel ve NoSQL modelleme, indeksleme, N+1 sorgu optimizasyonu ve dayanıklı API tasarım ilkelerini öğrenin.

25. İlişkisel (SQL) vs. İlişkisel Olmayan (NoSQL): Hangisi Ne Zaman?

PostgreSQL mi MongoDB mi? Ezbere Seçimlerden Kurtulma Kılavuzu
Bölüm 5 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
“Yeni başlayan geliştiriciler neden her projeye hemen MongoDB açmaya çalışır ve finansal işlemlerde NoSQL kullanmak nasıl bir felakettir?”

Bölümden Çıkarılacak 3 Altın Cümle:
- 📌 SQL ilişkisel bütünlüğü (ACID) ve katı şemaları korurken; NoSQL esnek şemalar ve yatay ölçeklenme için tasarlanmıştır. - 📌 Eğer verileriniz arasında ilişkiler (Kullanıcı -> Sipariş -> Fatura) varsa, varsayılan tercihiniz her zaman PostgreSQL olmalıdır. - 📌 Modern PostgreSQL, JSONB desteği sayesinde NoSQL’in esnekliğini SQL’in güvenilirliğiyle birleştirir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir banka kasasını ve bir oyuncak sepetini düşünün.

Banka kasasında (SQL) her kuruşun yeri bellidir. Kim kime ne kadar para gönderdiyse, Ahmet’in hesabından düşüldüğü saniyede Mehmet’in hesabına eklenmelidir. İkisi aynı anda gerçekleşmezse işlem iptal edilir (ACID).

Oyuncak sepetinde (NoSQL) ise kural yoktur! Pelüş ayı, lego parçası, uzaktan kumandalı araba hepsini aynı sepete atabilirsiniz. Hızlıca eşya atmak veya sepeti kucaklayıp başka bir odaya taşımak (yatay ölçeklenme) çok kolaydır.

Eğer banka parasını oyuncak sepetine koyarsanız, paralar kaybolur! Ancak IoT sensörlerinden saniyede 10.000 log topluyorsanız sepet tam aradığınız şeydir.

🧠 2. AI Çağında Yeni Zihniyet Modeli

SQL vs NoSQL Karşılaştırma Rehberi:

```
[İhtiyaç Analizi]
       |
       ├── Veri ilişkisel mi? (Tablolar arası Foreign Key var mı?)
       |   └──> SQL (PostgreSQL, MySQL)
       |
       ├── Finansal işlem veya katı tutarlılık (ACID) şart mı?
       |   └──> SQL (PostgreSQL)
       |
       ├── Veri şeması sürekli değişen dökümanlar mı? (CMS, Dinamik Formlar)
       |   └──> NoSQL (MongoDB, CouchDB) VEYA Postgres JSONB
       |
       └── Milyonlarca geçici oturum veya cache mi?
           └──> Key-Value NoSQL (Redis)
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Baş Veritabanı Mimarı
Yeni bir SaaS platformu kuruyoruz. Sistemde kullanıcılar, faturalar, abonelik
planları ve kullanıcıların yüklediği dinamik anket formları olacak.

## GÖREV:
1. Bu sistem için PostgreSQL mi yoksa MongoDB mi seçmeliyiz? Trade-off (ödünleşim)
analizi yap.
2. Neden PostgreSQL + JSONB kombinasyonunun her iki dünyanın en iyi yönlerini
birleştirdiğini açıkla.
3. Örnek bir kullanıcı ve anket şemasını SQL ve JSONB ile tasarla.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
-- Kötü NoSQL Yaklaşımı: İlişkisel veriyi MongoDB'de iç içe gömmek (Denormalization
Kabusu)
-- Kullanıcı bir adresini güncellediğinde, 50 farklı sipariş dökümanındaki gömülü
adresi elle güncellemek gerekir!
{
  "_id": "order_123",
  "user": { "id": "u1", "name": "Ali", "address": "Eski Adres" },
  "total": 450.00
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
-- PostgreSQL ile Hibrit Mükemmellik: Katı İlişkiler + Esnek JSONB
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE invoices (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    amount NUMERIC(10, 2) NOT NULL,
    status VARCHAR(50) DEFAULT 'PENDING',
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

-- Dinamik ve Şemasız Veri İçin JSONB Kolonu
CREATE TABLE survey_responses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    -- Anket soruları ve yanıtları dinamiktir; şema zorunluluğu yok!
    answers JSONB NOT NULL,
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

-- JSONB Alanı Üzerinde GIN İndeksi ile Işık Hızında Arama
CREATE INDEX idx_survey_answers ON survey_responses USING GIN (answers);
```

💡 **Kod Analizi:** PostgreSQL sayesinde hem finansal fatura verisi `REFERENCES users(id)` ile katı ilişkisel güvencede tutulur, hem de dinamik anket yanıtları `JSONB` kolonunda esnekçe saklanıp `GIN` indeksiyle aranabilir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Daha proje başlamadan ‘MongoDB daha modern’ deyip ilişkisel sistemi NoSQL’e taşımak.
  - **Çözüm:** Aksi için çok güçlü bir sebep yoksa varsayılan olarak her zaman PostgreSQL ile başlayın.
- **Tuzak:** NoSQL’de ilişkisel tabloları simüle etmeye çalışıp elle `populate` veya `lookup` yazmak.
  - **Çözüm:** Eğer `JOIN` yapmaya ihtiyaç duyuyorsanız yanlış veritabanını seçmişsiniz demektir; SQL’e geçin.
- **Tuzak:** SQL veritabanında para tutarlarını FLOAT olarak saklamak.
  - **Çözüm:** Para tutarları için daima `NUMERIC` veya `DECIMAL` tipi kullanın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 25: SQL vs NoSQL - Veritabanı Seçim Rehberi*

- 🖥️ **Slayt 1:** SQL ve NoSQL arasındaki felsefe farkı: ACID vs BASE.
- 🖥️ **Slayt 2:** İlişkisel Bütünlük (Foreign Key) neden hayat kurtarır?
- 🖥️ **Slayt 3:** PostgreSQL JSONB: NoSQL esnekliğini SQL'de kullanmak.
- 🖥️ **Slayt 4:** Canlı SQL: GIN indeksi ile JSON içinde sorgu çalıştırmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
PostgreSQL veritabanında JSONB kolonuna sahip bir tablo açın. `WHERE answers @> '{"rating": 5}'` sorgusunu çalıştırarak JSON içindeki bir alanın milisaniye altında nasıl filtrelendiğini gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki veritabanı şemasını açın. Eğer NoSQL kullanıyorsanız tablolar arası ilişki kurup kurmadığınızı; SQL kullanıyorsanız gereksiz tablolar yerine JSONB kullanılabilecek alanları analiz edin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Veritabanlarında 'ACID' kısaltmasının 'A' harfi (Atomicity) neyi garanti eder? - A) Veritabanının atom bombasına dayanıklı olduğunu - B) Bir transaction içindeki tüm adımların ya tamamen başarılı olacağını ya da tek bir hata durumunda tüm işlemlerin geri alınacağını (ya hep ya hiç) - C) Sadece Python ile çalıştığını - D) Tabloların otomatik silineceğini > **Doğru Yanıt:** B > **Açıklama:** Atomicity (Bütünlük), ilişkili işlemlerin bölünemez bir bütün olduğunu; başarısızlık durumunda veritabanının önceki temiz durumuna geri döndürüleceğini garanti eder.

Soru 2: PostgreSQL'in `JSONB` kolon tipi ile standart `JSON` tipi arasındaki temel fark nedir? - A) Hiçbir farkı yoktur - B) JSONB veriyi ikili (binary) formatta saklar ve GIN indeksleri ile alan bazlı hızlı arama yapılmasını sağlar - C) JSONB sadece sayılar saklayabilir - D) JSONB daha yavaş çalışır > **Doğru Yanıt:** B > **Açıklama:** JSONB, JSON verisini parçalanmış ikili formatta tutar ve indeksleme desteği sayesinde devasa JSON belgeleri içinde anında filtreleme imkanı sunar.

---

26. Veri Modelleme & ER Diyagramlarını AI ile Tasarlamak

1:1, 1:N ve N:M İlişkilerini Doğru Kurarak Veri Tutarlılığını Sağlamak
Bölüm 5 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Birçok junior mühendis N:M (Çoka-Çok) ilişkileri kurarken neden ara tabloyu (Junction Table) unutur ve AI ile kusursuz ER diyagramı nasıl çizdirilir?"

Bölümden Çıkarılacak 3 Altın Cümle:
- 📌 Veri modelleme, kod yazmadan önce gerçek dünyanın nesnelerini ve kurallarını tablolara aktarma sanatıdır. - 📌 1:N (Bire-Çok) ilişkide yabancı anahtar (Foreign Key) daima 'Çok' olan tarafa konur. - 📌 N:M (Çoka-Çok) ilişkiler doğrudan kurulamaz; arada iki foreign key tutan bir ara tablo (Junction Table) zorunludur.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir okul hayal edin: - Her öğrencinin tek bir kimlik kartı vardır: **Bire-Bir (1:1)** - Bir öğretmenin onlarca öğrencisi olabilir: **Bire-Çok (1:N)** - Peki öğrenciler ve dersler? Bir öğrenci 5 farklı derse girer, bir derste 30 farklı öğrenci vardır! İşte bu **Çoka-Çok (N:M)** ilişkidir!

Eğer öğrenci tablosunun içine dersin adını yazarsanız, öğrenci 5 ders aldığında öğrencinin adını 5 kez tekrar yazmak zorunda kalırsınız!

Çözüm: Ortaya bir sınıf yoklama defteri koymaktır (Junction Table). Defterde sadece: *"Öğrenci 101, Ders 201'e kayıtlıdır"* yazar.

Veri tabanı tasarımı, bu ilişkileri mantıklı defterlere bölme sanatıdır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

İlişki Türleri ve Yabancı Anahtar (Foreign Key) Kuralları:

```
1. BİRE-BİR (1:1):
   [Kullanıcı (User)] <——> [Kullanıcı Profili (Profile)]
   * Foreign Key profil tablosuna konur ve UNIQUE yapılır.

2. BİRE-ÇOK (1:N):
   [Müşteri (Customer)] 1 <—— N [Siparişler (Orders)]
   * Foreign Key daima 'Çok' tarafına (Orders tablosuna customer_id) konur.

3. ÇOKA-ÇOK (N:M):
   [Öğrenciler] N <—— [Kayıtlar (Enrollments)] ——> M [Dersler]
   * Ara tablo (Junction Table) her iki tablonun ID'sini tutar.
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Veritabanı Mimarı ve ER Uzmanı
Bir 'Online Kurs ve Eğitmen Platformu' (Udemy benzeri) için veritabanı şeması
tasarlayacağız.

## GEREKSİNİMLER:
- Bir eğitmenin birden fazla kursu olabilir (1:N).
- Bir kursun birden fazla kategorisi ve bir kategorinin birden fazla kursu olabilir
(N:M).
- Kullanıcılar kurslara kaydolabilmelidir (N:M).
- Her kursun video dersleri (Lessons) sırayla listelenmelidir (1:N).

## GÖREV:
1. Bu sistemin Mermaid ERD (Entity Relationship Diagram) şemasını çiz.
2. Prisma ORM formatında `schema.prisma` dosyasını foreign key ve index'leriyle
eksiksiz yaz.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ **Acemi / Kontrolsüz AI Çıktısı (Riskli):**

```
// Kötü Tasarım: Çoka-Çok ilişkiyi string array olarak saklamak
model Course {
  id          String   @id @default(uuid())
  title       String
  categories  String[] // HATA: Arama yapılamaz, kategori adı değişirse tüm kurslar
bozulur!
  studentIds  String[] // HATA: 50.000 öğrenci kaydolduğunda döküman patlar!
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
// Profesyonel Prisma Şeması: İlişkisel Bütünlük ve İndeksleme
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id           String       @id @default(uuid())
  email        String       @unique
  fullName     String
  taughtCourses Course[]    @relation("InstructorCourses")
  enrollments  Enrollment[]
  createdAt    DateTime     @default(now())
}

model Course {
  id           String       @id @default(uuid())
  title        String
  instructorId String
  instructor   User         @relation("InstructorCourses", fields: [instructorId],
references: [id])
  lessons      Lesson[]
  enrollments  Enrollment[]
  categories   CourseCategory[]
  createdAt    DateTime     @default(now())

  @@index([instructorId])
}

model Lesson {
  id        String   @id @default(uuid())
  courseId  String
  course    Course   @relation(fields: [courseId], references: [id], onDelete:
Cascade)
  title     String
  order     Int
  videoUrl  String

  @@index([courseId, order])
}

// N:M İlişkisi İçin Ara Tablo (Junction Table)
model Enrollment {
  userId     String
  courseId   String
  user       User     @relation(fields: [userId], references: [id], onDelete:
Cascade)
  course     Course   @relation(fields: [courseId], references: [id], onDelete:
Cascade)
  enrolledAt DateTime @default(now())

  @@id([userId, courseId]) // Bir kullanıcı aynı kursa iki kez kaydolamaz!
}
```

💡 **Kod Analizi:** `@@id([userId, courseId])` bileşik anahtarı (compound key) sayesinde bir öğrencinin aynı kursa mükerrer kaydolması veritabanı seviyesinde imkansız kılınmıştır. `onDelete: Cascade` ile kurs silindiğinde derslerinin öksüz (orphaned) kalması engellenir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Çoka-çok ilişkilerde ID listesini JSON veya virgülle ayrılmış string olarak tek bir kolona tıkıştırmak.
  - **Çözüm:** Her zaman iki foreign key içeren standart bir ara tablo (Junction Table) oluşturun.
- **Tuzak:** Foreign Key alanlarına indeks koymayı unutmak.
  - **Çözüm:** Prisma'da `@@index([foreignKey])` veya SQL'de `CREATE INDEX` ile arama hızını garanti edin.
- **Tuzak:** Silme (Delete) kurallarını belirlememek ve veritabanında öksüz (orphan) kayıtlar bırakmak.
  - **Çözüm:** İş mantığına göre `ON DELETE CASCADE` veya `ON DELETE RESTRICT` kurallarını açıkça tanımlayın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 26: Veri Modelleme - ER Diyagramları ve Prisma Şeması*

- 🖥️ **Slayt 1:** Veri modelleme nedir? Gerçek dünyayı tablolara dökmek.
- 🖥️ **Slayt 2:** 1:1, 1:N ve N:M ilişkilerinin matematiksel kuralları.
- 🖥️ **Slayt 3:** Junction Table (Ara Tablo) ve Compound Primary Key kullanımı.
- 🖥️ **Slayt 4:** Prisma ORM ile 5 dakikada kurumsal şema oluşturmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Mermaid.js ile bir e-ticaret ER diyagramı çizin. Ardından AI asistanına bu diyagramı verip hatasız bir `schema.prisma` üretmesini sağlayın ve `npx prisma format` ile doğrulayın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Kendi projeniz için Mermaid.js formatında en az 4 tablodan oluşan bir ER diyagramı çizin ve markdown önizlemesinde inceleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: İlişkisel veritabanlarında Çoka-Çok (Many-to-Many) bir ilişkiyi modellemenin standart yolu nedir? - A) Tüm ID'leri virgülle ayırıp tek bir hücreye yazmak - B) Her iki tablonun birincil anahtarlarını (Primary Key) yabancı anahtar (Foreign Key) olarak tutan bir ara tablo (Junction Table) oluşturmak - C) Veritabanını silip baştan kurmak - D) Sadece NoSQL kullanmak > **Doğru Yanıt:** B > **Açıklama:** N:M ilişkiler, her iki tarafın ilişkisini satır bazında tutan bir ara tablo (Junction Table) ile iki adet 1:N ilişkiye dönüştürülerek modellenir.

Soru 2: Bir tabloda `ON DELETE CASCADE` kuralı tanımlandığında ne gerçekleşir? - A) Bilgisayar yeniden başlar - B) Ana tablodaki kayıt silindiğinde, ona bağlı olan tüm alt tablodaki ilişkili kayıtlar da otomatik olarak güvenle silinir - C) Tablo şifrelenir - D) Silme işlemi tamamen yasaklanır > **Doğru Yanıt:** B > **Açıklama:** CASCADE silme kuralı, ebeveyn kayıt silindiğinde geride öksüz (orphan) kayıtlar kalmasını önlemek için çocuk kayıtları da siler.

27. SQL Ustalığı: Karmaşık Sorgular, İndeksleme ve AI Query Optimizer

EXPLAIN ANALYZE Okuyarak 10 Saniyelik Sorguları 5 Milisaniyeye Düşürmek
Bölüm 5 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Veritabanına indeks eklemek sorguları 1000 kat hızlandırıyorsa, neden tüm kolonlara indeks eklemiyoruz? Gizli bedel nedir?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 İndeks eklemek okuma (SELECT) hızını katbekat artırırken, her yazma (INSERT/UPDATE) işleminde ek CPU ve disk yükü getirir. - 📌 `EXPLAIN ANALYZE` çıktısındaki 'Seq Scan' (Ardışık Tarama) milyonlarca satırda sunucuyu boğan en büyük tehlikedir. - 📌 AI modelleri yavaş SQL sorgularını ve EXPLAIN planlarını analiz ederek en ideal B-Tree veya Composite indeksleri önerebilir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

1000 sayfalık kalın bir tıp ansiklopedisini düşünün.

Kitabın en arkasında alfabetik bir **Fihrist / Dizin (İndeks)** vardır. “Kalp krizi” kelimesini arıyorsanız, fihristten “K” harfine bakar ve doğrudan sayfa 412’yi açarsınız (İndeks Taraması - Index Scan). 5 saniye sürer!

Eğer o fihrist olmasaydı ne yapardınız? Sayfa 1’den başlayıp sayfa 1000’e kadar tüm sayfaları tek tek okumak zorunda kalırdınız! (Ardışık Tarama - Sequential Scan). 3 saat sürerdi!

Ama dikkat: Ansiklopediye yeni bir sayfa eklendiğinde (INSERT), arkadaki fihristi de baştan sona silip yeniden alfabetik düzenlemek gerekir!

İşte bu yüzden her kolona indeks konmaz; sadece sık aranan kritik kolonlara konur.

🧠 2. AI Çağında Yeni Zihniyet Modeli

SQL İndeksleme Mantığı (B-Tree):

```
Sorgu: SELECT * FROM users WHERE email = 'ali@test.com';

SENARYO A: İndeks Yok (Sequential Scan)
[Satır 1] ──> [Satır 2] ──> ... ──> [Satır 5.000.000] (Tüm disk okunur, 8 saniye sürer!)

SENARYO B: B-Tree İndeksi Var (Index Scan)
                  [ Root: L-Z ]
                 /             \
          [ A-K ]               [ M-Z ]
          /
   [ 'ali@test.com' ──> Satır Adresi: Block 42, Offset 8 ] (Sadece 3 adımda bulunur, 2 milisaniye!)
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: PostgreSQL Veritabanı Performans Tuning Uzmanı
Aşağıdaki SQL sorgusu 2 milyon satırlık `orders` tablosunda 7.5 saniye sürüyor.

## YAVAŞ SORGU:
```sql
SELECT id, user_id, total, created_at
FROM orders
WHERE status = 'PAID' AND created_at >= '2026-01-01'
ORDER BY created_at DESC
LIMIT 20;
```

EXPLAIN ANALYZE ÇIKTISI:

```
Seq Scan on orders (cost=0.00..45210.00 rows=152000 width=32) (actual
time=12.50..7450.21 ms)
```

GÖREV:

1. Bu sorgunun neden bu kadar yavaş olduğunu açıkla.
2. Bu sorguyu 5 milisaniyenin altına indirecek tam Composite Index (Bileşik İndeks) SQL komutunu yaz.
3. İndeks sıralamasının (status, created_at) neden önemli olduğunu açıkla.

```
---

## 🖥️ 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```sql
-- Kötü Çözüm: Ayrı ayrı iki bağımsız indeks oluşturmak (Bitmap Index Scan
karmaşası)
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_orders_created ON orders(created_at);
-- Veritabanı iki indeksi birleştirmeye çalışırken gereksiz CPU yakar!
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
-- Mükemmel Çözüm: Sorgu Filtresine ve Sıralamasına Birebir Uyumlu Composite Index
-- Sıralama: Önce eşitlik (equality: status), sonra aralık ve sıralama (range: created_at)
CREATE INDEX idx_orders_status_created
ON orders (status, created_at DESC);

-- İsteğe Bağlı: Index-Only Scan için kapsayıcı indeks (Covering Index)
-- Tabloya hiç gitmeden sadece indeksten yanıt döner:
CREATE INDEX idx_orders_optimized
ON orders (status, created_at DESC)
INCLUDE (user_id, total);
```

💡 **Kod Analizi:** `Composite Index` oluşturulurken altın kural: Önce `=` ile aranan eşitlik kolonu (`status`), ardından `>=` veya `ORDER BY` ile kullanılan kolondur (`created_at DESC`). `INCLUDE` eki ise tabloya (heap) gitme ihtiyacını tamamen ortadan kaldırır (Index-Only Scan).

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Tablodaki her kolona ezbere indeks koyup INSERT işlemlerini süründürmek.
  - **Çözüm:** Sadece WHERE, JOIN ve ORDER BY koşullarında sıkça kullanılan kolonları indeksleyin.
- **Tuzak:** İndekslenen kolon üzerinde fonksiyon çalıştırmak (Örn: `WHERE LOWER(email) = 'ali'`).
  - **Çözüm:** Fonksiyonlar standart B-Tree indeksini bozar; ya veriyi küçük harf kaydedin ya da Expression Index (`CREATE INDEX ON users(LOWER(email))`) oluşturun.
- **Tuzak:** Production ortamında kilitlenme (lock) yaratan indeks komutları çalıştırmak.
  - **Çözüm:** Canlı veritabanında indeks eklerken daima `CREATE INDEX CONCURRENTLY` kullanın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 27: SQL Ustalığı - EXPLAIN ANALYZE ve İndeks Optimizasyonu*

- 🖥️ **Slayt 1:** Veritabanı fihristi nedir? B-Tree indeks nasıl çalışır?
- 🖥️ **Slayt 2:** EXPLAIN ANALYZE okuma rehberi: Seq Scan vs Index Scan.
- 🖥️ **Slayt 3:** Bileşik İndekslerde (Composite Index) kolon sırası kuralı.
- 🖥️ **Slayt 4:** Production güvenliği: CREATE INDEX CONCURRENTLY.

Canlı Kodlama / Ekran Kaydı Senaryosu:
PostgreSQL'de 1.000.000 satırlık mock sipariş tablosu oluşturun. İndeks olmadan sorguyu çalıştırıp 8 saniye sürdüğünü ve 'Seq Scan' verdiğini gösterin. Ardından composite index ekleyip sürenin 3 milisaniyeye düştüğünü canlı kanıtlayın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki en yavaş çalışan veritabanı sorgusunu tespit edin. Başına `EXPLAIN ANALYZE` ekleyerek 'Seq Scan' olup olmadığını kontrol edin ve gerekiyorsa indeks ekleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Bir SQL sorgusunun yürütme planında 'Seq Scan' (Sequential Scan) görülmesi ne anlama gelir? - A) Sorgunun ışık hızında çalıştığını - B) Veritabanının uygun bir indeks bulamadığı için tablodaki tüm satırları baştan sona tek tek taramak zorunda kaldığını - C) Sadece Python'da hata olduğunu - D) Tablonun tamamen boş olduğunu > **Doğru Yanıt:** B > **Açıklama:** Seq Scan, aranan koşula uygun indeks bulunmadığında tablodaki tüm satırların diskten taranmasıdır; büyük tablolarda çok ciddi performans kaybıdır.

Soru 2: Canlı (production) bir PostgreSQL veritabanında tabloyu kilitlemeden ve yazma işlemlerini durdurmadan indeks eklemek için hangi anahtar kelime kullanılır? - A) FAST - B) CONCURRENTLY - C) ASYNC - D) NOWAIT > **Doğru Yanıt:** B > **Açıklama:** `CREATE INDEX CONCURRENTLY` komutu, tablo üzerinde okuma ve yazma kilitlerini engeller; arka planda güvenle indeks inşa edilmesini sağlar.

28. ORM'ler (Prisma, TypeORM, SQLAlchemy) ve Gizli Tehlike: N+1 Problemi

Tek Bir Sayfa İçin Veritabanına 1000 Sorgu Gönderme Tuzağından Kurtulmak
Bölüm 5 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 12 dk

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"ORM'ler geliştiricilerin hayatını kolaylaştıran harika araçlar mı, yoksa arka planda sessizce binlerce sorgu atan tehlikeli bir canavar mı?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 ORM (Object-Relational Mapping), nesne yönelimli kod ile ilişkisel veritabanı arasında köprü kurar; ancak SQL bilme zorunluluğunu ortadan kaldırmaz. - 📌 N+1 problemi, bir liste çekildikten sonra her elemanın ilişkili verisini döngü içinde ayrı ayrı sorgulama hatasıdır (1 + N sorgu). - 📌 `include` (Eager Loading) veya `JOIN` kullanarak N+1 sorgu felaketini tek bir birleşik sorguya indirebilirsiniz.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir süpermarkete gittiğinizi ve 50 farklı ürün alacağınızı düşünün.

Kasaya geldiniz. Sepetteki her ürünü tek tek geçirip ödemek yerine; her bir ürün için ayrı ayrı otoparka gidip arabanıza bir ürün bırakıp, tekrar kasaya dönüp ikinci ürünü alıp, tekrar otoparka gidip gelseydiniz ne olurdu? 50 kez arabaya gidip gelirdiniz ve gününüz biterdi!

İşte **N+1 Sorgu Problemi** tam olarak budur: Geliştirici 100 kullanıcının siparişlerini listelemek ister. 1. Önce 100 kullanıcıyı çeker (1 sorgu). 2. Sonra `for` döngüsünün içinde her kullanıcının siparişini tek tek veritabanına sorar (100 ayrı sorgu!). Toplamda 101 sorgu atılır! Veritabanı sunucusu nefes alamaz hale gelir.

Oysa *"Tüm kullanıcıları ve ilişkili siparişlerini tek seferde getir"* derseniz arabanıza tüm poşetleri tek seferde yüklersiniz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

N+1 Probleminin Tespiti ve Çözümü:

```
YAVAŞ (N+1 Sorgu):
1. Sorgu: SELECT * FROM users LIMIT 50;
2. Sorgu: SELECT * FROM orders WHERE user_id = 1;
3. Sorgu: SELECT * FROM orders WHERE user_id = 2;
...
51. Sorgu: SELECT * FROM orders WHERE user_id = 50;
─────────────────────────────────────────────────────
HIZLI (Eager Loading – Tek Sorgu):
SELECT users.*, orders.*
FROM users
LEFT JOIN orders ON orders.user_id = users.id
LIMIT 50;
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: ORM ve Veritabanı Optimizasyon Uzmanı
Aşağıdaki Prisma / TypeScript fonksiyonu veritabanına yüzlerce gereksiz sorgu
atmaktadır.

## YAVAŞ KOD:
```typescript
export async function getUsersWithPosts() {
  const users = await prisma.user.findMany({ take: 100 });
  const results = [];
  for (const user of users) {
    // N+1 tuzağı! Her kullanıcı için ayrı sorgu atılıyor!
    const posts = await prisma.post.findMany({ where: { authorId: user.id } });
    results.push({ ...user, posts });
  }
  return results;
}
````

GÖREV:

1. N+1 problemini açıkla ve arka planda neden 101 sorgu çalıştığını göster.
2. Prisma'nın `include` (Eager Loading) özelliğini kullanarak bu işlemi tek bir veritabanı turuna (round-trip) indirecek optimize kodu yaz.
````

````
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// N+1 Tuzağı: Döngü içinde await sorgusu çalıştırmak
export async function getArticlesWithCommentsNaive() {
  const articles = await prisma.article.findMany({ take: 50 });

  // 50 makale için 50 kez veritabanına gidip gelir (Toplam 51 sorgu!)
  return await Promise.all(articles.map(async (article) => {
    const comments = await prisma.comment.findMany({ where: { articleId:
article.id } });
    return { ...article, comments };
  }));
}
````

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```typescript
// Eager Loading ile Optimize Edilmiş Sürüm: Tek veya 2 birleşik sorgu
export async function getArticlesWithCommentsOptimized() {
  // Prisma arka planda tek bir JOIN veya IN sorgusuyla çözer
  return await prisma.article.findMany({
    take: 50,
    include: {
      comments: {
        select: {
          id: true,
          content: true,
          createdAt: true
        },
        orderBy: { createdAt: 'desc' },
        take: 5 // Sadece en son 5 yorumu alarak payload'ı hafiflet
      }
    }
  });
}
```

💡 **Kod Analizi:** `include` bloğu sayesinde Prisma tüm makaleleri ve ilişkili yorumları tek bir SQL sorgusunda çeker. 51 veritabanı turu (round-trip) 1 tura iner; sayfa yüklenme süresi 1500 ms’den 20 ms’ye düşer.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Döngü (for/map) içinde veritabanı sorgusu (`await db...`) çalıştırmak.
```

  - **Çözüm:** Döngü içinde asla sorgu çalıştırmayın; veriyi döngüden önce `include` veya `IN (...)` ile topluca çekin.
- **Tuzak:** ORM loglarını (query logging) kapatıp arkada ne kadar sorgu çalıştığını takip etmemek.
  - **Çözüm:** Geliştirme ortamında ORM SQL loglarını (`log: ['query']`) mutlaka açık tutun.
- **Tuzak:** İhtiyaç duyulmayan tüm devasa kolonları (`SELECT *`) çekmek.
  - **Çözüm:** `select` anahtarı ile sadece sayfada gösterilecek alanları çekerek bellek tasarrufu yapın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 28: ORM Optimizasyonu - N+1 Sorgu Tehlikesini Yok Etmek*

- 🖥️ **Slayt 1:** ORM nedir ve yazılımcıları nasıl tembelleştirir?
- 🖥️ **Slayt 2:** N+1 Probleminin anatomisi: 1 sorgu yerine 101 sorgu atmak.
- 🖥️ **Slayt 3:** Eager Loading (include) vs Lazy Loading mantığı.
- 🖥️ **Slayt 4:** Canlı SQL Log İncelemesi: 50 sorguyu 1 sorguya indirmek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Prisma'da `log: ['query']` aktif edin. Önce döngü içinde Post çeken fonksiyonu çalıştırıp terminali dolduran onlarca SQL sorgusunu gösterin. Ardından `include` ekleyip terminalde tek bir SQL sorgusu aktığını gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde ORM sorgu loglarını aktif edin. Bir sayfayı açtığınızda arka planda kaç adet SQL sorgusu tetiklendiğini terminalden sayın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Veritabanı geliştirmede 'N+1 Problemi' tam olarak neyi ifade eder? - A) Veritabanına 1 ekleme yapınca N tanesinin silinmesini - B) Ana kayıtları getirmek için 1 sorgu, ardından döngü içinde her bir kaydın ilişkili verisini getirmek için N adet ayrı sorgu çalıştırılması felaketini - C) Sadece Python'da geçerli bir matematik teoremini - D) Tabloların adlarının 1 artırılmasını > **Doğru Yanıt:** B > **Açıklama:** N+1 problemi, ilişkili verilerin toplu çekilmek yerine döngü içinde her satır için ayrı ayrı sorgulanarak veritabanının gereksiz yere boğulmasıdır.

Soru 2: Prisma veya TypeORM gibi ORM'lerde N+1 problemini çözmek için kullanılan temel yaklaşım hangisidir? - A) Eager Loading (`include` veya `relations` kullanarak ilişkili veriyi tek seferde çekmek)

- B) Veritabanını kapatıp açmak - C) Tüm verileri frontend'e aktarmak - D) Sadece string veri kullanmak > **Doğru Yanıt:** A > **Açıklama:** Eager Loading, ilişkili kayıtların ana sorguyla birlikte JOIN veya optimize toplu sorgularla tek seferde belleğe alınmasını sağlar.

29. Cache Mekanizmaları: Redis ile Yıldırım Hızında Yanıtlar

Cache-Aside Deseni ve 'Cache Invalidation' Zorluğunu Aşmak
Bölüm 5 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ **NotebookLM & Audio Overview Podcast Notları**

Podcast Tartışma Sorusu:
"'Bilgisayar bilimlerinde sadece iki zor şey vardır: Cache Invalidation ve isimlendirme.' Peki Redis ile önbellek yönetirken bayat (stale) veriyi nasıl önleriz?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 En hızlı veritabanı sorgusu, hiç yapılmayan veritabanı sorgusudur; sık okunan veriler RAM'de (Redis) tutulmalıdır. - 📌 Cache-Aside deseni: Önce Redis'e bak (Cache Hit), yoksa veritabanından oku ve Redis'e yaz (Cache Miss). - 📌 Her önbellek anahtarının mutlaka bir yaşam süresi (TTL - Time to Live) olmalıdır; sonsuz önbellek bellek patlamasına yol açar.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir kütüphane görevlisini hayal edin.

Günde 500 öğrenci gelip aynı kitabı soruyor: *"Harry Potter 1. kitap var mı?"*

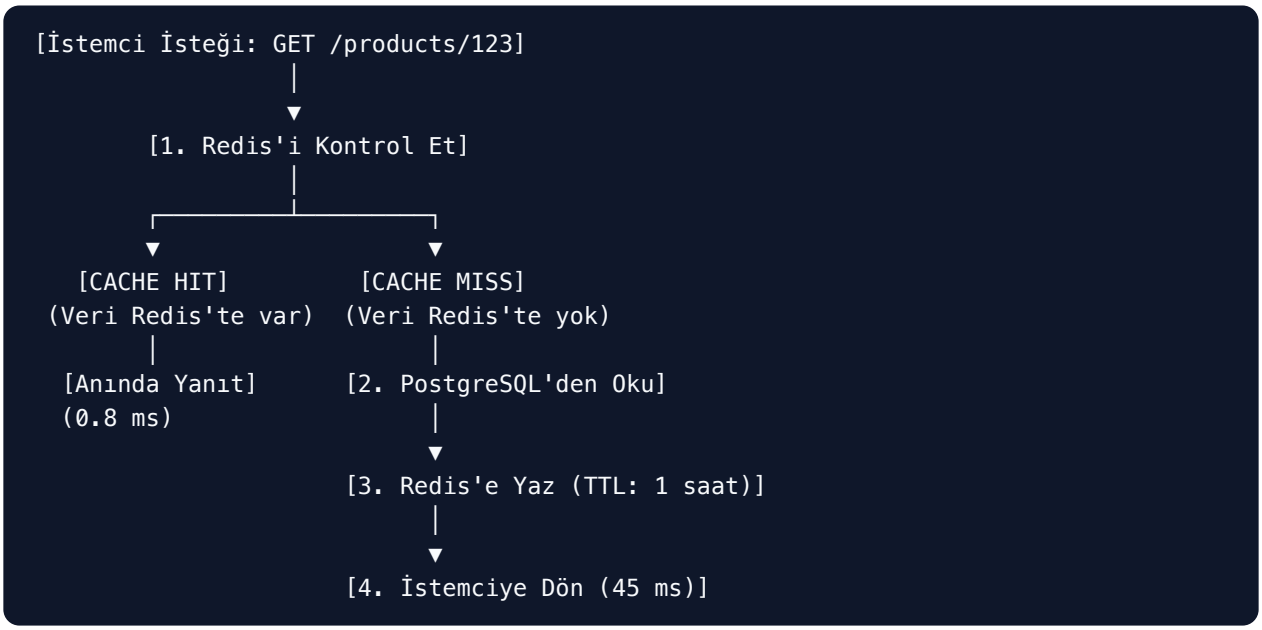
Eğer kütüphaneci her seferinde 3 kat yukarı çıkıp, tozlu rafların arasında 15 dakika yürüyüp kitabı kontrol edip gelseydi akşamı zor ederdi! (Veritabanı Disk Erişimi).

Akıllı bir kütüphaneci ne yapar? O çok sorulan kitabın bir kopyasını hemen masasının çekmecesine koyar! Biri sorduğunda çekmeceyi açıp 2 saniyede gösterir (Redis RAM Belleği).

Ancak bir kural vardır: Eğer kitap kütüphaneden atılırsa veya güncellenirse, çekmecedeki kitabı da hemen yenilemek gerekir (Cache Invalidation).

🧠 2. AI Çağında Yeni Zihniyet Modeli

Cache-Aside Deseni Akışı:



🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Dağıtık Sistemler ve Cache Mimarı
Node.js ve Redis (ioredis) kullanarak bir 'Ürün Detay Önbellek Servisi' yazacağız.

## GÖREV:
1. 'Cache-Aside' desenini uygulayan tip güvenli bir fonksiyon yaz.
2. Ürün verisini Redis'te saklarken 1 saatlik TTL (Time to Live) ekle.
3. Ürün güncellendiğinde (`updateProduct`) Redis'teki eski anahtarı anında silen
(Invalidation) fonksiyonu hazırla.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ **Acemi / Kontrolsüz AI Çıktısı (Riskli):**

```
// Kötü Cache: TTL yok (sonsuza kadar bellekte kalır) ve güncellemede silinmiyor
(bayat veri)
export async function getProductNaive(id: string) {
  const cached = await redis.get(`product:${id}`);
  if (cached) return JSON.parse(cached);

  const product = await db.product.findUnique({ where: { id } });
  // TTL YOK! Bellek dolunca Redis çöker; ürün fiyatı değişse de eski fiyat kalır!
  await redis.set(`product:${id}`, JSON.stringify(product));
  return product;
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
import Redis from "ioredis";

const redis = new Redis(process.env.REDIS_URL || "redis://localhost:6379");
const CACHE_TTL_SECONDS = 3600; // 1 Saat

export class ProductCacheService {
  private static getKey(id: string): string {
    return `cache:product:${id}`;
  }

  // 1. Cache-Aside Okuma
  static async getProduct(id: string): Promise<Product | null> {
    const key = this.getKey(id);

    try {
      const cached = await redis.get(key);
      if (cached) {
        return JSON.parse(cached) as Product; // Cache Hit (1 ms)
      }
    } catch (err) {
      console.warn("Redis okuma hatası, DB'ye yönlendiriliyor:", err);
    }

    // Cache Miss: Veritabanından oku
    const product = await db.product.findUnique({ where: { id } });
    if (!product) return null;

    // Redis'e TTL ile kaydet (Fail-safe)
    try {
      await redis.set(key, JSON.stringify(product), "EX", CACHE_TTL_SECONDS);
    } catch (err) {
      console.error("Redis yazma hatası:", err);
    }

    return product;
  }

  // 2. Cache Invalidation (Güncelleme Anında Temizleme)
  static async invalidate(id: string): Promise<void> {
    try {
      await redis.del(this.getKey(id));
    } catch (err) {
      console.error("Cache invalidation hatası:", err);
    }
  }
}
```

💡 **Kod Analizi:** Kıdemli yaklaşımda; Redis çöktüğünde uygulamanın durmaması için `try/catch` ile sessiz devralma (fallback to DB) sağlanmış, `EX` parametresi ile 1 saatlik yaşam süresi zorunlu kılınmış ve güncelleme anında `del` ile bayat veri riski önlenmiştir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Redis'e TTL (Time-To-Live) vermeden anahtar kaydetmek ve sunucu belleğini patlatmak.
  - **Çözüm:** İstisnasız her cache anahtarına makul bir TTL süresi verin.
- **Tuzak:** Redis çöktüğünde tüm backend uygulamasının da çökmesine izin vermek.
  - **Çözüm:** Redis çağrılarını try/catch ile sarın; Redis yoksa doğrudan veritabanına gidin (Graceful Degradation).
- **Tuzak:** Kullanıcıya özel hassas verileri genel (public) cache anahtarlarında saklamak.
  - **Çözüm:** Cache anahtarlarına kullanıcı ID'si ekleyin (`user:123:profile`) ve hassas verileri şifreleyin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 29: Cache Stratejileri - Redis ile Yıldırım Hızı*

- 🖥️ **Slayt 1:** Neden en hızlı sorgu hiç yapılmayan sorgudur?
- 🖥️ **Slayt 2:** Cache-Aside Deseni: Hit, Miss ve TTL kavramları.
- 🖥️ **Slayt 3:** Cache Invalidation kabusu ve çözüm yöntemleri.
- 🖥️ **Slayt 4:** Graceful Degradation: Redis çökerse sistem nasıl ayakta kalır?

Canlı Kodlama / Ekran Kaydı Senaryosu:
Docker üzerinde yerel bir Redis ayağa kaldırın. Postman ile API'ye istek atın: İlk istekte sürenin 120 ms olduğunu (DB okuma), ikinci istekte 2 ms'ye düştüğünü (Redis Hit) canlı gösterin. Ardından veriyi güncelleyip cache'in nasıl temizlendiğini kanıtlayın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Docker ile yerel makinenizde bir Redis başlatın (`docker run -p 6379:6379 redis:alpine`) ve projenizdeki en sık okunan endpoint'e Cache-Aside deseni ekleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Önbelleğe alınan bir veriye 'TTL' (Time to Live) süresi tanımlamanın temel amacı nedir? - A) Veriyi şifrelemek - B) Belirtilen süre dolduğunda verinin bellekten otomatik silinmesini sağlayarak hem RAM'in dolmasını engellemek hem de bayat veri riskini sınırlamak - C) Bilgisayarı kapatmak -

D) Sadece Python'da hata vermesini sağlamak > **Doğru Yanıt:** B > **Açıklama:** TTL, anahtarın yaşam süresini belirler; süresi biten anahtar silinir ve RAM'in taşması önlenir.

Soru 2: Cache-Aside deseninde 'Cache Miss' ne anlama gelir? - A) Sistemin tamamen çöktüğünü - B) Aranan verinin önbellekte (Redis) bulunamadığını; bu yüzden asıl veritabanından okunup önbelleğe yazılması gerektiğini - C) CSS dosyasının eksik olduğunu - D) Kullanıcının şifresini unuttuğunu > **Doğru Yanıt:** B > **Açıklama:** Cache Miss, aranan bilginin Redis'te olmaması durumudur; uygulama veritabanından veriyi çeker, Redis'e kaydeder ve kullanıcıya döner.

---

30. Kimlik Doğrulama ve Yetkilendirme: JWT, OAuth2 ve Session Yönetimi

Authentication (Sen Kimsin?) vs. Authorization (Neye İznin Var?) Ayrımı
Bölüm 5 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

---

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"JWT (JSON Web Token) neden 'localStorage' içinde saklanmamalıdır ve bir access token çalındığında sistem nasıl korunur?"

Bölümden Çıkarılacak 3 Altın Cümle:
- 📌 Authentication (Kimlik Doğrulama) 'Sen kimsin?' sorusudur; Authorization (Yetkilendirme) ise 'Neye erişebilirsin?' sorusudur. - 📌 JWT'ler şifrelenmiş değil, imzalanmış (signed) jetonlardır; içeriği herkes okuyabilir, bu yüzden içine asla şifre veya kredi kartı yazılmaz. - 📌 JWT'leri XSS saldırılarından korumak için `localStorage` yerine daima `HttpOnly, Secure` çerezlerde (Cookies) saklayın.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir havalimanına girdiğinizi düşünün.

1. **Authentication (Kimlik Doğrulama):** Pasaport kontrolüne gidersiniz. Polis memuru pasaportunuza ve yüzünüze bakar: *"Evet, sen Ali Turgut'sun."* Bu kimlik doğrulamadır.

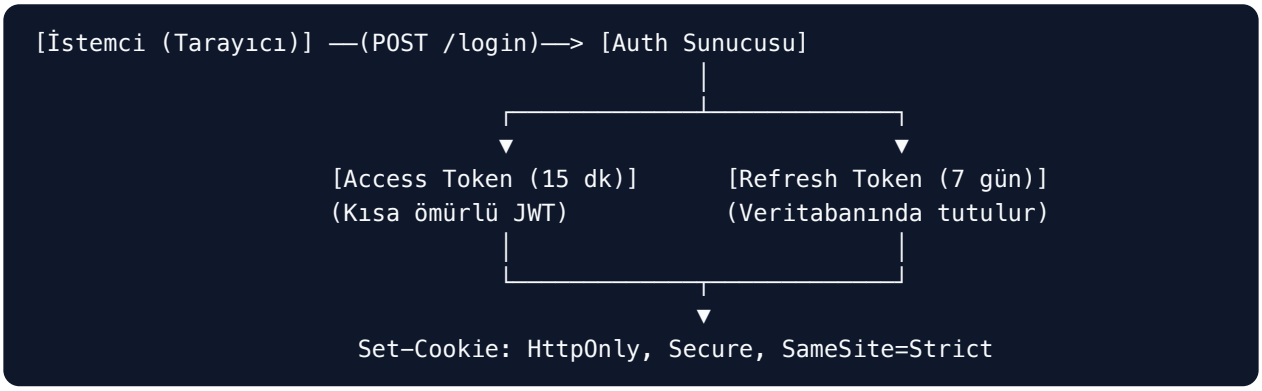
2. **Authorization (Yetkilendirme):** Uçağa binerken biletinizi gösterirsiniz. Biletiniz 'Ekonomi' ise Business Class bölümüne veya pilot kabinine giremezsiniz! Giriş kartınız (RBAC) sadece koltuk numaranıza izin verir.

Pasaportunuz var diye uçağı uçuramazsınız!

Yazılımda da bir kullanıcının sisteme giriş yapmış olması (AuthN), her veriyi silebileceği (AuthZ) anlamına gelmez. Rolleri ve izinleri (RBAC) sıkı bir kapı nöbetçisi gibi denetlemelisiniz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Güvenli Auth Mimarisi (HttpOnly Cookie + Refresh Token):



🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Web Güvenliği ve Kimlik Yönetimi Uzmanı
Express ve TypeScript kullanan bir API'ye Role-Based Access Control (RBAC)
middleware'i ekleyeceğiz.

## GÖREV:
1. Bir JWT doğrulayan ve kullanıcıyı `req.user` içine koyan `authenticate`
middleware'i yaz.
2. Sadece belirli rollere ('ADMIN', 'SUPERVISOR') izin veren `authorize(...roles)`
middleware'i tasarla.
3. Yetkisiz girişlerde (401 vs 403) doğru HTTP durum kodlarını fırlat.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ **Acemi / Kontrolsüz AI Çıktısı (Riskli):**

```
// Güvensiz Yaklaşım: Rolü frontend'den gelen parametreye göre kontrol etmek!
app.delete("/users/:id", async (req, res) => {
  // Saldırgan URL'e ?role=ADMIN yazarak herkesi silebilir!
  if (req.query.role === "ADMIN") {
    await db.user.delete({ where: { id: req.params.id } });
    return res.send("Silindi");
  }
  res.status(403).send("Yetkisiz");
});
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
import { Request, Response, NextFunction } from "express";
import jwt from "jsonwebtoken";

export interface UserTokenPayload {
  userId: string;
  email: string;
  role: "USER" | "ADMIN" | "SUPERVISOR";
}

// 1. Kimlik Doğrulama (Authentication – 401)
export function authenticate(req: Request, res: Response, next: NextFunction) {
  const token = req.cookies?.accessToken || req.headers.authorization?.split(" ")
[1];

  if (!token) {
    return res.status(401).json({ error: "Oturum açılmadı (Token eksik)." });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET!) as UserTokenPayload;
    (req as any).user = decoded;
    next();
  } catch (err) {
    return res.status(401).json({ error: "Geçersiz veya süresi dolmuş token." });
  }
}

// 2. Yetkilendirme (Authorization / RBAC – 403)
export function authorize(...allowedRoles: Array<UserTokenPayload["role"]>) {
  return (req: Request, res: Response, next: NextFunction) => {
    const user = (req as any).user as UserTokenPayload;

    if (!user || !allowedRoles.includes(user.role)) {
      return res.status(403).json({ error: "Yetersiz yetki: Bu kaynağa erişim
izniniz yok." });
    }

    next();
  };
}

// 3. Güvenli Rota Kullanımı:
// app.delete("/users/:id", authenticate, authorize("ADMIN"), deleteUserHandler);
```

💡 **Kod Analizi:** `401 Unauthorized` (Kimliğinizi doğrulamadınız, giriş yapın) ile `403 Forbidden` (Kimliğinizi biliyorum ama buraya girmeye yetkiniz yok) ayrımı tam olarak uygulanmıştır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** JWT içine kullanıcının şifresini veya kredi kartı numarasını koymak.
  - **Çözüm:** JWT şifrelenmiş değildir (Base64'tür); içine sadece zararsız kullanıcı ID ve rolü koyun.
- **Tuzak:** JWT'yi frontend'de `localStorage` içine kaydedip XSS saldırılarına açık kapı bırakmak.
  - **Çözüm:** Token'ları daima `HttpOnly` çerezlerde saklayarak JavaScript ile okunmasını engelleyin.
- **Tuzak:** 401 ve 403 durum kodlarını birbirine karıştırmak.
  - **Çözüm:** Giriş yapılmadıysa 401, giriş yapılmış ama rol yetersizse 403 dönün.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 30: Güvenli Kimlik Yönetimi - JWT ve RBAC Mimarisi*

- 🖥️ **Slayt 1:** Authentication (Kimlik) vs Authorization (Yetki) farkı.
- 🖥️ **Slayt 2:** JWT'nin anatomisi: Header, Payload, Signature.
- 🖥️ **Slayt 3:** Neden localStorage değil HttpOnly Cookie?
- 🖥️ **Slayt 4:** Bölüm 5 Özeti: Veritabanı, Modelleme ve Backend Ustalığı.

Canlı Kodlama / Ekran Kaydı Senaryosu:
jwt.io sitesini açın. Basit bir JWT oluşturup payload'ın şifresiz şekilde herkes tarafından okunabildiğini gösterin. Ardından Express'te `authorize('ADMIN')` middleware'i kurup normal kullanıcıyla istek atınca 403 Forbidden aldığını gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki bir rotayı inceleyin. Eğer kullanıcı rolü frontend'den gelen bir parametreye göre kontrol ediliyorsa, bunu JWT içindeki güvenli claims payload'ına göre kontrol edecek şekilde refactor edin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Bir web uygulamasında `401 Unauthorized` ile `403 Forbidden` arasındaki fark nedir? - A) Hiçbir farkı yoktur - B) 401 kullanıcının henüz oturum açmadığını (kimlik doğrulanmadığını); 403 ise kullanıcının oturum açtığını fakat o kaynağa erişim yetkisinin olmadığını belirtir - C) 401 sadece telefonlarda çıkar - D) 403 sunucunun bozulduğunu belirtir > **Doğru Yanıt:** B > **Açıklama:** 401 kimlik eksikliğini (AuthN), 403 ise yetki yetersizliğini (AuthZ) ifade eder.

Soru 2: JWT token'larının `localStorage` yerine `HttpOnly` çerezlerde saklanmasının birincil güvenlik sebebi nedir? - A) HttpOnly çerezlerin CSS'i daha hızlı render etmesi - B) Kötü niyetli JavaScript kodlarının (XSS saldırıları) çereze doğrudan erişip token'ı çalamamasını sağlamak - C) Sadece Chrome tarayıcısında çalışması - D) Token'ın boyutunu küçültmesi > **Doğru Yanıt:** B > **Açıklama:** `HttpOnly` bayrağı, tarayıcıda çalışan JavaScript kodunun (XSS) çerezi okumasını tarayıcı seviyesinde engelleyerek token hırsızlığını önler.

Bölüm 6: Test Odaklı Geliştirme (TDD) & Kalite Güvencesi

Birim, entegrasyon ve E2E test stratejileri kurun; yapay zekanın ürettiği kodun en büyük savunma kalkanı olan testleri yönetin.

31. Neden Test Yazarız? Test Piramidini Anlamak

'Kodum Zaten Çalışıyor' Tuzağından Kurtulup Gece Rahat Uyuma Sanatı
Bölüm 6 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Junior yazılımcılar neden test yazmayı vakit kaybı olarak görür ve test yazmayan bir şirketin iflas etmesi neden sadece bir zaman meselesidir?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Test yazmak bugünü değil; 6 ay sonra o kodu değiştirecek olan gelecekteki sizi korur (Regresyon Güvencesi). - 📌 Test Piramidi: Tabanda çok sayıda hızlı Unit Test, ortada Entegrasyon Testleri, tepede az sayıda E2E Test olmalıdır. - 📌 AI kod üretirken test yazımı artık saatler değil saniyeler sürer; test yazmamak için hiçbir mazeret kalmamıştır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir paraşüt imalathanesinde çalıştığınızı düşünün.

Yeni bir paraşüt diktiniz. Patronunuz gelip soruyor: *"Bu paraşüt açılacak mı?"* Siz de diyorsunuz ki: *"Valla patron kumaşı çok güzel diktim, rengi de harika, kesin açılır!"* Patron o paraşütle uçaktan atlar mı? Asla! Önce yerde rüzgar tünelinde binlerce kez açılma testine sokulur.

Yazılımda canlıya (production) kod atmak, o paraşütle 10.000 metreden atlamaktır.

Eğer test yazmadıysanız, her cuma akşamı canlıya güncelleme çıktığınızda kabus görürsünüz. Oysa test piramidine sahip bir mühendis, butona basıp kahvesini yudumlayarak hafta sonuna girer.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Test Piramidi Anatomisi:

```
           /\
          /  \     ──> 1. E2E (Uçtan Uca) Testler (%10)
         / E2E\    Tarayıcıyı açar, tıklar. Çok yavaş (dakikalar) ve kırılgan.
        /──────\
       /  INTEG \  ──> 2. Entegrasyon Testleri (%20)
      /  RATION  \ Veritabanı ve API bağlantılarını test eder. Orta hız.
     /────────────\
    /  UNIT TESTS  \ ──> 3. Birim (Unit) Testler (%70)
   /────────────────\ Saf fonksiyonları izole test eder. Işık hızında
(milisaniyeler).
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Test ve Kalite Güvencesi (QA) Mimarı
Bir e-ticaret sepet hesaplama fonksiyonu yazdık.

## GÖREV:
Test Piramidi felsefesine uygun olarak bu fonksiyon için 3 katmanda hangi testlerin
yazılması gerektiğini planla:
1. 3 adet kritik Unit Test senaryosu (saf fonksiyon girdileri ve sınır değerler)
2. 1 adet Entegrasyon Testi senaryosu (Veritabanından ürün fiyatı çekip sepeti
hesaplama)
3. 1 adet E2E Test senaryosu (Kullanıcının butona tıklayıp sepet toplamını ekranda
görmesi)
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Test Yazmayan Geliştirici: "Konsolda yazdırdım, çalışıyor!"
function calculateTotal(price: number, quantity: number) {
  return price * quantity;
}
console.log(calculateTotal(10, 2)); // "20 çıktı, tamamdır deploy edelim!"
// Negatif sayı gelirse ne olur? Float yuvarlama ne olur? Bilen yok!
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// Profesyonel Mühendis: Jest ile Kapsamlı Birim Test Paketi (calculator.test.ts)
import { calculateCartTotal } from "./calculator";

describe("calculateCartTotal - Birim Test Paketi", () => {
  it("standart ürünlerde sepet toplamını doğru hesaplamalıdır", () => {
    const items = [{ price: 100, quantity: 2 }, { price: 50, quantity: 1 }];
    expect(calculateCartTotal(items)).toBe(250);
  });

  it("boş sepette 0 döndürmelidir", () => {
    expect(calculateCartTotal([])).toBe(0);
  });

  it("negatif fiyat veya adet girildiğinde RangeError fırlatmalıdır", () => {
    const invalidItems = [{ price: -10, quantity: 1 }];
    expect(() => calculateCartTotal(invalidItems)).toThrow(RangeError);
  });

  it("ondalıklı kuruş tutarlarını 2 haneye doğru yuvarlamalıdır (0.1 + 0.2
hassasiyeti)", () => {
    const items = [{ price: 19.99, quantity: 3 }];
    expect(calculateCartTotal(items)).toBe(59.97);
  });
});
```

💡 **Kod Analizi:** Birim testler; sınır değerlerini (boş sepet, negatif sayılar, ondalık hassasiyet) saniyenin binde birinde test ederek kod tabanına kırılmaz bir güvenlik kalkanı örer.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Sadece mutlu senaryoyu (Happy Path) test edip hata durumlarını atlamak.
  - **Çözüm:** Testlerinizin en az %60'ı 'kullanıcı yanlış girdiğinde ne olur?' sorusuna odaklanmalıdır.
- **Tuzak:** Hiç unit test yazmadan sadece 50 tane aşırı yavaş E2E testi yazmak (Ters Piramit Antipattern).
  - **Çözüm:** Piramidi dik tutun: %70 Unit, %20 Integration, %10 E2E.
- **Tuzak:** Testler hata verdiğinde testi düzeltmek yerine `expect(true).toBe(true)` yazıp geçiştirmek.
  - **Çözüm:** Testi susturmayın; test sizin can simidinizdir, hatayı kaynakta çözün.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

🎥 **Ders Başlığı:** ***Ders 31: Neden Test Yazarız? - Test Piramidinin Gücü***

- 🖥️ **Slayt 1:** ‘Kodum çalışıyor’ yanılgısı ve regresyon riski.
- 🖥️ **Slayt 2:** Test Piramidi: Unit vs Integration vs E2E testler.
- 🖥️ **Slayt 3:** Happy Path vs Edge Case (Sınır Durum) dengesi.
- 🖥️ **Slayt 4:** Canlı Demo: Jest ile 10 milisaniyede 4 test çalıştırmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Terminalde Jest’i izleme modunda (`npm test -- --watch`) açın. Test dosyasında bilinçli olarak bir kuralı değiştirip testin anında kırmızıya döndüğünü, kodu düzeltince saniyesinde yeşile döndüğünü gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bugün yazdığınız bir fonksiyona en az 3 farklı senaryoyu (başarılı durum, boş girdi, hatalı girdi) kapsayan bir Jest veya PyTest dosyası yazın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Yazılım geliştirmede ‘Regresyon Hatası’ (Regression Bug) ne demektir? - A) Yeni bir özellik eklerken veya kodu refactor ederken, daha önce kusursuz çalışan eski bir özelliğin farkında olmadan bozulması - B) Bilgisayarın klavyesinin bozulması - C) Sadece Python’da çıkan söz dizimi hatası - D) İnternet hızının düşmesi > **Doğru Yanıt:** A > **Açıklama:** Regresyon hatası, yeni yapılan bir değişikliğin eski çalışan kodları sessizce kırmasıdır. Otomatik testlerin birincil varoluş sebebi regresyonları anında yakalamaktır.

Soru 2: Test Piramidi’ne göre bir projede en çok hangi test türünden bulunmalıdır? - A) E2E Testler - B) Birim (Unit) Testler (%70 civarında, hızlı ve izole) - C) Manuel Testler - D) Güvenlik Testleri > **Doğru Yanıt:** B > **Açıklama:** Birim testler milisaniyeler içinde çalıştığı ve spesifik mantık hatalarını doğrudan gösterdiği için piramidin en geniş tabanını oluşturur.

---

32. Birim Testlerini AI ile Hızlandırmak: Jest, PyTest ve Mocklama

Gerçek Veritabanına veya API’ye Bağlanmadan Test Yazma Sanatı
Bölüm 6 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Unit test koşarken gerçek veritabanına bağlanmak veya gerçek kredi kartından para çekmeye çalışmak neden bir felakettir ve 'Mock' kavramı sistemi nasıl kurtarır?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Birim testler tamamen deterministik ve dış dünyadan (ağ, disk, saat) yalıtılmış (izole) olmalıdır.
> - 📌 Mocklama (Mocking), dış bağımlılıkları (Stripe API, PostgreSQL, E-posta servisi) sahte ama kontrollü nesnelerle değiştirmektir. - 📌 AI modelleri, karmaşık servislerin mock objelerini ve test tablolarını (fixture) saniyeler içinde üretebilir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Tıp fakültesinde ameliyat eğitimi alan bir cerrah adayını düşünün.

İlk gün profesör öğrencinin eline neşteri verip: *"Hadi sedyedeki gerçek hastanın kalbini aç ve ameliyat et!"* der mi? Tabii ki hayır! Öğrenci önce silikon mankenler ve simülasyon cihazları üzerinde yüzlerce kez dikiş atar.

Yazılımda **Mock (Sahte Obje)** işte o silikon mankendir!

Eğer unit test yazarken gerçek Stripe API'sine gidip gerçek kredi kartından 100 dolar çekmeye kalkarsanız, test her çalıştığında şirket batar! Mock kullanarak Stripe'ın döndüğü cevabı taklit eder ve kodunuzun mantığını güvenle doğrularsınız.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Mocklama Mimarisi:

```
[Test Ortamı (Jest / PyTest)]
              |
              ▼
[Sipariş Servisi (Test Edilen Kod – SUT)]
              |
      ┌───────┴────────┐
      ▼                ▼
[GERÇEK DÜNYA]    [MOCK DÜNYASI]
❌ Canlı DB       ✅ jest.fn().mockResolvedValue({ id: 1, balance: 1000 })
❌ Canlı Stripe   ✅ jest.spyOn(stripe, 'charge').mockResolvedValue({ success: true })
(Çok yavaş,       (Işık hızında, sıfır maliyet, %100 kontrol)
bağlantı kopar)
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Test Mühendisi ve Jest Gurusu
Aşağıdaki ödeme servisi fonksiyonu için Jest birim testleri yazacağız.

## SERVİS KODU:
```typescript
export async function payInvoice(invoiceId: string, stripeClient: StripeSDK, db:
Database) {
  const invoice = await db.findInvoice(invoiceId);
  if (invoice.isPaid) throw new Error("Fatura zaten ödenmiş.");

  const charge = await stripeClient.charges.create({ amount: invoice.amount });
  await db.markAsPaid(invoiceId, charge.id);
  return { success: true, chargeId: charge.id };
}
```

GÖREV:

1. `stripeClient` ve `db` nesnelerini `jest.fn()` kullanarak mockla (sahte nesneler üret).
2. Başarılı ödeme durumunu test et (`markAsPaid` fonksiyonunun doğru argümanlarla çağrıldığını doğrula).
3. Fatura zaten ödendiğinde Stripe'a ASLA çağrı yapılmadığını (`expect(stripeClient.charges.create).not.toHaveBeenCalled()`) kanıtla.
```

````
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Kötü Test: Dış bağımlılıkları mocklamadan gerçek veritabanı beklemek
it("faturayı öder", async () => {
  // Eğer PostgreSQL sunucusu açık değilse test patlar!
  const res = await payInvoice("123", new RealStripe(), new RealDB());
  expect(res.success).toBe(true);
});
````
````

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
import { payInvoice } from "./paymentService";

describe("payInvoice - Birim Test Paketi (Mocklanmış)", () => {
  let mockDb: any;
  let mockStripe: any;

  beforeEach(() => {
    // Her test öncesi temiz mock nesneleri oluşturulur
    mockDb = {
      findInvoice: jest.fn(),
      markAsPaid: jest.fn().mockResolvedValue(true)
    };
    mockStripe = {
      charges: {
        create: jest.fn().mockResolvedValue({ id: "ch_mock_123" })
      }
    };
  });

  it("ödenmemiş faturayı başarıyla tahsil edip veritabanını günceller", async () =>
{
    mockDb.findInvoice.mockResolvedValue({ id: "inv_1", amount: 1500, isPaid:
false });

    const result = await payInvoice("inv_1", mockStripe, mockDb);

    expect(result.success).toBe(true);
    expect(result.chargeId).toBe("ch_mock_123");
    // Yan etkilerin doğrulanması (Assertion of side-effects)
    expect(mockStripe.charges.create).toHaveBeenCalledWith({ amount: 1500 });
    expect(mockDb.markAsPaid).toHaveBeenCalledWith("inv_1", "ch_mock_123");
  });

  it("zaten ödenmiş faturada Stripe çağrısı yapmadan hata fırlatmalıdır", async ()
=> {
    mockDb.findInvoice.mockResolvedValue({ id: "inv_2", amount: 500, isPaid:
true });

    await expect(payInvoice("inv_2", mockStripe, mockDb)).rejects.toThrow("Fatura
zaten ödenmiş.");

    // GÜVENLİK KONTROLÜ: Stripe asla çağrılmamalıdır!
    expect(mockStripe.charges.create).not.toHaveBeenCalled();
    expect(mockDb.markAsPaid).not.toHaveBeenCalled();
  });
});
```

💡 **Kod Analizi:** Bu testler ne internet bağlantısına ne de canlı bir veritabanına ihtiyaç duyar. 1000 adet böyle test toplam 1.5 saniyede koşar ve tüm iş kurallarını %100 doğrular.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Testler arasında mock durumlarını sıfırlamayı unutup testlerin birbirini kirletmesi (Flaky Tests).
  - **Çözüm:** Her testten önce `beforeEach(() => jest.clearAllMocks())` çalıştırın.
- **Tuzak:** Test edilen kodun mantığını değil, mock nesnesinin kendisini test etmek.
  - **Çözüm:** Mock sadece girdiyi sağlar; test ettiğiniz şey kendi yazdığınız servis fonksiyonudur.
- **Tuzak:** Mock kütüphanesinin gerçek API ile uyumsuz hale gelmesi (Stale Mock).
  - **Çözüm:** TypeScript interface'leri kullanarak mock nesnelerin gerçek tipleri implemente etmesini zorunlu kılın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 32: Birim Testleri & Mocklama - Jest ve PyTest Ustalığı*

- 🖥️ **Slayt 1:** Determinizm kuralı: Unit test neden dış dünyadan izole olmalıdır?
- 🖥️ **Slayt 2:** Mock, Stub ve Spy kavramları arasındaki farklar.
- 🖥️ **Slayt 3:** Jest ile bağımlılık enjeksiyonunu (DI) mocklamak.
- 🖥️ **Slayt 4:** Canlı Demo: 1000 testi 2 saniyede koşan mock altyapısı kurmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir hava durumu API'si çağıran fonksiyon yazın. İnterneti kapatın (offline moda geçin). Jest ile API yanıtını mocklayarak testin internet olmadan 5 milisaniyede geçtiğini canlı konsolda gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde harici bir API (hava durumu, ödeme veya e-posta) çağıran bir servisi bulun ve Jest mock (`jest.fn()`) kullanarak izole bir test yazın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Birim testlerde 'Mock' kullanmanın birincil teknik gerekçesi nedir? - A) Kodun satır sayısını artırmak - B) Ağ, veritabanı veya üçüncü taraf API gibi yavaş, maliyetli ve güvenilmez dış bağımlılıkları sahte ama kontrollü nesnelerle ikame ederek testleri hızlı ve deterministik kılmak - C) Sadece Python dilinde geçerli bir zorunluluk olması - D) CSS hatalarını engellemek > **Doğru Yanıt:** B > **Açıklama:** Mock nesneler dış bağımlılıkları yalıtır; testlerin internet veya veritabanı olmadan mikrosaniyeler içinde tutarlı çalışmasını sağlar.

Soru 2: Jest testlerinde `expect(mockFunction).not.toHaveBeenCalled()` assertion'ı neyi doğrular? - A) Fonksiyonun çöktüğünü - B) Belirtilen sahte fonksiyonun test senaryosu boyunca hiçbir koşulda tetiklenmediğini (istenmeyen yan etkinin oluşmadığını) - C) Bilgisayarın kapandığını - D) Testin başarısız sayılması gerektiğini > **Doğru Yanıt:** B > **Açıklama:** Bu ifade, istenmeyen durumlarda (örn. geçersiz şifre veya ödenmiş fatura) kritik fonksiyonların (örn. para çekme) asla çağrılmadığını garanti altına alır.

33. Entegrasyon ve E2E Testleri: Playwright ile Uçtan Uca Doğrulama

Tüm Parçaların Bir Araya Geldiğinde Doğru Çalıştığını Kanıtlamak
Bölüm 6 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Tüm unit testler yeşil yanarken (başarılıyken) bir web sitesinde 'Giriş Yap' butonu tıklandığında neden hiçbir şey olmaz? Entegrasyon testleri neden bu kör noktayı çözer?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 200 adet mükemmel çalışan birim test, iki modülün birbiriyle konuştuğu sınırda (interface) hata olmayacağını garanti etmez. - 📌 Playwright, modern web uygulamalarını gerçek Chromium, Firefox ve WebKit tarayıcılarında headless olarak test eden altın standarttır. - 📌 E2E testleri sadece en kritik 'kullanıcı yolculuklarına' (User Journey - Kayıt, Ödeme, Giriş) uygulanmalıdır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

İki mühendisin harika bir ev inşa ettiğini düşünün:

Biri dünyanın en kaliteli kapı kilidini üretti (Unit Test %100 başarılı). Diğeri ise dünyanın en sağlam ahşap kapısını yaptı (Unit Test %100 başarılı).

Fakat kapıyı yerine monte ettiklerinde bir baktılar: Kilit solda, kilit deliği sağda! Kapı kilitlenemiyor! İşte bu bir **Entegrasyon Hatasıdır**.

Birim testler parçaların tek başına sağlam olduğunu gösterir. **Entegrasyon ve E2E (Playwright) testleri** ise kapıyı duvara takıp anahtarı çeviren, kolu indiren ve kapının gerçekten kilitlendiğini gören nihai testtir.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Test Kapsamının Gerçeklik Seviyesi:

```
[Birim Test (Jest)]          —> "Fonksiyon matematiksel olarak doğru mu?"
            |
            ▼
[Entegrasyon Testi (Supertest)] —> "Express rotası veritabanına gerçekten yazıyor
mu?"
            |
            ▼
[E2E Testi (Playwright)]       —> "Kullanıcı tarayıcıda formu doldurup tıkladığında
başarı mesajını görüyor mu?"
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: E2E Test Otomasyon Mühendisi
Playwright ve TypeScript kullanarak bir e-ticaret sitesinin 'Kullanıcı Girişi (Login
Flow)' için E2E testi yazacağız.

## GEREKSİNİMLER:
1. `http://localhost:3000/login` sayfasına git.
2. E-posta ve şifre inputlarını doldur.
3. 'Giriş Yap' butonuna tıkla.
4. Başarılı girişte `/dashboard` sayfasına yönlendirildiğini ve sayfada 'Hoş geldin
Ali' yazısının belirdiğini doğrula.
5. Hatalı şifre girildiğinde ekranda beliren kırmızı hata kutusunu test et.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ **Acemi / Kontrolsüz AI Çıktısı (Riskli):**

```
// Kötü E2E Test: Kırılgan CSS seçicileri ve sabit bekleme (sleep) tuzağı
test("login naive", async ({ page }) => {
  await page.goto("http://localhost:3000/login");
  await page.click(".btn-primary-2"); // Kırılgan CSS! Buton stili değişirse test
çöker!
  await page.waitForTimeout(5000); // KÖTÜ: 5 saniye boşuna beklemek testi
yavaşlatır!
});
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
import { test, expect } from "@playwright/test";

test.describe("Kullanıcı Giriş Akışı (Login Flow)", () => {
  test.beforeEach(async ({ page }) => {
    await page.goto("/login");
  });

  test("geçerli bilgilerle giriş yapıldığında dashboard'a yönlendirmelidir", async
({ page }) => {
    // Erişilebilir (Accessible) Rol ve Label Seçicileri (Sağlam & Kırılmaz)
    await page.getByLabel("E-Posta Adresi").fill("ali@example.com");
    await page.getByLabel("Şifre").fill("GuvenliSifre123!");
    await page.getByRole("button", { name: "Giriş Yap" }).click();

    // Otomatik Bekleme (Auto-waiting): URL ve içerik gelene kadar dinamik bekler
    await expect(page).toHaveURL(/.*dashboard/);
    await expect(page.getByRole("heading", { name: "Hoş Geldin
Ali" })).toBeVisible();
  });

  test("hatalı şifrede açıklayıcı hata mesajı göstermelidir", async ({ page }) => {
    await page.getByLabel("E-Posta Adresi").fill("ali@example.com");
    await page.getByLabel("Şifre").fill("YanlisSifre!");
    await page.getByRole("button", { name: "Giriş Yap" }).click();

    const alert = page.getByRole("alert");
    await expect(alert).toBeVisible();
    await expect(alert).toContainText("Geçersiz e-posta veya şifre.");
  });
});
```

💡 **Kod Analizi:** `getByLabel` ve `getByRole` gibi erişilebilirlik (ARIA) seçicileri kullanılarak UI değişikliklerine karşı dayanıklılık sağlanmış; `page.waitForTimeout` yerine Playwright'ın dahili 'Auto-Waiting' özelliğiyle testler maksimum hızda çalıştırılmıştır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Testlerde `page.waitForTimeout(3000)` (sabit uyku) kullanıp test koşu sürelerini uzatmak.
  - **Çözüm:** Sabit sleep yerine Playwright'ın otomatik bekleyen `expect(locator).toBeVisible()` metodlarını kullanın.
- **Tuzak:** Kırılgan CSS seçicileri (`div > div.col-md-4 > button`) kullanmak.
  - **Çözüm:** Daima kullanıcı odaklı ARIA seçicileri (`getByRole` , `getByText`) veya `data-testid` kullanın.
- **Tuzak:** Her küçük buton için E2E testi yazıp CI pipeline süresini 45 dakikaya çıkarmak.
  - **Çözüm:** E2E testlerini sadece kritik kullanıcı akışları için yazın; detayları Unit testlere bırakın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 33: Uçtan Uca Testler - Playwright ile Tarayıcı Otomasyonu*

- 🖥️ **Slayt 1:** Unit testlerin göremediği entegrasyon kör noktaları.
- 🖥️ **Slayt 2:** Playwright mimarisi: Çoklu tarayıcı (Chromium, WebKit, Firefox).
- 🖥️ **Slayt 3:** Kırılmaz seçiciler: ARIA Rolleri ve data-testid kullanımı.
- 🖥️ **Slayt 4:** Canlı Demo: Playwright UI Mode ve Trace Viewer ile hata ayıklama.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Terminalde `npx playwright test --ui` komutunu çalıştırın. Playwright UI penceresinde testin tarayıcıyı nasıl açıp formu doldurduğunu, zaman çizelgesinde (timeline) her adımın ekran görüntüsünü nasıl kaydettiğini canlı gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenize Playwright ekleyin (`npm init playwright@latest`) ve ana sayfanızın açılıp başlığının doğru göründüğünü doğrulayan ilk E2E testinizi çalıştırın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: E2E testlerinde `page.locator('button.btn-blue-submit')` yerine `page.getByRole('button', { name: 'Giriş Yap' })` tercih edilmesinin sebebi nedir? - A) Daha az harf içermesi - B) CSS sınıfları değişse bile kullanıcının gördüğü metin ve erişilebilirlik (ARIA) rolü değişmediği sürece testin kırılmadan çalışmaya devam etmesi - C) Sadece Safari'de çalışması - D)

Bilgisayarın sesini açması > **Doğru Yanıt:** B > **Açıklama:** Kullanıcı odaklı (User-Facing) seçiciler, frontend tasarım revizyonlarından etkilenmez ve testleri son derece sağlam (resilient) kılar.

Soru 2: Playwright'ın 'Trace Viewer' aracının yazılımcılara sağladığı en büyük süper güç nedir? - A) Bedava kahve ısmarlaması - B) Testin koştuğu her milisaniyenin ekran görüntüsünü, konsol loglarını ve ağ (network) isteklerini zaman makinesi gibi geriye sararak inceleme imkanı vermesi - C) Kodları otomatik silmesi - D) Sadece Windows'ta çalışması > **Doğru Yanıt:** B > **Açıklama:** Trace Viewer, özellikle CI ortamında çöken E2E testlerinin tam olarak hangi saniyede ve hangi ağ hatasıyla patladığını video gibi incelemeyi sağlar.

34. Test Driven Development (TDD) + AI Döngüsü: Red-Green-Refactor

Yapay Zekanın Halüsinasyon Görmesini Engelleyen Nihai Geliştirme Ritmi
Bölüm 6 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Yapay zeka çağında TDD (Test-Driven Development) öldü mü, yoksa tam aksine AI'ı kontrol altında tutmanın tek bilimsel yolu haline mi geldi?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 TDD öldü söylemi yanlıştır; AI ile kod yazarken önce testi yazdırmak, AI'ın sınırlarını çizen en katı şartnamedir. - 📌 Red-Green-Refactor Döngüsü: 1. Başarısız test yaz (Kırmızı), 2. AI'a testi geçecek minimum kodu yazdır (Yeşil), 3. Kodu temizle (Refactor). - 📌 Bu döngüde yapay zeka sadece bir uygulayıcıdır; mimari kısıtları ve doğrulama kriterlerini belirleyen mühendistir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir okçuluk yarışmasını hayal edin.

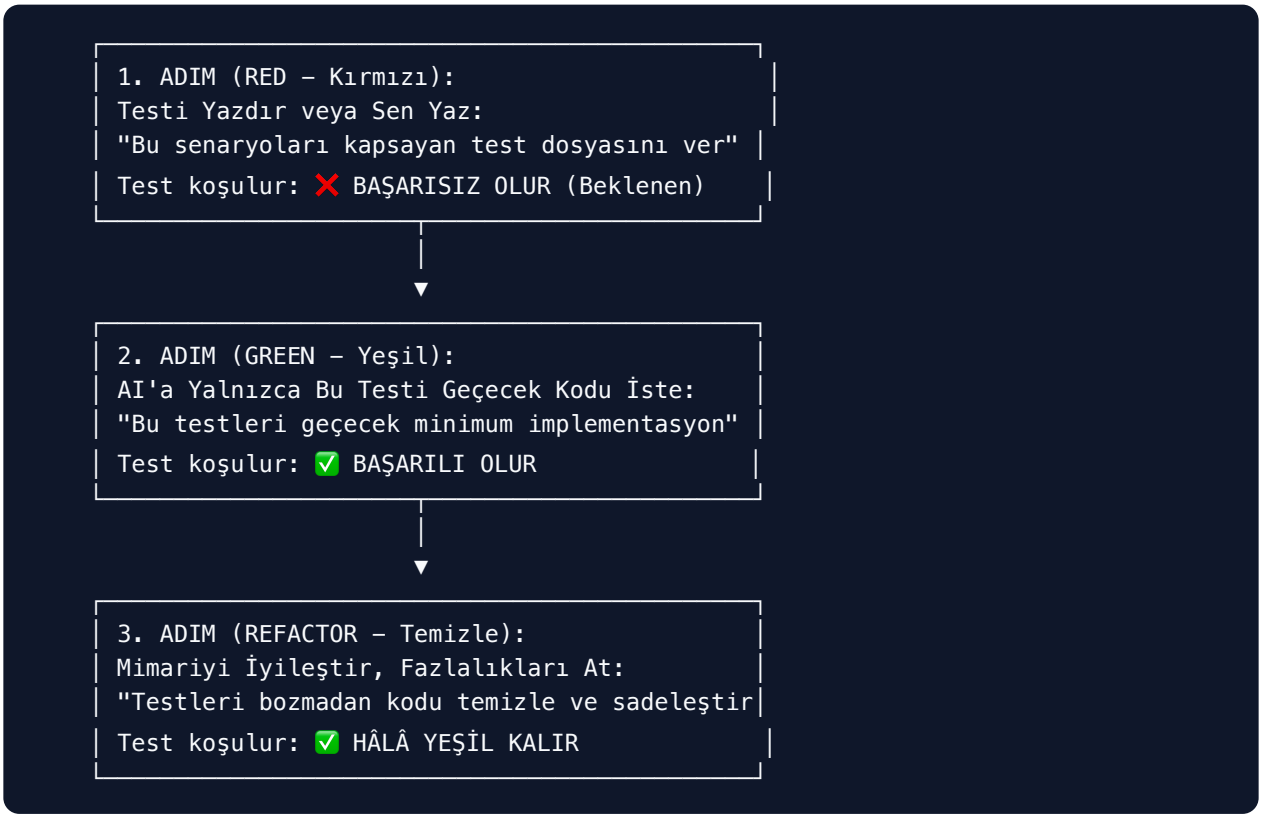
Geleneksel kötü yöntem: Oku rastgele ormana doğru fırlatırsınız. Okun düştüğü ağacın yanına gidip etrafına kırmızı bir hedef tahtası çizersiniz! *"Bakın tam hedeften vurdum!"* dersiniz. Komik, değil mi? Test yazmadan AI'a kod yazdırmak tam olarak budur.

TDD (Test-Driven Development) ise gerçek okçuluktur: 1. Önce 100 metre ileriye hedef tahtasını dikersiniz: **RED (Kırmızı)**. Henüz ok atılmadı, hedef boş. 2. Ardından oku tam merkeze fırlatırsınız: **GREEN (Yeşil)**. Test başarıyla geçti! 3. Duruşunuzu ve fiyonkları düzeltirsiniz: **REFACTOR (İyileştirme)**.

Yapay zekaya önce hedef tahtasını (testi) göstermezseniz, okun nereye gideceğini asla tahmin edemezsiniz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

AI Destekli TDD Döngüsü:



🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: TDD ve Temiz Kod Koçu
Bir 'İndirim Kuponu Doğrulama ve Uygulama' motoru geliştireceğiz.

## ADIM 1: RED (ÖNCE TEST)
Lütfen bana fonksiyonun kendisini YAZMA.
Sadece aşağıdaki gereksinimleri test eden Jest test dosyasını oluştur:
1. Kupon tarihi geçmişse `ExpiredCouponError` fırlatmalı.
2. Sepet tutarı minimum sepet limitinin altındaysa `MinimumSpendNotMetError`
fırlatmalı.
3. Yüzdelik indirimlerde (%20 indirim) tutarı doğru düşmeli.
4. Maksimum indirim tavanı varsa (örn: max 100 TL) indirimi o tutarla
sınırlandırmalı.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Geleneksel Hatalı Akış: Önce karmaşık kod yazıp sonra test yazmaya üşenmek
// Sonuç: Testi olmayan, sınır durumları belirsiz, refactor edilemeyen kod!
export function applyCouponBlind(cart: any, coupon: any) {
  return cart.total - coupon.amount;
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// TDD Yeşil Adımı: Testleri geçen temiz ve sağlam implementasyon
export interface Cart {
  total: number;
}

export interface Coupon {
  code: string;
  discountPercent: number;
  maxDiscountAmount: number;
  minSpend: number;
  expiresAt: Date;
}

export function applyPromotionalCoupon(cart: Cart, coupon: Coupon): number {
  if (new Date() > coupon.expiresAt) {
    throw new Error("ExpiredCouponError: Kuponun süresi dolmuş.");
  }

  if (cart.total < coupon.minSpend) {
    throw new Error("MinimumSpendNotMetError: Minimum sepet tutarı karşılanmadı.");
  }

  const calculatedDiscount = (cart.total * coupon.discountPercent) / 100;
  const actualDiscount = Math.min(calculatedDiscount, coupon.maxDiscountAmount);

  return Math.round((cart.total - actualDiscount) * 100) / 100;
}
```

💡 **Kod Analizi:** TDD yaklaşımında önce test yazıldığı için kupon tavan sınırı (`Math.min`), tarih kontrolü ve minimum tutar kuralı koda daha baştan defansif olarak entegre edilmiş olur.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** AI'a hem testi hem kodu aynı anda yazdırıp testin çalıştığını doğrulamamak.
  - **Çözüm:** Önce testi yazdırın, testin kırmızı yandığını (hata verdiğini) gözlerinizle görün, ardından kodu yazdırın.
- **Tuzak:** Refactor aşamasını atlayıp 'nasılsa yeşil yandı' diyerek çirkin kodu öylece bırakmak.
  - **Çözüm:** Yeşil yandıktan sonra kodu temizleyin; testleriniz yeşil kaldığı sürece korkusuzca düzenleyin.
- **Tuzak:** Test senaryolarını çok geniş ve belirsiz tutmak.
  - **Çözüm:** Her test fonksiyonunda sadece tek bir iş kuralını (`it should...`) test edin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 34: AI ile TDD - Red, Green, Refactor Disiplini*

- 🖥️ **Slayt 1:** Neden TDD yapay zekayı evcilleştirmenin tek kesin yoludur?
- 🖥️ **Slayt 2:** 3 Adımlı Ritim: Red (Kırmızı), Green (Yeşil), Refactor (İyileştirme).
- 🖥️ **Slayt 3:** AI'a önce test yazdırmanın yarattığı kusursuz şartname etkisi.
- 🖥️ **Slayt 4:** Canlı Demo: 5 dakikada TDD ile kupon motoru inşa etmek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Boş bir dosya açın. AI'a kupon testlerini yazdırın. Terminalde `npm test` çalıştırıp testin kırmızı yandığını (RED) gösterin. Ardından kodu üretip testin yeşile döndüğünü (GREEN) ve refactor sonrası yine yeşil kaldığını gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Geliştireceğiniz bir sonraki özellik için önce testi yazın veya AI'a sadece test dosyasını yazdırın. Testin başarısız olduğunu gördükten sonra kodu üretin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: TDD (Test-Driven Development) döngüsünün ilk adımı (RED) neden testin başarısız olduğunu görmeyi zorunlu kılar? - A) Geliştiricinin moralini bozmak için - B) Testin gerçekten çalıştığını, sahte (her zaman geçen) bir test olmadığını ve henüz yazılmamış özelliği doğru denetlediğini kanıtlamak için - C) Bilgisayarı test etmek için - D) Sadece C++ dilinde zorunlu olduğu için > **Doğru Yanıt:** B > **Açıklama:** Eğer test henüz kod yazılmadan önce yeşil yanıyorsa, o test hatalıdır veya hiçbir şey test etmiyordur. Kırmızı adımını görmek zorunludur.

Soru 2: TDD'nin üçüncü aşaması olan 'Refactor' adımında ne yapılır? - A) Proje silinir - B) Testlerin yeşil kalması güvencesi altında kod temizlenir, optimize edilir ve mimari olarak iyileştirilir - C) Yeni özellikler eklenir - D) Testler silinir > **Doğru Yanıt:** B > **Açıklama:** Refactor, çalışan sistemin dış davranışını değiştirmeden kod kalitesini, okunabilirliğini ve mimarisini iyileştirme aşamasıdır.

---

35. Edge Case (Uç Durum) Avcılığı: AI ile Olmayacak Senaryoları Bulmak

29 Şubat, Artık Saniyeler, Boş Stringler ve Sonsuz Sayılarla Sistemi Kırmak
Bölüm 6 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 12 dk

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Dünyanın en iyi yazılımcılarının yazdığı kodlar bile neden 29 Şubat'ta veya sıfıra bölme (division by zero) durumlarında çöker ve AI bu kör noktaları nasıl yakalar?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Kullanıcılar asla sizin düşündüğünüz gibi davranmaz; beklenmedik girdiler sistemin gerçek stres testidir. - 📌 Edge Case (Uç Durum): Boş stringler, maksimum integer limitleri, farklı zaman dilimleri (Timezones) ve özel karakterlerdir. - 📌 AI modellerine 'Bu sistemi patlatacak 10 anormal sınır durum üret' demek, en ucuz ve etkili güvenlik kalkanıdır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir bankomat (ATM) cihazı düşünün.

Mühendisler ATM'yi test ediyor: - Kartı sok, şifreyi gir, 100 TL çek -> Başarılı! - 500 TL çek -> Başarılı!

Peki sarhoş bir müşteri gelip ATM'ye: - Eksi 100 TL (-100 TL) çekmek isterse ne olur? (Yazılım eksi parayı çıkaramayacağı için kullanıcının hesabına para ekleyebilir!) - Kart yuvasına bozuk para veya kağıt mendil tıkarsa? - Aynı anda hem interneti kesip hem para çekme tuşuna basarsa?

İşte buna **Edge Case (Uç Durum)** denir. Acemi geliştirici sadece güneşli havaları düşünür. Kıdemli mühendis ise fırtınada çatının uçup uçmayacağını test eder.

🧠 2. AI Çağında Yeni Zihniyet Modeli

En Meşhur 5 Edge Case Kategorisi:

```
1. SAYISAL SINIRLAR (Numeric Limits):
   - Sıfır (0), Negatif Sayılar (-1), Ondalık Yuvarlama (0.1 + 0.2),
MAX_SAFE_INTEGER.

2. METİN VE KARAKTERLER (Strings & Unicode):
   - Boş string (""), Sadece boşluk ("   "), Null/Undefined, Emojiler (🤖), SQL/HTML
etiketleri.

3. ZAMAN VE TARİH (Temporal & Clocks):
   - 29 Şubat (Artık yıl), Gün ışığından yararlanma (Saatlerin 1 saat geri/ileri
alınması), UTC vs Yerel Saat.

4. KOLEKSİYONLAR (Collections & Arrays):
   - Boş liste ([]), Tek elemanlı liste ([1]), 1.000.000 elemanlı liste.

5. EŞZAMANLILIK (Concurrency & Race Conditions):
   - Aynı butona 10 milisaniye içinde 2 kez çift tıklama!
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: Kaos Mühendisi ve Uç Durum (Edge Case) Avcısı
Aşağıdaki kullanıcı yaşını ve doğum gününü doğrulayan fonksiyonu incele.

## KOD:
```typescript
function isEligibleToVote(birthDateStr: string): boolean {
  const birthYear = new Date(birthDateStr).getFullYear();
  const currentYear = new Date().getFullYear();
  return (currentYear - birthYear) >= 18;
}
````

GÖREV:

1. Bu koddaki en az 4 ölümcül sınır durumu (edge case) tespit et (Örn: Ay ve gün hesabı, 29 Şubat, geçersiz tarih formatı, gelecekteki tarihler).
2. Bu sınır durumları patlatacak Jest test senaryolarını yaz.
3. Fonksiyonu tüm bu uç durumlara karşı kurşun geçirmez şekilde refactor et.
````

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Kötü Kod: Sadece yılı çıkarır; ay ve günü hesaba katmaz!
// Örnek: Bugün Ocak 2026 ise ve kullanıcı Aralık 2008 doğumluysa (henüz 17
yaşındayken) 18 kabul eder!
export function isEligibleToVoteNaive(birthDateStr: string): boolean {
  return (new Date().getFullYear() - new Date(birthDateStr).getFullYear()) >= 18;
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```typescript
export function isEligibleToVoteRobust(
  birthDateStr: string,
  now: Date = new Date()
): boolean {
  const birthDate = new Date(birthDateStr);

  // 1. Uç Durum: Geçersiz tarih formatı (NaN)
  if (isNaN(birthDate.getTime())) {
    throw new TypeError("Geçersiz tarih formatı girildi.");
  }

  // 2. Uç Durum: Gelecekte bir doğum tarihi girilemez
  if (birthDate > now) {
    throw new RangeError("Doğum tarihi bugünden ileri bir tarih olamaz.");
  }

  // 3. Kesin Yaş Hesabı (Ay ve Gün Hassasiyeti)
  let age = now.getFullYear() - birthDate.getFullYear();
  const monthDiff = now.getMonth() - birthDate.getMonth();

  if (monthDiff < 0 || (monthDiff === 0 && now.getDate() < birthDate.getDate())) {
    age--; // Doğum günü bu yıl henüz gelmedi!
  }

  // 4. Uç Durum: 130 yaşından büyük mantıksız değerler
  if (age > 130) {
    throw new RangeError("Doğrulanamaz yaş değeri.");
  }

  return age >= 18;
}
```

💡 **Kod Analizi:** Kıdemli sürüm; geçersiz tarihleri (`NaN`), henüz günü gelmemiş ayları, gelecekteki doğum tarihlerini ve mantıksız sınırları tek tek kontrol ederek çökmeleri engeller.
```

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Kullanıcının her zaman doğru formatta veri gireceğini varsaymak.
  - **Çözüm:** Girdileri daima şüpheyle karşılayın; 'Boş gelirse ne olur?', 'Çok büyük gelirse ne olur?' diye sorun.
- **Tuzak:** Tarih hesaplamalarında `getFullYear() - getFullYear()` yapıp gün farkını unutmak.
  - **Çözüm:** Tarih karşılaştırmalarında gün ve saat hassasiyetini veya güvenilir kütüphaneleri (date-fns) kullanın.
- **Tuzak:** Sayısal işlemlerde sıfıra bölme (division by zero) kontrolü yapmamak.
  - **Çözüm:** Her bölme işleminden önce paydanın sıfır olmadığını teyit edin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 35: Edge Case Avcılığı - AI ile Olmayacak Senaryoları Bulmak*

- 🖥️ **Slayt 1:** Neden en büyük sistem çöküşleri Edge Case'lerden kaynaklanır?
- 🖥️ **Slayt 2:** 5 Temel Sınır Durum Kategorisi (Sayılar, Metinler, Tarihler, Listeler, Concurrency).
- 🖥️ **Slayt 3:** AI'ı 'Kötü Niyetli Hacker' rolünde sınır testleri için kullanmak.
- 🖥️ **Slayt 4:** Canlı Demo: Hatalı yaş hesaplama fonksiyonunu kırmak ve onarmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Basit bir metin kısaltma (truncate) fonksiyonunu AI'a verin. 'Bu fonksiyonu çökertecek 5 sınır durum testi yaz' deyin. AI'ın boş string, 100.000 karakterlik metin ve çoklu bayt emoji testlerini nasıl ürettiğini ve kodun patladığını canlı gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki en kritik bir fonksiyonu seçin. AI'a 'Bu fonksiyonu production'da çökertecek 5 çılgın edge case üret' deyin ve çıkan testleri çalıştırın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: JavaScript/TypeScript'te `0.1 + 0.2 === 0.3` ifadesinin sonucu neden `false` döner? - A) JavaScript bozuk bir dildir - B) Sayılar ikili (binary floating-point) tabanda temsil edildiği için 0.3 yerine 0.30000000000000004 hassasiyet kaybı oluşur - C) Sadece Google Chrome'da öyle olur - D) Sonuç true döner > **Doğru Yanıt:** B > **Açıklama:** IEEE 754 kayan noktalı standartta ikili taban kesirli sayıları tam temsil edemez; bu nedenle parasal işlemlerde kuruş yuvarlama kontrolleri şarttır.

Soru 2: Bir fonksiyonda 'Sıfıra Bölme' (Division by Zero) kontrolü yapılmadığında çoğu dilde ne gerçekleşir? - A) Kod daha hızlı çalışır - B) `Infinity` döner veya `ZeroDivisionError` fırlatılarak program anında çöker - C) Bilgisayarın ekranı kararır - D) Sayı otomatik olarak 1 olur > **Doğru Yanıt:** B > **Açıklama:** Sıfıra bölme matematiksel olarak tanımsızdır; diller ya çalışma zamanı hatası fırlatır ya da beklenmedik `Infinity` / `NaN` üreterek sistemi bozar.

36. Kod Kalite Metrikleri: SonarQube, Linter'lar ve Statik Analiz

Teknik Borç Faizini Ödemeden Sistemi Sürdürülebilir Kılmak
Bölüm 6 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

🎙️ **NotebookLM & Audio Overview Podcast Notları**

Podcast Tartışma Sorusu:
"Teknik borç (Technical Debt) nedir ve bir startup'ın ilk 6 ayında hızlı olmak için aldığı borçlar 2. yılda şirketi nasıl iflasa sürükler?"

Bölümden Çıkarılacak 3 Altın Cümle:
- 📌 Teknik borç, hızlı teslimat yapmak için kaliteden ödün vermektir; zamanında ödenmezse faizi tüm ekibi felç eder. - 📌 Statik analiz (ESLint, Biome, SonarQube), kodu çalıştırmadan potansiyel güvenlik açıklarını ve karmaşıklığı tespit eder. - 📌 Siklomotik Karmaşıklık (Cyclomatic Complexity), bir fonksiyondaki karar yollarının sayısıdır; 10'un üzerinde olması tehlike çanıdır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Kredi kartıyla alışveriş yaptığınızı düşünün.

Kredi kartı harika bir kolaylıktır: Paranız yokken bile istediğiniz ayakkabıyı alırsınız (Hızlı kod yazıp teslim etmek). Ancak her ay asgari tutarı bile ödemezseniz ne olur? Faiz biner, faizin faizi biner ve sonunda haciz gelir!

Yazılımda **Teknik Borç (Technical Debt)** işte o kredi kartıdır: - “Şimdilik test yazmayalım, sonra bakarız.” - “Buradaki if-else’i geçici yazdım, haftaya düzeltirim.” - “Kütüphane deprecated oldu ama şimdilik çalışıyor.”

1 yıl sonra bir butonu değiştirmek 3 hafta sürer hale gelir çünkü kimse o borç batağına dokunamaz!

Statik analiz ve linter araçları, her gün borcunuzu ödemenizi hatırlatan akıllı finans danışmanınızdır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Teknik Borç ve Kod Kalite Piramidi:

```
[KOD YAZILIRKEN]
       |
       ▼
[1. Linter & Formatlayıcı (Biome / ESLint / Prettier)]
   └── Söz dizimi hatalarını, kullanılmayan değişkenleri anında yakalar.
       |
       ▼
[2. Tip Kontrolü (TypeScript / MyPy)]
   └── Tip güvenliğini ve şema uyumunu garanti eder.
       |
       ▼
[3. Karmaşıklık ve Koku Analizi (SonarQube / Code Climate)]
   └── Siklomotik karmaşıklığı, mükerrer kodları (duplicate) ölçer.
       |
       ▼
[TEMİZ & SÜRDÜRÜLEBİLİR ÜRÜN]
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Kod Kalitesi ve Statik Analiz Uzmanı
Aşağıdaki yüksek karmaşıklığa sahip (Cyclomatic Complexity > 12) JavaScript
fonksiyonunu incele.

## KOD:
```typescript
function calculateUserTariff(user: any) {
  let rate = 10;
  if (user.isStudent) {
    if (user.age < 25) {
      rate -= 3;
    } else {
      rate -= 1;
    }
  } else if (user.isSenior) {
    if (user.hasCard) {
      rate -= 4;
    } else {
      rate -= 2;
    }
  } else {
    if (user.isVip) {
      rate -= 5;
    }
  }
  return rate;
}
```

GÖREV:

1. Bu koddaki Siklomotik Karmaşıklığı (Cyclomatic Complexity) analiz et.
2. 'Lookup Table' veya 'Kural Motoru (Rule Engine)' deseni kullanarak karmaşıklığı 2'nin altına düşüren temiz refactor'ü yap.
3. ESLint için karmaşıklık sınırını zorunlu kılan kural önerisini sun.
```

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Yüksek Siklomotik Karmaşıklık: 6 adet iç içe if-else ve 'any' kullanımı
export function calculateTariffNaive(user: any): number {
  if (user.isStudent && user.age < 25) return 7;
  if (user.isStudent && user.age >= 25) return 9;
  if (user.isSenior && user.hasCard) return 6;
  if (user.isSenior && !user.hasCard) return 8;
  if (user.isVip) return 5;
  return 10;
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```typescript
// Deklaratif ve Sıfır Karmaşıklık: Kural Listesi Deseni
export interface TariffUser {
  isStudent?: boolean;
  age?: number;
  isSenior?: boolean;
  hasCard?: boolean;
  isVip?: boolean;
}

type TariffRule = {
  predicate: (user: TariffUser) => boolean;
  discount: number;
};

const TARIFF_RULES: readonly TariffRule[] = [
  { predicate: (u) => Boolean(u.isVip), discount: 5 },
  { predicate: (u) => Boolean(u.isSenior && u.hasCard), discount: 4 },
  { predicate: (u) => Boolean(u.isStudent && (u.age ?? 0) < 25), discount: 3 },
  { predicate: (u) => Boolean(u.isSenior && !u.hasCard), discount: 2 },
  { predicate: (u) => Boolean(u.isStudent && (u.age ?? 0) >= 25), discount: 1 },
];

const BASE_RATE = 10;

export function calculateUserTariff(user: TariffUser): number {
  const matchedRule = TARIFF_RULES.find((rule) => rule.predicate(user));
  return BASE_RATE - (matchedRule?.discount ?? 0);
}
```

💡 **Kod Analizi:** İç içe if-else blokları tamamen ortadan kaldırılarak deklaratif bir kural listesine (`TARIFF_RULES`) dönüştürülmüştür. Fonksiyonun karmaşıklığı 1'e düşürülmüş ve yeni bir tarife kuralı eklemek tek bir satır eklemeye indirgenmiştir.
```

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Linter uyarılarını (warnings) önemsemeyip yüzlerce uyarının birikmesine izin vermek.
  - **Çözüm:** CI pipeline'ında uyarıları hata kabul eden kuralı (`--max-warnings 0`) devreye alın.
- **Tuzak:** Teknik borcu tamamen görmezden gelip her sprintte sadece yeni özellik yazmaya çalışmak.
  - **Çözüm:** Her sprint kapasitenizin en az %15-20'sini teknik borç temizliğine ve refactor'e ayırın.
- **Tuzak:** Linter kurallarını çok katı yapıp ekibi kod yazamaz hale getirmek.
  - **Çözüm:** Pragmatik olun; kuralları takım kararıyla belirleyin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 36: Kod Kalite Metrikleri - Teknik Borcu Kontrol Altına Almak*

- 🖥️ **Slayt 1:** Teknik Borç (Technical Debt) nedir ve bileşik faizi nasıl işler?
- 🖥️ **Slayt 2:** Siklomotik Karmaşıklık (Cyclomatic Complexity) nasıl ölçülür?
- 🖥️ **Slayt 3:** Biome ve ESLint ile otomatik kod hijyeni kurmak.
- 🖥️ **Slayt 4:** Bölüm 6 Özeti: Test Odaklı Geliştirme ve Kalite Güvencesi.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Karmaşık iç içe if-else içeren bir dosyada ESLint karmaşıklık eklentisini çalıştırıp uyarının yandığını gösterin. Kural tablosuna refactor ettikten sonra uyarının söndüğünü canlı gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde `eslint` veya `biome` çalıştırın. En karmaşık veya en çok uyarı veren dosyayı tespit edip en az 2 uyarısını temizleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Yazılım mühendisliğinde 'Siklomotik Karmaşıklık' (Cyclomatic Complexity) neyi ölçer? - A) Kodun kaç megabayt yer kapladığını - B) Bir fonksiyondaki bağımsız kod yürütme yollarının (if, else, while vb. dallanmaların) sayısını - C) Bilgisayarın ekran parlaklığını - D) GitHub'daki yıldız sayısını > **Doğru Yanıt:** B > **Açıklama:** Siklomotik karmaşıklık, koddaki mantıksal dallanma sayısını ölçer. Sayı ne kadar yüksekse kodu test etmek ve anlamak o kadar zorlaşır.

Soru 2: Teknik borcun (Technical Debt) zamanında ödenmemesinin ekipler üzerindeki en yıkıcı sonucu nedir? - A) Elektrik faturasının artması - B) Basit bir değişikliğin bile haftalar sürmesi, her güncellemede başka yerlerin kırılması ve geliştirici üretkenliğinin sıfıra yaklaşması - C) Sadece Python'da hata çıkması - D) Monitörün kapanması > **Doğru Yanıt:** B > **Açıklama:** Ödenmeyen teknik borç, kod tabanını kırıntı haline getirir; yeni özellik ekleme hızı çöker ve şirket teknik iflasa sürüklenir.

Bölüm 7: Hata Ayıklama (Debugging), Profiling & Gözlemlenebilirlik

Gelişmiş loglama, metrikler, trace takibi, bellek sızıntısı tespiti ve AI ile kök neden analizi gerçekleştirin.

37. Dedektif Gibi Düşünmek: Debugging'in Psikolojisi ve Bilimsel Metodu

Rastgele Kod Değiştirme Çaresizliğinden Hipotez Testine Geçiş
Bölüm 7 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Junior mühendisler bir bug çıktığında neden rastgele satırları değiştirip dua etmeye başlar? Sherlock Holmes gibi hipotez kurarak hata çözmek nasıl bir his?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
>
> - 📌 Debugging bir şans oyunu değil; bilimsel hipotez kurma ve eleme (Elimination) sürecidir. - 📌 İlk kural: Hatayı deterministik olarak yerel makinenizde yeniden üretebilir (reproduce) hale getirin.
> - 📌 AI modelleri çok iyi bir 'Sanal Dedektif Ortağıdır' (Watson); gözlemlediğiniz belirtileri verip olası şüphelileri listelemesini isteyin.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Sherlock Holmes'un bir cinayet mahalline girdiğini düşünün.

Holmes odaya girip: *"Hadi rastgele sokaktan bir adamı tutuklayalım, belki katil odur. Olmadı başka birini tutuklarız!"* der mi? Tabii ki demez! Çamur izine bakar, kırık vazoyu inceler, saati kontrol eder. İmkansız olan tüm olasılıkları eledikten sonra geriye kalan tek gerçek ne kadar inanılmaz olursa olsun doğru olandır!

Junior yazılımcıların en büyük hatası **"Pompalı Tüfek Debugging" (Shotgun Debugging)** yapmaktır: Oradan bir `+1` sil, şuraya bir `await` ekle, orayı `null` yap… Belki çalışır! Kod daha da batar.

Oysa bilimsel debugging şudur: 1. Gözlem: *"Sadece Salı günleri fatura toplamı 0 çıkıyor."* 2. Hipotez: *"Haftanın günü index'i 0'dan mı başlıyor?"* 3. Test: İzole bir test yaz ve hipotezi doğrula!

🧠 2. AI Çağında Yeni Zihniyet Modeli

Bilimsel Hata Ayıklama Döngüsü:

```
[1. GÖZLEM (Symptom)] ──> "Kullanıcı sepete 2. ürünü ekleyince fiyat ikiye
katlanıyor."
          |
          ▼
[2. TEKRAR ÜRET (Reproduce)] ──> "Adım 1: Giriş yap, Adım 2: X ekle, Adım 3: Y ekle
-> Bug çıktı!"
          |
          ▼
[3. HİPOTEZ KUR (Hypothesis)] ──> "Sepet state'i güncellenirken dizi mutasyona
uğruyor olabilir mi?"
          |
          ▼
[4. TEST ET (Experiment)] ──> "Breakpoint veya izole test ile değişkeni izle."
          |
     ┌────┴─────┐
     ▼          ▼
[DOĞRULANDI]  [ÇÜRÜTÜLDÜ]
(Kök nedeni   (Başka bir hipoteze geç:
 düzelt)       "API yanıtı mı bozuk?")
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Bilimsel Hata Ayıklama ve Dedektiflik Mentoru
Uygulamamızda açıklanamayan bir hata ile karşılaştık.

## BELİRTİLER (Gözlem):
- Kullanıcılar sepete kupon uyguladığında bazen indirim uygulanıyor, bazen de sepet
tutarı 0 TL'ye düşüyor.
- Sorun sadece birden fazla sekme açıkken veya hızlı tıklandığında yaşanıyor.

## GÖREV:
Rastgele kod yazma. Bunun yerine:
1. Bu hatayı açıklayabilecek en mantıklı 3 olası HİPOTEZİ listele (Örn: Race
condition, state mutasyonu, session çakışması).
2. Bu hipotezleri doğrulamak veya çürütmek için hangi adımları izlemem gerektiğini
planla.
3. Hatayı yerel makinemde %100 tekrarlayacak (reproduce) test senaryosunu tarif et.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ **Acemi / Kontrolsüz AI Çıktısı (Riskli):**

```
// Pompalı Tüfek Debugging: Sorunu anlamadan rastgele yamalar yapıştırmak!
export function applyDiscountShotgun(cart: any) {
  // "Bazen 0 oluyormuş, o zaman 0 ise eski haline getirelim bari!" (KÖK NEDENİ ÇÖZMEZ!)
  if (cart.total === 0) {
    cart.total = cart.previousTotal || 100;
  }
  return cart;
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// Kök Neden Analizi Sonucu: Asenkron Race Condition Tespit Edildi ve Çözüldü
export class CartManager {
  private activeDiscountPromise: Promise<number> | null = null;

  async applyDiscountWithLock(cartId: string, couponCode: string): Promise<CartResult> {
    // 1. Kök Neden: Kullanıcı art arda tıkladığında iki istek birbirinin üstüne yazıyordu (Race Condition)
    // Çözüm: Promise Memoization / Mutex Kilidi ile mükerrer istekleri engelle
    if (this.activeDiscountPromise) {
      const pendingTotal = await this.activeDiscountPromise;
      return { cartId, total: pendingTotal };
    }

    try {
      this.activeDiscountPromise = (async () => {
        const cart = await db.cart.findById(cartId);
        const discount = await couponService.calculate(cart, couponCode);
        const newTotal = Math.max(0, cart.subtotal - discount);
        await db.cart.updateTotal(cartId, newTotal);
        return newTotal;
      })();

      const finalTotal = await this.activeDiscountPromise;
      return { cartId, total: finalTotal };
    } finally {
      this.activeDiscountPromise = null; // Kilidi serbest bırak
    }
  }
}
```

💡 **Kod Analizi:** Belirtiyi gizleyen yamalar yapmak yerine kök neden (race condition) bilimsel olarak tespit edilmiş; promise kilitleme mekanizmasıyla sorun kalıcı olarak çözülmüştür.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Hatayı yerel makinede yeniden üretemeden körü körüne koda müdahale etmek.
  - **Çözüm:** Bir hatayı düzeltmeye başlamadan önce onu lokalde %100 tekrarlayan bir test yazın.
- **Tuzak:** Aynı anda 5 farklı dosyayı değiştirip hangisinin işe yaradığını bilememek.
  - **Çözüm:** Her seferinde sadece tek bir değişkeni veya hipotezi değiştirip test edin.
- **Tuzak:** Bug çözülünce regresyon testi yazmadan PR açmak.
  - **Çözüm:** Çözdüğünüz her bug için gelecekte tekrar etmesini engelleyen bir unit test ekleyin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 37: Dedektif Gibi Debugging - Bilimsel Hata Ayıklama*

- 🖥️ **Slayt 1:** Pompalı Tüfek (Shotgun) Debugging neden kariyerinizi bitirir?
- 🖥️ **Slayt 2:** Bilimsel Metot: Gözlem, Hipotez, Deney ve Doğrulama.
- 🖥️ **Slayt 3:** Hatayı %100 yeniden üretilebilir (Reproduce) kılmanın sırrı.
- 🖥️ **Slayt 4:** Canlı Vaka: Açıklanamayan bir sepet hatasını adım adım çözmek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
İçinde gizli bir race condition olan kodu açın. AI'a 'Bu kodda 3 potansiyel hipotez kur' deyin. Hipotezlerden birini test eden küçük bir script yazıp hatayı tetikleyin ve ardından çözümü gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Karşılaştığınız bir sonraki bug'da hemen kod değiştirmeyin. Bir kağıda: 'Gözlemim', 'Hipotezim' ve 'Doğrulama Adımım' başlıklarını yazıp adımları takip edin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Yazılım geliştirmede 'Shotgun Debugging' (Pompalı Tüfek Hata Ayıklama) antipattern'i ne anlama gelir? - A) Bilgisayara ateş etmek - B) Kök nedeni anlamadan ve hipotez kurmadan koddaki satırları rastgele değiştirip şans eseri çalışmasını ummak - C) Sadece Python'da çalışan bir kütüphane - D) Testlerin çok hızlı koşması > **Doğru Yanıt:** B > **Açıklama:** Shotgun debugging, rastgele deneme-yanılma yöntemidir; sorunun kök nedenini çözmediği gibi kodda yeni ve daha gizli bug'lar yaratır.

Soru 2: Bir hatayı çözmeye başlamadan önce yapılması gereken ilk ve en kritik kural nedir? - A) Hemen sunucuyu yeniden başlatmak - B) Hatayı yerel (local) ortamda deterministik olarak %100 yeniden üretilebilir (reproduce) hale getirmek - C) Şirket yöneticisine e-posta atmak - D) Tüm projeyi baştan yazmak > **Doğru Yanıt:** B > **Açıklama:** Tekrarlanamayan (reproduce edilemeyen) bir hatanın çözülüp çözülmediğini bilmek imkansızdır. Önce hatanın oluşma şartları izole edilmelidir.

38. Breakpoint'ler, Loglar ve Debugger Araçlarını Efektif Kullanmak

'console.log' Kirliliğinden Kurtulup Zamanı Durdurma Sanatı
Bölüm 7 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ **NotebookLM & Audio Overview Podcast Notları**

Podcast Tartışma Sorusu:
"Yazılımcılar neden koda 50 tane `console.log('buraya geldi 1')` yazmayı alışkanlık haline getirir ve interaktif debugger kullanmak bu süreyi nasıl 10 kat kısaltır?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 `console.log` ile hata aramak karanlık bir odada el feneriyle küçük bir noktaya bakmaktır; Debugger ise odanın ana ışığını yakmaktır. - 📌 Koşullu Breakpoint (Conditional Breakpoint): Sadece belirli bir koşul sağlandığında (örn: `i === 5000` veya `user.id === 'target'`) durur.
- 📌 Call Stack (Çağrı Yığını) penceresi, o fonksiyonun hangi fonksiyonlar tarafından çağrıldığını zamanda geriye giderek görmenizi sağlar.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir futbol maçını televizyonda izlediğinizi düşünün. Hakem ofsayt kararı verdi.

Eğer elinizde video durdurma teknolojisi olmasaydı ne yapardınız? Maçı baştan sona tekrar oynatıp, top ayağa değdiği anda gözünüzü kırpmadan ekrana bakmaya çalışırdınız! (İşte bu `console.log` ile kod çalıştırmaktır).

Peki VAR (Video Hakem) sistemi ne yapar? Tam topa vurulduğu milisaniyede görüntüyü **DURDURUR (Breakpoint)**. İleriye sarar (Step Over), geriye sarar, oyuncunun milimetrik pozisyonunu inceler (Scope Variables).

Modern IDE'lerin Debugger araçları sizin yazılımdaki VAR sisteminizdir. Zamanı dondurabilir, değişkenlerin içine girebilir ve hatayı nokta atışı yakalayabilirsiniz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Debugger Kontrol Tuşları Sözlüğü:

```
⏸️ PAUSE / RESUME (F5):
   Kodu bir sonraki breakpoint'e kadar normal hızında çalıştırır.

⏭️ STEP OVER (F10):
   Mevcut satırı çalıştırır ve alt satıra geçer (Fonksiyonun içine girmeden).

⬇️ STEP INTO (F11):
   Mevcut satırdaki fonksiyonun İÇİNE girer (İç mantığı satır satır görmek için).

⬆️ STEP OUT (Shift+F11):
   Bulunduğunuz fonksiyondan çıkar ve onu çağıran üst fonksiyona geri döner.

🔴 CONDITIONAL BREAKPOINT:
   Kodu sadece `user.age < 0` olduğunda durdurur (10.000 döngüyü tek tek gezmez!).
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Debugger ve Geliştirme Araçları Mentoru
VS Code veya Cursor üzerinde Node.js uygulamamızı debug edeceğiz.

## GÖREV:
1. `.vscode/launch.json` dosyasını TypeScript ve `tsx` (veya ts-node) ile anında
breakpoint yakalayacak şekilde hazırla.
2. 10.000 elemanlı bir döngüde sadece belirli bir hata anında duracak 'Conditional
Breakpoint' kullanımını anlat.
3. Call Stack penceresini kullanarak hatanın hangi ara katmandan tetiklendiğini
nasıl tespit edeceğimi göster.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ **Acemi / Kontrolsüz AI Çıktısı (Riskli):**

```
// Kötü / İlkel Yöntem: launch.json yok, kodun her yerine console.log saçmak
// console.log("Burada 1");
// console.log("User:", user);
// console.log("Burada 2");
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// Profesyonel VS Code / Cursor Debug Yapılandırması (.vscode/launch.json)
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "🐞 Debug Mevcut TypeScript Dosyası",
      "type": "node",
      "request": "launch",
      "runtimeExecutable": "npx",
      "runtimeArgs": ["tsx", "${file}"],
      "cwd": "${workspaceFolder}",
      "console": "integratedTerminal",
      "internalConsoleOptions": "neverOpen",
      "skipFiles": [
        "<node_internals>/**",
        "**/node_modules/**"
      ]
    },
    {
      "name": "🌐 Attach to Running Node Process",
      "type": "node",
      "request": "attach",
      "port": 9229,
      "restart": true,
      "skipFiles": ["<node_internals>/**"]
    }
  ]
}
```

💡 **Kod Analizi:** `skipFiles` ayarı sayesinde debug esnasında yanlışlıkla `node_modules` veya Node.js dahili kütüphanelerine düşmezsiniz; debugger sadece sizin kaynak kodunuz üzerinde adım adım ilerler.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Koda geçici `console.log` yazıp production'a commit etmek ve logları kirletmek.
  - **Çözüm:** Her zaman interaktif breakpoint kullanın; commit öncesi `git diff` ile log kalmadığını teyit edin.

- **Tuzak:** Büyük döngülerde normal breakpoint koyup 10.000 kez F5'e basmaya çalışmak.
  - **Çözüm:** Koşullu Breakpoint (Conditional Breakpoint) koyarak sadece `id === 'hatali-kayit'` anında durmasını sağlayın.
- **Tuzak:** Asenkron fonksiyonlarda `await` satırında Step Over yaparken thread'in atlamasını anlayamamak.
  - **Çözüm:** Event Loop mantığını hatırlayın; asenkron çağrılarda Step Into yerine bir sonraki satıra breakpoint koyun.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 38: Debugger Ustalığı - console.log'a Veda Etmek*

- 🖥️ **Slayt 1:** Neden console.log ile hata aramak zaman kaybıdır?
- 🖥️ **Slayt 2:** VS Code ve Cursor Debugger anatomisi: F5, F10, F11.
- 🖥️ **Slayt 3:** Koşullu Breakpoint (Conditional Breakpoints) ve Logpoints.
- 🖥️ **Slayt 4:** Canlı Demo: launch.json ile çalışan backend'e bağlanıp zamanı durdurmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
VS Code'da bir fonksiyona kırmızı breakpoint koyun. F5 ile debug modunu başlatın. Kod durduğunda sol paneldeki 'Variables' alanını açın; fonksiyonun içindeki tüm değişkenlerin canlı değerlerini, scope'unu ve Call Stack'i gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde `.vscode/launch.json` dosyasını oluşturun ve bir sonraki hatada `console.log` yazmak yerine kırmızı bir breakpoint koyarak değişkenleri inceleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Bir döngü içinde 5.000 eleman varken, sadece `i === 4250` olduğunda kodun durmasını sağlamak için hangi breakpoint türü kullanılmalıdır? - A) Standart Breakpoint - B) Koşullu Breakpoint (Conditional Breakpoint) - C) Logpoint - D) CSS Breakpoint > **Doğru Yanıt:** B > **Açıklama:** Koşullu breakpoint, içine yazılan mantıksal ifade true olana kadar kodu durdurmaz; böylece binlerce gereksiz adımı atlar.

Soru 2: Debugger panelinde 'Call Stack' (Çağrı Yığını) penceresinin geliştiriciye sunduğu en önemli bilgi nedir? - A) Bilgisayarın pil yüzdesi - B) O an durulan fonksiyonun hangi fonksiyonlar zinciri tarafından çağrıldığını (kimin kimi tetiklediğini) zamanda geriye dönük göstermesi - C) Kodun satır sayısı -

D) GitHub şifresi > **Doğru Yanıt:** B > **Açıklama:** Call Stack, o anki yürütme noktasına ulaşana kadar hangi fonksiyonların sırayla çağrıldığını gösteren geçmiş izidir.

39. Heisenbug'lar ve Race Condition: Asenkron Hataları Yakalamak

İncelemeye Çalıştığınızda Ortadan Kaybolan Hayalet Hatalarla Mücadele
Bölüm 7 • Zorluk Derecesi: İleri Seviye • Tahmini Okuma Süresi: 13 dk

🎙️ **NotebookLM & Audio Overview Podcast Notları**

Podcast Tartışma Sorusu:
"Koda `console.log` eklediğinizde veya breakpoint koyduğunuzda sihirli bir şekilde düzelen, ama canlıda patlayan 'Heisenbug' nedir ve nasıl çözülür?"

Bölümden Çıkarılacak 3 Altın Cümle:
- 📌 Heisenbug, gözlemlendiğinde (log veya debugger eklendiğinde gecikme değiştiği için) davranışını değiştiren veya kaybolan hatalardır. - 📌 Asenkron sistemlerde en büyük tehlike Race Condition'dır (Yarış Durumu); verinin hangi sırayla geleceği garanti edilemez. - 📌 `Promise.all` yerine sıralı bağımlılıkları `for..of` ile çözmek veya dağıtık kilitler (Redis Redlock / DB Locks) kullanmak yarış durumunu önler.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Kuantum fiziğindeki Heisenberg Belirsizlik İlkesini duymuşsunuzdur: Bir elektronun hızını ölçmeye çalıştığınız an, ölçüm aletinin ışığı elektronun yerini değiştirir!

Yazılımda da **Heisenbug** tam olarak budur: Canlı ortamda iki asenkron işlem yarışıyor. Aralarında 2 milisaniyelik bir fark var ve sistem çöküyor. Geliştirici hatayı yakalamak için araya `console.log` veya breakpoint koyuyor. Fakat o `console.log` diske yazarken sistemi 5 milisaniye yavaşlatıyor! Zamanlama değiştiği için hata birdenbire yok oluyor! Geliştirici çıldırır: *"Az önce hata veriyordu, şimdi çalışıyor!"*

Bu hayalet hatalarla baş etmenin tek yolu, şansa bırakılmış zamanlamalara değil, deterministik kilitlere ve sıralama kurallarına güvenmektir.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Race Condition (Yarış Durumu) Anatomisi:

```
BAŞLANGIÇ: Kullanıcı Bakiyesi: 100 TL
İki İstek Aynı Anda Gelir (Çift Tıklama):

İSTEK 1: (50 TL Çek)                İSTEK 2: (80 TL Çek)
    |                                    |
    ├─ Bakiyeyi Oku: 100 TL              ├─ Bakiyeyi Oku: 100 TL
    ├─ Kontrol: 100 >= 50? (EVET)        ├─ Kontrol: 100 >= 80? (EVET)
    ├─ Yeni Bakiye: 50 TL                ├─ Yeni Bakiye: 20 TL
    └─ DB'ye Yaz: 50 TL                  └─ DB'ye Yaz: 20 TL
_____________________________________________________________________

SONUÇ FELAKETİ:
Kullanıcı 130 TL para çekti, ama hesapta 20 TL kaldı! Banka soyuldu!
ÇÖZÜM: Atomic SQL Update: UPDATE accounts SET balance = balance - 50 WHERE balance
>= 50;
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Eşzamanlılık (Concurrency) ve Asenkron Sistemler Mimarı
Bir e-ticaret sitesinde bilet satışı yapıyoruz. Son 1 bilet kaldı ve iki kullanıcı
aynı anda 'Satın Al' butonuna bastı.

## GÖREV:
1. Bu durumdaki Race Condition tehlikesini ve 'Double-Spending' riskini açıkla.
2. PostgreSQL kullanarak 'Optimistic Locking' veya 'Atomic Update' ile bu sorunu
kökten çözen SQL / Prisma kodunu yaz.
3. Node.js tarafında `Promise.all` kullanılırken düşülen asenkron sıralama
tuzaklarını listele.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ **Acemi / Kontrolsüz AI Çıktısı (Riskli):**

```
// Felaket: Önce oku, sonra kontrol et, sonra yaz (Race Condition Açığı!)
export async function buyTicketNaive(ticketId: string, userId: string) {
  const ticket = await db.ticket.findUnique({ where: { id: ticketId } });

  // İki istek aynı anda burayı geçerse ikisi de bileti aldığını sanır!
  if (ticket.isSold) throw new Error("Bilet tükendi.");

  await db.ticket.update({
    where: { id: ticketId },
    data: { isSold: true, ownerId: userId }
  });
  return { success: true };
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// Kurşun Geçirmez: Atomik SQL Güncellemesi (Tek İşlemde Oku ve Kilitle)
export async function buyTicketAtomic(ticketId: string, userId: string) {
  // Veritabanı seviyesinde tek satırlık koşullu güncelleme (Row-level atomic write)
  const updatedRows = await db.$executeRaw`
    UPDATE tickets
    SET is_sold = TRUE, owner_id = ${userId}, version = version + 1
    WHERE id = ${ticketId} AND is_sold = FALSE
  `;

  // Eğer etkilenen satır sayısı 0 ise, yarışı diğer kullanıcı kazanmıştır!
  if (updatedRows === 0) {
    throw new ConflictError("Bilet sizden önce başka bir kullanıcı tarafından satın
alındı.");
  }

  return { success: true, message: "Bilet başarıyla adınıza rezerve edildi." };
}
```

💡 **Kod Analizi:** `UPDATE ... WHERE is_sold = FALSE` sorgusu veritabanı motorunun dahili satır kilidini (Row-level lock) kullanır. İki istek aynı milisaniyede gelse bile veritabanı istekleri kuyruğa sokar; ilki günceller, ikincisi 0 satır etkilediği için `ConflictError` fırlatır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Asenkron döngülerde `forEach` kullanmak (forEach asenkron await'leri beklemez!).
  - **Çözüm:** Sıralı asenkron işlemlerde daima `for (const item of items)` döngüsü kullanın.
- **Tuzak:** Para veya stok gibi kritik verileri hafızada (in-memory) tutup veritabanına sonradan yazmak.
  - **Çözüm:** Kritik kaynakları daima veritabanının atomik kilitleri veya Redis mutex ile koruyun.
- **Tuzak:** Heisenbug çıktığında 'şimdi geçti bir şey olmaz' diyerek sorunu görmezden gelmek.

- **Çözüm:** Zamanlama hataları canlıda bin kat daha sık çıkar; sorunu izole edip kilit mekanizması ekleyin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 39: Heisenbug ve Race Condition - Asenkron Hatalarla Savaş*

- 🖥️ **Slayt 1:** Heisenbug nedir? İncelendikçe kaybolan hayalet hatalar.
- 🖥️ **Slayt 2:** Race Condition (Yarış Durumu) ve Çift Harcama (Double-Spending) tehlikesi.
- 🖥️ **Slayt 3:** JavaScript Event Loop ve asenkron döngü tuzakları (forEach vs for..of).
- 🖥️ **Slayt 4:** Veritabanı seviyesinde Atomik Kilitler (Atomic Updates) ile mutlak koruma.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Node.js'te `Promise.all` ile aynı bakiye fonksiyonuna 10 eşzamanlı istek atan bir script çalıştırın. Naive kodda bakiyenin eksiye düştüğünü gösterin. Ardından atomik SQL kodunu çalıştırıp sadece ilk isteğin geçtiğini, kalan 9'unun engellendiğini kanıtlayın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde bakiye düşürme, stok güncelleme veya kupon kullanma gibi kritik bir fonksiyonu inceleyin. `find` ve `update` iki ayrı adımda yapılıyorsa bunu tek bir atomik sorguya dönüştürün.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: JavaScript kodunda `[1, 2, 3].forEach(async (id) => { await doTask(id); })` yazıldığında ne olur? - A) Görevler sırayla çalışır - B) `forEach` asenkron Promise dönüşünü beklemez; tüm görevler aynı anda yarışarak asenkron kaos yaratır - C) Kod hiç çalışmaz - D) Bilgisayar kapanır > **Doğru Yanıt:** B > **Açıklama:** `Array.prototype.forEach` geriye dönen Promise'leri beklemez (await etmez). Sıralı yürütme için `for..of` döngüsü kullanılmalıdır.

Soru 2: Veritabanında eşzamanlı iki isteğin aynı anda stok düşürmesini engellemenin en zarif ve performanslı yolu nedir? - A) Sunucuyu 1 saniye bekletmek - B) `UPDATE products SET stock = stock - 1 WHERE id = 1 AND stock > 0` gibi tek satırlık atomik koşullu güncelleme kullanmak - C) Kullanıcılara aynı anda tıklamamalarını rica etmek - D) Veritabanını silmek > **Doğru Yanıt:** B > **Açıklama:** Koşullu atomik SQL güncellemesi, veritabanı motorunun dahili satır kilidini kullanarak yarış durumunu (race condition) sıfır ek maliyetle çözer.

---

40. Bellek Sızıntısı (Memory Leak) ve CPU Boğulması Teşhisi

Sunucunun Her 4 Saatte Bir Şişip Çökmesini Node Profiler ile Önlemek
Bölüm 7 • Zorluk Derecesi: İleri Seviye • Tahmini Okuma Süresi: 13 dk

🎙️ **NotebookLM & Audio Overview Podcast Notları**

Podcast Tartışma Sorusu:
"Sunucunuz her gün saat 16:00'da RAM'i tüketip çöküyor ve yeniden başlıyor. Bu gizli bellek sızıntısını (Memory Leak) yapay zeka ve Heap Snapshot ile nasıl teşhis ederiz?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Bellek Sızıntısı (Memory Leak), artık ihtiyaç duyulmayan nesnelerin referansları silinmediği için Garbage Collector tarafından temizlenememesidir. - 📌 En yaygın sızıntı kaynakları: Temizlenmeyen Event Listener'lar, global dizilere sonsuz veri eklemek ve kapanış (Closure) tuzaklarıdır.
- 📌 Chrome DevTools / Node Inspector ile alınan iki 'Heap Snapshot' karşılaştırılarak hangi nesnelerin biriktiği saniyeler içinde bulunur.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Evinizde bir çöp kovası olduğunu düşünün.

Her gün eve alışveriş poşetleri, kargo kutuları geliyor. Siz bunları kullanıp çöpe atıyorsunuz ve haftada iki gün çöpçü gelip kovayı boşaltıyor (Garbage Collector).

Peki ya siz kargo kutularını çöpe atmak yerine: *"Aman belki lazım olur"* diyerek yatak odanızın tavanına doğru üst üste yığmaya başlarsanız ne olur? 3 ay sonra odaya sığamaz hale gelirsiniz, kapı kapanmaz ve ev çöker!

Yazılımda **Bellek Sızıntısı (Memory Leak)** tam olarak budur: Geliştirici global bir diziye her gelen isteğin logunu atar. 10 gün sonra sunucunun 8 GB RAM'i biter ve işletim sistemi uygulamayı acımasızca öldürür (OOM - Out of Memory Kill).

🧠 2. AI Çağında Yeni Zihniyet Modeli

Garbage Collection ve Referans Tutma Mantığı:

```
[Kök Nesne: Global Scope / Window / Process]
                    |
            (Aktif Referans Var)
                    ▼
          [GlobalCache Dizisi]
                    |
            (Binlerce Eski İstek)
                    ▼
          [UserSession Objesi] ──> GARBAGE COLLECTOR DİYOR Kİ:
                                   "Buna hâlâ globalden erişim var,
                                    bu yüzden bu nesneyi RAM'den SİLEMEM!"
──────────────────────────────────────────────────────────────────────
ÇÖZÜM: Referansı kopar (session = null) veya LRU Cache kullan!
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Node.js Performans ve Profiling Mimarı
Aşağıdaki Express kodunda gizli ve ölümcül bir bellek sızıntısı (Memory Leak)
bulunmaktadır.

## KOD:
```typescript
const requestHistory = []; // Global dizi

app.get("/api/data", (req, res) => {
  const context = {
    ip: req.ip,
    headers: req.headers,
    timestamp: new Date()
  };
  requestHistory.push(context); // Her istekte şişiyor!
  res.json({ message: "Başarılı" });
});
```

GÖREV:

1. Bu kodun neden OOM (Out Of Memory) çöküşüne yol açacağını açıkla.
2. Sınırsız büyüyen dizi yerine 'LRU (Least Recently Used) Cache' mantığını uygulayan güvenli kodu yaz.
3. Node.js'te `--inspect` ve Chrome DevTools ile Heap Snapshot alma adımlarını özetle.
```

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Felaket: Event Listener temizlenmiyor ve global dizi sınırsız büyüyor
const EventEmitter = require('events');
const eventBus = new EventEmitter();

app.get("/subscribe", (req, res) => {
  // Her istekte yeni bir dinleyici eklenir ama asla silinmez! (Memory Leak)
  eventBus.on("notification", (msg) => {
    res.write(msg);
  });
});
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```typescript
import { lru } from "tiny-lru";

// 1. Sınırlı Bellek: En fazla 1.000 kayıt tutan LRU Cache (Sabit RAM Kullanımı)
const requestCache = lru<RequestContext>(1000);

export function trackRequestSafe(req: Request) {
  const requestId = crypto.randomUUID();
  requestCache.set(requestId, {
    ip: req.ip,
    timestamp: Date.now()
  });
  // 1001. kayıt geldiğinde en eski kayıt otomatik bellekten atılır (Eviction)
}

// 2. Event Listener Temizliği (Hafıza Güvenliği)
export function attachCleanListener(req: Request, res: Response) {
  const onNotification = (data: string) => {
    res.write(data);
  };

  eventBus.on("notification", onNotification);

  // İstemci bağlantıyı kapattığında dinleyiciyi MUTLAKA kaldır!
  req.on("close", () => {
    eventBus.removeListener("notification", onNotification);
  });
}
```

💡 **Kod Analizi:** `lru(1000)` kullanılarak bellek tüketimi tavan bir sınırla sabitlenmiş; `req.on('close')` içinde `removeListener` çağrılarak bağlantı kapandığında nesnenin bellekten temizlenmesi garanti edilmiştir.
```

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Geçici verileri global değişkenlerde veya dosya kapsamındaki (module-level) dizilerde saklamak.
  - **Çözüm:** Global state'ten kaçının; saklanması gereken verileri Redis veya veritabanına yazın.
- **Tuzak:** Event listener ekleyip (`socket.on`) bağlantı koptuğunda temizlemeyi (`socket.off`) unutmak.
  - **Çözüm:** Her listener için mutlaka bir cleanup (temizlik) fonksiyonu tanımlayın.
- **Tuzak:** Büyük resim veya dosya işlemlerini belleğe (Buffer) tek parça yüklemek.
  - **Çözüm:** Büyük verileri işlerken RAM'i şişirmemek için Stream (Akış) API'lerini kullanın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 40: Bellek Sızıntısı & CPU Profiling - OOM Çöküşlerini Önlemek*

- 🖥️ **Slayt 1:** Bellek Sızıntısı (Memory Leak) nedir? V8 Garbage Collector nasıl çalışır?
- 🖥️ **Slayt 2:** En yaygın 3 sızıntı tuzağı: Global arrays, uncleaned listeners, closures.
- 🖥️ **Slayt 3:** Chrome DevTools ile Heap Snapshot alma ve karşılaştırma.
- 🖥️ **Slayt 4:** Stream kullanarak devasa dosyaları sıfır RAM ile işlemek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Node uygulamasını `node --inspect index.js` ile başlatın. Chrome'da `chrome://inspect` sayfasını açın. Heap Snapshot alın. Koda kasıtlı bir sızıntı ekleyip ikinci snapshot'ı alın. 'Comparison' görünümünde hangi objenin biriktiğini canlı kanıtlayın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizi `node --inspect` ile çalıştırıp Chrome DevTools üzerinde ilk Heap Snapshot'ınızı alın ve bellekte en çok yer tutan ilk 3 nesne tipini inceleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Node.js (V8) Garbage Collector'ı bellekteki bir nesneyi neden silemez ve bellek sızıntısına (Memory Leak) yol açar? - A) Bilgisayarın pili bittiği için - B) Nesneye hâlâ kök (Global Scope, aktif Closure veya temizlenmemiş Listener) üzerinden ulaşan aktif bir referans zinciri bulunduğu için - C)

Sadece C++ dilinde kod yazıldığı için - D) Dosya isimleri çok uzun olduğu için > **Doğru Yanıt:** B > **Açıklama:** Garbage Collector sadece hiçbir kök nesneden ulaşılamayan (unreachable) nesneleri temizler. Bir referans unutulduğunda nesne sonsuza kadar bellekte kalır.

Soru 2: 1 GB boyutundaki büyük bir CSV dosyasını sunucunun 512 MB RAM'ini patlatmadan işlemek için hangi mimari yaklaşım kullanılmalıdır? - A) Dosyayı tek seferde `fs.readFileSync` ile hafızaya almak - B) Node.js Streams (Akış) kullanarak dosyayı parça parça (chunk by chunk) okuyup işlemek - C) Dosyayı silmek - D) Tarayıcıyı kapatmak > **Doğru Yanıt:** B > **Açıklama:** Stream'ler veriyi tümüyle RAM'e almak yerine küçük tamponlar (buffer) halinde sırayla işler; böylece dosya ne kadar büyük olursa olsun RAM kullanımı sabit kalır.

41. Post-Mortem Kültürü: 'Kimi Suçlayacağız?' Değil, 'Nasıl Öğreneceğiz?'

Production Çöktüğünde Mühendisleri Suçlamayan Şirketlerin Yükselişi
Bölüm 7 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ **NotebookLM & Audio Overview Podcast Notları**

Podcast Tartışma Sorusu:
"Bir junior mühendis ilk haftasında yanlışlıkla canlı veritabanını sildiğinde kovulmalı mı, yoksa bir junior'ın canlı veritabanını silebildiği bir sisteme izin veren mimari mi hatalıdır?"

Bölümden Çıkarılacak 3 Altın Cümle:
- 📌 Suçsuz Post-Mortem (Blameless Post-Mortem), insan hatasını sistemik bir tasarım zaafı olarak ele alma disiplinidir. - 📌 Eğer bir mühendis bir tuşa basarak tüm sistemi çökertebiliyorsa, hata mühendiste değil; o tuşa korkuluk koymayan süreçtedir. - 📌 5 Neden (5 Whys) analizi ile yüzeysel bahaneleri aşıp kök mimari zafiyete ulaşılır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

2017 yılında Amazon AWS'de çalışan bir mühendis, faturalandırma sunucularındaki birkaç test sunucusunu kapatmak için terminale bir komut yazdı. Küçük bir yazım hatası (typo) yaptı ve Kuzey

Amerika'daki S3 depolama sunucularının devasa bir kısmını kapattı! Tüm dünya çapında binlerce popüler internet sitesi 4 saat boyunca çöktü.

Amazon o mühendisi kovdu mu? **HAYIR!**

Amazon yönetimi şunu söyledi: *"Mühendisimiz kötü niyetli değildi. Sorun, tek bir insanın yazdığı tek bir komutla bu kadar devasa bir sistemin çökmesine izin veren güvenlik bariyerlerimizin eksikliğidir."*

Komutun etkisini kısıtlayan yeni güvenlik sistemleri kuruldu ve o mühendis şirketin en kıdemli çalışanlarından biri olarak kalmaya devam etti.

İşte olgun bir mühendislik kültürü budur: İnsanları suçlamak hataları gizlemeye iter; sistemleri düzeltmek ise şirketi ölümsüz kılar.

🧠 2. AI Çağında Yeni Zihniyet Modeli

5 Neden (5 Whys) Kök Neden Analizi Örneği:

```
[OLAY: Canlı veritabanı 45 dakika boyunca çöktü]
  |
  ├─ 1. Neden çöktü? ──> Disk doldu.
  |
  ├─ 2. Disk neden doldu? ──> Debug log dosyaları 500 GB yer kapladı.
  |
  ├─ 3. Loglar neden silinmedi? ──> Log rotasyonu (logrotate) yapılandırılmamıştı.
  |
  ├─ 4. Neden yapılandırılmamıştı? ──> Yeni kurulan sunucu için checklist takip
edilmedi.
  |
  └─ 5. KÖK NEDEN: Sunucu kurulumları elle (manual) yapılıyor; Infrastructure as
Code (Terraform) otomasyonu yok!
──────────────────────────────────────────────────────────────────────────────
AKSIYON: "Mühendisi azarla" değil -> "Tüm sunucuları Terraform ile otomatik kur ve
log alarmı ekle!"
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Site Reliability Engineering (SRE) ve Olay Analiz Uzmanı
Dün gece yaşanan bir kesinti için 'Suçsuz Post-Mortem (Blameless Post-Mortem)'
raporu hazırlayacağız.

## OLAY BİLGİSİ:
- Tarih: 15 Ekim 2026, 22:15 - 23:00 (45 dk kesinti)
- Etki: Kullanıcılar ödeme yapamadı, yaklaşık 40.000 TL ciro kaybı oluştu.
- Tetikleyici: Yeni yayınlanan güncellemedeki hatalı bir `.env` ortam değişkeni.

## GÖREV:
Profesyonel bir Post-Mortem markdown raporu hazırla:
1. Olay Özeti & Zaman Çizelgesi (Timeline)
2. 5 Neden (5 Whys) Kök Neden Analizi
3. 'Neler İyi Gitti?' ve 'Neler Kötü Gitti?'
4. Önleyici Eylem Maddeleri (Action Items - Sahibi ve Bitiş Tarihi olan somut
görevler)
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# Kötü / Suçlayıcı Rapor:
Dün gece Ahmet yanlış .env dosyasını yüklediği için sistem çöktü.
Ahmet uyarıldı, bir daha yapmayacak.
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
# POST-MORTEM RAPORU: [INC-2026-10-15] Ödeme Servisi Kesintisi

## 📋 Yönetici Özeti
15 Ekim 2026 saat 22:15'te ödeme servisi, eksik bir ortam değişkeni
(`STRIPE_WEBHOOK_SECRET`) nedeniyle başlatılamamış ve 45 dakika boyunca ödeme
alınamamıştır. Toplam 120 işlem etkilenmiştir.

## ⏱️ Olay Zaman Çizelgesi
- **22:10:** v2.4.0 sürümü üretime dağıtıldı (deploy).
- **22:15:** Sentry üzerinde HTTP 500 alarmları tetiklendi.
- **22:25:** Nöbetçi mühendis (On-call) olaya müdahale etti ve rollback kararı aldı.
- **22:50:** Veritabanı geri alma adımı tamamlandı.
- **23:00:** Sistem tamamen sağlıklı duruma döndü.

## 🔍 5 Neden (5 Whys) Analizi
1. **Ödeme servisi neden çöktü?** -> `STRIPE_WEBHOOK_SECRET` değişkeni `undefined`
geldi.
2. **Değişken neden eksikti?** -> Prod ortamına değişken elle eklenmemişti.
3. **Neden elle eklenmesi gerekiyordu?** -> Ortam değişkenlerini yöneten merkezi bir
Secrets yöneticisi yoktu.
4. **Deploy öncesi neden fark edilmedi?** -> Uygulama ayağa kalkarken değişkenleri
kapıda doğrulayan bir kontrol (Zod env check) yoktu.
5. **KÖK NEDEN:** Uygulama eksik konfigürasyonla çalışmaya başlayabilmektedir;
'Fail-Fast' mekanizması eksiktir.

## 🛠️ Düzeltici Eylem Maddeleri (Action Items)
| Görev | Sorumlu | Termin |
| :--- | :--- | :--- |
| `env.mjs` şeması kurularak eksik değişkende derlemenin durdurulması | Can
(Backend) | 18 Ekim |
| Staging ortamında deploy öncesi duman testi (smoke test) eklenmesi | Elif (DevOps)
| 20 Ekim |
| Müşteri destek ekibine etkilenen kullanıcılara özür kuponu tanımlanması | Zeynep
(Ürün) | 16 Ekim |
```

💡 **Kod Analizi:** Rapor bireyleri suçlamak yerine; eksik konfigürasyonla sunucunun ayağa kalkmasını engelleyen ‘Fail-Fast’ doğrulama sistemi kurmayı hedefler. Böylece aynı hata gelecekte hiç kimse tarafından tekrarlanamaz.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Post-Mortem toplantılarında ‘Falanca kişi dikkatsiz davrandı’ diyerek insanları hedef almak.
  - **Çözüm:** Soruyu her zaman ‘Sistemimiz dikkatsizliği neden engellemedi?’ şeklinde sorun.
- **Tuzak:** Post-Mortem raporunu yazıp bir dokümana gömmek ve aksiyon maddelerini yapmamak.
  - **Çözüm:** Aksiyon maddelerini hemen Jira biletlerine dönüştürün ve bir sonraki sprinte dahil edin.
- **Tuzak:** Küçük kesintileri ‘nasılsa çabuk çözüldü’ diyerek belgelendirmemek.

- **Çözüm:** Her kesintiyi şirketin kurumsal hafızasını güçlendiren bir eğitim vakası olarak kaydedin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 41: Suçsuz Post-Mortem - Hatalardan Kurumsal Bilgelik Üretmek*

- 🖥️ **Slayt 1:** Neden suçlama kültürü şirketleri teknik iflasa sürükler?
- 🖥️ **Slayt 2:** Blameless Post-Mortem ilkeleri ve psikolojik güvenlik.
- 🖥️ **Slayt 3:** 5 Neden (5 Whys) analizi ile kök nedene inmek.
- 🖥️ **Slayt 4:** Örnek Vaka: 45 dakikalık prod kesintisini aksiyon planına dönüştürmek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Sentry'de patlayan bir hatanın loglarını açın. Yapay zekaya bu logları verip '5 Whys analizi yap ve suçsuz bir Post-Mortem taslağı oluştur' deyin. Çıkan rapordaki aksiyon maddelerini inceleyin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Geçmişte projenizde yaşadığınız küçük bir hatayı düşünün. Kendinize '5 Neden' sorusunu sorarak hatanın arkasındaki asıl sistemsel eksikliği bir kağıda yazın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: 'Suçsuz Post-Mortem' (Blameless Post-Mortem) yaklaşımının en temel felsefesi nedir? - A) Hata yapan çalışanlara ceza kesilmemesi - B) İnsan hatasını kaçınılmaz kabul edip, hataya izin veren sistemik ve mimari süreçleri iyileştirerek aynı sorunun bir daha ASLA yaşanmamasını sağlamak - C) Kesintileri müşterilerden gizlemek - D) Sadece haftalık toplantı yapmak > **Doğru Yanıt:** B > **Açıklama:** Suçsuz kültür, hatayı bireylere yüklemek yerine sistemik korumalar (linter, test, CI/CD) inşa ederek kurumun güvenilirliğini artırır.

Soru 2: Bir Post-Mortem raporunda en değerli ve kalıcı bölüm hangisidir? - A) Hatanın olduğu günün hava durumu - B) Somut sorumluları ve teslim tarihleri belirlenmiş 'Önleyici Eylem Maddeleri' (Action Items) - C) Şirket sahibinin imzası - D) Hatanın kaç satır sürdüğü > **Doğru Yanıt:** B > **Açıklama:** Aksiyon maddeleri olmadan bir post-mortem sadece bir şikayet metnidir. Kalıcı iyileşmeyi sağlayan somut düzeltici görevlerdir.

42. Canlı Ortam Gözlemlenebilirliği: Sentry, Datadog ve Log Analizi

Kullanıcılar 'Site Bozuk' Demeden Önce Hataları Canlıda Yakalamak
Bölüm 7 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Bir şirkette canlıdaki bir hatayı ilk olarak müşteriler Twitter'da şikayet edince öğreniyorsanız, gözlemlenebilirlik (Observability) açısından neden sınıfta kalmışsınızdır?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Gözlemlenebilirlik (Observability) 3 temel ayaktan oluşur: Metrikler (Metrics), Loglar (Logs) ve İzler (Traces). - 📌 Sentry, kullanıcı tarayıcısında veya sunucuda patlayan hataları anında yakalayarak tam stack trace ve kullanıcı oturumuyla önünüze serer. - 📌 AI modelleri, binlerce satırlık Datadog/CloudWatch log akışını tarayarak 'anomali' ve 'kök neden' özetini saniyeler içinde çıkarabilir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Hastanenin yoğun bakım ünitesini (ICU) hayal edin.

Hastanın parmağında nabız ölçer (Metrik), başında kalp elektrosu (Trace), hemşirenin elinde saatlik gözlem defteri (Log) vardır. Eğer hastanın kalbi durursa sirenler anında çalar ve doktorlar odaya dalar!

Peki ya yoğun bakımda hiçbir cihaz olmasaydı? Doktorlar hastanın durumunu ancak koridordaki bir ziyaretçi: *"Galiba 3. odadaki hasta nefes almıyor!"* dediğinde öğrenirdi!

İşte canlıya Sentry veya Datadog kurmadan kod atmak, o cihazsız yoğun bakım odasıdır.

Profesyonel bir ekip, kullanıcı hatayı henüz fark etmeden önce Sentry bildirimini alır, düzeltmeyi yapar ve güncellemeyi yayına alır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Gözlemlenebilirliğin (Observability) 3 Sacayağı:

```
┌──────────────────────────────────────────────────┐
│                 GÖZLEMLENEBİLİRLİK               │
└──────────────────────┬───────────────────────────┘
                       │
       ┌───────────────┼──────────────┐
       ▼               ▼              ▼
[1. METRİKLER (Metrics)] [2. LOGLAR (Logs)]  [3. İZLER (Traces)]
- "CPU kullanımı %92"   - Yapılandırılmış JSON- "İstek 1500 ms sürdü:
- "Hata oranı %0.05"    - "Kullanıcı giriş yaptı"  1200 ms DB sorgusu,
- Araç: Prometheus      - Araç: Winston / Pino    300 ms Stripe API"
                                                  - Araç: OpenTelemetry
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Observability ve SRE Uzmanı
Next.js / Node.js backend projemize kurumsal düzeyde Sentry entegrasyonu yapacağız.

## GÖREV:
1. `sentry.server.config.ts` yapılandırmasını oluştur (Tracing ve Profiling açık).
2. Hassas kullanıcı verilerinin (şifre, kredi kartı, auth token) Sentry'ye gitmesini
engelleyen `beforeSend` filtresini yaz (Sanitization).
3. Yakalanan bir hataya kullanıcı ID'si ve ek iş bağlamı (tags / extra context)
ekleme örneğini göster.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Kötü: console.error ile loglamak (Sunucu çökünce loglar kaybolur, kimse görmez!)
app.post("/checkout", async (req, res) => {
  try {
    await processPayment(req.body);
  } catch (err) {
    console.error("Hata oldu:", err); // Canlıda hiç kimse fark etmez!
    res.status(500).send("Sunucu hatası");
  }
});
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
import * as Sentry from "@sentry/node";

// 1. Sentry Başlatma ve Güvenlik Filtresi (sentry.server.ts)
Sentry.init({
  dsn: process.env.SENTRY_DSN,
  environment: process.env.NODE_ENV || "production",
  tracesSampleRate: 0.2, // Performans izleme için %20 örnekleme

  // GÜVENLİK: Hassas verileri Sentry'ye göndermeden önce temizle (Scrubbing)
  beforeSend(event) {
    if (event.request?.headers) {
      delete event.request.headers["authorization"];
      delete event.request.headers["cookie"];
    }
    return event;
  }
});

// 2. Güvenli Hata Yakalama ve Bağlam Ekleme
export async function handleCheckout(req: Request, res: Response) {
  try {
    await processPayment(req.body);
    res.json({ success: true });
  } catch (error) {
    // Hatayı zengin bağlamla Sentry'ye bildir
    Sentry.withScope((scope) => {
      scope.setUser({ id: req.user?.id, email: req.user?.email });
      scope.setTag("transaction_type", "subscription_renewal");
      scope.setExtra("cart_payload", req.body);
      Sentry.captureException(error);
    });

    res.status(500).json({ error: "İşlem sırasında beklenmedik bir hata oluştu." });
  }
}
```

💡 **Kod Analizi:** `beforeSend` filtresiyle hassas auth token ve çerezler temizlenerek KVKK / GDPR uyumu sağlanmış; `withScope` ile hatanın tam olarak hangi kullanıcıda ve hangi işlem tipinde patladığı etiketlenmiştir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Loglara şifre, TC kimlik no veya kredi kartı bilgilerini düz metin olarak basmak (PII Leak).
  - **Çözüm:** Hassas alanları log kütüphanelerinde (Pino/Winston) otomatik maskeleyen redactor kullanın.
- **Tuzak:** Sentry'ye her milisaniyede log gönderip kotayı 1 günde bitirmek.

  - **Çözüm:** `tracesSampleRate` oranını production'da makul bir seviyeye (%10 - %20) ayarlayın.
- **Tuzak:** Hata oluştuğunda try/catch ile hatayı sessizce yutup hiçbir yere bildirmemek.
  - **Çözüm:** Catch bloklarında hatayı mutlaka Sentry veya merkezi log sistemine gönderin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 42: Canlı Ortam Gözlemlenebilirliği - Sentry ve Telemetri*

- 🖥️ **Slayt 1:** Gözlemlenebilirliğin (Observability) 3 Sacayağı: Metrics, Logs, Traces.
- 🖥️ **Slayt 2:** Sentry kurulumu ve 'Breadcrumbs' ile hatanın adımlarını izlemek.
- 🖥️ **Slayt 3:** Hassas veri temizliği (PII Data Scrubbing) ile KVKK uyumu.
- 🖥️ **Slayt 4:** Bölüm 7 Özeti: Hata Ayıklama, Profiling ve Sistem Sağlığı.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Sentry dashboard'unu açın. Canlı API'de kasıtlı bir hata tetikleyin. Sentry'nin 2 saniye içinde hatayı nasıl yakaladığını, kullanıcının tıkladığı önceki butonları (breadcrumbs) ve stack trace'i harita gibi gösterdiğini inceleyin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Kişisel bir projenize ücretsiz bir Sentry hesabı bağlayın (`@sentry/node` veya `@sentry/react`) ve canlıda kasıtlı bir hata yakalatarak dashboard'da inceleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Yazılım sistemlerinde 'Gözlemlenebilirlik' (Observability) hangi 3 temel veriden oluşur? - A) HTML, CSS, JavaScript - B) Metrikler (Metrics), Loglar (Logs) ve Dağıtık İzler (Traces) - C) Klavye, Fare, Ekran - D) Git, Docker, Kubernetes > **Doğru Yanıt:** B > **Açıklama:** Modern gözlemlenebilirlik; sayısal metrikler, olay logları ve isteklerin tüm sistem boyunca izlediği dağıtık izlerden (traces) meydana gelir.

Soru 2: Sentry gibi hata izleme araçlarında 'Breadcrumbs' (Ekmek Kırıntıları) ne işe yarar? - A) Kuşlara yem vermek için - B) Hata patlamadan önceki son 10 adımda kullanıcının hangi butonlara bastığını, hangi URL'leri ziyaret ettiğini ve hangi ağ çağrılarının yapıldığını adım adım göstermek - C) Şifreleri kaydetmek - D) Dosyaları sıkıştırmak > **Doğru Yanıt:** B > **Açıklama:** Breadcrumbs, bir çöküş anından önce sistemde nelerin yaşandığını kronolojik olarak göstererek hatayı dakikalar içinde yeniden üretmeyi sağlar.

Bölüm 8: Güvenlik, OWASP & AI Kod Hijyeni

OWASP Top 10 açıkları, gizli anahtar yönetimi, prompt enjeksiyonları ve yapay zeka halüsinasyon risklerine karşı sisteminizi zırhlandırın.

43. AI'nın Ürettiği Kod Güvenli mi? Gizli Güvenlik Açıkları

Yapay Zekanın Koda Gizlice Enjekte Ettiği Güvenlik Açıklarını Avlamak
Bölüm 8 • Zorluk Derecesi: İleri Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Stanford Üniversitesi araştırmasına göre yapay zeka kullanan yazılımcıların daha çok güvenlik açığı ürettiği ve kendi kodlarının 'güvenli' olduğuna daha çok inandığı doğru mu?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Yapay zeka modelleri 'çalışan' kod üretmeye odaklıdır; 'güvenli' kod üretmek sizin şartnamelerinize bağlıdır. - 📌 En yaygın AI güvenlik kusurları: Hardcoded secret'lar, zayıf şifreleme algoritmaları ve girdi doğrulaması yapılmamış parametrelerdir. - 📌 Güvenlik bir eklenti değil; AI'a verilen promptların ve CI pipeline'ındaki SAST (Statik Güvenlik Analizi) araçlarının ayrılmaz parçası olmalıdır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir çilingir çağırdığınızı ve kapınıza kilit taktırmak istediğinizi düşünün.

Çilingir kapıya harika görünen, parlak pirinçten bir kilit takıyor. Anahtarı çeviriyorsunuz: *"Tık!"* diye dönüyor, kapı kapanıyor. Çilingir gülümsüyor: *"Harika çalışıyor!"*

Fakat çilingir gittikten sonra fark ediyorsunuz ki kilit dilini duvardaki yuvaya bağlamamış! Dışarıdan parmağınızla hafifçe dokunduğunuzda kapı ardına kadar açılıyor!

Yapay zekanın ürettiği kontrolsüz kod tam olarak o parlak pirinç kilittir: Kod hatasız derlenir, ekranda çalışır görünür. Ancak arkasında SQL Injection, yetkilendirme açığı (IDOR) veya zayıf şifreleme gibi devasa delikler bırakabilir.

Bir mühendis olarak göreviniz, anahtarın dönmesine değil, kilidin hırsızı durdurup durdurmadığına bakmaktır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

AI Kod Denetimi Kontrol Listesi (Security Checklist):

```
┌──────────────────────────────────────────────┐
│            AI KODU GÜVENLİK SÜZGECİ          │
└──────────────────────┬───────────────────────┘
                       │
  1. GİRDİ DOĞRULAMA: Dışarıdan gelen veri sanitize edildi mi?
  2. YETKİLENDİRME (AuthZ): Kullanıcı sadece kendi verisine mi erişiyor?
  3. KRİPTOĞRAFİ: MD5, SHA1, DES gibi kırılmış algoritmalar var mı?
  4. SIZINTI KONTROLÜ: API anahtarı veya şifre koda gömülmüş mü?
  5. HATA YÖNETİMİ: Stack trace veya DB şeması istemciye sızıyor mu?
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Kıdemli Siber Güvenlik ve Kod Güvenliği Müfettişi
Aşağıdaki Python fonksiyonu bir yapay zeka tarafından üretildi ve production'a
gönderilmek üzere.

## KOD:
```python
import hashlib
import sqlite3

def register_user(username, password):
    # AI önerisi:
    hashed_pwd = hashlib.md5(password.encode()).hexdigest()
    conn = sqlite3.connect('app.db')
    cursor = conn.cursor()
    cursor.execute(f"INSERT INTO users VALUES ('{username}', '{hashed_pwd}')")
    conn.commit()
```

GÖREV:

1. Bu koddaki 3 ölümcül güvenlik açığını açıkla (SQLi, zayıf hash, salt eksikliği).
2. Bu açıkları istismar eden (exploit) senaryoları göster.
3. Kodu Argon2id / bcrypt ve parametrik SQL kullanarak kurşun geçirmez şekilde yeniden yaz.

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```python
# Kötü / AI Üretimi Güvensiz Kod: MD5 kırılmıştır ve SQL Injection açığı vardır!
import hashlib, sqlite3

def register_user_naive(user, pwd):
    # MD5 1 saniyede kırılır; Rainbow tablosu saldırısına açıktır!
    h = hashlib.md5(pwd.encode()).hexdigest()
    # String formatlama ile SQL Injection: user = "admin', '123'); DROP TABLE users;
--"
    sqlite3.connect('app.db').cursor().execute(f"INSERT INTO users VALUES ('{user}',
'{h}')")
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```python
# Güvenli ve Kurumsal Standart: bcrypt + Parametrik SQL Sorgusu
import bcrypt
import sqlite3

def register_user_secure(username: str, password_plain: str) -> bool:
    # 1. Girdi Uzunluk Sınırı (DoS koruması)
    if len(password_plain) < 12 or len(password_plain) > 128:
        raise ValueError("Şifre 12 ile 128 karakter arasında olmalıdır.")

    # 2. Modern Hashleme: Rastgele Salt + Yavaşlatılmış İşlem Maliyeti (Cost factor
12)
    salt = bcrypt.gensalt(rounds=12)
    password_hash = bcrypt.hashpw(password_plain.encode("utf-8"),
salt).decode("utf-8")

    # 3. Parametrik SQL: Değişkenler asla doğrudan SQL içine gömülmez!
    with sqlite3.connect("app.db") as conn:
        cursor = conn.cursor()
        cursor.execute(
            "INSERT INTO users (username, password_hash) VALUES (?, ?)",
            (username.strip().lower(), password_hash)
        )
        conn.commit()

    return True
```

💡 **Kod Analizi:** MD5 yerine yavaş ve rastgele salt üreten `bcrypt` kullanılarak Rainbow Table saldırıları imkansız kılınmış; f-string yerine `(?, ?)` parametrik sorgusu kullanılarak SQL Injection tamamen bertaraf edilmiştir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Şifre hashlemek için MD5 veya SHA-256 gibi hızlı algoritmaları kullanmak.
  - **Çözüm:** Şifreler için daima yavaş ve bellek zorlayan algoritmalar (bcrypt, Argon2id, PBKDF2) kullanın.
- **Tuzak:** SQL sorgularına değişkenleri `f'{var}'` veya `+ var` şeklinde yapıştırmak.
  - **Çözüm:** İstisnasız her SQL sorgusunda parametrik sorgu (Parameterized Queries / Prepared Statements) kullanın.
- **Tuzak:** AI'a 'bunu güvenli yap' demeden doğrudan kod yazmasını istemek.
  - **Çözüm:** Promptlarınızda 'OWASP standartlarına uy, parametrik sorgu kullan' şartını açıkça belirtin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 43: AI ve Kod Güvenliği - Gizli Açıkları Yakalamak*

- 🖥️ **Slayt 1:** Stanford Araştırması: AI kullanan geliştiriciler neden daha çok açık üretiyor?
- 🖥️ **Slayt 2:** En tehlikeli 3 kör nokta: Parametresiz SQL, MD5 şifreleme, IDOR açıkları.
- 🖥️ **Slayt 3:** bcrypt vs Argon2id: Şifre güvenliğinin matematiksel temeli.
- 🖥️ **Slayt 4:** Canlı Hack Demosu: AI'ın yazdığı giriş fonksiyonuna SQL Injection saldırısı.

Canlı Kodlama / Ekran Kaydı Senaryosu:
String birleştirme ile yazılmış bir giriş fonksiyonu açın. Kullanıcı adı yerine `' OR '1'='1` yazarak şifre bilmeden sisteme giriş yapıldığını (SQL Injection) gösterin. Ardından parametrik sorguya dönüştürüp saldırıyı engelleyin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki tüm SQL sorgularını tarayın. Herhangi bir yerde `+` veya f-string ile sorguya değişken eklenmiş mi kontrol edin ve parametrik hale getirin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Kullanıcı şifrelerini veritabanında saklarken SHA-256 veya MD5 yerine neden `bcrypt` veya `Argon2id` tercih edilmelidir? - A) bcrypt'in daha az yer kaplaması - B) SHA-256 ve MD5'in çok hızlı olması sebebiyle saniyede milyarlarca parola deneyen kaba kuvvet (Brute-Force) saldırılarına dayanamaması; bcrypt'in ise bilinçli olarak yavaşlatılmış ve 'tuzlanmış' (salted) olması - C) Sadece Python'da çalışması - D) Bilgisayarı koruması > **Doğru Yanıt:** B > **Açıklama:** Şifre hash algoritmaları saldırganların deneme hızını düşürmek için bilinçli olarak yavaş çalışacak şekilde tasarlanır; hızlı algoritmalar şifre için güvensizdir.

Soru 2: SQL Injection saldırılarını tamamen engellemenin endüstri standardı kesin çözümü nedir? - A) Kullanıcılardan tırnak işareti yazmamalarını rica etmek - B) Parametrik Sorgular (Parameterized Queries / Prepared Statements) kullanmak - C) Veritabanı şifresini değiştirmek - D) Sayfayı her gün yeniden başlatmak > **Doğru Yanıt:** B > **Açıklama:** Parametrik sorgularda veritabanı motoru kullanıcı girdisini asla çalıştırılabilir SQL kodu olarak yorumlamaz; sadece saf veri (literal string) olarak işler.

---

44. OWASP Top 10 Pratik Kılavuzu: SQL Injection, XSS, CSRF

Web Güvenliğinin En Tehlikeli 10 Açığını Kod Seviyesinde Kapatmak
Bölüm 8 • Zorluk Derecesi: İleri Seviye • Tahmini Okuma Süresi: 13 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Bir hacker sadece bir yorum kutusuna `<script>` yazarak şirketinizin binlerce kullanıcısının oturumunu nasıl çalabilir? XSS ve CSRF'i nasıl tarihe gömeriz?"

Bölümden Çıkarılacak 3 Altın Cümle:
- 📌 OWASP Top 10, dünyada web uygulamalarına yönelik en yaygın ve yıkıcı 10 güvenlik riskini listeler. - 📌 XSS (Cross-Site Scripting), saldırganın web sayfasına zararlı JavaScript enjekte etmesidir; `dangerouslySetInnerHTML` kullanımından kaçının. - 📌 IDOR (Insecure Direct Object Reference), URL'deki ID'yi değiştirerek başkasının faturasına erişmektir; yetkilendirme kontrolü şarttır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir apartmanda oturduğunuzu düşünün.

1. **XSS:** Kapıdaki zil kutusuna bir saldırgan görünmez zehirli bir iğne yerleştiriyor. Komşunuz zile bastığında zehirleniyor ve anahtarlarını düşürüyor (Kötü niyetli JavaScript'in oturum çerezlerini çalması).
2. **CSRF:** Posta kutunuza sahte bir mektup geliyor: *"Bu zarfı açarsan 1000 TL kazanacaksın!"* Zarfı açtığınız anda arka kapıdaki kurye sizin adınıza bankaya talimat veriyor!
3. **IDOR:** Faturanıza bakmak için web sitesine giriyorsunuz. URL şöyle: `site.com/fatura/101`. Yukarıdaki 101 sayısını silip 102 yazıyorsunuz ve komşunuzun tüm kredi kartı ve harcama dökümü önünüze açılıyor!

OWASP Top 10 işte bu apartmandaki güvenlik açıklarının listesidir. Güvenli bir sistem, kapı numarasını değiştiren birine *"Dur bakalım, bu ev senin mi?"* diye soran sistemdir.

🧠 2. AI Çağında Yeni Zihniyet Modeli

En Kritik 3 OWASP Açığının Çözüm Haritası:

```
1. IDOR (Insecure Direct Object Reference):
   ❌ YANLIŞ: SELECT * FROM invoices WHERE id = req.params.id;
   ✅ DOĞRU: SELECT * FROM invoices WHERE id = req.params.id AND user_id =
req.user.id;

2. XSS (Cross-Site Scripting):
   ❌ YANLIŞ: <div dangerouslySetInnerHTML={{ __html: userComment }} />
   ✅ DOĞRU: <div>{userComment}</div> (React varsayılan olarak HTML'i sanitize eder)

3. CSRF (Cross-Site Request Forgery):
   ❌ YANLIŞ: Basit çerezler (SameSite ayarı yok)
   ✅ DOĞRU: SameSite=Strict ve CSRF Token doğrulaması
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: Web Uygulama Güvenliği Uzmanı (AppSec)
Aşağıdaki Express API endpoint'i tipik bir IDOR (Yetkisiz Nesne Erişimi) açığı
içermektedir.

## AÇIKLI KOD:
```typescript
app.get("/api/invoices/:id", authenticate, async (req, res) => {
  const invoice = await db.invoice.findUnique({
    where: { id: req.params.id }
  });
  res.json(invoice);
});
````

GÖREV:

1. Bu IDOR açığının bir saldırgan tarafından nasıl istismar edileceğini açıkla.
2. Kodda hem sahiplik kontrolünü (Ownership Check) hem de 404/403 durumlarını doğru yöneten güvenli sürümü yaz.
3. Bu açığı otomatik yakalayacak bir güvenlik test senaryosu (Jest/Supertest) tasarla.
````

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// IDOR Açığı: Kullanıcı giriş yapmış ama faturanın KENDİSİNE ait olduğu kontrol
edilmiyor!
app.get("/api/orders/:id", authenticate, async (req, res) => {
  // Saldırgan :id yerine rastgele sayılar deneyerek herkesin siparişini çalar!
  const order = await db.order.findUnique({ where: { id: req.params.id } });
  res.json(order);
});
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```typescript
// IDOR Güvenliği: Çift Koşullu Sorgu ve İzolasyon
app.get("/api/orders/:id", authenticate, async (req: Request, res: Response) => {
  const currentUserId = req.user.id;
  const orderId = req.params.id;

  // 1. Veritabanı sorgusunda hem sipariş ID'si hem de oturumdaki kullanıcı ID'si
aranır!
  const order = await db.order.findFirst({
    where: {
      id: orderId,
      userId: currentUserId // SAHİPLİK KONTROLÜ (Ownership Verification)
    }
  });

  // Bilgi sızdırmamak için 403 yerine 404 döneriz (Saldırgan siparişin var olup
olmadığını bilemez)
  if (!order) {
    return res.status(404).json({ error: "Sipariş bulunamadı." });
  }

  return res.json({ success: true, data: order });
});
```

💡 **Kod Analizi:** Sorguya `userId: currentUserId` koşulu eklenerek IDOR açığı kökten kapatılmıştır. Ayrıca bulunamadığında 403 yerine `404 Not Found` dönülerek saldırganın sistemdeki diğer sipariş ID'lerini taraması (Enumeration Attack) engellenmiştir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** URL'deki ID'nin kullanıcının kendi verisi olduğunu varsaymak (IDOR).
```

  - **Çözüm:** Her veritabanı sorgusunda `WHERE id = ? AND userId = ?` sahiplik kontrolü yapın.
- **Tuzak:** React'te `dangerouslySetInnerHTML` kullanarak kullanıcı girdisini doğrudan ekrana basmak (XSS).
  - **Çözüm:** Zorunlu olmadıkça asla ham HTML render etmeyin; DOMPurify ile temizleyin.
- **Tuzak:** Güvenlik açıklarını sadece frontend tarafında gizlemeye çalışmak.
  - **Çözüm:** Frontend güvenlik sağlamaz; tüm kontroller backend tarafında yapılmalıdır.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 44: OWASP Top 10 - Web Açıklarını Kapatmak*

- 🖥️ **Slayt 1:** OWASP Top 10 nedir ve neden endüstri standardıdır?
- 🖥️ **Slayt 2:** IDOR (Yetkisiz Nesne Erişimi) açığının anatomisi ve çözümü.
- 🖥️ **Slayt 3:** XSS (Cross-Site Scripting) ve React'te güvenli render alma.
- 🖥️ **Slayt 4:** SameSite Çerezler ve CSRF saldırılarını engellemek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Postman ile iki farklı kullanıcı token'ı oluşturun. Kullanıcı A'nın oluşturduğu siparişi, Kullanıcı B'nin token'ı ile çekmeye çalışın. Naive kodda siparişin sızdığını, sahiplik kontrolü eklenince 404 döndüğünü canlı gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki tüm `GET /resource/:id` ve `DELETE /resource/:id` endpoint'lerini inceleyin. Her birinde oturum açan kullanıcının sahiplik kontrolünün (ownership check) yapıldığını doğrulayın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: IDOR (Insecure Direct Object Reference) güvenlik açığı tam olarak nasıl meydana gelir? - A) Bilgisayarın klavyesinin bozulmasıyla - B) Uygulamanın, URL veya istek parametresindeki ID üzerinden veriyi çekerken, oturum açmış kullanıcının o veriye erişim hakkı olup olmadığını (sahiplik) denetlememesiyle - C) Sadece CSS dosyalarında - D) İnternet kablosu çekildiğinde > **Doğru Yanıt:** B > **Açıklama:** IDOR, bir kullanıcının sadece URL'deki ID numarasını (örn: 101 yerine 102 yazarak) değiştirip başkalarının verilerine erişebilmesidir.

Soru 2: React uygulamalarında kullanıcı girdilerini ekrana basarken XSS açığı oluşmasını varsayılan olarak engelleyen mekanizma nedir? - A) Otomatik HTML Entity Escaping (React süslü parantezler `{value}` içindeki tüm tehlikeli HTML karakterlerini otomatik olarak zararsız metne dönüştürür) - B)

Bilgisayarın antivirüsü - C) Sadece Google Chrome kullanılması - D) TypeScript derleyicisi > **Doğru Yanıt:** A > **Açıklama:** React, JSX içindeki değişkenleri ekrana basarken `<script>` gibi tehlikeli etiketleri otomatik olarak escape eder (`<script>`); bu sayede kod çalışamaz.

45. API Güvenliği & Rate Limiting: Sistemini Korumak

Brute-Force ve DoS Saldırılarına Karşı Hız Sınırları ve Güvenlik Başlıkları
Bölüm 8 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 12 dk

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Bir saldırgan `/api/login` endpoint'ine saniyede 10.000 şifre denemesi gönderdiğinde sunucunuz neden 20 saniyede çöker ve Rate Limiting sistemi nasıl kurtarır?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Rate Limiting (Hız Sınırlaması), bir IP veya kullanıcının belirli bir zaman diliminde yapabileceği istek sayısını sınırlayan emniyet sübabıdır. - 📌 Helmet.js, tarayıcıları koruyan kritik HTTP güvenlik başlıklarını (Content-Security-Policy, HSTS) tek satırda enjekte eder. - 📌 CORS (Cross-Origin Resource Sharing), rastgele yabancı web sitelerinin kullanıcılarınız adına API istekleri atmasını engeller.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir gece kulübünün kapısını düşünün.

Kapıda bir güvenlik görevlisi (Bouncer) var. Eğer güvenlik olmasaydı ne olurdu? 5.000 kişilik bir holigan grubu aynı anda kapıya yüklenir, kapıyı kırar ve kulübü yerle bir ederdi! (DDoS Saldırısı).

Güvenlik görevlisi ne yapar? - Kapıda sıra oluşturur: *"Her dakikada sadece 5 kişi içeri girebilir!"* (Rate Limiting). - Girenlerin üzerini arar, kesici aletleri toplar (Input Sanitization). - Davetiyesi olmayanları kapıdan çevirir (CORS).

Yazılımda bir API geliştirdiğinizde internetteki tüm botlar ve kötü niyetli tarayıcılar o kapıya koşar. Kapıya bir nöbetçi koymazsanız sunucunuz 1 dakikada çöker.

🧠 2. AI Çağında Yeni Zihniyet Modeli

API Güvenlik Duvarı Mimarisi:

```
[Gelen İstek: POST /api/login]
              |
              ▼
[1. HELMET: Güvenlik Başlıkları] (HSTS, No-Sniff, CSP)
              |
              ▼
[2. CORS KONTROLÜ] ──> "İstek izinli domainden (app.sirket.com) mi geliyor?"
              |
              ▼
[3. RATE LIMITER (Redis)] ──> "Bu IP son 1 dakikada 5'ten fazla denedi mi?"
              |
      ┌───────┴──────┐
      ▼              ▼
[LİMİT AŞILDI (429)] [ONAYLANDI (Geçebilir)]
(Too Many Requests)    |
                       ▼
            [4. ZOD DOĞRULAMASI]
            (Zararlı karakterleri filtrele)
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: API Güvenliği ve Altyapı Uzmanı
Express.js projemize kapsamlı bir güvenlik zırhı ekleyeceğiz.

## GÖREV:
1. `express-rate-limit` kullanarak `/api/auth/*` rotalarına dakikada en fazla 5
deneme sınırı koyan ve 429 Too Many Requests dönen middleware'i yaz.
2. `helmet` ve katı `cors` yapılandırmasını güvenli şekilde kur.
3. Redis tabanlı bir rate limiter'ın (Sliding Window) neden çoklu sunucularda
(Cluster) zorunlu olduğunu açıkla.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Kötü / Güvensiz Express Kurulumu: CORS her yere açık, hız sınırı yok, güvenlik başlığı yok!
import express from 'express';
const app = express();

app.use(cors()); // Tehlikeli: Tüm siteler (saldırganlar dahil) API'ye erişebilir!
// Rate limit yok: Saldırgan saniyede 100.000 şifre deneyebilir!
app.post('/login', (req, res) => { /* ... */ });
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
import express from "express";
import helmet from "helmet";
import cors from "cors";
import rateLimit from "express-rate-limit";

const app = express();

// 1. Güvenlik Başlıkları (Helmet)
app.use(helmet());

// 2. Katı CORS Yapılandırması
const allowedOrigins = ["https://app.sirketimiz.com"];
app.use(cors({
  origin: (origin, callback) => {
    if (!origin || allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error("CORS İlkesi İhlali: Bu domaine erişim izni yok."));
    }
  },
  credentials: true, // HttpOnly çerezlere izin ver
  methods: ["GET", "POST", "PUT", "DELETE"]
}));

// 3. Hassas Giriş Rotaları İçin Hız Sınırı (Rate Limiting)
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 Dakika
  max: 5, // IP başına maksimum 5 başarısız deneme
  standardHeaders: true, // RateLimit-* standart başlıklarını dön
  legacyHeaders: false,
  message: {
    status: 429,
    error: "Çok fazla başarısız giriş denemesi. Lütfen 15 dakika sonra tekrar
deneyin."
  }
});

// Giriş rotasına limitleyiciyi bağla
app.use("/api/auth/login", authLimiter);
```

💡 **Kod Analizi:** `helmet` ile kritik HTTP güvenlik başlıkları eklenmiş, `cors` ile sadece resmi şirket domainine izin verilmiş ve `authLimiter` ile şifre kaba kuvvet (Brute-Force) saldırıları 5 denemede durdurulmuştur.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Çoklu sunucu (cluster / Docker) ortamında yerel bellek (in-memory) rate limiter kullanmak.

  - **Çözüm:** Sunucu sayısı birden fazlaysa rate limiting durumunu merkezi bir Redis kümesinde tutun.
- **Tuzak:** CORS ayarında `origin: '*'` bırakıp kimlik doğrulamalı istekleri savunmasız bırakmak.
  - **Çözüm:** Origin olarak daima açıkça onaylanmış güvenilir frontend alan adlarını belirtin.
- **Tuzak:** DDoS saldırılarını sadece uygulama katmanında çözmeye çalışmak.
  - **Çözüm:** Büyük hacimli volumetrik saldırılar için Cloudflare veya AWS Shield gibi CDN katmanlarını kullanın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📹 Ders Başlığı: *Ders 45: API Güvenliği - Rate Limiting ve Helmet ile Korunma*

- 🖥️ **Slayt 1:** Brute-Force ve DoS saldırılarının maliyeti.
- 🖥️ **Slayt 2:** Rate Limiting mantığı: Sliding Window vs Token Bucket.
- 🖥️ **Slayt 3:** Helmet.js ile tek satırda 11 güvenlik başlığı enjekte etmek.
- 🖥️ **Slayt 4:** CORS (Cross-Origin Resource Sharing) nedir ve nasıl yapılandırılır?

Canlı Kodlama / Ekran Kaydı Senaryosu:
Express API'de login rotasına rate limit koyun. Bir for döngüsüyle arka arkaya 6 kez istek atın. İlk 5 isteğin 200 aldığını, 6. isteğin anında `429 Too Many Requests` ile engellendiğini canlı terminalde gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki API'ye `helmet` paketini ekleyin ve `/login` rotasına IP başına 15 dakikada en fazla 10 istek sınırı koyan bir rate limiter bağlayın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Bir API endpoint'ine belirlenen limitin üzerinde istek yapıldığında dönülmesi gereken standart HTTP durum kodu hangisidir? - A) 200 OK - B) 404 Not Found - C) 429 Too Many Requests - D) 500 Internal Server Error > **Doğru Yanıt:** C > **Açıklama:** `429 Too Many Requests` durum kodu, istemcinin hız limitini aştığını ve belirli bir süre beklemesi gerektiğini bildiren RFC standardıdır.

Soru 2: CORS (Cross-Origin Resource Sharing) politikasının temel görevi nedir? - A) İnterneti hızlandırmak - B) Bir web sitesinde çalışan zararlı JavaScript kodunun, kullanıcının bilgisi olmadan başka bir domaindeki API'ye yetkisiz istekler göndermesini tarayıcı seviyesinde engellemek - C) Veritabanını yedeklemek - D) CSS kodlarını küçültmek > **Doğru Yanıt:** B > **Açıklama:** CORS, tarayıcıların aynı

köken politikasını (Same-Origin Policy) esnetirken hangi yabancı domainlerin API'ye erişebileceğini denetleyen güvenlik kalkanıdır.

46. Gizli Bilgi (Secret) Yönetimi: API Anahtarlarını GitHub'a Yüklememek!

GitHub'a Yanlışlıkla AWS Anahtarı Yükleyip 1 Saatte 50.000 Dolar Fatura Yeme Kabusu
Bölüm 8 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"GitHub'da açık (public) bir depoya AWS veya OpenAI API anahtarı yüklendiğinde, saldırgan botların o anahtarı bulup kripto para madenciliği başlatması kaç saniye sürer?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Botlar GitHub'ın commit akışını gerçek zamanlı tarar; açık bir depoya düşen API anahtarı ortalama 60 saniye içinde istismar edilir. - 📌 `.env` dosyası asla Git deposuna commit edilmemelidir; `.gitignore` dosyası ilk günden ayarlanmalıdır. - 📌 Pre-commit hook'ları (TruffleHog, GitGuardian) hassas bilgilerin terminalden çıkmasını daha commit anında engeller.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Genç bir yazılımcının gerçek hikayesini dinleyin:

Bir cumartesi gecesi OpenAI API'si ile harika bir yapay zeka botu kodluyor. Kod çalışıyor, heyecanla terminale koşuyor:

```
git add .
git commit -m "ilk versiyon bitti"
git push origin main
```

Yatağa gidip uyuyor. Sabah uyandığında telefonunda 42 adet acil durum e-postası var! AWS faturası: **64.000 Dolar!**

Ne oldu? Koddaki AWS API anahtarını GitHub'a yüklemişti. Kötü niyetli tarayıcı botlar 40 saniye içinde o anahtarı kaptı ve dünya çapında 150 adet devasa sunucu açarak kripto para madenciliği başlattı!

Gizli bilgiler (Secrets) kod tabanına asla yazılmaz. Kod bir arabanın şasisidir; API anahtarı ise sürücünün cebindeki kontak anahtarıdır. Anahtarı torpido gözüne bantlayıp arabayı sokakta bırakmazsınız!

🧠 2. AI Çağında Yeni Zihniyet Modeli

Güvenli Secrets Mimarisi:

```
[GELİŞTİRİCİ MAKİNESİ]
├── src/code.ts ──> (Git'e Gönderilir)
├── .env.example ──> (Git'e Gönderilir: Boş şablon)
└── .env ──────────> (ASLA GİT'E GİTMEZ! .gitignore ile Kilitli)
      │
      ▼
[PRODUCTION / CI ORTAMI]
└── Ortam Değişkenleri (Environment Variables):
    - AWS Secrets Manager / Doppler / Infisical / Vercel Env
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: DevSecOps ve Secrets Güvenliği Mühendisi
Projemizde API anahtarlarının yanlışlıkla Git'e yüklenmesini %100 engelleyecek bir
güvenlik sistemi kuracağız.

## GÖREV:
1. Kapsamlı bir `.gitignore` dosyası hazırla (`.env`, `.env.local`, pem dosyaları
vb.).
2. Takım arkadaşları için güvenli bir `.env.example` şablonu oluştur.
3. Husky ve `gitleaks` kullanarak geliştirici commit atmadan önce gizli bilgi
taraması yapan 'Pre-commit Hook' kurulumunu göster.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ **Acemi / Kontrolsüz AI Çıktısı (Riskli):**

```
# Kötü .gitignore: .env dosyası unutulmuş!
node_modules/
dist/
# .env unutulduğu için tüm şifreler GitHub'a sızar!
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
# 1. Kapsamlı .gitignore Dosyası (.gitignore)
# Ortam değişkenleri ve gizli anahtarlar
.env
.env.*
!.env.example
*.pem
*.key
*.cert
credentials.json
id_rsa

# 2. Örnek Güvenli Şablon (.env.example - Git'e commit edilir)
DATABASE_URL="postgresql://user:password@localhost:5432/mydb"
JWT_SECRET="buraya_en_az_32_karakterlik_gizli_anahtar_yazin"
STRIPE_SECRET_KEY="sk_test_ornek_anahtar"

# 3. Gitleaks Pre-commit Hook Kurulumu (Husky ile)
# Terminalde çalıştırın:
# npx husky init
# echo "npx gitleaks protect --staged --verbose" > .husky/pre-commit
# Artık commit atarken gizli anahtar varsa Git işlemi anında durdurur!
```

💡 **Kod Analizi:** `.env.example` dosyası şifre içermez, sadece geliştiricilere hangi değişkenlerin tanımlanması gerektiğini gösterir. Husky pre-commit hook'u ise insan dalgınlığını ortadan kaldırarak şifre içeren commit'leri terminalde engeller.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** GitHub'a şifre yükledikten sonra 'Hemen bir commit atıp sildim' diyerek rahatlamak.
  - **Çözüm:** Git geçmişi silinen her şeyi saklar! Şifre bir kez commit edildiyse o anahtar ÖLMÜŞTÜR; anında revoke (iptal) edin.
- **Tuzak:** Frontend kodunun (React/Next.js) içine gizli API anahtarı yazmak.
  - **Çözüm:** Frontend koduna yazılan her şey tarayıcıda herkes tarafından okunabilir; gizli anahtarları sadece backend'de tutun.
- **Tuzak:** .env dosyasını Slack veya e-posta üzerinden takım arkadaşına göndermek.
  - **Çözüm:** Gizli bilgileri paylaşmak için 1Password, Bitwarden veya Doppler gibi güvenli şifre yöneticileri kullanın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 46: Secrets Yönetimi - API Anahtarlarını Korumak*

- 🖥️ **Slayt 1:** 64.000 Dolarlık Bulut Faturası: Bir sızıntının gerçek anatomisi.
- 🖥️ **Slayt 2:** .env vs .env.example felsefesi.
- 🖥️ **Slayt 3:** Gitleaks ve TruffleHog ile commit öncesi otomatik güvenlik kapıları.
- 🖥️ **Slayt 4:** Eğer anahtar sızarsa ne yapılmalı? Anında İptal (Revocation) protokolü.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Kasıtlı olarak sahte bir Stripe anahtarı içeren dosya oluşturup commit etmeye çalışın. Gitleaks pre-commit hook'unun devreye girip commit'i kırmızı alarm ile engellediğini canlı terminalde gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki `.gitignore` dosyasını açın. `.env` ve türevlerinin (`.env*.local`) kilitli olduğundan emin olun ve bir `.env.example` şablonu hazırlayın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Bir API anahtarını yanlışlıkla public (açık) bir GitHub reposuna push ettiğinizde yapılması gereken İLK ve ACİL işlem nedir? - A) GitHub'dan hemen yeni bir commit ile anahtarı silmek - B) İlgili servisin (AWS, OpenAI, Stripe) yönetim paneline girip o API anahtarını ANINDA İPTAL ETMEK (Revoke / Deactivate) ve yenisini üretmek - C) Bilgisayarı kapatıp beklemek - D) Depoyu private yapmak > **Doğru Yanıt:** B > **Açıklama:** GitHub'daki botlar saniyeler içinde anahtarı kaydeder; depoyu silseniz bile anahtar saldırganın elindedir. Tek kurtuluş anahtarı derhal iptal etmektir.

Soru 2: Bir projede `.env.example` dosyasının varoluş amacı nedir? - A) Gerçek şifreleri saklamak - B) Gerçek değerler içermeden, projeyi çalıştıracak diğer geliştiricilere hangi ortam değişkenlerinin gerekli olduğunu gösteren temiz bir şablon sunmak - C) CSS dosyalarını derlemek - D) Testleri hızlandırmak > **Doğru Yanıt:** B > **Açıklama:** `.env.example` , şifreleri ifşa etmeden yeni katılan geliştiricilerin hangi anahtarlara ihtiyaç duyduğunu gösteren güvenli dokümantasyon şablonudur.

47. Telif Hakları, Lisanslar ve Kodun Hukuki Güvenliği

GPL, MIT, Apache Lisansları ve Şirketinizi Davalardan Koruma Rehberi
Bölüm 8 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Bir açık kaynak projesinden kopyalanan 10 satırlık GPL lisanslı bir kod, şirketinizin tüm ticari kod tabanını açık kaynak yapmaya zorlayabilir mi?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Açık kaynak 'sahipsiz' demek değildir; her kütüphanenin yasal bir lisansı ve kullanım şartı vardır. - 📌 MIT ve Apache 2.0 lisansları son derece esnektir ve ticari yazılımlarda serbestçe kullanılabilir. - 📌 GPL (Copyleft) lisansı 'bulaşıcıdır' (Viral); ticari kapalı kaynak bir projeye GPL kod dahil edilirse tüm projenin açık kaynak yapılması gerekebilir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir otomobil fabrikası kurduğunuzu ve milyon dolarlık lüks bir araba ürettiğinizi düşünün.

Motor ustası bir cıvata eksik olunca yan sanayiden bedava bir cıvata bulup takıyor. Fakat o cıvatanın üzerinde küçük bir patent yazısı var: *"Bu parçayı kullanan her araba halka ücretsiz dağıtılmalıdır!"* (GPL Lisansı).

Şirketin avukatları bir gün kapınızı çalar ve der ki: *"O küçük parça yüzünden tüm fabrikanın planlarını ve arabalarımızı ücretsiz vermek zorundayız!"*

Yazılım dünyasında bu felaketi yaşayan dev şirketler oldu!

Yapay zeka modelleri bazen GPL lisanslı kodları eğitim verisinden hatırlayıp sizin projenize kopyalayabilir. Bir mühendisin görevi, projeye giren her kütüphanenin lisans pasaportunu kontrol etmektir.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Açık Kaynak Lisans Spektrumu:

```
[İTİBARLI & SERBEST (Permissive)]        [KATILIMCI & KISITLAYICI (Copyleft)]
          MIT / Apache 2.0 / BSD                          GPL v3 / AGPL
                    |                                          |
                    ▼                                          ▼
- Ticari projelerde serbestçe kullanılabilir. - Kodunu değiştirip dağıtırsan,
- Kodunu kapalı tutabilirsin.                 KENDİ KODUNU DA açık kaynak
- Sadece telif bildirimini sakla.             yapmak ZORUNDASIN! ("Bulaşıcı")
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Fikri Mülkiyet ve Yazılım Hukuku Danışmanı
Şirketimiz kapalı kaynak (proprietary) bir ticari SaaS ürünü geliştiriyor.

## GÖREV:
1. MIT, Apache 2.0 ve GPL v3 lisansları arasındaki temel hukuki farkları açıkla.
2. Neden şirket projelerinde GPL v3 lisanslı npm veya pip paketlerinin
kullanılmasının riskli olduğunu belirt.
3. CI pipeline'ımızda lisans uyumluluğunu otomatik denetleyen bir araç (`license-
checker` veya `fossa`) yapılandırma örneği sun.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Lisans Denetimi Yapmayan Şirket:
// Geliştirici bilmeden "gpl-calculator" paketini ekler ve tüm şirket kodu tehlikeye
girer!
"dependencies": {
  "super-secret-calculator": "gpl-licensed-package"
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// CI Pipeline Lisans Denetim Yapılandırması (package.json scripti)
{
  "scripts": {
    "audit:licenses": "npx license-checker --onlyAllow 'MIT;Apache-2.0;BSD-2-Clause;BSD-3-Clause;ISC' --summary"
  }
}

// GitHub Actions Adımı:
// - name: 🛡️ Yasal Lisans Denetimi
//   run: npm run audit:licenses
// Eğer birisi GPL veya AGPL lisanslı paket eklerse CI derlemeyi anında iptal eder!
```

💡 **Kod Analizi:** `license-checker` aracı ile sadece onaylı izinli lisanslar (`MIT` , `Apache-2.0` , `BSD`) kabul edilir; GPL gibi kapalı kaynak projeyi tehlikeye atacak bir lisans tespit edildiğinde derleme bloke edilir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** GitHub'da bulduğu her açık kaynak kodu lisansına bakmadan ticari projeye kopyalamak.
  - **Çözüm:** Her kütüphanenin `LICENSE` dosyasını kontrol edin; şüphe durumunda MIT veya Apache tercih edin.
- **Tuzak:** Yapay zekanın ürettiği büyük kod bloklarının telifli kaynaklardan birebir ezberlenmiş (memorized) olabileceğini unutmak.
  - **Çözüm:** AI araçlarında 'public code matching' / 'telemetri eşleme' filtresini daima açık tutun.
- **Tuzak:** AGPL lisanslı bir servisi ağ üzerinden çağırıp lisans şartlarına uymamak.
  - **Çözüm:** AGPL ağ üzerinden kullanımda dahi kaynak kodun açılmasını zorunlu kılar; şirketlerde çok dikkatli kullanılmalıdır.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 47: Yazılım Lisansları ve AI Etiği - Şirketi Korumak*

- 🖥️ **Slayt 1:** Açık kaynak lisanslarının anatomisi: Permissive vs Copyleft.
- 🖥️ **Slayt 2:** GPL v3 tuzağı: Şirket kodlarını açık kaynak yapmak zorunda kalmak.
- 🖥️ **Slayt 3:** AI modellerinin telifli kod kopyalama riski ve filtreler.
- 🖥️ **Slayt 4:** Canlı Demo: CI pipeline'ında otomatik lisans denetimi kurmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir Node projesine `license-checker` kurun. `npm run audit:licenses` çalıştırıp projedeki tüm kütüphanelerin lisans dökümünü ekranda listeleyin. Kasıtlı olarak GPL lisanslı bir paket ekleyip denetimin nasıl hata verdiğini gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki `package.json` veya `requirements.txt` dosyalarını açın. Projenizde hangi açık kaynak lisanslarının kullanıldığını bir listeleyici araçla kontrol edin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Aşağıdaki yazılım lisanslarından hangisi ticari ve kapalı kaynaklı bir projede kodu açma zorunluluğu olmadan en serbestçe kullanılabilir? - A) GPL v3 - B) AGPL v3 - C) MIT Lisansı - D) Lisanssız kodlar > **Doğru Yanıt:** C > **Açıklama:** MIT Lisansı, telif bildirimi korunduğu sürece kodun ticari kapalı kaynaklı projelerde serbestçe kullanılmasına, değiştirilmesine ve satılmasına izin verir.

Soru 2: GPL lisansının 'Bulaşıcı' (Viral / Copyleft) olarak adlandırılmasının sebebi nedir? - A) Bilgisayarlara virüs bulaştırması - B) GPL lisanslı bir kod parçasını veya kütüphaneyi kullanan bir yazılımın da zorunlu olarak GPL lisansıyla açık kaynak yapılmasını hukuken şart koşması - C) Sadece kış aylarında çalışması - D) Kodun silinmesine sebep olması > **Doğru Yanıt:** B > **Açıklama:** GPL, kendi kodundan türetilen tüm yazılımların da aynı şekilde özgür ve açık kaynak olmasını şart koşar (Reciprocal / Copyleft).

---

48. Savunmacı Programlama (Defensive Programming) Sanatı

'Murphy Kanunları'na Göre Kod Yazmak: Yanlış Gidebilecek Her Şey Yanlış Gidecektir
Bölüm 8 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

---

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Savunmacı programlama paranoyakça bir korku mudur, yoksa NASA'nın Mars araçlarını milyonlarca kilometre ötede hayatta tutan nihai mühendislik zihniyeti midir?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Savunmacı programlama: Girdilerin bozuk geleceğini, ağın kopacağını ve üçüncü taraf servislerin çökeceğini varsayarak kod yazmaktır. - 📌 Fail-Safe (Güvenli Başarısızlık): Sistem çöktüğünde veriyi bozmadan veya güvenlik kapısını açık bırakmadan zararsız bir duruma geçmelidir.
- 📌 Circuit Breaker (Devre Kesici) deseni, çöken bir servise sürekli istek atıp sunucuyu kilitlemek yerine sistemi korumaya alır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Elektrik panolarındaki sigortaları (Fuses) düşünün.

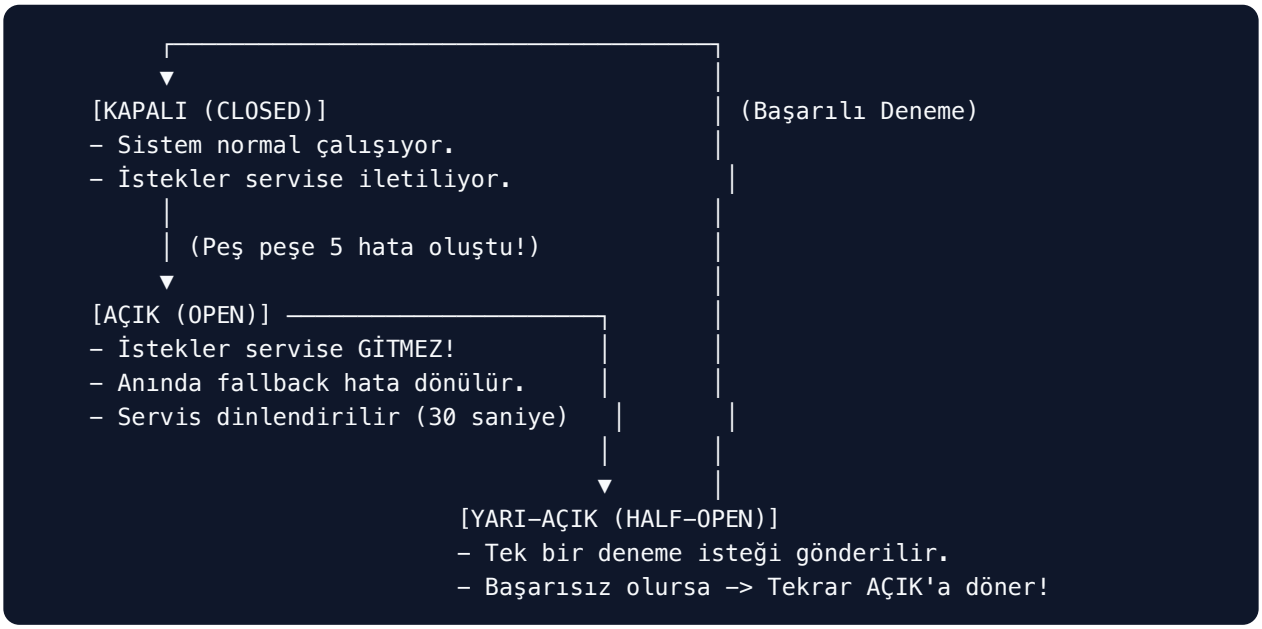
Evinize aniden 10.000 voltluk bir yıldırım düştüğünde ne olur? Sigorta anında **"ÇAT!"** diye atar ve evin elektriğini keser. Buzdolabınız veya televizyonunuz yanmaz; sadece ev karanlıkta kalır. Sigorta kendini feda ederek tüm evi korumuştur!

İşte **Savunmacı Programlama (Defensive Programming)** ve **Circuit Breaker** o elektrik sigortasıdır: Eğer ödeme yaptığınız bankanın sunucusu çöktüyse, uygulamanız kullanıcıya sonsuz yükleniyor (spinner) gösterip arkada binlerce istekle kendi sunucusunu boğmaz!

Sigortayı attırır: *"Şu an banka servislerinde geçici bir bakım var, lütfen 5 dakika sonra tekrar deneyin"* der ve sistemi güvenle ayakta tutar.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Circuit Breaker (Devre Kesici) Durum Makinesi:



🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Yüksek Güvenilirlikli Sistemler Mimarı
Üçüncü taraf bir döviz kuru API'sine bağımlı bir finansal servis yazıyoruz.

## GÖREV:
1. Savunmacı programlama ilkelerine uygun olarak bir Circuit Breaker deseni tasarla.
2. Servis çöktüğünde veya 2 saniye içinde yanıt vermediğinde (timeout), anında son
bilinen önbellekteki kuru dönen (Fallback) mekanizmasını kur.
3. Arka arkaya 3 hata olduğunda devreyi AÇIK (OPEN) duruma getiren tip güvenli
TypeScript kodunu yaz.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ **Acemi / Kontrolsüz AI Çıktısı (Riskli):**

```
// Savunmasız Kod: Timeout yok, hata kontrolü yok, sonsuz bekleme riski!
export async function getExchangeRateNaive() {
  // Dış servis çökerse bu fonksiyon dakikalarca asılı kalır ve tüm sunucuyu
kilitler!
  const res = await fetch("https://api.currency.com/rates");
  const data = await res.json();
  return data.rates.USD;
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// Savunmacı Mimari: Timeout + Circuit Breaker + Fallback Kalkanı

export class ResilientCurrencyService {
  private failureCount = 0;
  private circuitOpenUntil = 0;
  private lastKnownRate = 34.50; // Fallback varsayılanı

  async getUsdRate(): Promise<{ rate: number; isFallback: boolean }> {
    const now = Date.now();

    // 1. Sigorta Kontrolü (Circuit Breaker Açık mı?)
    if (now < this.circuitOpenUntil) {
      console.warn("[Circuit Breaker] Servis açık durumda, doğrudan fallback
dönülüyor.");
      return { rate: this.lastKnownRate, isFallback: true };
    }

    try {
      // 2. Zaman Aşımı Koruması (2 Saniye Sınırı)
      const res = await fetch("https://api.currency.com/rates", {
        signal: AbortSignal.timeout(2000)
      });

      if (!res.ok) throw new Error(`HTTP Hata: ${res.status}`);

      const data = await res.json();
      this.lastKnownRate = data.rates.USD;
      this.failureCount = 0; // Başarı durumunda sayacı sıfırla

      return { rate: this.lastKnownRate, isFallback: false };
    } catch (error) {
      this.failureCount++;
      console.error(`Döviz servisi hatası (${this.failureCount}/3):`, error);

      // 3. Eşik Aşımı: 3 hata üst üste gelirse sigortayı 1 dakika attır!
      if (this.failureCount >= 3) {
        this.circuitOpenUntil = now + 60 * 1000;
        console.error("[Circuit Breaker] Sigorta attı! 1 dakika boyunca dış çağrı
yapılmayacak.");
      }

      // 4. Fail-Safe: Sistem çökmez, son bilinen güvenli kuru döner
      return { rate: this.lastKnownRate, isFallback: true };
    }
  }
}
```

💡 **Kod Analizi:** `AbortSignal.timeout(2000)` ile isteklerin sonsuza kadar asılı kalması engellenmiş, 3 ardışık hatada devre açılarak çöken servise boşuna istek atılması durdurulmuş ve son bilinen kur ile kesintisiz hizmet (graceful degradation) sağlanmıştır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Dış API isteklerine asla zaman aşımı (timeout) koymamak.
  - **Çözüm:** Her dış ağ çağrısına mutlaka 2-5 saniyelik bir `timeout` sınırı koyun.
- **Tuzak:** Hata anında uygulamanın beyaz ekran verip çökmesine izin vermek.
  - **Çözüm:** Kullanıcıya daima dostça bir hata ekranı ve alternatif bir yol (fallback) sunun.
- **Tuzak:** Savunmacı programlamayı abartıp her satıra gereksiz 10 kat kontrol eklemek.
  - **Çözüm:** Savunmayı sistem sınırlarına (API girişleri, dış servis çağrıları) yerleştirin; iç fonksiyonlarda tip güvenliğine güvenin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 48: Savunmacı Programlama - Circuit Breaker ve Fallback Tasarımı*

- 🖥️ **Slayt 1:** Murphy Kanunu: Yanlış gidebilecek her şey yanlış gidecektir.
- 🖥️ **Slayt 2:** Fail-Safe Tasarım: Çöküş anında güvenli duruma geçmek.
- 🖥️ **Slayt 3:** Circuit Breaker (Devre Kesici) durum makinesi: Closed, Open, Half-Open.
- 🖥️ **Slayt 4:** Bölüm 8 Özeti: Güvenlik, OWASP ve Savunmacı Kod Hijyeni.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir döviz servisi simülasyonu yapın. Dış servisi kasıtlı olarak kapatın. İlk 3 istekte timeout olduğunu, 4. istekte devrenin açılıp 0 milisaniyede fallback değer döndüğünü canlı konsolda gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde harici bir API çağıran fonksiyona 3 saniyelik bir `AbortSignal.timeout` ve servis çöktüğünde çalışacak bir `fallback` değeri ekleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Yazılım mimarisinde 'Circuit Breaker' (Devre Kesici) deseninin temel amacı nedir? - A) Bilgisayara yeni donanım bağlamak - B) Sürekli hata veren veya çökmüş olan bir dış servise ısrarla istek atıp sistem kaynaklarını tüketmeyi durdurmak ve sistemi güvenli bir bekleme durumuna almak - C) Sadece CSS dosyalarını birleştirmek - D) Kullanıcı şifrelerini değiştirmek > **Doğru Yanıt:** B > **Açıklama:** Circuit

Breaker, çöken bir servise yapılan çağrıları geçici olarak keserek hem o servisin toparlanmasına fırsat tanır hem de ana sistemin kilitlenmesini önler.

Soru 2: Savunmacı Programlama (Defensive Programming) felsefesine göre bir yazılımcının varsayımı ne olmalıdır? - A) Kullanıcıların her zaman doğru veri gireceği ve internetin asla kopmayacağı - B) Dış girdilerin bozuk olabileceği, ağın her an kopabileceği ve bağımlılıkların çökebileceği; sistemin bu durumlara hazırlıklı olması gerektiği - C) Kodun sadece cuma günleri test edilmesi gerektiği - D) Sadece tek bir tarayıcıyı desteklemek gerektiği > **Doğru Yanıt:** B > **Açıklama:** Savunmacı programlama 'İyimser Kodlama'nın zıddıdır; her türlü felaket senaryosunu baştan hesaba katarak sistemi dayanıklı (resilient) kılar.

Bölüm 9: DevOps, Konteynerizasyon & Bulut Dağıtımı

Docker çok aşamalı derlemeler, CI/CD pipeline otomasyonu ve bulut dağıtım mimarilerini hayata geçirin.

49. 'Benim Makinemde Çalışıyordu': Docker ve Konteyner Mantığı

Yazılım Dünyasının En Meşhur Mazeretini Tarihe Gömen Konteyner Devrimi
Bölüm 9 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Bir junior mühendisin 'Ama benim bilgisayarımda çalışıyordu!' demesi neden bir senior lider için en sinir bozucu cümledir ve Docker bunu nasıl çözer?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Docker, uygulamanızı tüm bağımlılıkları, işletim sistemi kütüphaneleri ve konfigürasyonlarıyla birlikte izole bir kutuya (Container) paketler. - 📌 İmaj (Image) mimari plandır (class); Konteyner (Container) ise o planın çalışan canlı örneğidir (instance). - 📌 Çok aşamalı derleme (Multi-Stage Build), üretim imajlarını 1 GB'tan 50 MB'a düşürerek hem güvenlik hem hız sağlar.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

1950'li yıllardan önce uluslararası deniz ticaretini hayal edin.

Muzlar ahşap kasalara konur, pamuklar çuvallara basılır, viski şişeleri samanların arasına sarılırdı. Bir limandan diğerine aktarılırken şişeler kırılır, muzlar çürür, her limanın vinci farklı olduğu için yükleme günlerce sürerdi!

Sonra Malcolm McLean adında bir adam çıktı ve **Standart Çelik Taşımacılık Konteynerini** icat etti! İçinde ne olursa olsun (araba, muz, bilgisayar), dışındaki çelik kutunun boyutları ve kilitleri standarttır. Dünyadaki her vinç, her gemi, her tır bu konteyneri aynı hızla taşır.

Docker işte o çelik konteynerdir!

Uygulamanızı Docker kutusuna koyduğunuzda; kodunuzun Mac'te mi, Windows'ta mı yoksa AWS Ubuntu sunucusunda mı çalıştığının hiçbir önemi kalmaz. Her yerde aynı milisaniyede çalışır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Docker Kavramlar Sözlüğü:

```
1. DOCKERFILE:
   Konteynerin yemek tarifidir. "Hangi Linux sürümü, hangi Node.js, hangi dosyalar
kopyalanacak?"

2. DOCKER IMAGE (İmaj):
   Tarife göre fırından çıkmış dondurulmuş kek kalıbıdır. Değiştirilemez
(Immutable), taşınabilir.

3. DOCKER CONTAINER (Konteyner):
   İmajın RAM üzerinde çalışan canlı halidir. Başlatılabilir, durdurulabilir,
çoğaltılabilir.

4. DOCKER REGISTRY (Docker Hub / AWS ECR):
   İmajların internette saklandığı ve paylaşıldığı kütüphanedir.
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: DevOps ve Konteynerleştirme Uzmanı
Next.js ve Node.js 20 kullanan bir web uygulaması için production-ready `Dockerfile`
yazacağız.

## GÖREV:
1. 'Multi-Stage Build' (Çok Aşamalı Derleme) kullanarak derleme araçlarını nihai
imajdan temizle.
2. `node_modules` ve kaynak kodları non-root (root olmayan) güvenli bir kullanıcı
ile çalıştır.
3. İmaj boyutunu en aza indiren hafif `node:20-alpine` tabanını kullan.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# Kötü / Ağır Dockerfile: 1.2 GB boyut, root kullanıcısı, devDependencies
temizlenmemiş!
FROM node:20
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build
CMD ["npm", "start"] # Root olarak çalışır (BÜYÜK GÜVENLİK RİSKİ)
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
# Çok Aşamalı (Multi-Stage) Üretim Dockerfile

# AŞAMA 1: Bağımlılıkları Kur
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

# AŞAMA 2: Uygulamayı Derle
FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

# AŞAMA 3: Hafif & Güvenli Üretim Ortamı (Minimal Runner)
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production

# Güvenlik Kuralı: Asla 'root' kullanıcısı ile çalıştırma!
RUN addgroup --system --gid 1001 nodejs && \
    adduser --system --uid 1001 nextjs

# Sadece derlenmiş çıktıları kopyala (İmaj boyutu 1.2 GB yerine 80 MB!)
COPY --from=builder /app/public ./public
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./.next/static

USER nextjs
EXPOSE 3000
ENV PORT=3000

CMD ["node", "server.js"]
```

💡 **Kod Analizi:** Multi-stage yapı sayesinde derleyici ve dev paketler `builder` aşamasında çöpe atılmış, sadece saf JavaScript ve asset'ler `runner` aşamasına kopyalanarak imaj boyutu 1.2 GB'tan 80 MB'a düşürülmüş ve `USER nextjs` ile root çalıştırma riski önlenmiştir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Konteyneri varsayılan `root` kullanıcısı ile çalıştırıp sunucuda güvenlik açığı yaratmak.
  - **Çözüm:** Her zaman `USER node` veya özel bir non-root kullanıcı tanımlayın.
- **Tuzak:** `.dockerignore` dosyası oluşturmayıp `node_modules` ve `.env` dosyalarını imaja kopyalamak.

  - **Çözüm:** Proje köküne mutlaka kapsamlı bir `.dockerignore` dosyası ekleyin.
- **Tuzak:** Konteyner içinde veri saklayıp konteyner silindiğinde tüm veritabanını kaybetmek.
  - **Çözüm:** Kalıcı veriler için daima Docker Volumes (`-v db_data:/var/lib/postgresql/data`) kullanın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 49: Docker 101 - 'Benim Makinemde Çalışıyordu' Efsanesine Son*

- 🖥️ **Slayt 1:** Neden 'Benim makinemde çalışıyor' mazereti artık kabul edilemez?
- 🖥️ **Slayt 2:** Konteyner vs Sanal Makine (VM): Hafifliğin sırrı.
- 🖥️ **Slayt 3:** Multi-Stage Build ile 1.2 GB'lık imajı 80 MB'a düşürmek.
- 🖥️ **Slayt 4:** Canlı Demo: Sıfırdan Dockerfile yazıp imaj derlemek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir Next.js projesinde `docker build -t my-app .` çalıştırın. `docker images` çıktısında imaj boyutunun 80 MB olduğunu gösterin. `docker run -p 3000:3000 my-app` ile tarayıcıda uygulamanın ayağa kalktığını canlı gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Kişisel bir projeniz için çok aşamalı (multi-stage) bir `Dockerfile` ve `.dockerignore` dosyası yazarak yerel makinenizde imajı derleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Docker'da 'Multi-Stage Build' (Çok Aşamalı Derleme) kullanmanın en büyük faydası nedir? - A) Bilgisayara yeni monitör bağlamak - B) Derleme aşamasında kullanılan ağır araçları (compiler, TypeScript, npm cache) üretim imajından çöpe atarak nihai imajı hem çok hafif (MB düzeyinde) hem de saldırılara karşı daha güvenli kılmak - C) Sadece C++ dilinde çalışması - D) İnternet gerektirmemesi > **Doğru Yanıt:** B > **Açıklama:** Multi-stage builds, gereksiz kaynak kodları ve derleme araçlarını nihai imajın dışında bırakarak hafif ve güvenli konteynerler üretir.

Soru 2: Docker konteynerleri ile geleneksel Sanal Makineler (Virtual Machine - VM) arasındaki temel fark nedir? - A) Hiçbir farkı yoktur - B) Sanal makineler her biri için ayrı bir misafir işletim sistemi (Guest OS) kurarken; Docker konteynerleri ana makinenin Linux çekirdeğini (Kernel) ortaklaşa kullanarak saniyeler içinde ayağa kalkar - C) Docker sadece mobilde çalışır - D) VM'lerin ücretsiz olması > **Doğru**

Yanıt: B > **Açıklama:** Docker ana çekirdeği (Host Kernel) paylaştığı için çok daha az RAM harcar ve milisaniyeler içinde başlar.

50. Docker Compose ile Çok Servisli Yerel Geliştirme Ortamı

Tek Bir 'docker compose up' Komutuyla Tüm Şirket Altyapısını Ayağa Kaldırmak
Bölüm 9 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ **NotebookLM & Audio Overview Podcast Notları**

Podcast Tartışma Sorusu:
"Yeni katıldığınız bir şirkette projeyi çalıştırmak için 3 gün boyunca PostgreSQL, Redis, RabbitMQ kurup boğuşmak mı, yoksa tek bir 'docker compose up' ile 1 dakikada kodlamaya başlamak mı?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Docker Compose, çoklu konteyner uygulamalarını tek bir YAML dosyasında (`compose.yaml`) tanımlayıp yönetme aracıdır. - 📌 Konteynerler aynı Docker köprüsünde (bridge network) birbirlerine IP adresi yerine doğrudan servis adlarıyla (örn: `postgres:5432`) erişir. - 📌 Named Volume'lar, konteynerler silinse veya yeniden başlatılsa bile veritabanı verilerinin kaybolmasını engeller.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir orkestrayı yönettiğinizi düşünün.

Konser saati geldiğinde kemancıya *"Sen hazırlan"*, 10 dakika sonra piyaniste *"Sen gel"*, sonra davulcuya *"Sen ayarla"* derseniz konser fiyaskoyla sonuçlanır!

Orkestra şefi sahneye çıkar, bagetini kaldırır: **"1, 2, 3, BAŞLA!"** dediği anda 50 müzisyen aynı anda tek bir armoniyle çalmaya başlar!

Docker Compose işte o orkestra şefidir! - Backend sunucusu - PostgreSQL veritabanı - Redis önbelleği - MailHog e-posta test sunucusu

`docker compose up` dediğiniz anda hepsi aynı saniyede ayağa kalkar, birbirine bağlanır ve siz anında kod yazmaya başlarsınız.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Docker Compose Ağ ve Veri Mimarisi:

```
[docker compose up]
        |
        ▼
┌──────────────────────────────────────────────────┐
|           ÖZEL DOCKER AĞI (Internal Bridge)      |
|                                                  |
|  [Servis: backend] ──(postgres:5432)──> [Servis: db]   |
|         |                                  |     |
|         | (redis:6379)                     ▼     |
|         ▼                                [NAMED VOLUME] |
|  [Servis: redis]                         (postgres_data)|
|                                           (Veri Kalıcı!)|
└──────────────────────────────────────────────────┘
        |
  (Port 3000:3000)
        ▼
[Geliştirici Tarayıcısı: localhost:3000]
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: DevOps ve Geliştirici Deneyimi (DevEx) Mimarı
Yerel geliştirme ortamımız için eksiksiz bir `compose.yaml` (Docker Compose) dosyası hazırlayacağız.

## GEREKSİNİMLER:
1. `app`: Mevcut Dockerfile'ı derlesin, 3000 portunu dışa açsın ve kod değişikliklerini anında yakalasın (volume mount).
2. `postgres`: PostgreSQL 16 Alpine kullansın, verileri `pg_data` named volume'unda saklasın ve sağlık kontrolü (healthcheck) içersin.
3. `redis`: Redis Alpine kullansın.
4. `app` servisi sadece PostgreSQL sağlık kontrolünden (healthy) geçtikten sonra başlasın (`depends_on`).
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# Kötü Compose Dosyası: Volume yok (veriler silinir), healthcheck yok, şifreler açık
services:
  db:
    image: postgres
  web:
    image: my-web-app
# db hazır olmadan web başlarsa bağlantı hatasıyla çöker!
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```yaml
services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgresql://postgres:gizlisifre123@postgres:5432/uygulamadb
      - REDIS_URL=redis://redis:6379
    volumes:
      - .:/app
      - /app/node_modules # Host makinenin node_modules'ünü ezmeyi engeller
    depends_on:
      postgres:
        condition: service_healthy # Veritabanı TAM hazır olana kadar bekle!

  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: gizlisifre123
      POSTGRES_DB: uygulamadb
    ports:
      - "5432:5432"
    volumes:
      - pg_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d uygulamadb"]
      interval: 5s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

volumes:
  pg_data: # Konteyner silinse bile veritabanı verisi diske kaydedilir!
```

💡 **Kod Analizi:** `depends_on: condition: service_healthy` sayesinde backend servisi veritabanı gerçekten hazır olana kadar bekleterek başlangıç çöküşlerini engeller; `pg_data` named volume ile veritabanı kalıcılığı garanti edilir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Konteyner içi bağlantılarda `localhost:5432` yazıp bağlanamamak.
  - **Çözüm:** Konteynerler birbirine `localhost` ile değil, servis adıyla (`postgres:5432`) bağlanır.
- **Tuzak:** Named volume kullanmayıp `docker compose down` yapınca tüm veritabanının silinmesi.
  - **Çözüm:** Veritabanı servislerinde mutlaka kalıcı named volume tanımlayın.
- **Tuzak:** Büyük projelerde `docker compose up` yapıp logları okuyamamak.
  - **Çözüm:** `docker compose up -d` ile arka planda başlatıp sadece ilgilendiğiniz servisin loglarını izleyin (`docker compose logs -f app`).

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

🎥 Ders Başlığı: *Ders 50: Docker Compose - Tek Komutta Tüm Altyapı*

- 🖥️ **Slayt 1:** Docker Compose nedir ve çoklu servisleri nasıl orkestre eder?
- 🖥️ **Slayt 2:** Konteynerler arası iç ağ (Internal Network) ve servis adlandırma.
- 🖥️ **Slayt 3:** Service Healthchecks ve depends_on ile sıralı başlatma.
- 🖥️ **Slayt 4:** Canlı Demo: Tek komutla Next.js + Postgres + Redis ayağa kaldırmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Terminalde `docker compose up` çalıştırın. PostgreSQL'in sağlık kontrolünden geçip yeşil yandığını, ardından backend servisinin ayağa kalktığını gösterin. `docker compose down` ve tekrar `up` yaparak veritabanı kayıtlarının korunduğunu gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projeniz için bir `compose.yaml` dosyası oluşturun. İçine bir PostgreSQL ve bir Redis servisi ekleyip `docker compose up -d` ile ayağa kaldırın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Docker Compose ağında bir backend servisi PostgreSQL veritabanına bağlanırken host adresi olarak ne yazmalıdır? - A) localhost - B) 127.0.0.1 - C) compose.yaml dosyasında PostgreSQL servisine verilen isim (Örn: `postgres` veya `db`) - D) Bilgisayarın MAC adresi > **Doğru Yanıt:** C > **Açıklama:** Docker Compose dahili DNS sunucusuna sahiptir; servisler birbirlerine doğrudan servis isimleriyle (örn: `postgres:5432`) erişir.

Soru 2: Docker Compose'da veritabanı servisinin hazır olmasını beklemek için hangi mekanizma kullanılır? - A) setTimeout - B) `healthcheck` ve `depends_on: condition: service_healthy` - C) Bilgisayarı kapatıp açmak - D) Sadece `ports` tanımı > **Doğru Yanıt:** B > **Açıklama:** Healthcheck veritabanının soket bağlantılarını kabul ettiğini teyit eder; `depends_on` ise bağımlı servisi o an gelene kadar bekletir.

51. Bulut Bilişime Giriş: AWS, GCP ve Azure Temel Kavramları

Kendi Sunucunu Yönetme Zahmetinden Bulutun Sınırsız Gücüne Geçiş
Bölüm 9 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"AWS konsolunda 200'den fazla servis var. Junior bir mühendis bunların hepsini bilmek zorunda mı, yoksa sadece 4 temel yapı taşını bilmek yeterli mi?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Tüm bulut servisleri 4 ana yapı taşı üzerine kuruludur: Compute (İşlemci), Storage (Depolama), Database (Veritabanı) ve Networking (Ağ). - 📌 EC2 kiralık sanal bir bilgisayardır; S3 sınırsız bir dosya deposudur; RDS yönetilen bir veritabanıdır. - 📌 Bulut maliyetleri kullandıkça öde (Pay-as-you-go) modelidir; kapatılmayan test sunucuları devasa faturalar çıkarabilir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Evinizde elektrik üretmek istediğinizi düşünün.

Bahçenize jeneratör kurup, her gün benzin alıp, motorun yağını değiştirip, gece sesinden uyuyamaz mıydınız? Tabii ki hayır! Duvardaki prize bir fiş takarsınız. Elektrik idaresi elektriği üretir, siz sadece ay sonunda harcadığınız kilovatsaat kadar fatura ödersiniz.

Bulut Bilişim (Cloud Computing) işte o elektrik prizidir!

Eskiden şirketler ofislerinin bodrum katına devasa sunucu odaları kurar, klima arızalarıyla boğuşur, hard disk değiştirirdi (On-Premise). Bugün AWS, Google Cloud veya Azure sayesinde tek tıkla dünyanın en güçlü veri merkezlerinden prizi takıyor ve sadece kullandığımız saniye kadar ödeme yapıyoruz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Bulutun 4 Temel Yapı Taşı Karşılaştırması:

Yapı Taşı	AWS Karşılığı	GCP Karşılığı	Ne İşe Yarar?
Compute (İşlem)	EC2 / ECS / Lambda	Compute Engine / Cloud Run	Kodu çalıştıran sanal işlemciler ve RAM
Storage (Depolama)	S3 (Simple Storage)	Cloud Storage	Resimler, videolar, yedekler (Sınırsız nesne deposu)
Database (Veri)	RDS / Aurora / DynamoDB	Cloud SQL / Firestore	Otomatik yedeklenen, bakımı yapılan veritabanı
Networking (Ağ)	VPC / Route 53 / ALB	VPC / Cloud DNS / Load Balancing	Güvenlik duvarları, alan adı ve yük dengeleme

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Bulut Çözüm Mimarı (AWS Certified Solutions Architect)
Bir startup'ın ilk web uygulamasını AWS üzerinde ayağa kaldıracağız.

## GEREKSİNİMLER:
- 1 adet Docker konteynerinde çalışan Node.js API
- 1 adet PostgreSQL veritabanı
- Kullanıcı profil fotoğraflarının yükleneceği dosya deposu

## GÖREV:
1. En ucuz ve yönetimi en kolay AWS servis kombinasyonunu öner (Örn: App Runner / ECS Fargate + RDS + S3).
2. S3 bucket'ına dosya yüklerken güvenliği sağlayan 'Presigned URL' mimarisini açıkla.
3. Maliyet alarmı (AWS Budget Alert) kurmanın adımlarını özetle.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Kötü Bulut Mimarisi: Kullanıcı resimlerini doğrudan web sunucusunun diskine kaydetmek
// Sunucu yeniden başladığında veya 2. sunucu açıldığında tüm resimler kaybolur!
app.post("/upload", (req, res) => {
  fs.writeFileSync(`/var/www/uploads/${req.body.name}`, req.body.file);
});
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
import { S3Client, PutObjectCommand } from "@aws-sdk/client-s3";
import { getSignedUrl } from "@aws-sdk/s3-request-presigner";

const s3 = new S3Client({ region: process.env.AWS_REGION });

// Mükemmel Mimari: S3 Presigned URL ile Doğrudan Güvenli Yükleme
// Sunucu dosya yüküyle yorulmaz; istemci doğrudan S3'e güvenle yükler!
export async function generateAvatarUploadUrl(userId: string): Promise<string> {
  const fileKey = `avatars/${userId}-${Date.now()}.png`;

  const command = new PutObjectCommand({
    Bucket: process.env.S3_BUCKET_NAME,
    Key: fileKey,
    ContentType: "image/png"
  });

  // 15 Dakika geçerli tek kullanımlık imzalı yükleme linki üret
  return await getSignedUrl(s3, command, { expiresIn: 900 });
}
```

💡 **Kod Analizi:** Presigned URL deseni sayesinde kullanıcı dosyayı web sunucunuza değil, doğrudan S3'e yükler. Web sunucunuzun işlemcisi ve belleği dosya yükleme trafiğiyle boğulmaz.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Kullanıcı dosyalarını sunucunun yerel diskine kaydetmek (Konteyner kapanınca silinir!).
  - **Çözüm:** Statik medyayı daima S3 veya Google Cloud Storage gibi Nesne Depolarında (Object Storage) saklayın.
- **Tuzak:** AWS hesabında bütçe alarmı (Budget Alert) kurmayı unutmak.
  - **Çözüm:** İlk gün 10$ ve 50$ eşiklerinde e-posta gönderen bir bütçe alarmı kurun.
- **Tuzak:** S3 bucket'larını internete tamamen açık (Public Read/Write) bırakıp veri sızdırmak.

- **Çözüm:** Bucket'larda 'Block Public Access' ayarını açık tutun; erişimi Presigned URL ile sınırlayın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 51: Bulut Bilişim 101 - AWS, GCP ve Temel Yapı Taşları*

- 🖥️ **Slayt 1:** Neden artık kimse bodrumda sunucu kurmuyor? Bulutun gücü.
- 🖥️ **Slayt 2:** 4 Temel Taş: Compute, Storage, Database, Networking.
- 🖥️ **Slayt 3:** S3 Presigned URL ile sunucuyu yormadan güvenli dosya yükleme.
- 🖥️ **Slayt 4:** AWS Bütçe Alarmları ile sürpriz faturalardan korunmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
AWS konsoluna girin (veya LocalStack ile simüle edin). S3 üzerinde bir bucket açın. Node.js SDK ile 15 saniyede bir presigned URL üretip curl ile dosyayı doğrudan S3'e yüklemeyi canlı gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

AWS veya Google Cloud hesabınızda ilk bütçe alarmınızı (Budget Alert: 10$) kurun ve bildirim e-postanızı ekleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: AWS S3 (Simple Storage Service) gibi Nesne Depoları (Object Storage) geleneksel sunucu disklerinden hangi özelliğiyle ayrılır? - A) Sadece müzik saklayabilmesi - B) Sınırsız depolama kapasitesi sunması, yüksek dayanıklılık (%99.999999999) sağlaması ve dosyalara doğrudan HTTP üzerinden erişilebilmesi - C) Bilgisayara takılan bir USB bellek olması - D) Sadece Python ile çalışması > **Doğru Yanıt:** B > **Açıklama:** S3, sabit disk boyutu kısıtlaması olmadan petabaytlarca veriyi güvenle ve yüksek dayanıklılıkla saklayan dağıtık bir nesne deposudur.

Soru 2: Bir web uygulamasında dosya yüklerken 'Presigned URL' kullanmanın sağladığı en büyük mimari fayda nedir? - A) Dosyanın rengini değiştirmesi - B) İstemcinin dosyayı doğrudan S3'e yüklemesini sağlayarak backend sunucusunun işlemci, bellek ve bant genişliği darboğazına girmesini engellemesi - C) Sadece Chrome'da çalışması - D) Veritabanını silmesi > **Doğru Yanıt:** B > **Açıklama:** Presigned URL, dosya trafiğini uygulama sunucusundan arındırarak yükü doğrudan S3 altyapısına devreder.

52. Sunucusuz (Serverless) Mimariler & Edge Computing

Sıfır Sunucu Yönetimi, Soğuk Başlangıçlar (Cold Start) ve Küresel Dağıtım
Bölüm 9 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Serverless mimaride gerçekten 'sunucu yok' mu? AWS Lambda ve Vercel Edge Functions kodu dünyanın dört bir yanındaki kullanıcılara 5 milisaniyede nasıl ulaştırır?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Serverless 'sunucusuz' demek değildir; sunucuların yönetimini ve ölçeklenmesini bulut sağlayıcısına devretmektir. - 📌 Kullanılmadığında 0 maliyet üretir; trafik geldiğinde saniyenin onda birinde otomatik olarak 1'den 10.000 örneğe fırlar. - 📌 Soğuk Başlangıç (Cold Start) sorununu aşmak için hafif edge çalışma zamanları (Cloudflare Workers / Vercel Edge) tercih edilmelidir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir taksi durağını ve kiralık bir arabayı düşünün.

Eğer bir araba kiralarsanız (Geleneksel Sunucu / EC2): Arabayı sürseniz de, garajda park halinde dursa da her ay 1000 dolar kira ödersiniz.

Peki ya taksi çağırırsanız (Serverless / Lambda)? Taksi kapınıza gelir, sizi gitmek istediğiniz yere götürür ve sadece gittiğiniz kilometre kadar ücret ödersiniz. Taksi bittikten sonra gider; arabanın vergisiyle, sigortasıyla, bakımıyla siz uğraşmazsınız!

Edge Computing ise durağın hemen her köşe başında olmasıdır: Taksi 30 saniye içinde kapınızdadır!

Günde sadece 50 istek alan bir servis için ayda 50 dolar sunucu parası ödemek yerine, serverless ile ayda 0.01 dolar ödersiniz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Geleneksel Sunucu vs Serverless vs Edge:

```
┌──────────────────────────────────────────────────┐
│                MİMARİ KARŞILAŞTIRMASI            │
└──────────────────────────────────────────────────┘
                          │
                          │
        ┌─────────────────┼─────────────────┐
        ▼                 ▼                 ▼
[1. GELENEKSEL SUNUCU] [2. SERVERLESS (Lambda)] [3. EDGE COMPUTING]
- 7/24 çalışır, ödersin- İstek gelince uyanır   - Kullanıcıya en yakın
- Node.js tam ortamı   - Boştayken 0$ fatura     veri merkezinde koşar
- Cold Start yok       - Cold Start: 200-500 ms - Cold Start: < 5 ms
- Konum: Tek bir bölge - Konum: Bölgesel        - Konum: 300+ Şehir
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Serverless ve Edge Mimarı
Vercel Edge Functions veya Cloudflare Workers kullanarak küresel ölçekte çalışan bir
'Geolokasyon Yönlendirme Servisi' yazacağız.

## GÖREV:
1. Kullanıcının hangi ülkeden geldiğini tespit eden bir Edge Function kodu yaz.
2. V8 Isolate mimarisinin neden Node.js sanal makinelerine göre 100 kat daha hızlı
ayağa kalktığını açıkla.
3. Serverless fonksiyonlarda veritabanı bağlantı havuzu (Connection Pooling)
patlamasını önlemek için Prisma Accelerate / Neon Serverless kullanımını özetle.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Kötü Serverless Kullanımı: Her fonksiyon çağrısında yeni DB bağlantısı açmak
// 1000 eşzamanlı istek geldiğinde veritabanının bağlantı havuzu (Max Connections)
patlar!
import { Client } from 'pg';

export async function handler() {
  const client = new Client(process.env.DB_URL);
  await client.connect(); // HATA: Her lambdada yeni bağlantı açılırsa PostgreSQL
çöker!
  const res = await client.query('SELECT * FROM config');
  await client.end();
  return res.rows;
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
// Modern Edge Fonksiyonu: Sub-10ms Gecikme ile Küresel Yürütme (api/geo.ts)

export const config = {
  runtime: "edge" // V8 Isolate üzerinde çalışır (Cold Start < 5ms)
};

export default async function handler(request: Request) {
  // Edge runtime otomatik olarak kullanıcının coğrafi konumunu başlıkta sunar
  const country = request.headers.get("x-vercel-ip-country") || "TR";
  const city = request.headers.get("x-vercel-ip-city") || "Istanbul";

  const localizedGreeting = country === "TR"
    ? "Hoş geldiniz!"
    : "Welcome to our global platform!";

  return new Response(
    JSON.stringify({
      message: localizedGreeting,
      location: { country, city },
      servedAt: new Date().toISOString()
    }),
    {
      status: 200,
      headers: {
        "content-type": "application/json",
        "cache-control": "public, s-maxage=60, stale-while-revalidate=30"
      }
    }
  );
}
```

💡 **Kod Analizi:** `runtime: 'edge'` direktifi ile kod standart ağır container yerine Cloudflare/Vercel V8 Isolate motorunda koşar. Dünyanın neresinden istek gelirse gelsin kullanıcıya en yakın veri merkezinde 5 milisaniyede yanıt üretilir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Serverless fonksiyonlarda veritabanı bağlantı havuzu (Connection Pooler) kullanmamak.
  - **Çözüm:** Sunucusuz ortamlarda daima PgBouncer, Prisma Accelerate veya Neon gibi HTTP bağlantı havuzlayıcıları kullanın.
- **Tuzak:** Büyük ve uzun süren (>15 dakika) işlemleri Serverless fonksiyonlarda çalıştırmaya çalışmak.
  - **Çözüm:** Uzun süren işler (video işleme, büyük veri) için Lambda yerine arka plan işçileri (ECS/SQS) kullanın.
- **Tuzak:** Edge fonksiyonlarında Node.js'e özel modülleri (örn: `fs` veya `child_process`) çağırmak.
  - **Çözüm:** Edge çalışma zamanları standart Web API'lerini (fetch, Request, Response) destekler; dosya sistemi modüllerini çağırmayın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 52: Serverless ve Edge Computing - Sunucusuz Gelecek*

- 🖥️ **Slayt 1:** Serverless nedir? Sıfır maliyetten küresel ölçeğe geçiş.
- 🖥️ **Slayt 2:** Cold Start problemi ve V8 Isolate mimarisi.
- 🖥️ **Slayt 3:** Edge Runtimes: Kullanıcının 5 milisaniye yanında kod çalıştırmak.
- 🖥️ **Slayt 4:** Serverless veritabanı bağlantı havuzu yönetimi (Connection Pooling).

Canlı Kodlama / Ekran Kaydı Senaryosu:
Vercel üzerinde basit bir Edge Function deploy edin. Farklı ülkelerden VPN ile istek atarak yanıt süresinin her ülkede 10 milisaniye altında kaldığını ve x-vercel-ip-country başlığının doğru çalıştığını canlı gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Next.js veya Cloudflare Workers projenizde basit bir Edge rotası oluşturun ve kullanıcının istek başlıklarını inceleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Serverless (Sunucusuz) mimarinin geleneksel sanal makinelere (VM) göre en büyük ekonomik avantajı nedir? - A) Elektrik faturası ödememek - B) 'Scale-to-Zero' (Sıfıra Ölçeklenme) özelliği saye-

sinde hiç istek gelmediğinde 0 TL maliyet üretmesi; sadece çağrıldığı saniyeler için ödeme yapılması - C) Sadece C++ dilinde çalışması - D) Kodların silinmesi > **Doğru Yanıt:** B > **Açıklama:** Serverless servisler kullanılmadığı anlarda tamamen kapanır ve sıfır maliyet yaratır; trafik geldiğinde ise milisaniyeler içinde ayağa kalkar.

Soru 2: Serverless fonksiyonlarda 'Cold Start' (Soğuk Başlangıç) ne demektir? - A) Sunucunun kışın donması - B) Uzun süre çağrılmayan bir fonksiyon ilk kez tetiklendiğinde bulut sağlayıcının yeni bir mikro-konteyner ayağa kaldırması sebebiyle ilk istekte yaşanan küçük gecikme - C) Bilgisayarın fanının çalışması - D) Kodun derlenememesi > **Doğru Yanıt:** B > **Açıklama:** Boştaki fonksiyonlar uyutulur. Yeni bir istek geldiğinde ortamın sıfırdan başlatılması birkaç yüz milisaniyelik soğuk başlangıç gecikmesine yol açar.

53. Infrastructure as Code (IaC): Terraform ve Pulumi'ye AI Bakışı

Bulut Konsolunda Elle Tıklamayı Bırakıp Altyapıyı Kod Olarak Sürümlemek
Bölüm 9 • Zorluk Derecesi: İleri Seviye • Tahmini Okuma Süresi: 13 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"AWS konsolunda fareyle tıklayarak sunucu açmak (ClickOps) neden şirketler için saatli bir bombadır ve Terraform altyapıyı nasıl silinip yeniden kurulabilir kılar?"

Bölümden Çıkarılacak 3 Altın Cümle:
- 📌 ClickOps (Konsolda elle tıklayarak altyapı kurmak) belgelenemez, tekrarlanamaz ve insan hatasına davetiye çıkarır. - 📌 Infrastructure as Code (IaC), sunucuları, veritabanlarını ve güvenlik duvarlarını tıpkı uygulama kodu gibi Git'te versiyonlamaktır. - 📌 Terraform'un `plan` komutu, gerçek buluta dokunmadan önce hangi kaynakların oluşturulacağını veya silineceğini önizleme imkanı sunar.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir şehir kurduğunuzu hayal edin.

Belediye başkanı sokakları, su borularını ve elektrik hatlarını mühendislerin aklına estikçe elle kazarak yapmasına izin verirse ne olur? 6 ay sonra kimse hangi borunun nereden geçtiğini bilmez! Yeni bir bina yapmak için kazmayı vurduğunuz an ana su borusunu patlatırsınız! İşte konsolda elle tıklayarak sunucu kurmak (**ClickOps**) tam olarak bu kaostur.

Infrastructure as Code (Terraform) ise o şehrin 3 boyutlu CAD mimari planıdır.

Tek bir dosyada: - 1 adet PostgreSQL veritabanı - 2 adet yük dengeleyici - 1 adet S3 depolama alanı kod olarak yazılır. Şehir bir depremde yıkılsa bile tek bir komutla (`terraform apply`) 5 dakikada tüm şehri başka bir kıtada birebir yeniden inşa edebilirsiniz!

🧠 2. AI Çağında Yeni Zihniyet Modeli

Terraform Çalışma Döngüsü:

```
[main.tf Kodunu Yaz]
        |
        ▼
[1. terraform init] ──> Bulut sağlayıcı eklentilerini (AWS Provider) indirir.
        |
        ▼
[2. terraform plan] ──> "Neler değişecek?" (Önizleme yapar, ASLA değişiklik yapmaz!)
        |
        ▼
[3. terraform apply] ──> Gerçek buluta gider ve kaynakları inşa eder!
        |
        ▼
[terraform.tfstate] ──> Bulutun mevcut durumunu hafızasında tutan kilit dosya.
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: DevSecOps ve Terraform Mimarı
AWS altyapımız için Terraform (HCL) kodu yazacağız.

## GÖREV:
Aşağıdaki altyapıyı oluşturan modüler bir `main.tf` hazırla:
1. Şifrelenmiş (AES-256) ve public erişime kapalı bir AWS S3 bucket'ı.
2. Bu bucket'a dosya yüklendiğinde versiyonlamayı (versioning) aktif et.
3. Altyapıyı test etmek için `terraform plan` çıktısının nasıl okunacağını açıkla.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# Kötü Terraform: Güvensiz, şifreleme yok, public erişim açık!
resource "aws_s3_bucket" "insecure_bucket" {
  bucket = "sirket-gizli-veriler"
  # Şifreleme yok, versiyonlama yok!
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
# Profesyonel ve Güvenli Terraform Mimarisi (main.tf)

terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

# 1. S3 Depolama Alanı
resource "aws_s3_bucket" "secure_assets" {
  bucket        = "sirket-guvenli-depo-${var.environment}"
  force_destroy = false

  tags = {
    Environment = var.environment
    ManagedBy   = "Terraform"
  }
}

# 2. Versiyonlama Koruması (Yanlışlıkla silinmeyi önler)
resource "aws_s3_bucket_versioning" "versioning" {
  bucket = aws_s3_bucket.secure_assets.id
  versioning_configuration {
    status = "Enabled"
  }
}

# 3. Sunucu Taraflı Şifreleme (AES-256)
resource "aws_s3_bucket_server_side_encryption_configuration" "crypto" {
  bucket = aws_s3_bucket.secure_assets.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

# 4. İnternete Tamamen Kapat (Block Public Access)
resource "aws_s3_bucket_public_access_block" "public_block" {
  bucket = aws_s3_bucket.secure_assets.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

💡 **Kod Analizi:** Altyapı kodla tanımlandığı için şifreleme, versiyonlama ve public erişim kilidi gibi güvenlik standartları zorunlu kılınmış; insan dalgınlığıyla açık bir S3 bucket bırakma riski sıfırlanmıştır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** `terraform.tfstate` dosyasını GitHub'a commit etmek (Şifreler ve API anahtarları içerebilir!).
  - **Çözüm:** State dosyasını daima AWS S3 ve DynamoDB kilitli uzak backend'de (Remote Backend) saklayın.
- **Tuzak:** `terraform plan` çıktısını okumadan körü körüne `terraform apply` çalıştırmak.
  - **Çözüm:** Plan çıktısındaki kırmızı silme (`- destroy`) uyarılarını mutlaka tek tek inceleyin.
- **Tuzak:** Konsoldan (ClickOps) elle değişiklik yapıp Terraform koduyla uyumsuzluk (Drift) yaratmak.
  - **Çözüm:** Bir kaynak Terraform ile yönetiliyorsa, konsoldan elle müdahaleyi kesinlikle yasaklayın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 53: Infrastructure as Code - Terraform ile Bulutu Kodlamak*

- 🖥️ **Slayt 1:** ClickOps felaketi: Neden konsolda elle tıklamak yasaklanmalıdır?
- 🖥️ **Slayt 2:** IaC felsefesi: Altyapıyı Git ile versiyonlamak.
- 🖥️ **Slayt 3:** Terraform yaşam döngüsü: init, plan, apply, destroy.
- 🖥️ **Slayt 4:** Canlı Demo: 60 saniyede kodla AWS S3 bucket ayağa kaldırmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir `main.tf` dosyası oluşturun. `terraform init` ve `terraform plan` çalıştırın. Terminalde yeşil `+ create` satırlarını gösterin. `terraform apply` diyerek AWS'de (veya LocalStack'te) saniyeler içinde kaynağın oluştuğunu gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bilgisayarınıza Terraform kurun (`brew install terraform`) ve LocalStack veya AWS kullanarak basit bir depolama alanı oluşturan ilk `.tf` dosyanızı yazıp `terraform plan` alın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Terraform'da `terraform plan` komutunun temel işlevi nedir? - A) Buluttaki tüm sunucuları silmek - B) Yazılan kod ile bulutun mevcut durumunu karşılaştırarak, hangi kaynakların ekleneceğini, değiştirileceğini veya silineceğini buluta DOKUNMADAN güvenle önizlemek - C) Sadece fatura miktarını hesaplamak - D) Bilgisayarı kapatmak > **Doğru Yanıt:** B > **Açıklama:** `terraform plan`, altyapı üzerinde hiçbir yıkıcı işlem yapmadan önce geliştiriciye tam bir değişiklik önizlemesi (dry-run) sunar.

Soru 2: Terraform'un ürettiği `terraform.tfstate` dosyasını neden public bir Git reposuna commit etmemeliyiz? - A) Dosya çok büyük olduğu için - B) State dosyasının veritabanı şifreleri, API anahtarları gibi hassas bilgileri düz metin olarak içerebilmesi ve güvenlik açığı yaratması - C) Sadece Linux'ta çalışması - D) Kodun derlenmesini engellemesi > **Doğru Yanıt:** B > **Açıklama:** Terraform state dosyası buluttaki kaynakların tüm detaylarını (şifreler dahil) tutar; bu nedenle daima şifrelenmiş uzak backend'lerde (S3 vb.) saklanmalıdır.

54. Sıfır Kesintili Dağıtım (Zero-Downtime Deployment): Blue-Green & Canary

Cuma Akşamı Bile Korkusuzca Deploy Çıkma Sanatı
Bölüm 9 • Zorluk Derecesi: İleri Seviye • Tahmini Okuma Süresi: 13 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Milyonlarca kullanıcısı olan bir banka veya e-ticaret sitesi yeni versiyon yayınlarken 'Sitemiz bakım nedeniyle kapalıdır' yazısı koyamaz! Sıfır kesinti nasıl sağlanır?"

Bölümden Çıkarılacak 3 Altın Cümle:
- 📌 Zero-Downtime Deployment, kullanıcılar hiçbir kesinti yaşamadan yeni sürümün yayına alınmasıdır. - 📌 Blue-Green Dağıtım: Eski sürüm (Blue) ve yeni sürüm (Green) yan yana çalışır; yönlendirici tek bir anahtarla trafiği yeniye aktarır. - 📌 Canary (Kanarya) Dağıtım: Yeni sürüm önce kullanıcıların sadece %5'ine sunulur; hata yoksa kademeli olarak %100'e çıkarılır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Eski zamanlarda kömür madencilerini düşünün.

Madenciler yerin yüzlerce metre altına inerken yanlarında küçük sarı bir kanarya kuşu (Canary) taşırlardı. Kömür madeninde zehirli gaz (karbonmonoksit) sızarsa, kanarya kuşunun bünyesi küçük olduğu için hemen bayılır ve madencileri uyarırdı! Madenciler kuşu görünce hemen maskelerini takar ve canlarını kurtarırdı!

Yazılımda **Canary Deployment** işte o madenci kuşudur!

Yeni bir güncelleme yazdığınızda bunu hemen tüm kullanıcılara açmazsınız. Önce trafiğin sadece %1'ine (kanaryalara) açarsınız. Eğer Sentry'de hata oranı artmazsa güncellemeyi %10, %50 ve %100'e çıkarırsınız. Bir hata olursa sadece o %1 etkilenir ve sistem 3 saniyede eski haline geri döner (Rollback).

🧠 2. AI Çağında Yeni Zihniyet Modeli

Blue-Green vs Canary Dağıtım Mimarisi:

```
BLUE-GREEN DAĞITIM:
[Yük Dengeleyici (Router)]
         |
         ├──(Trafik: %100)──> [BLUE: Mevcut Sürüm v1.0] (Canlıda)
         |
         └──(Trafik: %0)─────> [GREEN: Yeni Sürüm v2.0] (Test Ediliyor)
[Geçiş Anı: Router yönü değiştirir; Green %100 olur, Blue yedekte bekler!]

_____________________________________________________________________

CANARY DAĞITIM:
[Yük Dengeleyici]
         |
         ├──(Trafik: %95)─────> [STABLE: Mevcut Sürüm v1.0]
         |
         └──(Trafik: %5)─────> [CANARY: Yeni Sürüm v2.0] (Gözlemleniyor)
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: SRE ve Sürüm Dağıtım Yöneticisi
Üretim ortamımızda sıfır kesintili (Zero-Downtime) bir Canary dağıtım stratejisi
kurgulayacağız.

## GÖREV:
1. Blue-Green ile Canary dağıtım arasındaki mimari ve maliyet farklarını
karşılaştır.
2. Otomatik geri alma (Automated Rollback) tetikleyicisi olan bir senaryoyu tasarla
(Örn: HTTP 500 oranı %1'i geçerse anında eski sürüme dön).
3. Veritabanı şema güncellemelerinde (Migrations) sıfır kesinti sağlamak için
'Expand and Contract' desenini açıkla.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# Kötü Dağıtım: Rolling update veya anında kapatıp açma (Kesinti yaşanır!)
# Eski sunucu kapanır, yenisi 45 saniyede ayağa kalkarken gelen kullanıcılar 502 Bad
Gateway alır!
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```yaml
# Kubernetes / Cloud Canary Dağıtım Yapılandırması (Canary Rollout)
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: api-service
spec:
  replicas: 10
  strategy:
    canary:
      # Kademeli Trafik Dağıtımı
      steps:
        - setWeight: 5    # 1. Adım: Sadece %5 trafik yönlendir
        - pause: { duration: 10m } # 10 dakika boyunca Sentry / Prometheus izle!
        - setWeight: 20   # 2. Adım: %20'ye çıkar
        - pause: { duration: 10m }
        - setWeight: 50   # 3. Adım: %50'ye çıkar
        - pause: { duration: 5m }

      # Otomatik Geri Alma (Automated Rollback Kuralı)
      analysis:
        templates:
          - templateName: error-rate-check
        args:
          - name: max-error-rate
            value: "0.01" # Hata oranı %1'i geçerse ANINDA ESKİYE DÖN!
```

💡 **Kod Analizi:** `Argo Rollouts` kullanılarak trafik önce %5'e verilir ve Prometheus metrikleri incelenir. Hata oranı %1'i aşarsa sistem insan müdahalesine gerek kalmadan milisaniyeler içinde eski sürüme geri döner.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Veritabanında kolon silip eski sürümün çökmesine yol açmak (Breaking Migration).
  - **Çözüm:** Expand-Contract desenini uygulayın: Önce yeni kolonu ekleyin, iki sürümü de destekleyin, eski kolonu en son silin.
- **Tuzak:** Rollback (Geri alma) stratejisini önceden test etmeyip kriz anında paniklemek.
  - **Çözüm:** Her deploy öncesi tek tuşla geri alma planınızın çalıştığından emin olun.
- **Tuzak:** Canary dağıtımda logları ve metrikleri izlemeden trafiği hemen %100'e çekmek.
  - **Çözüm:** Her kademede en az 5-10 dakika metriklerin oturmasını bekleyin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 54: Sıfır Kesintili Dağıtım - Blue-Green ve Canary Stratejileri*

- 🖥️ **Slayt 1:** Neden 'Sitemiz bakımda' dönemi tarihe karıştı?
- 🖥️ **Slayt 2:** Blue-Green Dağıtım: Anında anahtarlama ve yedekleme.
- 🖥️ **Slayt 3:** Canary Dağıtım: Madenci kanaryası gibi hata avlamak.
- 🖥️ **Slayt 4:** Bölüm 9 Özeti: DevOps, Konteynerler ve Bulut Mimarisi.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Yerel Kubernetes (Minikube) veya Docker üzerinde iki versiyon ayağa kaldırın. Bir yük dengeleyici üzerinden gelen 100 isteğin 95'inin v1'e, 5'inin v2'ye gittiğini canlı log akışında kanıtlayın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizin deployment sürecini inceleyin. Bir hata olduğunda eski sürüme dönmenin (Rollback) kaç dakika sürdüğünü ve bu sürecin otomatikleştirilip otomatikleştirilemeyeceğini analiz edin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Canary (Kanarya) Dağıtım stratejisinin birincil amacı nedir? - A) Kuş resimleri göstermek - B) Yeni yazılım sürümünü tüm kullanıcılara açmadan önce küçük bir kullanıcı grubuna (%1-%5) sunarak, olası hataların tüm sistemi çökertmesini engellemek ve riski minimize etmek - C) Sadece gece yarısı güncelleme yapmak - D) Sunucu masraflarını iki katına çıkarmak > **Doğru Yanıt:** B > **Açıklama:** Canary dağıtım, yeni sürümü küçük bir kullanıcı grubunda test eder; bir hata varsa sadece o küçük grup etkilenir ve sistem anında eski sürüme geri döner.

Soru 2: Veritabanı şemasında sıfır kesintili (Zero-Downtime) değişiklik yapmayı sağlayan mimari desene ne ad verilir? - A) Kapat-Aç Deseni - B) Expand and Contract (Genişlet ve Daralt) Deseni - C) SQL Silme Deseni - D) Rastgele Güncelleme > **Doğru Yanıt:** B > **Açıklama:** Expand and Contract deseni, önce yeni yapıyı ekleyip hem eski hem yeni kodu destekler (Expand), ardından eski kodu ve alanı temizler (Contract); böylece kesinti yaşanmaz.

Bölüm 10: Kariyer Hızlandırma, İletişim & Kıdemli Zihniyeti

Teknik liderlik, mülakat hazırlığı, mentörlük ve AI çağında aranan 10x mühendis olma vizyonunu edinin.

55. Junior’dan Mid-Level’a Geçişin Haritası: Hangi Metriklerle Ölçülürsün?

Sürekli Yardım İsteyen Geliştiriciden Bağımsız Çözüm Üreten Mühendise Dönüşmek
Bölüm 10 • Zorluk Derecesi: İleri Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Bir yazılımcının unvanını 'Junior'dan 'Mid-Level'a taşıyan şey yazdığı kod satırı sayısı mıdır, yoksa aldığı sorumluluğun ve özerkliğin (autonomy) seviyesi midir?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Junior adım adım yönlendirme (hand-holding) bekler; Mid-Level belirsiz bir problemi alıp bağımsız çözer. - 📌 Terfi etmenin en hızlı yolu, sadece kodu değil; testini, dokümantasyonunu ve deployment'ını sahiplenmektir (End-to-End Ownership). - 📌 Senior mühendisin vaktini koruyan junior, ekipteki en değerli junior'dır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

İki marangoz çırağını hayal edin:

Çırak Can: Ustanın yanına her 5 dakikada bir gelir: - *"Usta bu tahtayı hangi testereyle keseyim?"* - *"Usta bu vidayı nereye sıkayım?"* - *"Usta cetveli bulamadım, ne yapayım?"* Usta iş yapamaz hale gelir, akşama kadar Can'ı yönetir!

Çırak Elif: Usta sabah gelir ve der ki: *"Bize bu odaya sığacak bir kitaplık lazım."* Elif odayı ölçer, taslağı çizer, tahtaları keser, monte eder ve akşam ustaya bitmiş kitaplığı sunar: *"Usta kitaplığı yaptım, sağlamlığını test ettim, verniğini attım. İncelemek ister misin?"*

Usta Elif'e bakar ve der ki: *"Sen artık çırak değilsin, sen artık bir kalfasın (Mid-Level)!"*

Yazılımda da terfi formülü budur: Soru sormak ayıp değildir; ama ödevini yapıp çözümlerle gelen mühendis ışık hızında yükselir.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Mühendislik Seviyeleri ve Sorumluluk Matrisi:

Seviye	Problem Tanımı	Çözüm Yolu	Beklenen Çıktı
Junior	Çok net, küçük bilet (Örn: “Butonun rengini mavi yap”)	Senior yönlendirmesiyle adım adım	Çalışan temiz kod
Mid-Level	Orta belirsizlik (Örn: “Kupon doğrulama servisini yaz”)	Bağımsız mimari araştırma + AI ile çözüm	Test edilmiş, deploy edilmiş servis
Senior	Yüksek belirsizlik (Örn: “Ödeme altyapısını ölçeklendir”)	Stratejik mimari karar, risk yönetimi	Takımın yönü ve sistem dayanıklılığı

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Mühendislik Direktörü ve Kariyer Mentoru
Ben bir teknoloji şirketinde 1 yıldır çalışan ve Mid-Level seviyesine terfi etmek
isteyen bir Junior mühendisim.

## GÖREV:
1. Bir sonraki 6 ay içinde Mid-Level seviyesine çıkmamı sağlayacak 4 temel somut
yetkinliği listele (Bağımsızlık, Test Kalitesi, İletişim, Hata Çözme).
2. Senior mühendislere soru sorarken 'Senior Vaktini Koruyan Soru Sorma Şablonu'nu
hazırla.
3. Yönetici ile yapacağım 1-on-1 toplantısında terfi beklentimi profesyonelce
konuşma senaryosu sun.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# Kötü Soru Sorma Şekli (Senior'ı Tüketen):
"Selam abi, benim kod hata veriyor baksana bi."
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
# Profesyonel ve Senior Vaktini Koruyan Soru Sorma Şablonu:

"Selam Can, [Jira-405] biletinde ödeme webhook'unu test ederken bir sorunla
karşılaştım.

1. **GÖZLEM:** Stripe'tan gelen webhook'ta imza doğrulaması başarısız oluyor
(`SignatureVerificationError`).
2. **DENEDİKLERİM:**
   - Ham body'nin (`rawBody`) bozulup bozulmadığını kontrol ettim.
   - Webhook secret anahtarının doğruluğunu test ettim.
   - Sentry loglarında gelen header'ı inceledim.
3. **HİPOTEZİM:** Express'teki `bodyParser.json()` middleware'i raw body buffer'ını
parse ettiği için imza eşleşmiyor olabilir.
4. **SORUM:** Daha önce bu projede webhook rotası için `bodyParser.raw()` muafiyeti
uygulanmış bir örnek var mı, nereden bakmamı önerirsin?"
```

💡 **Kod Analizi:** Bu şablon sayesinde senior mühendis sorunu 30 saniyede anlar; junior'ın çaba gösterdiğini ve hipotez kurduğunu görerek büyük saygı duyar.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Takıldığı an hiçbir araştırma yapmadan anında senior'ın masasına koşmak.
  - **Çözüm:** Soruyu sormadan önce en az 20 dakika AI ve dökümantasyon ile araştırma yapıp hipotezlerinizi listeleyin.
- **Tuzak:** Haftalarca sessizce takılıp kalıp (stuck) kimseye hiçbir şey söylememek.
  - **Çözüm:** Eğer bir problemde 45 dakika boyunca hiçbir ilerleme kaydedemediyseniz profesyonelce yardım isteyin.
- **Tuzak:** Sadece kendisine verilen görevi yapıp sistemin bütünüyle ilgilenmemek.
  - **Çözüm:** Deploy süreçlerini, logları ve müşteri geri bildirimlerini merak edip takip edin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 55: Junior'dan Mid-Level'a Terfi - Özerklik ve Sahiplik*

- 🖥️ **Slayt 1:** Junior vs Mid-Level: Kod yazma hızından bağımsızlık seviyesine.
- 🖥️ **Slayt 2:** Senior vaktini koruyan soru sorma sanatı.
- 🖥️ **Slayt 3:** Uçtan uca sahiplik (End-to-End Ownership) felsefesi.
- 🖥️ **Slayt 4:** 1-on-1 toplantılarında terfi beklentisini yönetmek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir slack mesajı simülasyonu yapın. Kötü ve tembel bir soru ile profesyonel '4 Adımlı Soru Şablonu'nu yan yana koyarak senior mühendislerin verdiği tepkiyi ve kazanılan saygıyı tartışın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Takıldığınız bir sonraki teknik problemde 'Senior Vaktini Koruyan Soru Şablonu'nu kullanarak mesajınızı hazırlayın ve öyle yardım isteyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Bir yazılım mühendisinin 'Mid-Level' seviyesine ulaştığını gösteren en belirgin performans sinyali nedir? - A) Günde 2000 satır kod yazması - B) Kendisine verilen orta karmaşıklıktaki bir görevi, sürekli yönlendirmeye ihtiyaç duymadan mimari araştırmasını yaparak, testlerini yazarak ve uçtan uca sahiplenerek bağımsız (autonomous) çözebilmesi - C) Şirketin en eski çalışanı olması - D) Klavyeye bakmadan yazabilmesi > **Doğru Yanıt:** B > **Açıklama:** Mid-level seviyesinin temel tanımı 'Özerklik'tir (Autonomy). Sürekli yönlendirme gerektirmeden işi teslim edebilen mühendis artık junior değildir.

Soru 2: Kıdemli bir mühendise soru sorarken yapılması gereken en profesyonel davranış nedir? - A) Sadece 'kod çalışmıyor' yazıp beklemek - B) Sorunu özetlemek, neleri denediğini belirtmek, kendi hipotezini sunmak ve spesifik bir konuda görüş istemek - C) Soru sormaktan tamamen vazgeçip işi bırakmak - D) Gece yarısı aramak > **Doğru Yanıt:** B > **Açıklama:** Ödevini yapmış, denediği yolları ve hipotezini sunan geliştirici hem senior'ın vaktini korur hem de profesyonel olgunluğunu kanıtlar.

56. Teknik İletişim: Ürün Yöneticileri ve Tasarımcılarla Konuşma Sanatı

'Jargon' Duvarını Yıkıp İş Değeriyle Konuşan Mühendis Olmak
Bölüm 10 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Yazılımcılar neden 'Bu kod çok çirkin, refactor etmemiz lazım' dediğinde patron bütçe vermez de, 'Bu refactor ödeme dönüşüm oranını %12 artıracak' dediğinde anında onaylar?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Şirketler kod için değil; müşterilere değer üretmek ve ciro kazanmak için vardır. - 📌 Ürün Yöneticisi (PM) ve Tasarımcılarla konuşurken teknik jargonu (ORM, Big-O, Pod) bırakıp iş etkisine (Hız, Maliyet, Kullanıcı Kaybı) odaklanın. - 📌 Bir şeye 'Olmaz' demek yerine her zaman 'Şu maliyetle ve şu takasla (trade-off) yapılabilir' alternatifini sunun.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir oto tamircisine gittiğinizi düşünün.

Tamirci kaputu açıyor ve size şöyle diyor: *"Efendim krank milinin eksantrik segmanındaki mikrometre toleransı 0.04 bar sürtünme oluşturuyor, subapları rektifiye etmemiz lazım!"* Ne anlarsınız? Hiçbir şey! Korkarsınız ve tamirciye güvenmezsiniz.

Peki tamirci şöyle derse: *"Aracınızın motorundaki şu parça aşınmış. Eğer şimdi 500 TL'ye değiştirmezsek, haftaya uzun yolda motor yanar ve 50.000 TL masraf çıkar."* Anında onaylarsınız!

Yazılımda Ürün Yöneticileri (Product Manager) ve CEO'lar tam olarak o araba sahibidir. Onlara: *"PostgreSQL connection pool tükendi"* derseniz anlamazlar. *"Sunucu yoğunlukta siparişleri kaçırıyor ve günde 10.000 TL kaybediyoruz"* derseniz bütçeyi hemen alırsınız.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Jargondan İş Değerine Tercüme Sözlüğü:

```
TEKNİK JARGON (Yazılımcı Dili)          ——> İŞ DEĞERİ (Yönetici Dili)
-------------------------------------------------------------------------
"Teknik borcumuz çok birikti."          ——> "Geliştirme hızımız yarıya düştü;
                                             yeni özellikleri 2 kat yavaş
çıkarıyoruz."

"Database index'i eklememiz lazım."     ——> "Arama sayfası 6 saniyeden 0.5 saniyeye
                                             inecek ve sepeti terk edenler azalacak."

"Test yazmamız gerekiyor."              ——> "Canlıya güncelleme çıkarken sistemin
                                             çökmesini ve müşteri kaybını
önleyeceğiz."

"AWS sunucu boyutunu büyütelim."        ——> "Aylık 20$ maliyetle cuma yoğunluğundaki
                                             kesintileri sıfıra indireceğiz."
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Teknik Ürün Yöneticisi ve İletişim Koçu
Ben bir backend mühendisiyim. Ürün Yöneticimize (Product Manager) 2 haftalık bir
'Teknik Borç ve Performans İyileştirme' sprinti için bütçe/zaman vermesi gerektiğini
anlatmak istiyorum.

## GÖREV:
1. Mühendislerin yaptığı en yaygın 3 iletişim hatasını listele.
2. Teknik borcu 'Para, Hız ve Müşteri Memnuniyeti' metrikleriyle anlatan ikna edici
1 sayfalık bir toplantı brifi hazırla.
3. PM'in 'Şu an yeni özellik çıkarmamız lazım, bunu erteleyelim' itirazına verilecek
profesyonel ve yapıcı karşı argümanı yaz.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# Kötü Mühendis İletişimi:
"Kodlar spagetti oldu, kimse anlamıyor. Bu sprint yeni özellik yazamam, refactor
yapmam lazım."
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
# İş Odaklı Mühendislik Teklifi:

## 🎯 Konu: Ödeme Altyapısı Hızlandırma ve Çökme Önleme Yatırımı

Sayın Ürün Ekibi,

Son 3 sprint verilerimizi incelediğimizde:
- Ödeme sayfamız ortalama **4.2 saniyede** yükleniyor (Sektör ideali: < 1.5 saniye).
- Yavaşlık sebebiyle sepetten dönen kullanıcı oranımız yaklaşık **%8**.
- Ayrıca ödeme kodunun karmaşıklığı nedeniyle yeni bir ödeme yöntemi eklemek 2 hafta sürüyor.

### 💡 2 Haftalık Odaklanmış İyileştirme Planımız:
1. Veritabanı sorgularını optimize ederek sayfa yüklenme süresini **1.2 saniyeye** indireceğiz.
2. Ödeme kodunu modüler hale getirerek bir sonraki çeyrekte planlanan 'Apple Pay' entegrasyon süresini **2 haftadan 3 güne** düşüreceğiz.

### 📈 Beklenen İş Getirisi:
- Sepet tamamlama oranında tahmini **%5-%7 ciro artışı**.
- Gelecek sprintlerde yeni özellik teslim hızında **%40 ivmelenme**.
```

💡 **Kod Analizi:** Bu dille konuşan bir mühendis, şirketin büyümesini dert edinen bir iş ortağıdır. PM bu teklifi reddetmez; tam aksine heyecanla destekler.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Toplantılarda teknik jargona boğulup PM'in veya tasarımcının gözünü korkutmak.
  - **Çözüm:** Metaforlar, basit analojiler ve kullanıcı deneyimi metrikleriyle konuşun.
- **Tuzak:** Tasarımcının çizdiği zor bir UI için 'Bu imkansız, yapılmaz' demek.
  - **Çözüm:** 'Bunu birebir yapmak 3 hafta sürer; ancak şu küçük değişiklikle 2 günde %90 benzerini yapabiliriz' deyin.
- **Tuzak:** Şirketin para kazanma modelini (Business Model) hiç merak etmemek.
  - **Çözüm:** Şirketinizin nereden para kazandığını ve hangi özelliğin ciro getirdiğini mutlaka öğrenin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 56: Teknik İletişim - Mühendislikten İş Değerine Köprü*

- 🖥️ **Slayt 1:** Neden şirketler kod için değil, değer için yaşar?

- 🖥️ **Slayt 2:** Jargon duvarını yıkmak: Teknik dilden iş diline tercüme.
- 🖥️ **Slayt 3:** Tasarımcılar ve PM'lerle 'Yapıcı Takas (Trade-off)' müzakeresi.
- 🖥️ **Slayt 4:** Teknik borcu yönetime satma ve onay alma stratejisi.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir sprint planlama toplantısı canlandırın. 'Kodu yeniden yazmalıyız' diyen mühendis ile 'Bu optimizasyon müşteri dönüşümünü %8 artıracak' diyen mühendisin toplantıdaki etkisini karşılaştırın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Şirketinizdeki veya projenizdeki bir teknik ihtiyacı alın. Bunu tek bir teknik terim kullanmadan, bir işletme sahibine anlatır gibi 3 cümlelik bir iş teklifine dönüştürün.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Bir yazılım mühendisi ürün yöneticisine (PM) teknik bir borcu açıklarken hangi yaklaşımı seçmelidir? - A) 'Kodlar çok kötü, sen anlamazsın' demek - B) Teknik borcun geliştirme hızını nasıl yavaşlattığını, yeni özelliklerin çıkış süresini ne kadar geciktirdiğini ve çözüldüğünde şirkete sağlayacağı hız/ciro getirisini anlatmak - C) Sadece kızgın emojiler atmak - D) Hiçbir şey söylemeden gizlice kodu değiştirmek > **Doğru Yanıt:** B > **Açıklama:** Yöneticiler iş sonuçlarıyla ilgilenir. Teknik borcu geliştirme hızı, hata riski ve finansal getiri üzerinden açıklamak anında onay almanızı sağlar.

Soru 2: Bir tasarımcının sunduğu arayüzü kodlamak çok zorsa profesyonel bir mühendis nasıl yanıt vermelidir? - A) 'Bu imkansız, çizimi çöpe at' demek - B) Tasarımın neden zor olduğunu açıklayıp, aynı kullanıcı hedefine hizmet eden ve çok daha kısa sürede yapılabilecek pratik bir alternatif önermek - C) Tasarımcıyla küsmek - D) Çizimi görmezden gelip kafasına göre yapmak > **Doğru Yanıt:** B > **Açıklama:** Mühendislik takasları (trade-off) yönetme sanatıdır. 'Olmaz' demek yerine alternatif çözüm sunmak saygı uyandırır.

---

57. AI Araçlarıyla Portföy ve Açık Kaynak Projesi İnşa Etmek

Sıradan 'To-Do App' Projelerini Çöpe Atıp Fark Yaratan İşler Üretmek
Bölüm 10 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 12 dk

---

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"İşe alım yöneticileri neden adayın GitHub'ında 10 tane yarım kalmış tutorial projesi görmekten bıktı ve tek bir gerçek açık kaynak katkısı nasıl iş teklifi getirir?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Tutorial çöplüğü (Tutorial Hell) projeleri (To-Do list, basit klonlar) artık hiçbir şirketi etkilemiyor.
- 📌 İyi bir portföy projesi: Gerçek bir problemi çözen, canlıda çalışan (deployed), testleri olan ve temiz dokümante edilmiş tek bir projedir. - 📌 AI araçları, popüler açık kaynak kütüphanelerindeki (open source) bekleyen hata biletlerini (Issues) analiz edip ilk PR'ınızı açmanızı 10 kat kolaylaştırır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir resim sergisine gittiğinizi düşünün.

Duvarda 50 tane boyama kitabı sayfası asılı. Çocuklar numaralara göre boyamış: 1 mavi, 2 sarı… O ressamı Louvre Müzesi'ne davet eder misiniz? Asla! Çünkü o sadece başkasının çizgilerini doldurmuştur (Tutorial Hell).

Oysa köşede duran, orijinal, fırça darbeleri kendine has, çerçevesi özenle yapılmış tek bir tablo tüm sergiyi büyüler!

Yazılımda da GitHub profiliniz sizin resim serginizdir. YouTube'dan kopyalanmış 20 tane yarım 'Netflix Klonu' yerine; gerçek bir problemi çözen, Docker ile paketlenmiş, testleri yeşil yanan ve canlıda çalışan tek bir açık kaynak araç sizi yüzlerce adayın önüne geçirir.

🧠 2. AI Çağında Yeni Zihniyet Modeli

İşe Alım Yöneticisini Etkileyen Portföy Formülü:

```
┌──────────────────────────────────────────────┐
│              10x PORTFÖY PROJESİ FORMU       │
└──────────────────────┬───────────────────────┘
                       │
  1. GERÇEK PROBLEM: "Geliştiricilerin .env dosyalarındaki eksikleri bulan CLI
aracı"
  2. CANLI ÇALIŞMA: Tek tıkla çalışan demo veya npm/pip paketi.
  3. TEST KAPSAMI: Jest testleri ve yeşil yanan GitHub Actions CI rozeti.
  4. MÜKEMMEL README: Mimari şema (Mermaid), kurulum kılavuzu, GIF animasyonu.
  5. TEMİZ GİT GEÇMİŞİ: Conventional Commits standardında 30+ düzenli commit.
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Açık Kaynak Topluluk Lideri ve Portföy Danışmanı
GitHub'da açık kaynak bir CLI aracı yayınlayacağız. Proje: 'Git commit mesajlarını
analiz edip Conventional Commits standartlarına uygunluğunu denetleyen linter'.

## GÖREV:
1. Bu proje için dünya standartlarında, rozetler (Badges) ve Mermaid diyagramı
içeren bir `README.md` şablonu hazırla.
2. Açık kaynak projelere ilk katkıyı (First Contribution) yapmak için 'Good First
Issue' etiketli biletleri AI ile nasıl çözeceğimi planla.
3. Bir iş görüşmesinde bu projeyi anlatırken öne çıkarılacak 3 mimari noktayı
belirle.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# commit-linter
Bu proje commitleri kontrol eder.
Kurmak icin npm install yazin.
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

````
# 🛡️ CommitShield: Intelligent Conventional Commit Linter

[![CI Quality Gate](https://github.com/username/commit-shield/actions/workflows/ci.yml/badge.svg)](https://github.com)
[![npm version](https://badge.fury.io/js/commit-shield.svg)](https://badge.fury.io/js/commit-shield)
[![License: MIT](https://img.shields.io/badge/License-MIT-blue.svg)](https://opensource.org/licenses/MIT)

**CommitShield**, Git commit mesajlarınızı Conventional Commits standartlarına göre AST seviyesinde denetleyen, hafif ve sıfır bağımlılıklı bir CLI aracıdır.

---

## ⚡ Hızlı Kurulum

```bash
# Global Kurulum
npm install -g commit-shield

# Git Pre-commit Hook Olarak Ekleme
commit-shield install-hook
````

🏗️ Mimari Akış Şeması

```
flowchart LR
    A[git commit] --> B[CommitShield CLI]
    B --> C{Geçerli Format mı?}
    C -->|Evet| D[✅ Commit Kabul Edildi]
    C -->|Hayır| E[❌ Hata: Conventional Commit Kuralı İhlali]
```

🧪 Testler ve Doğrulama

```
npm test -- --coverage # %98 Birim Test Kapsamı
```
```

💡 Kod Analizi:
Böyle bir README, CI rozetleri ve Mermaid şeması gören bir işe alım lideri adayın kodlama kalitesinden ve profesyonelliğinden anında emin olur.

⚠ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** GitHub'da sadece ödev veya YouTube klon projeleri barındırmak.
  - **Çözüm:** Profilinizi temizleyin; sadece gurur duyduğunuz en iyi 2-3 projeyi 'Pinned' (Sabitlenmiş) yapın.
- **Tuzak:** README dosyası olmayan veya sadece 'Proje Başlığı' yazan boş repolar bırakmak.
  - **Çözüm:** README projenizin vitrinidir; ekran görüntüleri, kurulum adımları ve test rozetleri ekleyin.
- **Tuzak:** Açık kaynak projelere sadece dokümantasyondaki bir harf hatasını düzelterek katkı yapmaya çalışmak.
  - **Çözüm:** 'Good First Issue' etiketli gerçek bir bug'ı testleriyle birlikte çözerek değer üretin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

🎥 Ders Başlığı: *Ders 57: Portföy ve Açık Kaynak – Fark Yaratan Mühendis Olmak*
- 🖥️ **Slayt 1:** Neden 'To-Do App' projeleri artık kimseyi etkilemiyor?
- 🖥️ **Slayt 2:** Mükemmel bir GitHub portföyünün 5 bileşeni.
- 🖥️ **Slayt 3:** AI kullanarak açık kaynak projelere ilk PR'ı açma kılavuzu.
- 🖥️ **Slayt 4:** Vitrin oluşturma: Mükemmel README ve mimari diyagramlar.

Canlı Kodlama / Ekran Kaydı Senaryosu:
GitHub'da açık kaynaklı bir depodaki (örn: Express veya Fastify) bir 'Good First Issue' biletini açın. AI asistanına bu hatayı analiz ettirip test senaryosunu nasıl hazırlayacağınızı canlı gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:
GitHub profilinizdeki en iyi projenizi seçin. Ona profesyonel bir `README.md`, rozetler (badges) ve bir Mermaid akış şeması ekleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: İşe alım yapan bir teknoloji liderinin bir adayın GitHub profilinde en çok görmek istediği şey nedir?
  - A) 100 farklı yarım bırakılmış tutorial klonu
  - B) Gerçek bir problemi çözen, testleri olan, CI/CD boru hattı yeşil yanan ve canlıda çalışan 1-2 adet yüksek kaliteli proje
  - C) Profil resminin yapay zeka olması
  - D) Sadece yeşil commit kutuları

> **Doğru Yanıt:** B

1. REPEAT (Tekrar Et & Anla): “Problemi doğru anladığımdan emin olmak için tekrar edeyim: Girdi pozitif tamsayılar mı olacak?”
2. EXAMPLES (Örnek Girdiler & Edge Caseler Çıkar): “Peki boş dizi veya tek elemanlı dizi gelirse çıktı ne olmalı?”
3. APPROACH (Sözde Kod ve Yaklaşım Tartışması): “Burada iki yol var: O(N^2) naive yaklaşım ya da O(N) Hash Map. İkincisini öneriyorum çünkü…”
4. CODE (Temiz ve Okunabilir Kod Yaz): Anlamlı değişken isimleri, early return ve modüler yapı.
5. TEST (Kodu Kendi Kendine Adım Adım Koştur): “Şimdi örnek bir ‘5’ girdisiyle fonksiyonu satır satır takip edelim…”
6. OPTIMIZE (Karmaşıklığı Açıkla): “Zaman karmaşıklığı O(N), alan karmaşıklığı O(N)‘dir.“

````
---

## 🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda
doğrudan kullanabilirsiniz:

```markdown
# ROL: FAANG / Tier-1 Şirket Kıdemli Mülakatçısı
Ben mülakata giren bir junior yazılım mühendisiyim. Bana 'Canlı Kodlama ve Sistem
Tasarımı' mülakatı simülasyonu yapacaksın.

## MÜLAKAT KURALLARI:
1. Bana gerçekçi, orta zorlukta bir problem sor (Örn: URL Kısaltıcı Servis veya Rate
Limiter tasarımı).
2. Doğrudan çözümü söyleme; sadece benim sorduğum netleştirme sorularına yanıt ver.
3. Çözümümü REACTO metoduna göre (İletişim, Edge Caseler, Big-O Analizi) puanla ve
yapıcı geri bildirim ver.
````

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Mülakatta Elenme Sebebi: Sessizce oturup kontrolsüz kod yazmak
function solve(a: any, b: any) {
  // Mülakatçı ne yaptığınızı bilmiyor, değişken isimleri anlamsız, edge case yok!
  let x = [];
  for (let i = 0; i < a.length; i++) { x.push(a[i] + b); }
  return x;
}
```
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// Mülakat Kazandıran Kod: İletişimli, Tip Güvenli, Sınır Değerleri Kontrollü
export function mergeSortedArrays(
  primaryList: readonly number[],
  secondaryList: readonly number[]
): number[] {
  // 1. Sınır Durum Koruması (Edge Cases)
  if (!primaryList.length) return [...secondaryList];
  if (!secondaryList.length) return [...primaryList];

  const mergedResult: number[] = [];
  let pointerA = 0;
  let pointerB = 0;

  // 2. İki İşaretçi (Two-Pointer) Deseni - O(N + M) Zaman, O(N + M) Alan
  while (pointerA < primaryList.length && pointerB < secondaryList.length) {
    if (primaryList[pointerA] <= secondaryList[pointerB]) {
      mergedResult.push(primaryList[pointerA]);
      pointerA++;
    } else {
      mergedResult.push(secondaryList[pointerB]);
      pointerB++;
    }
  }

  // 3. Kalan elemanları ekle
  while (pointerA < primaryList.length) {
    mergedResult.push(primaryList[pointerA++]);
  }
  while (pointerB < secondaryList.length) {
    mergedResult.push(secondaryList[pointerB++]);
  }

  return mergedResult;
}
```

💡 **Kod Analizi:** Mülakatçıya iki işaretçi (Two-Pointer) yaklaşımının neden O(N+M) zaman karmaşıklığıyla sıralama fonksiyonlarından (`sort` : O(N log N)) daha üstün olduğu sesli olarak açıklanır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Soruyu alır almaz hiç düşünmeden ve soru sormadan klavyeye sarılmak.
  - **Çözüm:** İlk 3-5 dakikayı sorular sorarak, sınır durumları netleştirerek geçirin.
- **Tuzak:** Mülakat esnasında dakikalarca tamamen sessiz kalmak.
  - **Çözüm:** Zihninizden geçenleri sesli düşünün: 'Şu an bir döngü kurmayı düşünüyorum çünkü…'
- **Tuzak:** Kodu bitirir bitirmez 'bitti' demek.

- **Çözüm:** Kodu mülakatçıya teslim etmeden önce bir örnek girdiyle satır satır kendi kendinize test edin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 58: AI Çağında Teknik Mülakatlar - REACTO Metodu*

- 🖥️ **Slayt 1:** LeetCode ezberciliğinin sonu: Şirketler artık neyi ölçüyor?
- 🖥️ **Slayt 2:** REACTO Metodu: Mülakatta adım adım kazanma stratejisi.
- 🖥️ **Slayt 3:** Sesli Düşünme (Thinking Out Loud) sanatı.
- 🖥️ **Slayt 4:** Canlı Mülakat Simülasyonu: Two-Pointer algoritması üzerinden vaka.

Canlı Kodlama / Ekran Kaydı Senaryosu:
AI'ı mülakatçı rolüne sokun. Bir LeetCode orta seviye soru isteyin. Sesli düşünerek promptlar yazın; AI'ın sınır durum sorularınızı nasıl yanıtladığını ve sizi nasıl puanladığını canlı gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bugün AI asistanınıza: 'Bana orta seviye bir algoritma mülakatı simülasyonu yap' deyin ve REACTO metodunu uygulayarak bir mülakat simülasyonunu tamamlayın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Teknik bir mülakatta bir soruyu çözmeye başlamadan önce yapılması gereken en doğru ilk adım nedir? - A) Hemen hızlıca kod yazmaya başlamak - B) Problemi kendi cümlelerinizle tekrar edip, girdi sınırları ve edge caseler hakkında netleştirici sorular sormak - C) Mülakatçıdan cevabı istemek - D) Bilgisayarı yeniden başlatmak > **Doğru Yanıt:** B > **Açıklama:** Mülakatçılar gerçek dünyadaki gibi belirsiz problemler verir; iyi bir mühendis önce sorular sorarak gereksinimleri netleştirir.

Soru 2: Mülakatta kod yazmayı bitirdikten hemen sonra yapılması gereken profesyonel hareket nedir? - A) 'Bitti' deyip susmak - B) Mülakatçının test etmesini beklemeden, belirlediğiniz örnek bir girdiyle kodu satır satır kendi kendinize çalıştırıp (dry run) doğrulamak - C) Kodu silmek - D) Odayı terk etmek > **Doğru Yanıt:** B > **Açıklama:** Kendi kodunu test eden mühendis, yazdığı işe sahip çıktığını ve dikkatli olduğunu kanıtlar; mülakatçının puanını en çok artıran adımdır.

---

59. Sürekli Öğrenme Sistemi: Değişen Teknolojide Geri Kalmama Formülü

Her Hafta Yeni Bir Framework Çıkarken Akıl Sağlığını Korumak
Bölüm 10 • Zorluk Derecesi: İleri Seviye • Tahmini Okuma Süresi: 12 dk

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
"Her ay yeni bir JavaScript framework'ü veya yeni bir AI modeli çıkıyor. Bir mühendis tükenmişlik (burnout) yaşamadan nasıl her zaman güncel kalabilir?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Teknolojiler değişir, prensipler kalıcıdır. React veya Vue gelip geçicidir; veri yapıları, HTTP ve temiz mimari ebedidir. - 📌 T-Shaped Mühendislik: Bir alanda çok derin uzmanlık (T'nin dikey çizgisi), diğer alanlarda genel vizyon (T'nin yatay çizgisi). - 📌 Haftalık 'Öğrenme Radarı' kurarak yapay zekayı haftalık teknoloji özetleyiciniz haline getirin.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Okyanustaki dalgaları düşünün.

Her gün binlerce dalga gelir, köpürür, kıyıya vurur ve yok olur. Bir sörfçü her gelen küçük dalgaya atlamaya çalışırsa 1 saatte yorulur, nefesi tükenir ve boğulur! (Teknoloji Yorgunluğu / Hype Chasing).

Usta bir sörfçü ne yapar? Okyanusun derinliklerindeki ana akıntıları (Currents) okur. Küçük köpüklere aldırmaz; arkadan gelen devasa ve güçlü dalgayı bekler ve üzerinde ustalıkla kayar.

Yazılımda o küçük dalgalar her hafta çıkan yeni kütüphanelerdir. Okyanusun derin dip akıntıları ise: - Veri Yapıları ve Algoritmalar - Dağıtık Sistemler Mimarisi - Ağ Protokolleri (TCP, HTTP) - İş Problemlerini Çözme Becerisi

Temellere odaklanan bir mühendis, teknoloji ne kadar değişirse değişsin asla boğulmaz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

T-Shaped Mühendis Modeli:



🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Baş Teknoloji Yöneticisi (CTO) ve Yaşam Boyu Öğrenme Koçu
Yazılım dünyasındaki hızlı değişim karşısında tükenmişlik (burnout) hissetmeden
sürdürülebilir bir 'Kişisel Gelişim Sistemi' kurmak istiyorum.

## GÖREV:
1. 'FOMA (Fear of Missing Out – Geri Kalma Korkusu)' tuzağından kurtulmak için hangi
teknolojileri derinlemesine, hangilerini sadece genel kültür olarak öğrenmem
gerektiğini filtreleyen karar matrisini kur.
2. Haftada sadece 3 saat ayırarak güncel kalmayı sağlayan 'Haftalık Öğrenme Rutini'
tasarla.
3. Bir makale veya yeni teknolojiyi yapay zekaya 5 dakikada 'İlk İlkeler (First
Principles)' seviyesinde özetletme promptu oluştur.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# Kötü / Yoran Öğrenme Yaklaşımı:
Twitter'da gördüğüm her yeni kütüphaneyi hemen indirip gece 3'e kadar tutorial
izlemek!
(2 hafta sonra yorgunluktan kod yazamaz hale gelmek...)
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
# Sürdürülebilir Öğrenme Sistemi: 3 Saatlik Haftalık Blok

## 📅 Haftalık Öğrenme Rutini (Haftada 3 Saat):
- **Cuma 16:00 - 17:00 (1 Saat - Radar Taraması):**
  - AI ile sektör bültenlerini özetlet (Changelog, Hacker News, InfoQ).
  - Sadece "Bu teknoloji bizim hangi problemimizi çözer?" sorusuna yanıt ara.

- **Cumartesi 10:00 - 12:00 (2 Saat - Derin Prototip):**
  - Seçilen tek bir kavramı (Örn: Redis Pub/Sub veya WebAssembly) izole bir repoda
kodla dene.
  - Notlarını kendi dijital bahçene (Obsidian / GitHub) markdown olarak kaydet.

## 🤖 AI 'İlk İlkeler (First Principles)' Promptu:
"Aşağıdaki yeni duyurulan teknolojiyi açıkla:
1. Hangi temel fiziksel/mimari darboğazı çözmek için icat edildi?
2. 20 yıl önceki hangi klasik bilgisayar bilimi kavramının modern bir uygulamasıdır?
3. Bu teknolojinin arkasındaki temel ödünleşim (Trade-off: Neyi feda ediyor?)
nedir?"
```

💡 **Kod Analizi:** Yeni çıkan her aracı değil, arkasındaki bilgisayar bilimi prensiplerini anlamak; 10 yıl sonra da dimdik ayakta kalan bir mühendis olmanın sırrıdır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Hype peşinde koşup her yeni aracı hemen şirket projelerine entegre etmeye çalışmak.
  - **Çözüm:** Yeni araçları önce kişisel projelerinizde (sandbox) deneyin; olgunlaşmadan prod'a sokmayın.
- **Tuzak:** Sürekli tutorial izleyip tek satır bağımsız kod yazmamak (Tutorial Hell).
  - **Çözüm:** İzlediğiniz her 30 dakikalık içerik için mutlaka 1 saat kendi başınıza kod yazın.
- **Tuzak:** Dinlenmeyi ve uyku düzenini ihmal ederek tükenmişlik (burnout) yaşamak.
  - **Çözüm:** Yazılımcılık bir maratondur, 100 metre koşusu değil; düzenli dinlenme en büyük üretkenlik aracıdır.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📹 Ders Başlığı: *Ders 59: Sürekli Öğrenme ve Teknoloji Yorgunluğunu Aşmak*

- 🖥️ **Slayt 1:** Neden teknolojiler değişir ama prensipler kalıcıdır?
- 🖥️ **Slayt 2:** T-Shaped Mühendislik: Derinlik ve genişlik dengesi.
- 🖥️ **Slayt 3:** Haftada 3 saat ile tükenmeden güncel kalma rutini.

• 🖥️ **Slayt 4:** Yapay zekayı kişisel araştırma ve özetleme asistanı yapmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Karmaşık 40 sayfalık teknik bir dökümanı (örneğin Raft Consensus algoritması) AI'a verin. 'İlk İlkeler (First Principles) promptu' ile konunun özünü 2 dakikada çıkardığını ve nasıl kalıcı öğrenileceğini canlı gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bu hafta takviminizde sadece 2 saatlik kesintisiz bir 'Öğrenme Bloğu' ayırın ve uzun zamandır merak ettiğiniz bir temel konuyu (örn: veritabanı indeksleri nasıl çalışır) kodla deneyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: 'T-Shaped Mühendis' kavramı neyi ifade eder? - A) Sadece T harfiyle başlayan dilleri bilen kişiyi - B) Bir alanda çok derin uzmanlığa (T'nin dikey bacağı) sahipken, diğer ilgili mühendislik disiplinlerinde de genel vizyon ve iletişim yeteneğine (T'nin yatay çizgisi) sahip olan mühendisi - C) Sadece tişört giyen yazılımcıyı - D) Monitörü dikey kullanan kişiyi > **Doğru Yanıt:** B > **Açıklama:** T-Shaped mühendisler hem kendi ana uzmanlıklarında yüksek katma değer üretir hem de DevOps, frontend, backend gibi komşu ekiplerle kusursuz işbirliği yapar.

Soru 2: Hızla değişen yazılım teknolojileri karşısında tükenmişlik (burnout) yaşamamanın en sağlam kalkanı nedir? - A) Tüm yeni çıkan teknolojileri aynı anda öğrenmeye çalışmak - B) Geçici kütüphaneler yerine değişmeyen temel prensiplere (Veri yapıları, Algoritmalar, Ağ, Mimari) odaklanmak - C) Yazılımcılığı bırakmak - D) Sadece gece çalışmak > **Doğru Yanıt:** B > **Açıklama:** Temel bilgisayar bilimi prensiplerini sağlam öğrenen bir mühendis için yeni çıkan diller veya framework'ler sadece birkaç günlük söz dizimi (syntax) adaptasyonundan ibarettir.

---

60. Mezuniyet ve Manifesto: AI'ın Efendisi Olan Yeni Nesil Mühendis

Yapay Zekanın Değiştiremediği Tek Şey: İnsan Merakı, Vizyonu ve Yargı Yeteneği
Bölüm 10 • Zorluk Derecesi: Uzmanlık Seviyesi • Tahmini Okuma Süresi: 12 dk

---

🎙️ NotebookLM & Audio Overview Podcast Notları

Podcast Tartışma Sorusu:
“60 bölümlük bu yolculuğun sonunda: Yapay zeka yazılımcıların yerini mi alacak, yoksa yapay zekayı bir orkestra gibi yöneten mühendisler diğer herkesin yerini mi alacak?”

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Geleceğin kazananı yapay zekadan korkanlar değil; yapay zekayı bir enstrüman gibi çalan ‘Orkestra Şefi Mühendisler’dir. - 📌 Kodlama araçları ne kadar gelişirse gelişsin; empati kurma, problem tanımlama ve etik karar alma yeteneği daima insana aittir. - 📌 Bu kitap bir son değil; ömür boyu sürecek merak, üretim ve ustalık yolculuğunuzun ilk adımıdır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Kitabımızın 1. bölümündeki o ilk günü hatırlıyor musunuz? Ekranda yanıp sönen o yalnız imleç ve kulağınızdaki fısıltı: *“Yazılımcılık bitti mi?”*

Şimdi arkanıza yaslanın ve geçtiğiniz 60 bölümlük yolu izleyin: - Algoritmaları sözde kodla tasarladınız. - Big-O ile darboğazları ezdiniz. - Git ile çatışmaları çözüp CI/CD boru hatları kurdunuz. - Modüler monolitler ve tasarım kalıplarıyla sistemleri ölçeklendirdiniz. - Veritabanlarında indekslerle sorguları 5 milisaniyeye indirdiniz. - Test Piramidi ile kurşun geçirmez kalkanlar ördünüz. - Post-Mortem ile hatalardan bilgelik ürettiniz. - Ve güvenlik duvarlarıyla sistemleri siber saldırılara karşı kilitlediniz.

Gördünüz mü? Yazılımcılık ölmedi. Yazılımcılık kabuk değiştirdi! Klavye hamallığından, milyarlarca parametreli yapay zeka modellerini yöneten bir **Orkestra Şefliğine** yükseldi.

Baget artık sizin elinizde. Sahneye çıkın ve dünyanın en güzel yazılım senfonisini besteleyin!

🧠 2. AI Çağında Yeni Zihniyet Modeli

Yeni Nesil AI-Driven Mühendisin 10 Maddelik Manifestosu:

```
1. Ben bir klavye amelesi değilim; ben sistem kuran bir Problem Çözücüyüm.
2. Yapay zeka benim yerimi alamaz; çünkü doğruluğu denetleyen akıl benim.
3. Körü körüne güvenmem; her çıktıyı testlerle ve bilimsel şüpheyle doğrularım.
4. Mazeret üretmem; 'Makinemde çalışıyordu' demek yerine konteynerle garanti ederim.
5. Hatalardan korkmam; suçsuz post-mortem kültürüyle her çöküşü bilgeliğe
dönüştürürüm.
6. Kodun bekçisiyim; güvenlik ve veri mahremiyetini asla sonraya bırakmam.
7. Basitliği severim; erken optimizasyon ve gereksiz mikroservis çılgınlığından
kaçınırım.
8. Takımıma değer katarım; jargondan uzak, iş değerini ve empatiyi önceleyen bir
dille konuşurum.
9. Sürekli öğrenirim; geçici trendlere değil, zamansız bilgisayar bilimi temellerine
odaklanırım.
10. Ben yapay zekanın çırağı değil; onu yöneten Orkestra Şefiyim!
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Geleceğin Yazılım Mimarı ve Mentor
Tebrikler! 'AI Çağında Junior Yazılım Mühendisliği' kitabının tüm 60 bölümünü
tamamladın.

## MEZUNİYET GÖREVİ:
1. Bu kitaptan öğrendiğin en kritik 3 zihniyet değişimini özetle.
2. Kendine ait 1 yıllık 'Kariyer ve Uzmanlaşma Hedef Kartı'nı hazırla.
3. Bir sonraki projenin ilk commit mesajına şu sözü yaz:
   `feat(core): initialize project with AI orchestration and human craftsmanship`
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Dün: Yapay zekadan korkan, ne yaptığını bilmeden kopyala-yapıştır yapan acemi
// const code = ai.magic(); // "İnşallah çalışır..."
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
/**
 * Bugün: Ne istediğini bilen, bağlamı yöneten, test eden ve sistemi orkestre eden
mühendis.
 *
 * @manifesto
 * "Teknoloji değişir, ilkeler ebedidir.
 *  Yapay zeka hızlandırır, insan aklı yönetir."
 */

export interface FutureEngineer {
  readonly mindset: "Conductor";
  readonly verification: "Deterministic Tests";
  readonly empathy: "Business & User Focused";
  readonly velocity: "10x Accelerated by AI";
}

export const graduate: FutureEngineer = {
  mindset: "Conductor",
  verification: "Deterministic Tests",
  empathy: "Business & User Focused",
  velocity: "10x Accelerated by AI"
};

console.log("🎓 Tebrikler! AI Çağının Yeni Nesil Yazılım Mühendisi Olarak Mezun
Oldunuz!");
```

💡 **Kod Analizi:** 60 bölüm boyunca edindiğiniz bilgi birikimi; sizi sadece kod yazan biri değil, geleceğin teknoloji dünyasında söz sahibi olan bir sistem kurucu yaptı.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Kitabı bitirip ‘artık her şeyi öğrendim’ diyerek öğrenmeyi bırakmak.
  - **Çözüm:** Öğrenme asla bitmez; her yeni proje yeni bir öğrenme sahasıdır.
- **Tuzak:** Yapay zekanın hızına kapılıp temelleri (algoritmalar, testler) tekrar unutmak.
  - **Çözüm:** Her zaman ‘Bu kodu AI olmadan da anlayabiliyor muyum?’ sorusunu canlı tutun.
- **Tuzak:** Yalnız çalışıp topluluklardan ve açık kaynaktan kopmak.
  - **Çözüm:** Bilginizi başkalarıyla paylaşın, topluluklara katılın ve yeni gelen junior’lara mentorluk yapın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

🎥 **Ders Başlığı:** ***Ders 60: Mezuniyet & Manifesto - AI Senfonisinin Orkestra Şefi***

- 🖥️ **Slayt 1:** 60 Bölümlük Yolculuğun Büyük Resmi: Nereden Nereye Geldik?
- 🖥️ **Slayt 2:** Yeni Nesil Mühendisin 10 Maddelik Manifestosu.
- 🖥️ **Slayt 3:** Yapay zekanın asla değiştiremeyeceği insani süper güçler.
- 🖥️ **Slayt 4:** Kapanış ve Geleceğe Adım: Baget Artık Sizin Elinizde!

Canlı Kodlama / Ekran Kaydı Senaryosu:
Terminalinizi açın. Sıfırdan bir depo oluşturun, ilk commit'i manifesto mesajıyla atın ve bu 60 bölümlük eğitimin tamamlanışını kutlayın!

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

10 Maddelik Mühendislik Manifestosu'nu bir çıktısını alıp veya not uygulamanıza sabitleyip masanızın görebileceğiniz bir yerine asın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: AI çağında bir yazılım mühendisinin yapay zekaya karşı en büyük ve yeri doldurulamaz üstünlüğü nedir? - A) Daha hızlı klavye kullanması - B) İnsan empati yeteneği, gerçek hayat problemlerini anlama vizyonu, etik yargı ve sistemik doğrulama kabiliyeti - C) Asla uyumaması - D) Bilgisayardan daha az elektrik harcaması > **Doğru Yanıt:** B > **Açıklama:** Yapay zeka ne kadar gelişirse gelişsin; neyin inşa edileceğine karar verme, kullanıcıyla empati kurma ve etik doğruluk daima insanın alanıdır.

Soru 2: Bu 60 bölümlük eğitimin ana felsefesi olan 'Orkestra Şefi Mühendis' kavramı neyi özetler? - A) Sadece müzik yazılımları geliştirmeyi - B) Yapay zeka araçlarını birer enstrüman ve yardımcı stajyer olarak görüp; mimariyi, testleri ve sistemin yönünü tam hakimiyetle yöneten bilinçli mühendisi - C) Hiç kodlama bilmeyen yöneticileri - D) Sadece konserde çalışan yazılımcıları > **Doğru Yanıt:** B > **Açıklama:** Orkestra şefi mühendis, AI'ın gücünü arkasına alarak mimariyi, kaliteyi ve iş değerini yüksek vizyonla yöneten yeni nesil lider geliştiricidir.